

Оглавление

Билет №1	7
1. Числовая последовательность и её предел. Свойства пределов. Критерий Коши сходимости числовой последовательности.	7
1. Стандартные типы данных; представление основных структур программирования; типы данных, определяемые пользователем.	14
Билет №2	26
1. Предел функции и его свойства. Первый и второй замечательные пределы. Правило Лопиталя раскрытия неопределенностей.	26
2. Реляционные, объектно-ориентированные, реляционно-ориентированные БД; распределенные БД.	34
Билет №3	43
1. Производная функции одной переменной, дифференциал, их свойства и геометрический смысл. Дифференцирование сложной, обратной функции, заданной неявно и параметрически.	43
2. Системы Управления БД (СУБД); концептуальная модели БД; языки БД.	48
Билет №4	56
1. Дифференцирование функций нескольких переменных. Дифференциал и частные производные, Производная по направлению.	56
2. Постановка задачи и спецификация программы; способы записи алгоритма. Базовые конструкции в блок-схемах.	63
Билет №5	73
1. Необходимые и достаточные условия экстремума функции одной переменной Теоремы Ферма, Ролля, Лагранжа.	73

2. Интеллектуальный анализ данных. Понятие DATA mining. Предметно ориентированные аналитические системы. Статистические пакеты. Деревья решений. Системы для визуализации многомерных данных. Виды прогнозов и методы прогнозирования.....	77
Билет №6	80
1. Исследование функций одной переменной. Монотонность, экстремум, выпуклость, точки перегиба, асимптоты.	80
2. Понятие "неструктурированная информация", ее отличительные черты. Методы и технологии обработки неструктурированной информации. Средства моделирования информационных потоков и структур данных.	83
Билет №7	86
1. Первообразная и неопределенный интеграл. Основные правила интегрирования. Интегрирование рациональных функций (разложение на простейшие дроби) и некоторых иррациональных.	86
2. Понятие технического зрения. Назначение технического зрения. Проблемы регистрации цифровых изображений. Компоненты технического зрения. Реконструкция изображения методом решения обратных задач.....	95
Билет №8	98
1. Понятие определенного интеграла. Основные свойства и критерий интегрируемости, Классы интегрируемых функций.	98
2. Нейросетевой подход к растровому распознаванию. Нейросетевой классификатор. Искусственные нейронные сети для распознавания границ, форм и опознание объекта, сегментации и трекинга.	102
Билет №9	106

1. Понятие двойного интеграла и его свойства. Сведение к повторному интегралу. Нахождение объемов.....	106
2.Методологии структурного системного анализа и проектирования: SADT, структурного системного анализа Гейна - Сарсона, структурного анализа и проектирования Йордона / Де марко.....	111
Билет №10	115
1. Числовые ряды и их сумма; Критерии сходимости. Абсолютная и условная сходимость. Признаки сравнения, Коши, Даламбера, Лейбница сходимости числовых рядов.....	115
2. UML - унифицированный язык моделирования. Понятие диаграмм, процесса проектирования.	120
Билет №11	125
1. Степенные ряды. Область сходимости. Радиус сходимости. Разложение функции в степенной ряд.....	125
2. Основные этапы проектирования ИЭС. Каскадная и модифицированная, спиральная модели этапов проектирования (ЖЦ) ИЭС. Пять способов создания систем.	128
Билет №12	132
1. Матрицы, операции над ними. Определители n-го порядка, теорема Лапласа. Обратная матрица, ранг матрицы, базисный минор.	132
2. Виды обеспечения ИЭС. Организационно, лингвистическое, правовое. Состав и структура информационного обеспечения ИЭС. Состав и роли работников группы разработки проекта.	142
БИЛЕТ 13	146
2. Решение систем линейных уравнений по правилу Крамера, по методу Гаусса.	146

3. Собственные векторы и собственные значения оператора...	154
1. Состав и структура программного обеспечения ИЭС	159
2. Структура программных модулей	160
3. Отладка программ и программных комплексов.....	161
4. Верификация и документирование программного обеспечения	162
БИЛЕТ 14.....	163
1. Собственные векторы и собственные значения оператора...	163
1. Методы представления и обработки нечётких знаний в продукционных системах	168
2. Достоинства и недостатки правил-продукций	169
3. Примеры систем-оболочек продукционного типа	170
БИЛЕТ 15.....	171
1. Квадратичные формы, их невырожденные преобразования. Канонический вид. квадратичной формы. Приведение квадратичной формы к главным осям.....	171
1. Нейронные сети.....	179
1.1 Общие понятия.....	179
1.2 Элементы нейронных сетей	179
1.3 Архитектура нейронных сетей	180
1.4 Обучение нейронных сетей	180
2. Методы построения правил классификации: деревья решений	181
2.1 Основная идея.....	181
2.2 Индукция дерева (построение).....	181
2.3 Обработка и оптимизация.....	182
БИЛЕТ 16.....	183

1. Дифференциальные уравнения первого порядка, разрешенные относительно производной. Уравнения с разделяющимися переменными, однородные уравнения, уравнения в полных дифференциалах.	183
1. Формирование и сбор данных	188
2. Предобработка и очистка данных	188
3. Трансформация и обогащение данных.....	189
4. Построение моделей и анализ	190
5. Итоги подготовки данных	190
БИЛЕТ 17	192
1. Линейное однородное дифференциальное уравнение n -го порядка.	192
БИЛЕТ 18	197
1. Линейное однородное дифференциальное уравнение с постоянными коэффициентами.	197
БИЛЕТ 19	201
2. Линейное неоднородное дифференциальное уравнение n -ого порядка. Метод вариации постоянных нахождения решения. ..	201
1. Понятие системы. Классификация систем	204
БИЛЕТ 20	207
1. Линейное неоднородное дифференциальное уравнение n -ого порядка с постоянными коэффициентами, подбор частных решений при специальном виде правой части.	208
2. Основные положения теории моделирования: понятие модели, последовательность разработки математических моделей,	210
БИЛЕТ 21	222

1. Условная вероятность. Формула полной вероятности, формула Байеса.....	222
1. Основные положения и области применения имитационного моделирования.	225
БИЛЕТ 22	234
1. Независимые испытания. Формула Бернулли. Локальная предельная теорема. Теорема Муавра-Лапласа.	234
2. Случайные величины и функции распределения. Дискретные случайные величины, абсолютные непрерывные случайные величины.....	238
Беспроводная связь. Передача радиосигнала, передача в видимом световом диапазоне и ИК-диапазоне. Спутниковые системы связи. Мобильная связь.	244
БИЛЕТ 23	254
1. Случайные величины и функции распределения. Дискретные случайные величины, абсолютные непрерывные случайные величины.....	254
Методы повышения достоверности передачи.	261
БИЛЕТ 24	277
Алгоритмы сжатия информации.	277

1. Числовая последовательность и её предел. Свойства пределов.
Критерий Коши сходимости числовой последовательности.

Числовой последовательностью называется бесконечное множество чисел $x_1, x_2 \dots x_n$, следующих одно за другим в определенном порядке и построенных по определенному закону, с помощью которого x_n задается как функция целочисленного аргумента n , то есть $x_n = f(n)$.

Число a называется **пределом последовательности** $\{x_n\}$, если для любого числа $\varepsilon > 0$ существует такой номер $n_0 = n_0(\varepsilon)$, что при $n \geq n_0$ выполняется неравенство $|x_n - a| < \varepsilon$.

Если число a – это предел последовательности $\{x_n\}$, то это обозначают как $\lim_{n \rightarrow \infty} x_n = a = \lim_n x_n = a$, или $x_n \rightarrow a$ при $n \rightarrow \infty$, или $x_n \xrightarrow{n \rightarrow \infty} a$

Теоремы числовых последовательностей:

Теорема 1: Числовая последовательность не может иметь более одного предела.

Последовательность, имеющая предел, называется сходящейся. В противном случае числовая последовательность называется расходящейся.

Для сходящихся числовых последовательностей имеют место следующие теоремы.

Теорема 2: Предел суммы/разности двух последовательностей равен сумме/разности пределов от каждой из них, если последние существуют: $\lim_{n \rightarrow \infty} (x_n \pm y_n) = \lim_{n \rightarrow \infty} x_n \pm \lim_{n \rightarrow \infty} y_n$

Пример:

$$\lim_{n \rightarrow \infty} \left(\frac{1}{n} + \frac{1}{n^2} \right) = \lim_{n \rightarrow \infty} \frac{1}{n} + \lim_{n \rightarrow \infty} \frac{1}{n^2} = 0 + 0 = 0$$

ТЕОРЕМА 3

Предел произведения двух последовательностей равен произведению пределов от каждой из них, если пределы сомножителей существуют:

$$\lim_{n \rightarrow \infty} (x_n \cdot y_n) = \lim_{n \rightarrow \infty} x_n \cdot \lim_{n \rightarrow \infty} y_n$$

Пример:

$$\lim_{n \rightarrow \infty} \left(\frac{2}{n} \right) = \lim_{n \rightarrow \infty} \left(2 \cdot \frac{1}{n} \right) = \lim_{n \rightarrow \infty} 2 + \lim_{n \rightarrow \infty} \frac{1}{n} = 2 + 0 = 2$$

ТЕОРЕМА

Предел отношения двух последовательностей равен отношению пределов от каждой из них, если эти пределы существуют и предел знаменателя не равен нулю:

$$\lim_{n \rightarrow \infty} \frac{x_n}{y_n} = \frac{\lim_{n \rightarrow \infty} x_n}{\lim_{n \rightarrow \infty} y_n}, \quad \lim_{n \rightarrow \infty} y_n \neq 0$$

Основные свойства пределов

1. Функция в данной точке может иметь только один предел.
2. Предел постоянной равен самой этой постоянной: $\lim_B c = c$, c – постоянная.

3. Предел суммы функций равен сумме пределов этих функций:

$$\lim_B (f_1(x) + f_2(x)) = \lim_B f_1(x) + \lim_B f_2(x)$$

4. Предел произведения функций равен произведению пределов этих функций:

$$\lim_B (f_1(x) * f_2(x)) = \lim_B f_1(x) * \lim_B f_2(x)$$

1. Отсюда следует, что постоянный множитель можно выносить за знак предела:

$$\lim_B (c * f(x)) = c * \lim_B f(x)$$

5. Предел частного двух функций равен частному пределов этих функций (если предел делителя не равен нулю):

$$\lim_B \left(\frac{f_1(x)}{f_2(x)} \right) = \lim_B f_1(x) / \lim_B f_2(x), \quad \text{при } \lim_B f_2(x) \neq 0$$

6. (свойство предела сложной функции) Если $\lim_{x \rightarrow x_0} f_2(x) = u_0$, $\lim_{u \rightarrow u_0} f_1(u) = A$, то предел сложной функции

$$\lim_{x \rightarrow x_0} f_1(f_2(x)) = A.$$

7. Если при базе B (т.е. в некоторой окрестности точки x_0 или при достаточно больших x) $f_1(x) < f_2(x)$, то $\lim_B f_1(x) \leq \lim_B f_2(x)$.

Отметим, что в перечисленных свойствах предполагается существование пределов функций $f_1(x)$ и $f_2(x)$, из чего следуют заключения о значениях пределов суммы, произведения или частного этих функций. Но при этом из существования предела суммы, произведения или частного функций не обязательно следует, что существуют пределы самих слагаемых, сомножителей или делимого и делителя.

Например, $\lim_{x \rightarrow \frac{\pi}{2}} (\operatorname{tg} x + \operatorname{ctg} x) = \lim_{x \rightarrow \frac{\pi}{2}} 1 = 1$, но при этом $\lim_{x \rightarrow \frac{\pi}{2}} \operatorname{tg} x$ не существует.

Свойства предела функции.

- Функция в данной точке может иметь только один предел.
- Если функция $f(x)$, $g(x)$ в точке $x=a$ имеют пределы, соответственно A, B то
- $\lim(f(x)+g(x))=A+B$
- $\lim(f(x)*g(x))=A*B$
- $\lim(f(x)/g(x))=A/B, B \neq 0$
- $\lim(cf(x))=c\lim f(x)$

Критерий Коши сходимости числовой последовательности.

Для того, чтобы последовательность имела конечный предел, необходимо и достаточно, чтобы она удовлетворяла условию Коши.

Доказательство необходимости

Пусть последовательность $\{x_n\}$ сходится к конечному пределу a :

$$\lim_{n \rightarrow \infty} x_n = a$$

Это означает, что имеется некоторая функция $N_1(\varepsilon_1)$, так что для любого $\varepsilon_1 > 0$ выполняется неравенство:

$$(1.1) \quad |x_n - a| < \varepsilon_1 \text{ при } n > N_1(\varepsilon_1).$$

Покажем, что последовательность $\{x_n\}$ удовлетворяет условию Коши. Для этого нам нужно найти такую функцию $N(\varepsilon)$, , при которой, для любого $\varepsilon > 0$, выполняются неравенства:

$$|x_n - x_m| < \varepsilon$$

при $n > N(\varepsilon), m > N(\varepsilon)$.

Воспользуемся свойствами неравенств и применим (1.1):

$$\begin{aligned} |x_n - x_m| &= |x_n - a - (x_m - a)| \leq |x_n - a| + |x_m - a| < \varepsilon_1 + \varepsilon_1 \\ &= 2\varepsilon_1 \end{aligned}$$

Последнее неравенство выполняется при $n > N_1(\varepsilon_1), m > N_1(\varepsilon_1)$.

Заменим ε_1 на $\varepsilon/2$. Тогда для любого $\varepsilon > 0$ имеем:

$$|x_n - x_m| < \varepsilon$$

при $n > N(\varepsilon), m > N(\varepsilon)$.

где $N(\varepsilon) = N_1(\varepsilon/2)$.

Необходимость доказана.

Доказательство достаточности

Пусть последовательность $\{x_n\}$ удовлетворяет условию Коши.

Докажем, что она сходится к конечному числу. Доказательство разделим на три части. Сначала докажем, что последовательность ограничена. Затем применим теорему Больцано – Вейерштрасса, согласно которой у ограниченной последовательности существует подпоследовательность, сходящаяся к конечному числу. И наконец, покажем, что к этому числу сходится вся последовательность.

Докажем, что последовательность

$\{x_n\}$, удовлетворяющая условию Коши, ограничена. Для этого, в условии Коши, положим $\varepsilon = 1$. Тогда существует такое натуральное число N_1 , при котором выполняются неравенства:

$$|x_n - x_m| < 1$$

(2.1.1) при $n > N_1, m > N_1$.

Возьмем любое натуральное число $m > N_1$ и зафиксируем член последовательности x_m . Обозначим его как x_0 , чтобы подчеркнуть, что это постоянное, не зависящее от индекса n число.

Подставляем в (2.1.1) и выполняем преобразования.

При $n > N_1$ имеем:

$$|x_n - x_0| < 1;$$

$$-1 < x_n - x_0 < 1;$$

$$-1 + x_0 < x_n < 1 + x_0;$$

$$-1 - |x_0| \leq -1 + x_0 < x_n < 1 + x_0 \leq 1 + |x_0|;$$

$$|x_n| < 1 + |x_0|.$$

Отсюда видно, что при $n > N_1$ члены последовательности ограничены. Поскольку, при $n \leq N_1$, имеется только конечное число членов, то и вся последовательность $\{x_n\}$ ограничена.

Применим теорему Больцано – Вейерштрасса. Обозначим такую подпоследовательность как $\{x_{nk}\}$. Тогда

$$\lim_{k \rightarrow \infty} x_{nk} = a.$$

Покажем, что к числу a сходится вся последовательность.

Поскольку последовательность $\{x_n\}$ удовлетворяет условию Коши, то имеется некоторая функция $N(\varepsilon_1)$, при которой для любого $\varepsilon_1 > 0$ выполняются неравенства:

$$|x_n - x_m| < \varepsilon_1$$

при $n > N(\varepsilon_1), m > N(\varepsilon_1)$.

В качестве возьмем член сходящейся подпоследовательности и

заменим ε_1 на $\varepsilon/2$:

$$(2.3.1) \quad \text{при } n > N\left(\frac{\varepsilon}{2}\right), \quad n_k > N(\varepsilon/2).$$

Зафиксируем n . Тогда (2.3.1) является неравенством, содержащим последовательность $\{x_{nk}\}$, у которой исключено конечное число первых членов с $n_k \leq N(\varepsilon/2)$. Конечное число первых членов не влияет на сходимость. Поэтому предел при $k \rightarrow \infty$ усеченной последовательности $\{x_{nk}\}$ по-прежнему равен a . Применяя свойства пределов, связанные с неравенствами и арифметические свойства пределов, при $k \rightarrow \infty$, из (2.3.1) имеем:

$$|x_n - a| \leq \varepsilon/2 \quad \text{при } n > N(\varepsilon/2)$$

Воспользуемся очевидным неравенством: $\varepsilon/2 < \varepsilon$. Тогда $|x_n - a| < \varepsilon$

$$\text{при } n > N(\varepsilon/2).$$

То есть для любого $\varepsilon > 0$ существует натуральное число $N_1(\varepsilon) = N(\varepsilon/2)$, так что $|x_n - a| < \varepsilon$
при $n > N_1(\varepsilon)$..

Это означает, что число a является пределом всей последовательности $\{x_n\}$ (а не только ее подпоследовательности $\{x_{nk}\}$).

Теорема доказана

Условие Коши

Последовательность $\{x_n\}$ удовлетворяет условию Коши, если для любого положительного действительного числа $\varepsilon > 0$ существует такое натуральное число N_ε , что

$$|x_n - x_m| < \varepsilon \quad \text{при } n > N_\varepsilon, m > N_\varepsilon$$

Последовательности, удовлетворяющие условию Коши, называют фундаментальными последовательностями.

Число N_ε зависит от ε . Т.е. $N(\varepsilon)$. если оно является функцией от действительной переменной ε , областью значений которой является множество натуральных чисел.

Условие Коши

Последовательность $\{x_n\}$ удовлетворяет **условию Коши**, если для любого положительного действительного числа $\varepsilon > 0$ существует такое натуральное число N_ε , что

$$(1) \quad |x_n - x_m| < \varepsilon \text{ при } n > N_\varepsilon, m > N_\varepsilon.$$

Последовательности, удовлетворяющие условию Коши, также называют **фундаментальными последовательностями**.

Условие Коши можно представить и в другом виде. Пусть $m > n$. Если $m < n$, то поменяем n и m местами. Случай $n = m$ нас не интересует, поскольку при этом неравенство (1) выполняется автоматически. Имеем:

$$m = n + m - n = n + p;$$

$$|x_n - x_m| = |x_m - x_n| = |x_{n+p} - x_n|.$$

Здесь p – натуральное число.

Тогда условие Коши можно сформулировать так:

Последовательность $\{x_n\}$ удовлетворяет **условию Коши**, если для любого $\varepsilon > 0$ существует такое натуральное число N_ε , что

$$(2) \quad |x_{n+p} - x_n| < \varepsilon \text{ при } n > N_\varepsilon \text{ и любых натуральных } p.$$

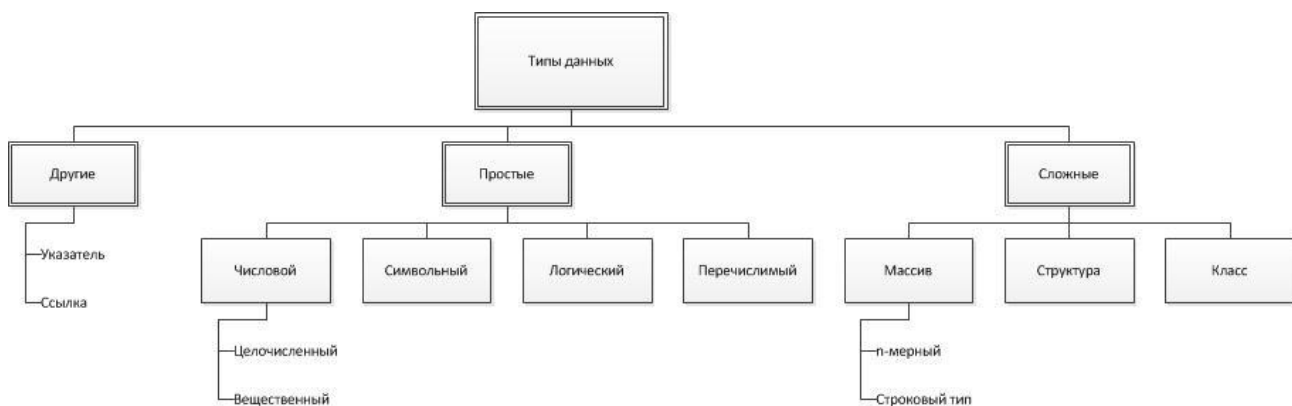
Число N_ε , фигурирующее в условии Коши, зависит от ε . То есть оно является функцией от действительной переменной ε , областью значений которой является множество натуральных чисел

Число N_ε также можно записать в виде $N(\varepsilon)$, как это принято для обозначения функций.

1. Стандартные типы данных; представление основных структур программирования; типы данных, определяемые пользователем.

Тип данных - фундаментальное понятие языка программирования. Тип данных определяет, что именно представляют собой данные, как они хранятся в памяти, какие операции с ними можно выполнять.

Классификация типов данных



Изначально, типы данных делятся на простые и составные.

Простой - это тип данных, объекты (переменные или постоянные) которого не имеют доступной программисту внутренней структуры.

Для объектов **составного** типа данных, в противовес простому, программист может работать с элементами внутренней его структуры.

Числовой тип данных разработан для хранения естественно чисел.

Символьный - для хранения одного символа.

Логический тип имеет два значения: истина и ложь.

Перечислимый тип может хранить только те значения, которые прямо указаны в его описании.

Для простых типов данных определяются границы диапазона и количество байт, занимаемых ими в памяти компьютера.

В большинстве языков программирования, простые типы жестко связаны с их представлением в памяти компьютера. Компьютер хранит данные в виде последовательности битов, каждый из которых может иметь значение 0 и 1. Фрагмент данных в памяти может выглядеть следующим образом

00011011011100010110010000111011 . . .

Данные на битовом уровне (в памяти) не имеют ни структуры, ни смысла. Как интерпретировать данные, как целочисленное число, или вещественное, или символ, зависит от того, какой тип имеют данные, представленные в этой и последующих ячейках памяти.

Стандартные типы данных

Числовые типы данных

Целочисленные типы данных

Выделяют знаковые и беззнаковые. Как видно из названия, знаковые предназначены для хранения как положительных, так и отрицательных значений, нуль, а беззнаковые - чисел, не меньше нуля.

Беззнаковые типы данных, в отличие от соответствующих знаковых, имеют в два раза больший диапазон. Это из-за их машинного представления. В знаковых типах первый бит указывает на знак числа: 1 - отрицательное, 0 - положительное.

Исходя из машинного представления целого числа, в ячейке памяти из n бит может храниться 2^n для беззнаковых, и 2^{n-1} для знаковых типов.

Типы данных в C#:

Тип	Разрядность в битах	Диапазон чисел
byte	8	0 - 255
sbyte	8	-128 - 127
short	16	-32 768 - 32 767
ushort	16	0 - 65 535
int	32	-2 147 483 648 - 2 147 483 647
uint	32	0 - 4 294 967 295
long	64	-9 223 372 036 854 775 808 - 9 223 372 036 854 775 807
ulong	64	0 - 18 446 744 073 709 551 615

Типы данных в C++:

Тип	Разрядность в битах	Диапазон чисел
unsigned short int / unsigned short	16	0 - 65 535
short int	16	-32 768 - 32 767
unsigned long int / unsigned long	32	0 - 4 294 967 295
long int / long	32	-2 147 483 648 - 2 147 483 647
int (16 разрядов)	16	-32 768 - 32 767
int (32 разряда)	32	-2 147 483 648 - 2 147 483 647
unsigned int (16 разрядов)	16	0 - 65 535
unsigned int (32 разряда)	32	0 - 4 294 967 295

У некоторых типов есть приписка "16 разрядов" или "32 разряда". Это означает, что в зависимости от разрядности операционной системы и компилятора данный тип будет находиться в соответствующем диапазоне. Поэтому, рекомендуется не использовать int, unsigned int, а использовать их аналоги, но уже жестко определенные, short, long, unsigned short, unsigned long.

Вещественные типы данных

Числа вещественного типа данных задаются в форме чисел с плавающей запятой.

Плавающая запятая — форма представления действительных чисел, в которой число хранится в форме мантиссы и показателя степени. В случае языков программирования, любое число может быть представлено в следующем виде

(1)

$$N = M * 10^p$$

где N — записываемое число;

M — мантисса;

p (целое) — порядок.

Например: $14441544 = 1,4441544 * 10^7$; $0,0004785 = 4,785 * 10^{-4}$.

Компьютер же на экран выведет следующие числа:

1,4441544E+7; 4,785E-4

Следовательно, в отведенной памяти хранится мантисса и порядок записываемого числа. Рассмотрим на примера типа данных, который хранится в 8 байтах или 64 битах. В данном случае, мантисса составляет 53 бита: 1 для знака числа и 52 для её значения; порядок 10 битов: 1 бит для знака и 10 для значения. Мы можем в данном случае говорить о диапазоне точности, то есть насколько малое и насколько большое число может хранить данный тип данных: $4,94 \times 10^{-324}$ до $1,79 \times 10^{308}$. Но, поскольку, память компьютера не безразмерна, да и далеко не всегда нужно, храниться несколько первых разрядов мантиссы, которые называются значащими.

Вывод: вещественные типы данных, в отличии от целочисленных, характеризуются диапазоном точности и количеством значащих разрядов.

Вещественные типы данных в C#:

Тип	Разрядность в битах	Количество значащих цифр	Диапазон точности
float	32	7	от $1,5 * 10^{-45}$ до $3,4 * 10^{38}$
double	64	15	от $4,9 * 10^{-324}$ до $1,7 * 10^{308}$
decimal	128	28	от $1,0 * 10^{-28}$ до $7,9 * 10^{28}$

Вещественные типы данных в C++:

Тип	Разрядность в битах	Количество значащих цифр	Диапазон точности
float	32	7	от $1,5 * 10^{-45}$ до $3,4 * 10^{38}$
double	64	15	от $4,9 * 10^{-324}$ до $1,7 * 10^{308}$

Символьный тип данных

Значение переменной этого типа данных представляет собой один символ. В действительности, это есть целое число. В зависимости от кодировки, это число превращается в некий символ. Данные типы данных характеризуются лишь размером выделяемой под них памяти.

Логический тип данных

Это тип данных, значения которых могут быть: true (правда) или false (ложь). Логическому типу данных соответствуют в C# и C++ тип bool, в Java - boolean.

Перечислимый тип данных

Во внутреннем представлении, это целочисленный тип данных, только здесь пользователь вместо числа использует заранее определенные строковые значения.

Чтобы прочувствовать эту концепцию, приведем пример на языке C++ (в C# и Java аналогично)

```
enum Forms {shape, sphere, cylinder, polygon};
```

Теперь переменные перечислимого типа Forms могут принимать лишь значения, определенные в примере кода. Это очень удобно, ведь мы уже оперируем не с числами, а с некими смысловыми значениями, замечу лишь, что для компьютера эти значения всё равно являются целыми числами.

Массив

Далее перейдем к сложным типам данных. **Массив** - это набор однотипных переменных, расположенных в памяти непосредственно друг за другом, доступ к которым осуществляется по индексу. Индекс массива — целое число, указывающее на конкретный элемент массива.

Каждый массив характеризуется типом данных его элементов, который может быть как простым, так и сложным, то есть любым.

В языках программирования нельзя оперировать всем массивом, работают с конкретным элементом.

```
array[0]
```

Индекс имеет тип данных чаще всего int.

Пользовательские типы данных

Структура

Структура - конструкция, позволяющая содержать в себе набор переменных различных типов.

Структуры реализованы в языке программирования, чтобы собрать некие близкие по смыслу вещи воедино.

Например, есть колесо автомобиля. У колеса есть диаметр, толщина, шина. Шина в свою очередь является структурой, у которой есть свои параметры: материал, марка, чем заполнена. Естественно, для каждого параметра можно создать свою переменную или константу, у нас появится большое количество переменных, которые, чтобы понять к чему они относятся, нужно в именах общую часть выделять. Имена будут нести лишнюю смысловую нагрузку. Получается запутанная история. А так мы определяем две структуры, а затем параметры в них.

```
struct Tyre
```

```
{
```

```
    Material material;
```

```
    int mark;
```

```
};  
  
struct Wheel  
{  
    double diameter;  
    double thickness;  
    Tyre tyre;  
}
```

Класс

Еще одним пользовательским типом данных является класс. Класс умеет всё, что и структура, но кроме параметров, у него есть и методы, и поддерживает большое количество вещей, связанных с объектно-ориентированным программированием.

Представление основных структур программирования.

Операторы действия

Инструкция или оператор – наименьшая автономная часть языка программирования; команда. Программа обычно представляет собой последовательность инструкций.

Например C#: if, if-else, while, do-while, for, switch-case, return и т.д. Также операторам можно считать объявление переменной (int a).

Оператор ветвления

Оператор ветвления (условная инструкция, условный оператор) — оператор, конструкция языка программирования, обеспечивающая выполнение определённой команды (набора команд) только при условии истинности некоторого логического выражения, либо

выполнение одной из нескольких команд (наборов команд) в зависимости от значения некоторого выражения.

Конструкция *if/else*

Оператор **if** определяет, какой оператор будет выполняться при выполнении условия, заданного логическим выражением.

Переключатель (*switch*)

switch — это оператор выбора, который выбирает для выполнения один раздел switch из списка кандидатов, сравнивая их с выражением соответствия.

Конструкция if/else	Конструкция switch (переключатель)
<pre>if (a == b) { ... } else { ... }</pre>	<pre>Console.WriteLine("Нажмите Y или N"); string selection = Console.ReadLine(); switch (selection) { case "Y": Console.WriteLine("Вы нажали букву Y"); break; case "N": Console.WriteLine("Вы нажали букву N"); break;</pre>

	<pre> default: Console.WriteLine("Вы нажали неизвестную букву"); break; } </pre>
--	---

Оператор цикла

Цикл — разновидность управляющей конструкции в высокоуровневых языках программирования, предназначенная для организации многократного исполнения набора инструкций.

Цикл for. Первая часть объявления цикла - <code>int i = 0</code> - создает и инициализирует счетчик <code>i</code>. Перед выполнением цикла его значение будет равно 0. Вторая часть - условие, при котором будет выполняться цикл И третья часть - приращение счетчика.	<pre> for (int i = 0; i < 9; i++) { Console.WriteLine(\$"Квадрат числа {i} равен {i*i}"); } </pre>
---	---

Цикл do: сначала выполняется код цикла, а потом происходит проверка условия в инструкции while. И пока это условие истинно, цикл повторяется.	<pre>int i = 6; do { Console.WriteLine(i); i--; } while (i > 0);</pre>
Цикл while: в отличие от цикла do цикл while сразу проверяет истинность некоторого условия, и если условие истинно, то код цикла выполняется.	<pre>int i = 6; while (i > 0) { Console.WriteLine(i); i--; }</pre>

Подпрограмма

Подпрограмма - автономная часть программы, реализующая определенный алгоритм, и допускающая обращение к себе из различных частей основной программы. Подпрограмма может быть многократно вызвана из разных частей программы. В языке C# имеется две разновидности подпрограмм: процедуры и функции.

Напомним еще раз, что в С# процедуры и функции существуют только как методы некоторого класса, они не существуют вне класса.

В языке С# нет специальных ключевых слов - procedure и function, но присутствуют сами эти понятия. Синтаксис объявления метода позволяет однозначно определить, чем является метод - процедурой или функцией.

Функция отличается от процедуры двумя особенностями:

Она всегда вычисляет некоторое значение, возвращаемое в качестве результата функции;

И вызывается в выражениях.

Процедура С# имеет свои особенности:

Она возвращает формальный результат void, указывающий на отсутствие результата;

Вызов процедуры является оператором языка;

И она имеет входные и выходные аргументы, причем выходных аргументов - ее результатов - может быть достаточно много.

Функция	<pre>static string GetHello() { return "Hello"; } static int GetSum() { int x = 2; int y = 3; int z = x + y;</pre>
----------------	---

	<pre>return z; }</pre>
Процедура	<pre>static void SayHello() { Console.WriteLine("Hello"); }</pre>

1. Предел функции и его свойства. Первый и второй замечательные пределы. Правило Лопиталя раскрытия неопределенностей.

Пусть функция $f(x)$ задана в окрестности точки a , за исключением может быть самой точки a .

Число A называется **пределом функции** $f(x)$ в точке a (при $x \rightarrow a$), если для любого $\varepsilon > 0$ (в частности, сколь угодно малого) найдется такое $\delta > 0$, что для всех точек x удовлетворяется

$|x - a| < \delta$, ($x \neq a$), выполняется неравенство

$|f(x) - A| < \varepsilon$, в этом случае предел

$$\lim_{x \rightarrow a} f(x) = A$$

Все значения функции $f(x) \approx A$, если $x \approx a$

Свойства предела функции.

Функция в данной точке может иметь только один предел.

Если функция $f(x)$, $g(x)$ в точке $x=a$ имеют пределы, соответственно A, B то

$$\lim_{x \rightarrow a} (f(x) + g(x)) = A + B$$

$$\lim_{x \rightarrow a} (f(x) * g(x)) = A * B$$

$$\lim_{x \rightarrow a} (f(x)/g(x)) = \frac{A}{B}, \quad B \neq 0$$

$$\lim_{x \rightarrow a} (cf(x)) = c \lim_{x \rightarrow a} f(x)$$

Первый и второй замечательные пределы.

Первый замечательный: $\lim_{x \rightarrow 0} \frac{\sin x}{x} = 1$

Второй замечательный: $\lim_{x \rightarrow \infty} \left(1 + \frac{1}{x}\right)^x = e$

Правило Лопиталя раскрытия неопределенностей.

Производная от функции недалеко падает, а в случае правил Лопиталя она падает точно туда же, куда падает исходная функция. Это обстоятельство помогает в раскрытии неопределённостей вида $0/0$ или ∞/∞ и некоторых других неопределённостей, возникающих при вычислении **предела** отношения двух бесконечно малых или бесконечно больших функций. Вычисление значительно упрощается с помощью этого правила (на самом деле двух правил и замечаний к ним):

$$\lim_{x \rightarrow a} \frac{f(x)}{g(x)} = \lim_{x \rightarrow a} \frac{f'(x)}{g'(x)}$$

Как показывает формула выше, при вычислении предела отношений двух бесконечно малых или бесконечно больших функций предел отношения двух функций можно заменить пределом отношения их производных и, таким образом, получить определённый результат

Правило Лопиталя для случая предела двух бесконечно малых величин. Пусть функции $f(x)$ и $g(x)$ имеют производные (то есть дифференцируемы) в некоторой окрестности точки a . А в самой точке a они могут и не иметь производных. При этом в окрестности точки a производная функции $g(x)$ не равна нулю ($g'(x) \neq 0$) и пределы этих функций при стремлении x к значению функции в точке a равны между собой и равны нулю:

$$\lim_{x \rightarrow a} f(x) = \lim_{x \rightarrow a} g(x) = 0$$

Тогда предел отношения этих функций равен пределу отношения их производных:

$$\lim_{x \rightarrow a} \frac{f(x)}{g(x)} = \lim_{x \rightarrow a} \frac{f'(x)}{g'(x)}$$

Правило Лопиталя для случая предела двух бесконечно больших величин. Пусть функции $f(x)$ и $g(x)$ имеют производные (то есть дифференцируемы) в некоторой окрестности точки a . А в самой точке a они могут и не иметь производных. При этом в окрестности точки a производная функции $g(x)$ не равна нулю ($g'(x) \neq 0$) и пределы этих функций при стремлении x к значению функции в точке a равны между собой и равны бесконечности:

$$\lim_{x \rightarrow a} f(x) = \lim_{x \rightarrow a} g(x) = \infty$$

Тогда предел отношения этих функций равен пределу отношения их производных:

$$\lim_{x \rightarrow a} \frac{f(x)}{g(x)} = \lim_{x \rightarrow a} \frac{f'(x)}{g'(x)}$$

То есть для неопределённостей вида $0/0$ или ∞/∞ предел отношения двух функций равен пределу отношения их производных, если последний существует (конечный, то есть равный определённому числу, или бесконечный, то есть равный бесконечности).

Пример:

Найти предел $\lim_{x \rightarrow 0} \frac{e^x - e^{-x} - 2x}{x - \sin x}$. При подстановке $x = 0$ получается неопределенность вида $\frac{0}{0}$. Применим правило Лопиталя. Найдем производные числителя и знаменателя: $f'(x) = e^x + e^{-x} - 2$; $g'(x) = 1 - \cos x$ и

подставим их в предел: $\lim_{x \rightarrow 0} \frac{e^x + e^{-x} - 2}{1 - \cos x} = \frac{1+1-2}{1-1} = \frac{0}{0}$ - опять получилась неопределенность. Применим правило Лопиталя еще раз. $f''(x) = e^x - e^{-x}$; $g''(x) = \sin x$;

$\lim_{x \rightarrow 0} \frac{e^x - e^{-x}}{\sin x} = \frac{1-1}{0} = \frac{0}{0}$ - применяем правило Лопиталя еще раз.

$f'''(x) = e^x + e^{-x}$; $g'''(x) = \cos x$; $\lim_{x \rightarrow 0} \frac{e^x + e^{-x}}{\cos x} = \frac{2}{1} = 2$.

Для раскрытия других неопределенностей $0 \cdot \infty$, $\infty - \infty$, 1^∞ , 0^0 их следует предварительно преобразовать к неопределенности вида $\frac{0}{0}$ или $\frac{\infty}{\infty}$.

Замечания.

1. Правила Лопиталя применимы и тогда, когда функции $f(x)$ и $g(x)$ не определены при $x = a$.
2. Если при вычисления предела отношения производных функций $f(x)$ и $g(x)$ снова приходим к неопределённости вида $0/0$ или ∞/∞ , то правила Лопиталя следует применять многократно (минимум дважды).
3. Правила Лопиталя применимы и тогда, когда аргумент функций (икс) стремится не к конечному числу a , а к бесконечности ($x \rightarrow \infty$).

К неопределёностям видов $0/0$ и ∞/∞ могут быть сведены и неопределённости других видов.

Доказательства замечательных пределов

Понятие **замечательных пределов** используется для обозначения хорошо известных математических тождеств со

взятием предела. Замечательны они потому, что они уже доказаны великими математиками и нам нам остается лишь пользоваться ими для удобства нахождения пределов. Из них наиболее известны первый и второй замечательные пределы.

Первый замечательный предел имеет вид $\lim_{x \rightarrow 0} \frac{\sin x}{x} = 1$.

Вместо переменной x могут присутствовать различные функции, главное, чтобы они стремились к 0.

Пример.

Необходимо вычислить предел $\lim_{x \rightarrow 0} \frac{\sin 7x}{3x}$

Как видно, данный предел очень похож на первый замечательный, но это не совсем так. Вообще, если Вы замечаете в пределе $\sin$, то надо сразу задуматься о том, возможно ли применение первого замечательного предела.

Согласно нашему правилу №1 подставим вместо x ноль:

$$\lim_{x \rightarrow 0} \frac{\sin 7x}{3x} = \frac{0}{0}$$

Получаем неопределенность $\frac{0}{0}$.

Теперь попробуем самостоятельно организовать первый замечательный предел. Для этого проведем нехитрую комбинацию:

$$\lim_{x \rightarrow 0} \frac{\sin 7x}{3x} = \frac{0}{0} = \lim_{x \rightarrow 0} \frac{\sin 7x}{3 \cdot \frac{1}{7} \cdot 7x}$$

Таким образом мы организовываем числитель и знаменатель так, чтобы выделить $7x$. Вот уже и проявился знакомый замечательный предел. Желательно при решении выделять его:

(1-й замечательный предел)

$$\lim_{x \rightarrow 0} \frac{\sin 7x}{3x} = \frac{0}{0} = \lim_{x \rightarrow 0} \frac{\sin 7x}{3 \cdot \frac{1}{7} \cdot 7x}$$

Подставим решение первого замечательного примера и получаем:

(1-й замечательный предел)

$$\lim_{x \rightarrow 0} \frac{\sin 7x}{3x} = \frac{0}{0} = \lim_{x \rightarrow 0} \frac{\sin 7x}{3 \cdot \frac{1}{7} \cdot 7x} = \frac{1}{3}$$

Упрощаем дробь:

(1-й замечательный предел)

$$\lim_{x \rightarrow 0} \frac{\sin 7x}{3x} = \frac{0}{0} = \lim_{x \rightarrow 0} \frac{\sin 7x}{3 \cdot \frac{1}{7} \cdot 7x} = \frac{1}{3} = \frac{7}{3}$$

Ответ: 7/3.

Как видите – все очень просто.

Второй замечательный предел имеет вид $\lim_{\alpha \rightarrow \infty} \left(1 + \frac{1}{\alpha}\right)^\alpha = e$, где $e = 2,718281828...$ – это иррациональное число.

Вместо переменной x могут присутствовать различные функции, главное, чтобы они стремились к ∞ .

Пример.

Необходимо вычислить предел $\lim_{x \rightarrow \infty} \left(1 + \frac{1}{3x}\right)^{4x}$

Здесь мы видим наличие степени под знаком предела, значит возможно применение второго замечательного предела.

Как всегда воспользуемся правилом №1 – подставим ∞ вместо x :

Видно, что при $x \rightarrow \infty$ основание степени $\left(1 + \frac{1}{3x}\right) \rightarrow 1$, а показатель – $4x > \infty$, т.е. получаем неопределенность вида 1^∞ :

$$\lim_{x \rightarrow \infty} \left(1 + \frac{1}{3x}\right)^{4x} = 1^{\infty}$$

Воспользуемся вторым замечательным пределом для раскрытия нашей неопределенности, но сначала надо его организовать. Как видно – надо добиться присутствия в показателе, для чего возведем основание в степень $3x$, и одновременно в степень $1/3x$, чтобы выражение не менялось:

$$\lim_{x \rightarrow \infty} \left(1 + \frac{1}{3x}\right)^{4x} = 1^{\infty} = \lim_{x \rightarrow \infty} \left(\left(1 + \frac{1}{3x}\right)^{3x} \right)^{\frac{1}{3x} \cdot 4x}$$

Не забываем выделять наш замечательный предел:

$$\lim_{x \rightarrow \infty} \left(1 + \frac{1}{3x}\right)^{4x} = 1^{\infty} = \lim_{x \rightarrow \infty} \left(\left(1 + \frac{1}{3x}\right)^{3x} \right)^{\frac{1}{3x} \cdot 4x}$$

e (2-ой замечательный предел)

Дальше знак предела перемещаем в показатель:

$$\lim_{x \rightarrow \infty} \left(1 + \frac{1}{3x}\right)^{4x} = 1^{\infty} = \lim_{x \rightarrow \infty} \left(\left(1 + \frac{1}{3x}\right)^{3x} \right)^{\frac{1}{3x} \cdot 4x} = e^{\lim_{x \rightarrow \infty} \frac{4x}{3x}} = e^{\frac{4}{3}}$$

e (2-ой замечательный предел)

Ответ: $e^{\frac{4}{3}}$.

$$f(x) \geq g(x),$$

то если $\lim_{x \rightarrow x_0} g(x) = +\infty$, то и $\lim_{x \rightarrow x_0} f(x) = +\infty$;

если $\lim_{x \rightarrow x_0} f(x) = -\infty$, то и $\lim_{x \rightarrow x_0} g(x) = -\infty$.

Если на некоторой проколотой окрестности $\overset{\circ}{U}(x_0)$ точки x_0 :

$$\varphi(x) \leq f(x) \leq \psi(x),$$

и существуют конечные (или бесконечные определенного знака) равные пределы:

$$\lim_{x \rightarrow x_0} \varphi(x) = \lim_{x \rightarrow x_0} \psi(x) = a, \text{ то}$$

$$\lim_{x \rightarrow x_0} f(x) = a.$$

Доказательства основных свойств приведены на странице

«[Основные свойства предела функции](#)».

Арифметические свойства предела функции

Пусть функции $f(x)$ и $g(x)$ определены в некоторой проколотой окрестности точки x_0 . И пусть существуют конечные пределы:

$$\lim_{x \rightarrow x_0} f(x) = a \text{ и } \lim_{x \rightarrow x_0} g(x) = b.$$

И пусть C – постоянная, то есть заданное число. Тогда

$$\lim_{x \rightarrow x_0} (C \cdot f(x)) = C \cdot a;$$

$$\lim_{x \rightarrow x_0} (f(x) \pm g(x)) = a \pm b;$$

$$\lim_{x \rightarrow x_0} (f(x) \cdot g(x)) = a \cdot b;$$

$$\lim_{x \rightarrow x_0} \frac{f(x)}{g(x)} = \frac{a}{b}, \text{ если } b \neq 0.$$

Если $\lim_{x \rightarrow x_0} f(x) = a$, то $\lim_{x \rightarrow x_0} |f(x)| = |a|$.

Пример: Найти предел $\lim_{x \rightarrow 0} \frac{e^x - e^{-x} - 2x}{x - \sin x}$.

При подстановке $x = 0$ получается неопределенность вида $\frac{0}{0}$.

Применим правило Лопиталя. Найдем производные числителя и знаменателя: $f'(x) = e^x + e^{-x} - 2$; $g'(x) = 1 - \cos x$ и подставим их в

предел: $\lim_{x \rightarrow 0} \frac{e^x + e^{-x} - 2}{1 - \cos x} = \frac{1 + 1 - 2}{1 - 1} = \frac{0}{0}$ - опять получилась неопределенность.

Применим правило Лопиталя еще раз. $f''(x) = e^x - e^{-x}$; $g''(x) = \sin x$;

$\lim_{x \rightarrow 0} \frac{e^x - e^{-x}}{\sin x} = \frac{1-1}{0} = \frac{0}{0}$ - применяем правило Лопиталя еще раз.

$$f'''(x) = e^x + e^{-x}; \quad g'''(x) = \cos x; \quad \lim_{x \rightarrow 0} \frac{e^x + e^{-x}}{\cos x} = \frac{2}{1} = 2.$$

Для раскрытия других неопределенностей $0 \cdot \infty$, $\infty - \infty$, 1^∞ , 0^0 их следует предварительно преобразовать к неопределенности вида $\frac{0}{0}$ или $\frac{\infty}{\infty}$.

$$\lim_{x \rightarrow 0} x \ln x = [0 \cdot \infty] = \lim_{x \rightarrow 0} \frac{\ln x}{\frac{1}{x}} = \left[\frac{\infty}{\infty} \right] = \lim_{x \rightarrow 0} \frac{\frac{1}{x}}{-\frac{1}{x^2}} = \lim_{x \rightarrow 0} (-x) = 0$$

Пример

Неопределенности вида 0^0 , 1^∞ , ∞^0 можно раскрыть с помощью логарифмирования. Такие неопределенности встречаются при нахождении пределов функций вида $y = [f(x)]^{g(x)}$, $f(x) > 0$ в окрестности точки a при $x \rightarrow a$. Для нахождения предела такой функции достаточно найти предел функции $\ln y = g(x) \ln f(x)$.

Пример: Найти предел $\lim_{x \rightarrow 0} x^x$.

Здесь $y = x^x$, $\ln y = x \ln x$.

$$\lim_{x \rightarrow 0} \ln y = \lim_{x \rightarrow 0} x \ln x = \lim_{x \rightarrow 0} \frac{\ln x}{\frac{1}{x}} = \left\{ \begin{array}{l} \text{правило} \\ \text{Лопиталя} \end{array} \right\} = \lim_{x \rightarrow 0} \frac{1/x}{-1/x^2} = -\lim_{x \rightarrow 0} x = 0;$$

Тогда

$$\lim_{x \rightarrow 0} \ln y = \ln \lim_{x \rightarrow 0} y = 0; \Rightarrow \lim_{x \rightarrow 0} y = \lim_{x \rightarrow 0} x^x = 1$$

Следовательно

2. Реляционные, объектно-ориентированные, реляционно-ориентированные БД; распределенные БД.

Реляционная модель – совокупность данных, состоящая из набора двумерных таблиц. В теории множеств таблице соответствует термин отношение (relation), физическим представлением которого является таблица, отсюда и название модели – реляционная.

В сравнении с иерархической и сетевой моделью данных, реляционная модель отличается более высоким уровнем абстракции данных. Реляционная модель является удобной и наиболее привычной формой представления данных, так в настоящее время эта модель является фактическим стандартом, на который ориентируются практически все современные коммерческие СУБД. На реляционной модели данных строятся реляционные базы данных.

Реляционная база данных представляет собой хранилище данных, организованных в виде двумерных таблиц. Любая таблица реляционной базы данных состоит из строк (называемых также записями) и столбцов (называемых также полями).

Строки таблицы содержат сведения о представленных в ней фактах (или документах, или людях, одним словом, - об однотипных объектах). На пересечении столбца и строки находятся конкретные значения содержащихся в таблице данных.

Данные в таблицах удовлетворяют следующим принципам:

1. Каждое значение, содержащееся на пересечении строки и столбца, должно быть атомарным.
2. Значения данных в одном и том же столбце должны принадлежать к одному и тому же типу, доступному для использования в данной СУБД.
3. Каждая запись в таблице уникальна, то есть в таблице не существует двух записей с полностью совпадающим набором значений ее полей.
4. Каждое поле имеет уникальное имя.
5. Последовательность полей в таблице несущественна.
6. Последовательность записей в таблице несущественна.

Поле или комбинацию полей, значения которых однозначно идентифицируют каждую запись таблицы, называют **возможным ключом** (или просто **ключом**).

Если таблица имеет более одного возможного ключа, тогда один ключ выделяют в качестве первичного. Первичный ключ любой таблицы обязан содержать уникальные непустые значения для каждой строки.

Поле, указывающее на запись в другой таблице, связанную с данной записью, называется **внешним ключом** .

Подобное взаимоотношение между таблицами называется **связью** . Связь между двумя таблицами устанавливается путем присвоения значений внешнего ключа одной таблицы значениям первичного ключа другой.

Группа связанных таблиц называется **схемой базы данных**.

Информация о таблицах, их полях, первичных и внешних ключах, а также иных объектах базы данных, называется **метаданными** .

Достоинство реляционной модели данных заключается в простоте, понятности и удобстве физической реализации на ЭВМ. Именно простота и понятность для пользователя явились основной причиной ее широкого использования.

Недостатки: медленный доступ к данным, затрата больших объемов памяти.

Объектно-ориентированные БД

Объектно-ориентированные базы данных – базы данных, в которых информация представлена в виде объектов, как в объектно-ориентированных языках программирования. Объектно-ориентированные базы данных обычно рекомендованы для тех

случаев, когда требуется высокопроизводительная обработка данных, имеющих сложную структуру.

Причиной появления систем объектно-ориентированных баз данных была потребность в более адекватном представлении и моделировании сущностей реального мира, поскольку ООБД обеспечивают гораздо более развитую модель данных, нежели традиционные — реляционные базы данных.

Основные трудности объектно-ориентированного моделирования данных проистекают из того, что такого развитого математического аппарата, на который могла бы опираться общая объектно-ориентированная модель данных, не существует. В большой степени поэтому до сих пор нет базовой объектно-ориентированной модели.

Объектно-ориентированный подход базируется на концепциях:

- объекта и идентификатора объекта;
- атрибутов и методов;
- классов;
- иерархии и наследования классов.

Не смотря на огромную популярность парадигмы ООП в программировании, в технологии разработки баз данных эта парадигма пока не особо популярна. И тому есть как объективные, так и субъективные причины.

Популярность. Под реляционные базы создано множество замечательных продуктов, которые необходимо поддерживать и развивать. В эти продукты уже вложены большие деньги, и заказчики готовы еще вкладывать деньги в их развитие. Напротив, с использованием ООБД разработано сравнительно мало

серьезных коммерческих продуктов, существует мало мощных ООСУБД.

Язык запросов и его стандартизация. Еще в далеком 1986 году был принят первый стандарт SQL-86, который определил всю судьбу реляционных БД. После принятия стандарта все разработчики реляционных СУБД обязаны были следовать ему. Для объектно-ориентированных баз данных пока стандарта языка запросов нет. Сейчас среди разработчиков даже нет единого мнения о том, что этот язык запросов должен делать, не говоря уже о том, как он это должен делать.

Математический аппарат. Для реляционных БД в свое время Эдгар Кодд заложил фундамент математического аппарата реляционной алгебры. Этот мат. аппарат объясняет, как должны выполняться основные операции над отношениями в базе данных, доказывает их оптимальность (либо из него видно, где надо оптимизировать). С другой стороны, для ООБД нет такого аппарата, даже несмотря на то, что работы в этой области ведутся с 80-х годов. Таким образом, в ООБД пока нет строгих терминов, таких как декартово произведение, отношение и т. д.

Проблема хранения данных и методов. В реляционных БД хранятся только голые данные. Что с ними будет делать приложение, зависит уже от приложения. В ООБД же, напротив должны храниться объекты, а объект — это совокупность его свойств (параметры объекта) и методов (интерфейс объекта). Здесь так же нет единого мнения, как ООБД должна осуществлять хранение объектов и как разработчик должен эти объекты разрабатывать и проектировать. Здесь же возникает и проблема хранения иерархии объектов, хранение абстрактных классов и т.п.

Объектно-реляционные БД

Эпоха объектно-реляционных баз данных началась в декабре 1996 года, когда компания Informix выпустила объектно-реляционную систему управления базами данных (ОРСУБД) Informix Universal Server. Вслед за ней в 1997 г. на рынке появились ОРСУБД компаний Oracle (Oracle8) и IBM (DB2 Universal Database). До появления ОРСУБД ведущих компаний термин «объектно-реляционная СУБД» связывался с системами Postgres-Illustra и UniSQL, разработанными под руководством Майкла Стоунбрейкера и Вона Кима соответственно.

Объектно-реляционная модель данных является реляционной моделью с некоторыми свойствами объектной модели данных.

Объектные расширения реализуются в виде надстроек, которые динамически подключаются к ядру. На основе этой идеи под руководством М.Стоунбрейкера в университете Беркли (Калифорния, США) была разработана СУБД Postgres, которая имеет следующие ключевые возможности:

Типы, операторы и методы доступа, определяемые пользователем. Вспомним пример с земельными участками из главы 6.1. Для решения этой задачи мы можем определить новый тип данных "участок", необходимые операции над ним (например, вычисление площади), а также метод доступа, поскольку с помощью В-дерева нельзя выполнить двумерный поиск в задаче о перекрывающихся многоугольниках. Здесь целесообразно использовать дерево более высокой размерности (R-дерево) или другие методы.

Поддержка сложных объектов, представляющих собой наборы других объектов.

Перегрузка операторов манипулирования данными. Например, возможно создание такой конструкции

SELECT владелец FROM участки WHERE расположение_участка IN заданная_область

Создание функций, определяемых пользователем.

Динамическое (т.е. без прерывания работы СУБД) добавление новых типов данных, операторов, функций и методов доступа. Описание всех этих возможностей создается на языке C и компилируется в объектный файл, который может динамически загружаться сервером СУБД.

Наследование данных и функций. Например, от типа "участок" мы можем породить потомков "обычный участок" (сумма_налога, поступление_платежей) и "участок освобожденный от налога" (сумма_налога, причина_освобождения). При этом функция задолженность(участок) должна выполняться для всех типов участков.

Использование массивов как значений полей кортежей.

Достоинства:

Расширенные возможности SQL, в особенности, средства серверного программирования, обеспечивающие возможности определения UDT (User Data Type), хранимых процедур и функций, триггеров и т.д. позволяют переносить на сервер баз данных все большую часть логики приложений.

При проектировании приложения базы данных имеется три альтернативы: можно реализовать логику приложения на стороне клиента, на сервере приложений и на сервере баз данных. Очевидно, что каждая альтернатива имеет право на жизнь, и каждая из них может оказаться выигрышной в конкретной ситуации.

Недостатки:

Обширные возможности можно так же отнести и к недостаткам так как некоторые возможности в значительной степени противоречит учению Эдгара Кодда, в котором обосновывалась целесообразность независимости базы данных от приложений. Независимость базы данных от приложений часто выглядит очень привлекательной идеей, но для ее применения разумно отказаться от многих расширений SQL.

Распределенные БД

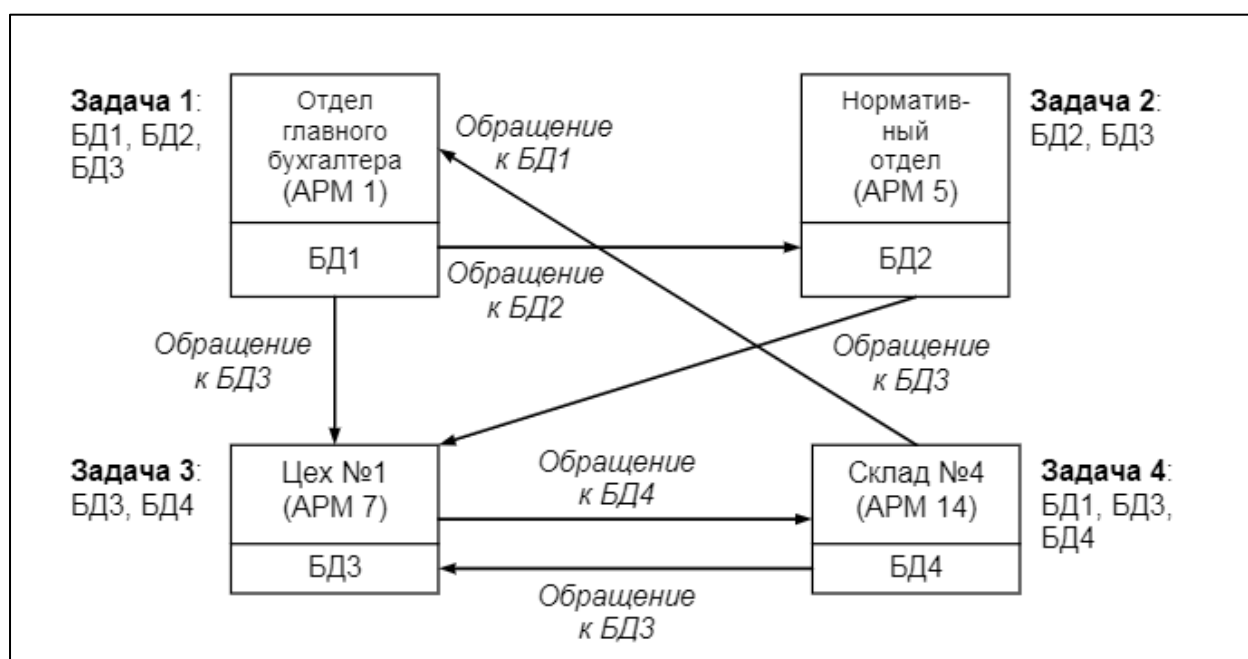
По организации и технологии обработки данных базы данных подразделяются на централизованные и распределенные.

При централизованной архитектуре все необходимые для работы специалистов данные и СУБД размещены на центральном компьютере вместе с приложением, принимающим входную информацию с пользовательского ПК (терминала) и отображающим данные на экране пользователя.

Для снижения передачи большого потока данных централизованной БД создают распределенные базы данных, т.е. БД, состоящие из нескольких, возможно пересекающихся или дублирующих друг друга частей, которые находятся в различных узлах сети. Так как предприятия, организации сами по себе имеют распределенную структуру, поэтому данные фактически распределены по структурным подразделениям. Отсюда ИС должны содержать распределенную базу данных, которая должна отражать структуру предприятия, организации.

Распределенная БД (РБД) – это набор файлов (таблиц), хранящихся в разных узлах информационной сети и логически связанных таким образом, чтобы составлять единую совокупность данных. При этом связь может быть функциональной или через копии одного и того же файла.

РБД предполагает хранение и выполнение функций управления данными в нескольких узлах и передачу данных между этими узлами в процессе выполнения запросов. Разбиение данных может быть таким, что на разных компьютерах могут храниться разные таблицы или разные части одной таблицы (части столбцов или строк). Это не должно иметь значения для пользователей или приложений, т.е. работа с распределённой БД для пользователей или приложений не должна отличаться от работы с централизованной БД.



Пример полностью распределенной БД

Для РБД необходима схема, определяющая местонахождение данных в сети, чтобы не нужно было указывать, куда переслать запрос, для получения требуемых данных. При этом нужен сложный механизм управления одновременной обработкой, обеспечивающий синхронизацию при каждом обновлении информации.

1. Производная функции одной переменной, дифференциал, их свойства и геометрический смысл. Дифференцирование сложной, обратной функции, заданной неявно и параметрически.

Приращение функции $y=f(x)$ в точке x_0 , соответствующее приращению Δx , называется $\Delta f(x_0) = f(x_0 + \Delta x) - f(x_0) = \Delta y$

Предел отношения приращения функции к приращению аргумента

$$\lim_{\Delta x \rightarrow 0} \frac{\Delta y}{\Delta x} = \lim_{\Delta x \rightarrow 0} \frac{f(x_0 + \Delta x) - f(x_0)}{\Delta x}, \text{ если он существует в точке } x_0$$

называется **производной функции**.

Функция $y=f(x)$ называется дифференцируемой в точке x_0 , если ее приращение в точке x_0 можно представить в виде $\Delta y = A * \Delta x + \theta(\Delta x)$, где $\frac{\theta(\Delta x)}{\Delta x} \rightarrow 0 (\Delta x \rightarrow 0)$

Главная линейная часть приращения $\Delta y = A * \Delta x$ называется **дифференциалом** функции $y=f(x)$ и обозначается dy или $df(x_0)$

Свойства производных:

1. Если производные функции f и g в точке x_0 существуют, то существует и производная суммы $f+g$ в этой точке, причем

$$(f+g)'(x_0) = f'(x_0) + g'(x_0);$$

(производная суммы равна сумме производных)

2. Если производные функции f и g в точке x_0 существуют, то существует и производная произведения fg в этой точке, причем

$$(fg)'(x_0) = f'(x_0)g(x_0) + f(x_0)g'(x_0);$$

3. Если производные функции f и g в точке x_0 существуют и $g(x_0) \neq 0$, то существует и производная частного f/g в этой точке, причем

$$w'_4(x) = \left(\frac{f(x)}{g(x)}\right)' = \frac{f'(x)g(x) - g'(x)f(x)}{(g(x))^2}$$

В частности, если $g(x)=c$, то

$$(fc)'(x_0) = (cf)'(x_0) = cf'(x_0)$$

(постоянный множитель можно вынести за знак производной).

Геометрический смысл производной:

Производная функции в точке x_0 равна угловому коэффициенту касательной к графику функции, проведенной в точке с абсциссой x_0 . ($f' = k = \tan \alpha$)

Физический смысл производной: производная функции $y = f(x)$ в точке x_0 - это скорость изменения функции $f(x)$ в точке x_0

Пусть функция $y = f(x)$ дифференцируема на отрезке $[a; b]$.

Производная функции в некоторой точке x_0 принадлежит $[a; b]$

определяется равенством $\lim_{\Delta x \rightarrow 0} \frac{\Delta y}{\Delta x} = f'(x)$. Тогда по свойству

предела можно записать: $\frac{\Delta y}{\Delta x} = f'(x) + \alpha$, где $\alpha \rightarrow 0$, при $\Delta x \rightarrow 0$ т.е.

является бесконечно малой, $f'(x)$ остается постоянной величиной

при $\Delta x \rightarrow 0$. Следовательно: $\Delta y = f'(x) \cdot \Delta x + \alpha \cdot \Delta x$

Итак, приращение дифференцируемой функции $y = f(x)$ может быть представлено в виде суммы двух слагаемых, из которых первое (при $f'(x) \neq 0$) линейно относительно Δx и при $\Delta x \rightarrow 0$ является бесконечно малой того же порядка малости, что Δx . Поэтому говорят, что первое слагаемое является **главной частью**

приращения, линейной относительно Δx . Второе слагаемое – бесконечно малая величина более высокого порядка, чем Δx .

Дифференциалом функции $f(x)$ в точке x называется главная линейная часть приращения функции. Дифференциал функции $y = f(x)$ равен произведению её производной на приращение независимой переменной x (аргумента).

Обозначается dy или $df(x)$.

Таким образом, если функция $y = f(x)$ имеет производную $f'(x)$ в точке x , то произведение производной $f'(x)$ на приращение Δx аргумента называют **дифференциалом функции**.

Найдем дифференциал функции $y = x$. В этом случае $y' = (x)' = 1$ и, следовательно, $dy = dx = \Delta x$. Значит, дифференциал dx независимой переменной x совпадает с ее приращением Δx .

Поэтому можем записать: $dy = f'(x)dx$. Можно также записать:

$$f'(x) = \frac{dy}{dx}.$$

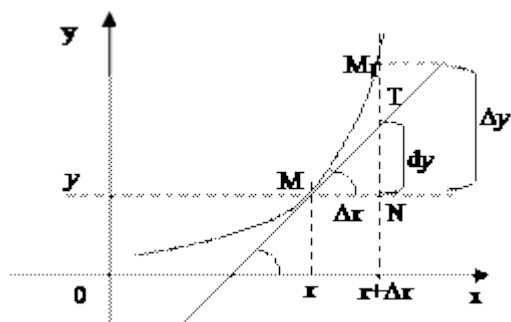
Следовательно, производную $f'(x)$ можно рассматривать как отношение дифференциала функции к дифференциалу независимой переменной.

Замечание. Из дифференцируемости функции в точке следует существование дифференциала в этой точке. Справедливо и обратное утверждение: для функции $y = f(x)$ существует дифференциал $dy = A \cdot dx$ в некоторой точке x , то эта функция имеет производную в точке x и $f'(x) = A$.

Геометрический смысл дифференциала:

Дифференциал функции $f(x)$, соответствующей данным значениям x и Δx , равен приращению ординаты касательной к кривой $y = f(x)$ в данной точке x .

Доказательство:



Рассмотрим функцию $y = f(x)$ и соответствующую ей кривую. Возьмем на кривой произвольную точку $M(x; y)$, проведем касательную к кривой в этой точке и обозначим через α угол, который касательная образует с положительным направлением оси Ox . Дадим независимой переменной x приращение Δx , тогда функция получит приращение $\Delta y = NM_1$. Значениям $x + \Delta x$ и $y + \Delta y$ на кривой $y = f(x)$ будет соответствовать точка $M_1(x + \Delta x; y + \Delta y)$.

Из $\triangle MNT$ находим $NT = MN \cdot \operatorname{tg} \alpha$. Т.к. $\operatorname{tg} \alpha = f'(x)$, а $MN = \Delta x$, то $NT = f'(x) \cdot \Delta x$. Но, по определению дифференциала, $dy = f'(x) \cdot \Delta x$, поэтому $dy = NT$.

Дифференцирование сложной, обратной функции, заданной неявно и параметрически.

Пусть функция $u = g(x)$ определена на множестве X , и U — область её значений.

Пусть далее функция $y = f(u)$ определена на множестве U .

Поставим в соответствие каждому x из X число $f(g(x))$.

Тем самым на множестве X будет задана функция $y = f(g(x))$.

Её называют **сложной функцией**.

Если известна производная функции $f(x)$, то производную сложной функции $f(u)$ можно вычислить с помощью следующей формулы:

$$(f(u))' = f'(u) \cdot u'.$$

Неявно заданная функция

Если функция задана уравнением $y=f(x)$, разрешенным относительно y , то функция задана в явном виде (явная функция).

Под **неявным заданием** функции понимают задание функции в виде уравнения $F(x;y)=0$, не разрешенного относительно y .

Всякую явно заданную функцию $y=f(x)$ можно записать как неявно заданную уравнением $f(x)-y=0$, но не наоборот.

Не всегда легко, а иногда и невозможно разрешить уравнение относительно y (например, $y+2x+\cos y-1=0$ или $2^y-x+y=0$).

Если неявная функция задана уравнением $F(x; y)=0$, то для нахождения производной от y по x нет необходимости разрешать уравнение относительно y : **достаточно продифференцировать это уравнение по x , рассматривая при этом y как функцию x , и полученное затем уравнение разрешить относительно y' .**

Производная неявной функции выражается через аргумент x и функцию y .

Пример 21.1

Найти производную функции y , заданную уравнением $x^3+y^3-3xy=0$.

Решение: Функция y задана неявно. Дифференцируем по x равенство $x^3+y^3-3xy=0$. Из полученного соотношения

$$3x^2+3y^2 \cdot y'-3(1 \cdot y+x \cdot y') = 0$$

следует, что $y^2y'-xy'=y-x^2$, т. е. $y'=(y-x^2)/(y^2-x)$.

Функция, заданная параметрически

Пусть зависимость между аргументом x и функцией y задана параметрически в виде двух уравнений

$$\begin{cases} x = x(t), \\ y = y(t), \end{cases} \quad (21.1)$$

где t — вспомогательная переменная, называемая параметром.

Найдем производную y'_x , считая, что функции (21.1) имеют производные и что функция $x=x(t)$ имеет обратную $t=\varphi(x)$. По правилу дифференцирования обратной функции

$$t'_x = \frac{1}{x'_t}. \quad (21.2)$$

Функцию $y=f(x)$, определяемую параметрическими уравнениями (21.1), можно рассматривать как сложную функцию $y=y(t)$, где $t=\varphi(x)$. По правилу дифференцирования сложной функции имеем: $y'_x = y'_t \cdot t'_x$. С учетом равенства (21.2) получаем

$$y'_x = y'_t \cdot \frac{1}{x'_t}, \quad \text{т. е.} \quad y'_x = \frac{y'_t}{x'_t}.$$

Полученная формула позволяет находить производную y'_x от функции, заданной параметрически, не находя непосредственной зависимости y от x .

Пример 21.2

Пусть
$$\begin{cases} x = t^3, \\ y = t^2. \end{cases}$$

Найти y'_x .

Решение: имеем $x'_t = 3t^2$, $y'_t = 2t$. Следовательно, $y'_x = 2t/t^2$, т. е. $y'_x = \frac{2}{3t}$.

В этом можно убедиться, найдя непосредственно зависимость y от x .

Действительно, $t = \sqrt[3]{x}$. Тогда $y = \sqrt[3]{x^2}$. Отсюда $y'_x = \frac{2}{3\sqrt[3]{x}}$, т. е. $y'_x = \frac{2}{3t}$.

2. Системы Управления БД (СУБД); концептуальная модели БД; языки БД.

Система управления базами данных (СУБД) — совокупность программных и лингвистических средств общего или специального

назначения, обеспечивающих управление созданием и использованием баз данных.

Основные функции СУБД

управление данными во внешней памяти (на дисках);

управление данными в оперативной памяти с использованием дискового кэша;

журнализация изменений, резервное копирование и восстановление базы данных после сбоев;

поддержка языков БД (язык определения данных, язык манипулирования данными).

Обычно современная СУБД содержит следующие **компоненты**:

ядро, которое отвечает за управление данными во внешней и оперативной памяти и журнализацию,

процессор языка базы данных, обеспечивающий оптимизацию запросов на извлечение и изменение данных и создание, как правило, машинно-независимого исполняемого внутреннего кода,

подсистему поддержки времени исполнения, которая интерпретирует программы манипуляции данными, создающие пользовательский интерфейс с СУБД

а также **сервисные программы** (внешние утилиты), обеспечивающие ряд дополнительных возможностей по обслуживанию информационной системы.

По способу доступа к БД различают

Файл-серверные

В файл-серверных СУБД файлы данных располагаются централизованно на файл-сервере. СУБД располагается на каждом клиентском компьютере (рабочей станции). Доступ СУБД к данным

осуществляется через локальную сеть. Синхронизация чтений и обновлений осуществляется посредством файловых блокировок.

Преимуществом этой архитектуры является низкая нагрузка на процессор файлового сервера.

Недостатки: потенциально высокая загрузка локальной сети; затруднённость или невозможность централизованного управления; затруднённость или невозможность обеспечения таких важных характеристик, как высокая надёжность, высокая доступность и высокая безопасность. Применяются чаще всего в локальных приложениях, которые используют функции управления БД; в системах с низкой интенсивностью обработки данных и низкими пиковыми нагрузками на БД.

На данный момент файл-серверная технология считается устаревшей, а её использование в крупных информационных системах — недостатком.

Примеры: Microsoft Access, Paradox, dBase.

Клиент-серверные

Клиент-серверная СУБД располагается на сервере вместе с БД и осуществляет доступ к БД непосредственно, в монопольном режиме. Все клиентские запросы на обработку данных обрабатываются клиент-серверной СУБД централизованно.

Недостаток клиент-серверных СУБД состоит в повышенных требованиях к серверу.

Достоинства: потенциально более низкая загрузка локальной сети; удобство централизованного управления; удобство обеспечения таких важных характеристик, как высокая надёжность, высокая доступность и высокая безопасность.

Примеры: Oracle Database, MS SQL Server, Sybase Adaptive Server Enterprise, PostgreSQL, MySQL, Caché, ЛИНТЕР.

Встраиваемые

Встраиваемая СУБД — СУБД, которая может поставляться как составная часть некоторого программного продукта, не требуя процедуры самостоятельной установки. Встраиваемая СУБД предназначена для локального хранения данных своего приложения и не рассчитана на коллективное использование в сети.

Физически встраиваемая СУБД чаще всего реализована в виде подключаемой библиотеки. Доступ к данным со стороны приложения может происходить через SQL либо через специальные программные интерфейсы.

Примеры: OpenEdge, SQLite, BerkeleyDB, Firebird Embedded, Microsoft SQL Server Compact, ЛИНТЕР.

Концептуальная модель базы данных — это некая наглядная диаграмма, нарисованная в принятых обозначениях и подробно показывающая связь между объектами и их характеристиками.

Создается концептуальная модель для дальнейшего проектирования базы данных и перевод ее, например, в реляционную базу данных. На концептуальной модели в визуально удобном виде прописываются связи между объектами данных и их характеристиками.

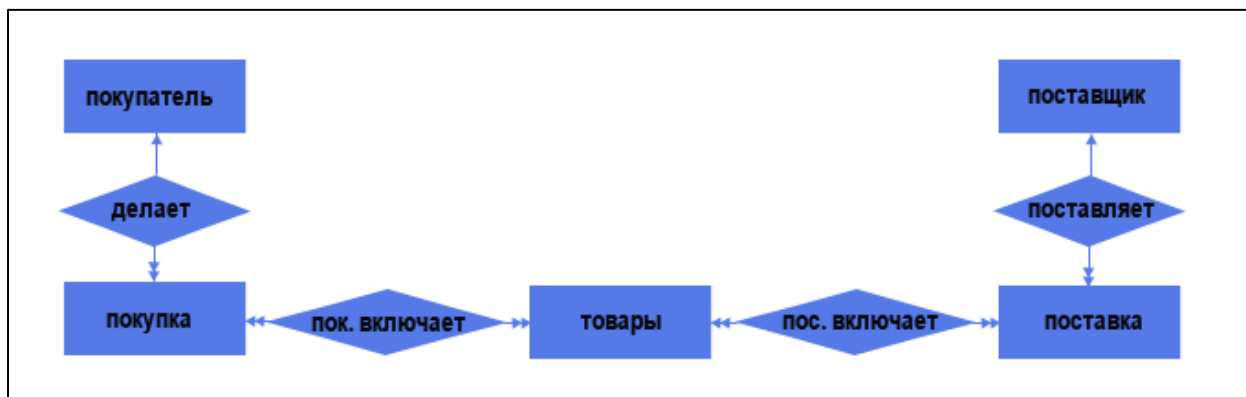
Принятые определения в концептуальной базе данных

Для единообразия программирования баз данных введены следующие понятия для концептуальных баз данных:

Объект или сущность. Это фактическая вещь или объект (для людей) за которой пользователь (заказчик) хочет наблюдать. Например, Иванов Иван Иванович;

Атрибут – это характеристика объекта, соответствующая его сущности.

Третье понятие в проектировании концептуальной базы данных это **связь или отношения** между объектами.



Преобразование концептуальной модели в логическую модель, как правило, осуществляется по формальным правилам. Этот этап может быть в значительной степени автоматизирован.

Термины «семантическая модель», «концептуальная модель» и «инфологическая модель» являются синонимами. Кроме того, в этом контексте равноправно могут использоваться слова «модель базы данных» и «модель предметной области» (например, «концептуальная модель базы данных» и «концептуальная модель предметной области»), поскольку такая модель является как образом реальности, так и образом проектируемой базы данных для этой реальности.

Конкретный вид и содержание концептуальной модели базы данных определяется выбранным для этого формальным аппаратом. Обычно используются графические нотации, подобные ER-диаграммам.

На этапе логического проектирования учитывается специфика конкретной модели данных, но может не учитываться специфика конкретной СУБД.

Для работы с базами данных используются специальные языки, в целом называемые языками баз данных. В ранних СУБД поддерживалось несколько специализированных по своим функциям языков. Чаще всего выделялись два - язык определения схемы БД (SDL - Schema Definition Language) и язык манипулирования данными (DML - Data Manipulation Language).

SDL служил главным образом для определения логической структуры БД, т.е. той структуры БД, какой она представляется пользователям.

DML содержал набор операторов манипулирования данными, т.е. операторов, позволяющих заносить данные в БД, удалять, модифицировать или выбирать существующие данные.

В современных СУБД обычно поддерживается единый интегрированный язык, содержащий все необходимые средства для работы с БД, начиная от ее создания и обеспечивающий базовый пользовательский интерфейс с базами данных.

Стандартным языком наиболее распространенных в настоящее время реляционных СУБД является язык **SQL** (Structured Query Language).

Прежде всего язык SQL сочетает средства SDL и DML, т.е. позволяет определять схему реляционной БД и манипулировать данными.

Изначально SQL был основным способом работы пользователя с базой данных и позволял выполнять следующий набор операций:

создание в базе данных новой таблицы;

добавление в таблицу новых записей;

изменение записей;

удаление записей;

выборка записей из одной или нескольких таблиц (в соответствии с заданным условием);

изменение структур таблиц.

Со временем SQL усложнился — обогатился новыми конструкциями, обеспечил возможность описания и управления новыми хранимыми объектами (например, индексы, представления, триггеры и хранимые процедуры) — и стал приобретать черты, свойственные языкам программирования.

При всех своих изменениях SQL остаётся самым распространённым лингвистическим средством для взаимодействия прикладного программного обеспечения с базами данных.

Операторы SQL делятся на:

операторы определения данных (Data Definition Language, DDL):

CREATE создаёт объект базы данных (саму базу, таблицу, представление, пользователя и так далее),

ALTER изменяет объект,

DROP удаляет объект;

операторы манипуляции данными (Data Manipulation Language, DML):

SELECT выбирает данные, удовлетворяющие заданным условиям,

INSERT добавляет новые данные,

UPDATE изменяет существующие данные,

DELETE удаляет данные;

операторы определения доступа к данным (Data Control Language, DCL):

GRANT предоставляет пользователю (группе) разрешения на определённые операции с объектом,

REVOKE отзывает ранее выданные разрешения,

DENY задаёт запрет, имеющий приоритет над разрешением;

операторы управления транзакциями (Transaction Control Language, TCL):

COMMIT применяет транзакцию,

ROLLBACK откатывает все изменения, сделанные в контексте текущей транзакции,

SAVEPOINT делит транзакцию на более мелкие участки.

1. Дифференцирование функций нескольких переменных. Дифференциал и частные производные, Производная по направлению.

Дифференцируемые функции нескольких переменных.

Пусть функция двух переменных $z = f(x, y)$ определена в некоторой открытой области D плоскости XOY , $M_0(x_0, y_0)$ – точка области D . Придавая переменным приращения Δx и Δy , перейдем из точки M_0 в какую-нибудь точку $M(x_0 + \Delta x, y_0 + \Delta y)$ той же области. При этом функция $z = f(x, y)$ получит приращение $\Delta z(M_0) = f(x_0 + \Delta x, y_0 + \Delta y) - f(x_0, y_0)$.

В отличие от частных приращений $\Delta x^{z(M_0)}$ и $\Delta y^{z(M_0)}$ это приращение называется *полным приращением функции $z = f(x, y)$ в точке M_0* , соответствующим приращениям Δx и Δy независимых переменных.

Функция $z = f(x, y)$ называется дифференцируемой в точке $M_0(x_0, y_0)$ если ее полное приращение в этой точке может быть записано в виде

$$\Delta z(M_0) = A * \Delta x + B * \Delta y + a_1 * \Delta x + a_2 * \Delta y, \quad (4.1)$$

где A, B – некоторые числа, a_1, a_2 – бесконечно малые при $\Delta x \rightarrow 0, \Delta y \rightarrow 0$.

Замечание. Функции a_1, a_2 зависят от $x_0, y_0, \Delta x, \Delta y$.

Функция, дифференцируемая в каждой точке некоторой области, называется дифференцируемой в этой области.

ПРИМЕР. Функция $z = x^2 y$ будет дифференцируемой в любой точке (x, y) , так как

$$\Delta z(x, y) = (x + \Delta x)^2 (y + \Delta y) - x^2 y =$$

$$= 2xy\Delta x + x^2\Delta y + 2x\Delta x\Delta y + y(\Delta x)^2 + (\Delta x)^2\Delta y$$

Здесь $2xy\Delta x + x^2\Delta y$ – главная часть полного приращения функции, а слагаемое $2x\Delta x\Delta y + y(\Delta x)^2 + (\Delta x)^2\Delta y = (2x\Delta y + y\Delta x)\Delta x + (\Delta x)^2\Delta y$ есть бесконечно малая более высокого порядка по сравнению с Δx и Δy .

Данное выше определение дифференцируемости функции двух переменных является естественным обобщением определения дифференцируемости функции одной переменной.

Следовательно, можно поставить вопрос о том, какие из свойств дифференцируемых функций одной переменной сохраняются для функции двух переменных. Так, было установлено, что если функция одной переменной дифференцируема в некоторой точке, то она непрерывна и имеет производную в этой точке. Последнее условие оказалось и достаточным, т. е. из существования производной функции одной переменной в данной точке следует дифференцируемость функции в этой точке. На функции двух переменных эти свойства переносятся в следующем виде.

Необходимые условия дифференцируемости. Если функция дифференцируема в точке $M_0(x_0, y_0)$, то она непрерывна в этой точке и имеет в ней частные производные по обоим независимым переменным. Причем $f'_x(x_0, y_0) = A$, а $f'_y(x_0, y_0) = B$.

Дифференциал и частные производные

Частной производной функции нескольких переменных по одной из этих переменных называется предел отношения

соответствующего частного приращения функции к приращению данной переменной, когда последнее стремится к нулю (если этот предел существует).

Для функции $z=f(x,y)$ имеем:

• $\frac{\partial z}{\partial x} = z_x = f_x(x, y) = \lim_{\Delta x \rightarrow 0} \frac{f(x+\Delta x, y) - f(x, y)}{\Delta x}$ (частная производная по переменной x);

• $\frac{\partial z}{\partial y} = z_y = f_y(x, y) = \lim_{\Delta y \rightarrow 0} \frac{f(x, y+\Delta y) - f(x, y)}{\Delta y}$ (частная производная по переменной y).

Из определения частных производных следует, что для нахождения производной z_x надо считать постоянной переменную y , а для нахождения z_y – переменную x .

При нахождении частной производной пользуются правилами дифференцирования функции одной переменной, считая все другие аргументы постоянными.

Полным дифференциалом функции называется сумма произведений частных производных этой функции на приращения соответствующих независимых переменных, т. е. $dz = z_x * \Delta x + z_y * \Delta y$

Для независимых переменных x и y любые приращения Δx и Δy будем считать их дифференциалами, т. е. $\Delta x = dx$ и $\Delta y = dy$.

Тогда *полный дифференциал функции* $z=f(x,y)$ вычисляется по следующей формуле:

$$dz = \frac{\partial z}{\partial x} * dx + \frac{\partial z}{\partial y} * dy$$

а для функции трех переменных $u = f(x, y, z)$:

$$du = \frac{\partial u}{\partial x} * dx + \frac{\partial u}{\partial y} * dy + \frac{\partial u}{\partial z} * dz$$

Полный дифференциал часто используется для *приближенных вычислений* значений функции, т. е.

$$f(x_0 + \Delta x, y_0 + \Delta y) \approx f(x_0, y_0) + f_x(x_0, y_0) * \Delta x + f_y(x_0, y_0) * \Delta y$$

Существование частных производных является лишь *необходимым*, но недостаточным условием дифференцируемости функции.

Следующая теорема выражает *достаточное условие дифференцируемости* функции двух переменных.

Теорема. Для того чтобы функция $z = f(x; y)$ была дифференцируемой в данной точке, достаточно, чтобы она обладала частными производными, непрерывными в этой точке.

Пример 11. Вычислить частные производные и полный дифференциал функции $z = \frac{y}{x+y}$.

Решение

Считая y постоянным, найдем производную по x

$$\begin{aligned} \frac{\partial z}{\partial x} = z_x &= \left[\frac{y}{x+y} \right]_x = y \cdot \left[\frac{1}{x+y} \right]_x = y \cdot \left[-\frac{1}{(x+y)^2} \right] \cdot (x+y)_x = \\ &= -\frac{y}{(x+y)^2} \cdot (1+0) = -\frac{y}{(x+y)^2}. \end{aligned}$$

Считая x постоянным и дифференцируя по y , находим

$$\begin{aligned} \frac{\partial z}{\partial y} = z_y &= \left[\frac{y}{x+y} \right]_y = \frac{(y)_y \cdot (x+y) - y \cdot (x+y)_y}{(x+y)^2} = \\ &= \frac{1 \cdot (x+y) - y \cdot (0+1)}{(x+y)^2} = \frac{x}{(x+y)^2}. \end{aligned}$$

Полный дифференциал: $dz = -\frac{y}{(x+y)^2} \cdot dx + \frac{x}{(x+y)^2} \cdot dy.$

Частными производными второго порядка называют частные производные, взятые от частных производных первого порядка

$$z_{xx} = \frac{\partial^2 z}{\partial^2 x} = \frac{\partial}{\partial x} \frac{\partial z}{\partial x} = f_{xx}(x, y)$$

$$z_{xy} = \frac{\partial^2 z}{\partial x \partial y} = \frac{\partial}{\partial y} \frac{\partial z}{\partial x} = f_{xy}(x, y)$$

Аналогично определяются частные производные третьего и более высоких порядков. Запись $\frac{\partial^n z}{\partial x^k \partial y^{n-k}}$ означает, что функция z k раз продифференцирована по переменной x и $n - k$ раз по переменной y .

Частные производные $f_{xy}(x, y)$ и $f_{yx}(x, y)$ называются *смешанными*. Значения смешанных производных равны в тех точках, в которых эти производные непрерывны.

Полный дифференциал второго порядка d^2z функции $z = f(x, y)$ выражается формулой

$$d^2z = \frac{\partial^2 z}{\partial^2 x} dx^2 + 2 \frac{\partial^2 z}{\partial x \partial y} dx dy + \frac{\partial^2 z}{\partial^2 y} dy^2$$

Дифференциалы высших порядков определяются по аналогии

$$\partial^n z = \left(\frac{\partial}{\partial x} dx + \frac{\partial}{\partial y} dy \right)^n (z)$$

$$d^n z = \left[\frac{\partial}{\partial x} \cdot dx + \frac{\partial}{\partial y} \cdot dy \right]^n (z).$$

Пример 13. Найти частные производные второго порядка функции $z = e^{x^2 y^2}$.

Решение

Вначале найдем частные производные первого порядка

$$\frac{\partial z}{\partial x} = e^{x^2 y^2} \cdot 2xy^2; \quad \frac{\partial z}{\partial y} = e^{x^2 y^2} \cdot 2x^2 y.$$

Продифференцировав их еще раз, получим

$$\frac{\partial^2 z}{\partial x^2} = e^{x^2 y^2} \cdot 4x^2 y^4 + e^{x^2 y^2} \cdot 2y^2;$$

$$\frac{\partial^2 z}{\partial y^2} = e^{x^2 y^2} \cdot 4x^4 y^2 + e^{x^2 y^2} \cdot 2x^2;$$

$$\frac{\partial^2 z}{\partial x \partial y} = e^{x^2 y^2} \cdot 4x^3 y^3 + e^{x^2 y^2} \cdot 4xy;$$

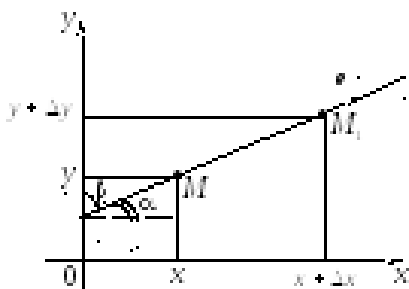
$$\frac{\partial^2 z}{\partial y \partial x} = e^{x^2 y^2} \cdot 4x^3 y^3 + e^{x^2 y^2} \cdot 4xy.$$

Сравнивая последние два выражения, видим, что $\frac{\partial^2 z}{\partial x \partial y} = \frac{\partial^2 z}{\partial y \partial x}$.

Производная по направлению. Градиент

Частные производные $\frac{\partial f}{\partial x}$ и $\frac{\partial f}{\partial y}$ представляют собой производные

от функции $z = f(x; y)$ по двум частным направлениям осей Ox и Oy



Пусть функция $z = f(x; y)$ определена в некоторой окрестности точки $M(x; y)$, t – некоторое направление, задаваемое единичным вектором $e = (\cos \alpha, \cos \beta)$, где $|e| = \cos^2 \alpha + \cos^2 \beta = 1$ либо $\alpha + \beta = \frac{\pi}{2}$ (или $\frac{3\pi}{2}$); $\cos \alpha, \cos \beta$ – косинусы углов, образуемых вектором e с осями координат и называемые *направляющими косинусами*.

При перемещении в данном направлении t точки $M(x; y)$ в точку $M_1(x + \Delta x; y + \Delta y)$ функция z получит приращение $\Delta t_z = f(x + \Delta x; y + \Delta y) - f(x; y)$, называемое *приращением функции z в данном направлении t*

Если $MM_1 = \Delta t$ то, очевидно, что $\Delta x = \Delta t * \cos \alpha, \Delta y = \Delta t * \cos \beta$ следовательно, $\Delta_t z = f(x + \Delta t * \cos \alpha, y + \Delta t * \cos \beta) - f(x, y)$.

Производной z_t по направлению t функции двух переменных $z = f(x; y)$ называется предел отношения приращения функции в этом направлении к величине перемещения Δt при стремлении последней к нулю.

Производная z_t характеризует скорость изменения функции в направлении t

Формула для производной функции $z = f(x; y)$ по направлению имеет вид

$$z_t = z_x * \cos \alpha + z_y * \cos \beta$$

Пример 16. Дана функция $z = x^2 + y^2$, в точке $M(1; 1)$ направление составляет с осью Ox угол $\alpha = \frac{\pi}{3}$. Найти производную функции по указанному направлению в этой точке.

Решение

Так как $\alpha = \frac{\pi}{3}$, то угол $\beta = \frac{\pi}{2} - \alpha = \frac{\pi}{6}$. По формуле производной функции по направлению получим

$$\frac{\partial z}{\partial \ell} = \frac{\partial z}{\partial x} \cdot \cos \frac{\pi}{3} + \frac{\partial z}{\partial y} \cdot \cos \frac{\pi}{6} = 2x \cdot \frac{1}{2} + 2y \cdot \frac{\sqrt{3}}{2} = x + \sqrt{3}y.$$

В точке $M(1; 1)$ получаем: $\left. \frac{\partial z}{\partial \ell} \right|_{(1;1)} = 1 + \sqrt{3}$.

Градиентом $grad z$ функции $z = f(x; y)$ называется вектор с координатами $(z_x; z_y)$.

Рассмотрим скалярное произведение векторов $grad z = (z_x; z_y)$ и единичного вектора $e = (\cos \alpha; \cos \beta)$.

Получим

$$(grad z; e) = z_x \cdot \cos \alpha + z_y \cdot \cos \beta.$$

и $\text{grad } z$ и единичного вектора, задающего направление t

Градиент функции $\text{grad } z$ в данной точке характеризует направление максимальной скорости изменения функции в этой точке.

2. Постановка задачи и спецификация программы; способы записи алгоритма. Базовые конструкции в блок-схемах.

Постановка задачи — это процесс формулировки назначения программного обеспечения и основных требований к нему. Описываются *функциональные требования*, определяющие функции, которые должно выполнять программное обеспечение, и *эксплуатационные требования*, определяющие характеристики его функционирования. Этап постановки задачи заканчивается разработкой *технического задания* с принятием основных проектных решений.

Техническое задание в соответствии со стандартом ГОСТ 19.201—78 «Техническое задание. Требования к содержанию и оформлению» имеет следующие основные разделы:

введение: наименование и краткая характеристика программного обеспечения;

основание для разработки;

назначение разработки: описание функционального и эксплуатационного назначения, функций;

требования к программному изделию: к функциональным характеристикам, к надежности, к техническим средствам;

требования к программной документации;

технологические требования.

Другое описание:

Постановка задачи - важнейший этап в разработке программы. Результатом должна быть спецификация программы.

Программная спецификация - точное описание того результата, который необходимо получить с помощью программы — это описание должно точно устанавливать, что должна делать программа, не указывая как она должна это делать.

Для программ, заканчивающих свою работу каким-либо результатом, программная спецификация может иметь форму спецификации ввода-вывода, которая описывает желаемое отображение множества входных величин и множества выходных величин.

Для циклических программ, в которых нельзя указать точку завершения невозможно дать спецификацию ввода-вывода, поэтому специфицируются отдельные функции, реализуемые программой в ходе циклических операций.

Постановка задачи разработки программного обеспечения является самым **первым и наиболее важным этапом** при проектировании программного обеспечения. Именно здесь закладывается фундамент будущей программы. Если на этом этапе допущены ошибки или не предусмотрены какие-то важные детали, то это отразится на конечном результате, а вносить изменения и исправления в такой проект будет крайне проблематично. Более того, если задача поставлена неверно, то ее дальнейшее решение не имеет смысла. **На этапе постановки задачи необходимо ответить на следующие вопросы:**

существуют ли методы или способы решения задачи без использования ЭВМ, нужно ли решать данную задачу на ЭВМ;

какую пользу принесет использование ЭВМ для решения задачи, что будет улучшено, ускорено, оптимизировано, сэкономлено при использовании ЭВМ, какова основная цель применения ЭВМ;

что необходимо из оборудования, кроме универсальной ЭВМ, какой тип ЭВМ необходим для решения задачи, какая платформа(ы) ЭВМ может подойти, какая операционная система будет управлять работой ЭВМ, какое дополнительное программное обеспечение нужно;

каковы бизнес-процессы и документооборот прикладной предметной области, где будет применяться разрабатываемое программное обеспечение;

каков формат исходных (входных) данных, какие данные и в какой форме необходимы для решения задачи;

каков формат промежуточных и выходных данных, какие данные и в какой форме необходимо получить;

какой интерфейс пользователя программного обеспечения нужно обеспечить, какой интерфейс с дополнительным оборудованием необходим;

какова «глубина» проработки пользовательской, инженерной, программной и конструкторской документации, эксплуатационных документов, методических пособий и руководств по использованию программы, кто и как будет применять результаты решения.

Способы записи алгоритма

Вербальный – алгоритм описывается на человеческом языке (*Рецепт любого блюда*).

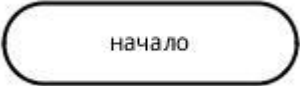
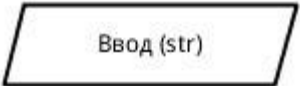
Символьный – алгоритм описывается с помощью набора символов (*Листинг программы*).

Графический – алгоритм описывается с помощью набора графических изображений (*Блок схема*).

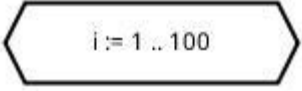
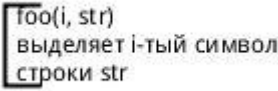
Общепринятыми способами записи являются графическая запись с помощью блок-схем и символьная запись с помощью какого-либо алгоритмического языка.

Описание алгоритма с помощью блок схем осуществляется рисованием последовательности геометрических фигур, каждая из которых подразумевает выполнение определенного действия алгоритма. Порядок выполнения действий указывается стрелками.

Основные функциональные элементы блок-схем

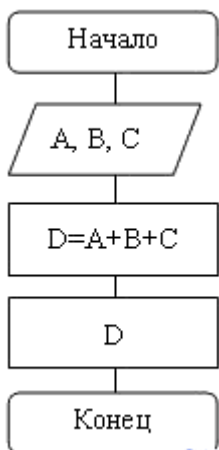
 Терминатор начала и конца работы функции	Терминатором начинается и заканчивается любая функция. Тип возвращаемого значения и аргументов функции обычно указывается в комментариях к блоку терминатора.
 Операции ввода и вывода данных	В ГОСТ определено множество символов ввода/вывода, например вывод на магнитные ленты, дисплеи и т.п. Если источник данных не принципиален, обычно используется символ параллелограмма. Подробности ввода/вывода могут быть указаны в комментариях.

Выполнение операций над данными	В блоке операций обычно размещают одно или несколько (ГОСТ не запрещает) операций присваивания, не требующих вызова внешних функций.
Блок, иллюстрирующий ветвление алгоритма	Блок в виде ромба имеет один вход и несколько подписанных выходов. В случае, если блок имеет 2 выхода (соответствует оператору ветвления), на них подписывается результат сравнения — «да/нет». Если из блока выходит большее число линий (оператор выбора), внутри него записывается имя переменной, а на выходящих дугах — значения этой переменной.
Вызов внешней процедуры	Вызов внешних процедур и функций помещается в прямоугольник с дополнительными вертикальными линиями.
Начало и конец цикла	Символы начала и конца цикла содержат имя и условие. Условие может отсутствовать в одном из символов пары. Расположение условия, определяет тип оператора, соответствующего символам на языке высокого уровня — оператор с предусловием (while) или постусловием (do ... while).

 Подготовка данных	Символ «подготовка данных» в произвольной форме (в ГОСТ нет ни пояснений, ни примеров), задает входные значения. Используется обычно для задания циклов со счетчиком.
 Соединитель	В случае, если блок-схема не уместится на лист, используется символ соединителя, отражающий переход потока управления между листами. Символ может использоваться и на одном листе, если по каким-либо причинам тянуть линию не удобно.
 Комментарий	Комментарий может быть соединен как с одним блоком, так и группой. Группа блоков выделяется на схеме пунктирной линией.

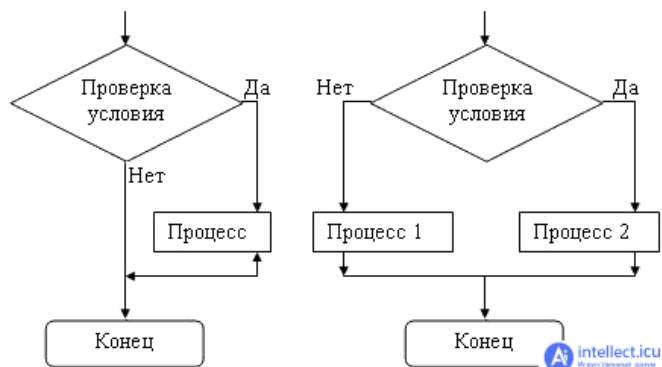
Базовые конструкции в блок-схемах

1. Базовая структура «следование» (линейный алгоритм). Образуется последовательностью действий, следующих одно за другим:



2. Базовая структура «ветвление» (разветвляющийся алгоритм).

Обеспечивает в зависимости от результата проверки условия (**да** или **нет**) выбор одного из альтернативных путей работы алгоритма. Каждый из путей ведет к **общему выходу**, так что работа алгоритма будет продолжаться независимо от того, какой путь будет выбран.



3. Базовая структура «цикл» (циклический алгоритм). Обеспечивает многократное выполнение некоторой совокупности действий, которая называется телом цикла.

В цикле всегда имеется четыре действия:

подготовка – задание начального значения параметру цикла;

основные действия (тело цикла) – реализация необходимых вычислений;

подготовка к следующему циклу (модификация) – изменение параметра цикла;

проверка условия – проверка условия окончания цикла.

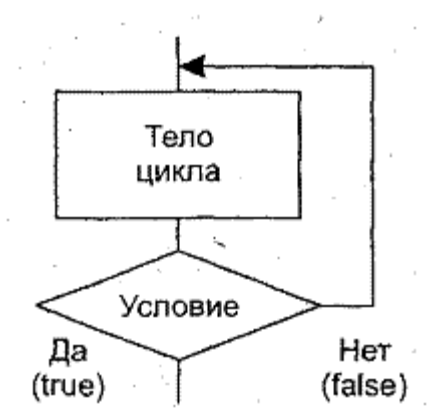
Способ организации цикла зависит от условия задачи. Иногда указывается количество повторений цикла. Это так называемые **циклы со счетчиками** (или *арифметические алгоритмы*).

Типы циклических алгоритмов:

Цикл с *предусловием*. Перед выполнением цикла проверяется условие выполнения цикла. Если условие истинно, то цикл выполняется. При ложности условия цикл заканчивается.

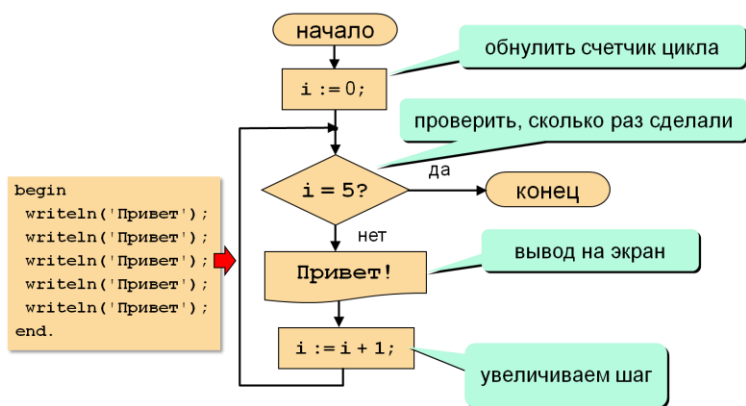


Цикл с *постусловием*. Условие продолжения цикла проверяется уже после того, как выполнено тело цикла.



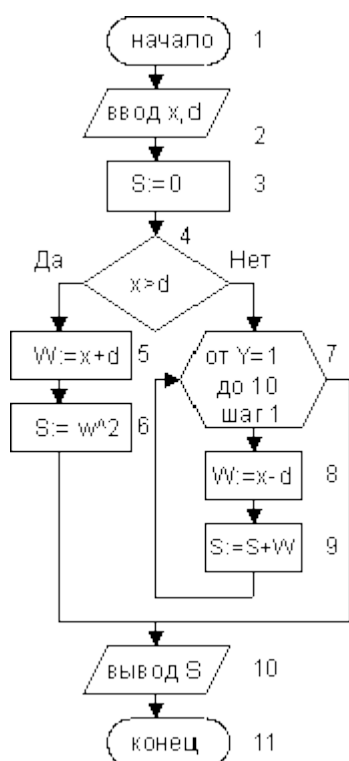
Основное различие: во втором случае цикл выполняется, по крайней мере, один раз, а в первом – может получиться, что цикл вообще не выполняется.

Цикл с заданным числом повторений, когда указывается количество повторений цикла. Это так называемые циклы со *счетчиками* (или арифметические циклы).



Алгоритм, который содержит в себе несколько структур одновременно, называется **комбинированным**.

Пример комбинированного алгоритма:



Следование: 1-3, 5-6, 8-9, 10-11 блок.

Ветвление: 4-9 блок.

Цикл: 7-9 блок.

1. Необходимые и достаточные условия экстремума функции одной переменной Теоремы Ферма, Ролля, Лагранжа.

Необходимое условие экстремума

Если функция $y=f(x)$ имеет экстремум в точке x_0 , то ее производная $f'(x_0)$ либо равна нулю, либо не существует.

Точки, в которых производная равна нулю: $f'(x)=0$, называются стационарными точками функции.

Точки, в которых выполняется необходимое условие экстремума для непрерывной функции, называются **критическими точками** этой функции. То есть **критические точки** - это либо стационарные точки (решения уравнения $f'(x)=0$), либо это точки, в которых производная $f'(x)$ не существует.

Замечание

Не в каждой своей критической точке функция обязательно имеет максимум или минимум.

Первое достаточное условие экстремума

Теорема

Пусть для функции $y=f(x)$ выполнены следующие условия:

функция непрерывна в окрестности точки x_0 ;

$f'(x_0)=0$ или $f'(x_0)$ не существует;

производная $f'(x)$ при переходе через точку x_0 меняет свой знак.

Тогда в точке $x=x_0$ функция $y=f(x)$ имеет экстремум, причем это минимум, если при переходе через точку x_0 производная меняет свой знак с минуса на плюс; максимум, если при переходе через точку x_0 производная меняет свой знак с плюса на минус.

Если производная $f'(x)$ при переходе через точку x_0 не меняет знак, то экстремума в точке $x=x_0$ нет.

Таким образом, для того чтобы исследовать функцию $y=f(x)$ на экстремум, необходимо:

найти производную $f'(x)$;

найти критические точки, то есть такие значения x , в которых $f'(x)=0$ или $f'(x)$ не существует;

исследовать знак производной слева и справа от каждой критической точки;

найти значение функции в экстремальных точках.

Второе достаточное условие экстремума

Теорема

Пусть для функции $y=f(x)$ выполнены следующие условия:

она непрерывна в окрестности точки x_0 ;

первая производная $f'(x)=0$ в точке x_0 ;

$f''(x) \neq 0$ в точке x_0 .

Тогда в точке x_0 достигается экстремум, причем, если $f''(x_0) > 0$, то в точке $x=x_0$ функция $y=f(x)$ имеет минимум; если $f''(x_0) < 0$, то в точке $x=x_0$ функция $y=f(x)$ достигает максимум.

Теорема Ферма (О равенстве нулю производной)

Пусть функция $y = f(x)$ удовлетворяет следующим условиям:

она дифференцируема на интервале $(a; b)$;

достигает наибольшего или наименьшего значения в точке $x_0 \in (a; b)$.

Тогда производная в этой точке равна нулю, то есть $f'(x_0) = 0$.

Следствие. (Геометрический смысл теоремы Ферма)

В точке наибольшего и наименьшего значения, достигаемого внутри промежутка, касательная к графику функции параллельна оси абсцисс.

Доказательство. Пусть функция $y=f(x)$ достигает в точке C локального максимума. Тогда $f(C) \geq f(x) \quad \forall x \in O(C, \delta)$, или $\Delta y = f(x) - f(C) = f(C + \Delta x) - f(C) \leq 0 \quad \forall x \in O(C, \delta)$. Очевидно, отношение

$$\frac{\Delta y}{\Delta x} = \frac{f(C + \Delta x) - f(C)}{\Delta x} \leq 0, \text{ если } \Delta x > 0,$$

$$\frac{\Delta y}{\Delta x} = \frac{f(C + \Delta x) - f(C)}{\Delta x} \geq 0, \text{ если } \Delta x < 0, \Delta x = x - C.$$

Переходя к пределу, найдём

$$\lim_{\Delta x \rightarrow +0} \frac{\Delta y}{\Delta x} = f'_+(C) \leq 0, \quad \lim_{\Delta x \rightarrow -0} \frac{\Delta y}{\Delta x} = f'_-(C) \geq 0.$$

Т.к. производная $f'(C)$ по условию теоремы существует, то $f'(C) = f'_-(C) = f'_+(C)$, что возможно только в случае, когда $f'(C) = 0$.

Доказательство аналогично, если функция достигает в точке C локального минимума.

Теорема Ролля (О нуле производной функции, принимающей на концах отрезка равные значения)

Пусть функция $y = f(x)$

непрерывна на отрезке $[a; b]$;

дифференцируема на интервале $(a; b)$;

на концах отрезка $[a; b]$ принимает равные значения $f(a) = f(b)$.

Тогда на интервале $(a; b)$ найдется, по крайней мере, одна точка x_0 , в которой $f'(x_0) = 0$.

Следствие. (Геометрический смысл теоремы Ролля)

Найдется хотя бы одна точка, в которой касательная к графику функции будет параллельна оси абсцисс.

Следствие.

Если $f(a) = f(b) = 0$, то теорему Ролля можно сформулировать следующим образом: между двумя последовательными нулями дифференцируемой функции имеется, хотя бы один, нуль производной.

Доказательство. Т.к. функция непрерывна на отрезке $[a, b]$, то она достигает на нем своих наименьшего m и наибольшего M значений (см. теорему 4 в Лекции 2.4). Если $m = M$, то функция $f(x)$ постоянная и её производная в любой точке равна нулю. Теорема в этом случае справедлива. Пусть $m \neq M$. Поскольку $f(a) = f(b)$, то по крайней мере одно из чисел m или M отлично от $f(a)$. Пусть $M \neq f(a)$, т.е. наибольшее значение достигается во внутренней точке отрезка.

$M = f(C)$, $C \in (a, b)$. Т.к. в точке $x = C$ функция достигает локального максимума и $f'(C)$ существует, то по теореме Ферма $f'(C) = 0$. Теорема доказана.

Теорема Лагранжа (О конечных приращениях)

Пусть функция $y = f(x)$

непрерывна на отрезке $[a; b]$;

дифференцируема на интервале $(a; b)$.

Тогда на интервале $(a; b)$ найдется по крайней мере одна точка x_0 , такая, что

$$\frac{f(b) - f(a)}{b - a} = f'(x_0)$$

Замечание

Теорема Ролля есть частный случай теоремы Лагранжа, когда $f(a) = f(b)$.

Следствие. (Геометрический смысл теоремы Лагранжа)

На кривой $y = f(x)$ между точками a и b найдется точка $M(x_0; f(x_0))$, такая, что через эту точку можно провести касательную, параллельную хорде AB (рис. 1).

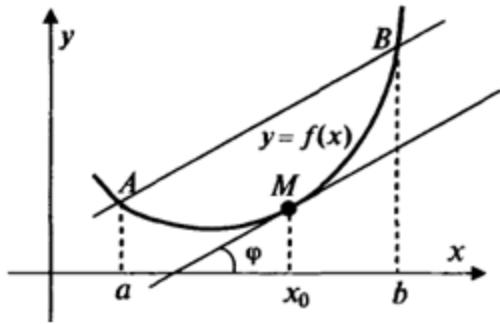


рис 1

Доказанная формула называется **формулой Лагранжа** или **формулой конечных приращений**. Она может быть переписана в виде:

$$f(b) - f(a) = f'(x_0)(b - a)$$

2. Интеллектуальный анализ данных. Понятие DATA mining. Предметно ориентированные аналитические системы. Статистические пакеты. Деревья решений. Системы для визуализации многомерных данных. Виды прогнозов и методы прогнозирования.

Data Mining (добыча данных) представляет собой процесс глубокого погружения в данные для извлечения ключевых знаний, для превращения необработанных больших данных в полезную информацию.

Первые системы Data Mining предназначались для обработки данных о продажах в супермаркетах по нескольким параметрам, включая их объем по регионам и тип продукта.

Задачи: прогнозирование: оценка продаж, предсказание нагрузки сервера или его времени простоя, рекомендации: определение продуктов, которые будут продаваться вместе, создание рекомендательных сообщений, поиск последовательностей: анализ выбора заказчиков во время совершения покупок, прогнозирование их поведения.

Цель Data Mining – превращение "сырых" данных в знания, которые можно использовать для принятия решений.

Предметно-ориентированные аналитические системы

Это программы, заточенные под конкретную сферу. Например:

в банке — системы скоринга, которые оценивают платёжеспособность;

в магазине — системы товарных рекомендаций;

на заводе — SCADA-аналитика, подсказывающая, когда оборудование выйдет из строя.

Такая система содержит: специальное хранилище данных, набор моделей, знакомых с отраслевыми нюансами, и удобный интерфейс для аналитиков.

Статистические пакеты

Это программные средства, предназначенные для проведения статистического анализа данных.

Коммерческие: SAS, SPSS — «всё в одном», за лицензии нужно платить.

Свободные: R и Python (с библиотеками Pandas, SciPy, scikit-learn) — бесплатны, гибки, требуют больше навыков программирования.

Онлайн-блокноты: Jupyter, Google Colab — позволяют писать код и сразу видеть результат.

Деревья решений

Дерево решений — средство поддержки принятия решений, использующееся в машинном обучении, анализе данных и статистике, которая предсказывает значение на основе ряда входных данных (признаков), путем последовательного выбора условий (ветвлений). Он помогает в решении задач по классификации и регрессии.

Инструмент помогает решать следующие задачи:

Классификация. Отнесение объектов к одному из заранее известных классов. Целевая переменная должна иметь дискретные задачи. Классификация = целевая переменная принимает ограниченное число категорий (дискретные значения).

Регрессия (численное предсказание). Предсказание числового значения независимой переменной для заданного входного вектора. Регрессия = целевая переменная — непрерывная величина (числа, проценты и т.д.)

Описание объектов. Набор правил в дереве решений позволяет компактно описывать объекты. Поэтому вместо сложных структур, используемых для описания объектов, можно хранить деревья решений.

Многомерные данные

Многомерные данные — это такие данные, у которых много признаков (характеристик) для каждого объекта.

Зачем нужны:

Упростить восприятие сложных данных

Найти скрытые закономерности

Представить результаты анализа

Виды прогнозов

По форме результата:

Количественный прогноз — предсказывается число

Качественный прогноз — предсказывается категория / событие

По горизонту прогнозирования: Краткосрочный, Среднесрочный, Долгосрочный

По степени определенности: Детерминированный прогноз — один чёткий результат, Вероятностный прогноз — с учетом вероятностей

Методы прогнозирования: Статистика (Простые тренды, понятная структура данных), Машинное обучение (Много данных, сложные зависимости), Экспертные методы (Мало данных, нужен человеческий опыт).

Билет №6

1. Исследование функций одной переменной. Монотонность, экстремум, выпуклость, точки перегиба, асимптоты.

Исследование функции целесообразно вести в определённой последовательности.

- 1) Найти область определения функции.
- 2) Найти точки пересечения графика с осями координат.
- 3) Найти интервалы знакопостоянства функции (промежутки, на которых $f(x) > 0$ или $f(x) < 0$).
- 4) Исследовать функцию на чётность и нечётность; на периодичность.
- 5) Исследовать функцию на непрерывность, найти точки разрыва и выяснить характер разрывов.
- 6) Найти промежутки монотонности функции.
- 7) Найти экстремумы функции.
- 8) Найти промежутки выпуклости и вогнутости графика функции. Найти точки перегиба.
- 9) Найти асимптоты графика функции.

Используя результаты исследования, построить график функции.

Монотонная функция — это функция, приращение которой не меняет знака, то есть либо всегда отрицательное, либо всегда положительное. Если в дополнение приращение не равно нулю, то

функция называется **строго монотонной**. Монотонная функция — это функция, меняющаяся в одном и том же направлении.

Функция возрастает, если большему значению аргумента соответствует большее значение функции. Функция убывает, если большему значению аргумента соответствует меньшее значение функции.

Определения

Пусть дана функция $f : M \subset \mathbb{R} \rightarrow \mathbb{R}$. Тогда

функция f называется **возрастающей** на M , если

$$\forall x, y \in M, x > y \Rightarrow f(x) \geq f(y).$$

функция f называется **строго возрастающей** на M , если

$$\forall x, y \in M, x > y \Rightarrow f(x) > f(y).$$

функция f называется **убывающей** на M , если

$$\forall x, y \in M, x > y \Rightarrow f(x) \leq f(y).$$

функция f называется **строго убывающей** на M , если

$$\forall x, y \in M, x > y \Rightarrow f(x) < f(y).$$

(Строго) возрастающая или убывающая функция называется (строго) монотонной.

Условия монотонности функции

Пусть функция $f \in C((a, b))$ непрерывна на (a, b) , и имеет в каждой точке $x \in (a, b)$ производную $f'(x)$. Тогда

f не убывает на (a, b) тогда и только тогда, когда $\forall x \in (a, b) f'(x) \geq 0$;

f не возрастает на (a, b) тогда и только тогда, когда $\forall x \in (a, b) f'(x) \leq 0$.

Пусть функция $f \in C((a, b))$ непрерывна на (a, b) , и имеет в каждой точке $x \in (a, b)$ производную $f'(x)$. Тогда

если $\forall x \in (a, b) f'(x) > 0$, то f строго возрастает на (a, b) ;

если $\forall x \in (a, b) f'(x) < 0$, то f строго убывает на (a, b) .

Пусть $f \in C((a, b))$, и всюду на интервале определена производная $f'(x)$. Тогда f строго возрастает на интервале (a, b) тогда и только тогда, когда выполнены следующие два условия:

$$\forall x \in (a, b) f'(x) \geq 0;$$

$$\forall (c, d) \subset (a, b) \exists x \in (c, d) f'(x) > 0.$$

Аналогично, f строго убывает на интервале (a, b) тогда и только тогда, когда выполнены следующие два условия:

$$\forall x \in (a, b) f'(x) \leq 0;$$

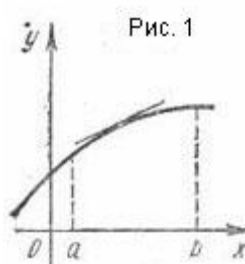
$$\forall (c, d) \subset (a, b) \exists x \in (c, d) f'(x) < 0.$$

Экстремум в

математике — *максимальное* или *минимальное* значение функции на заданном множестве. Точка, в которой достигается экстремум, называется *точкой экстремума*. Соответственно, если достигается минимум — точка экстремума называется *точкой минимума*, а если максимум — *точкой максимума*.

Выпуклости

Дифференцируемая функция называется выпуклой вниз на интервале X , если ее график расположен не ниже касательной к нему в любой точке интервала X .



Дифференцируемая функция называется выпуклой вверх на интервале X , если ее график расположен не выше касательной к нему в любой точке интервала X .

Выпуклую вверх функцию часто называют выпуклой, а выпуклую вниз – вогнутой.

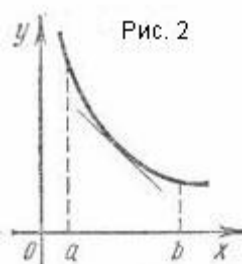


Рис. 2

Точка перегиба функции $f : \mathbb{R} \rightarrow \mathbb{R}$ внутренняя точка x_0 области определения f , такая что f непрерывна в этой точке, существует конечная или определенного знака бесконечная производная в этой точке, и x_0 является одновременно концом интервала строгой выпуклости вверх и началом интервала строгой выпуклости вниз, или наоборот.

Точка графика [непрерывной функции](#), в которой изменяется выпуклость на вогнутость или наоборот, называется **точкой перегиба**.

Асимптота кривой с бесконечной ветвью — прямая, обладающая тем свойством, что расстояние от точки кривой до этой прямой стремится к нулю при удалении точки вдоль ветви в бесконечность.

2. Понятие "неструктурированная информация", ее отличительные черты. Методы и технологии обработки неструктурированной информации. Средства моделирования информационных потоков и структур данных.

Неструктурированная информация — это данные, не имеющие заранее заданной табличной структуры и не вписывающиеся в стандартные реляционные БД.

Черта	Объяснение
Отсутствие формата	Нет единого шаблона, структура произвольная
Большой объем	Огромные массивы данных (Big Data)
Сложность обработки	Трудно автоматизировать анализ
Семантическая нагрузка	Информация часто "спрятана" в смысле, а не в структуре
Необходимость ИИ и NLP	Требуются умные алгоритмы для анализа текста, речи, изображений

Методы и технологии обработки неструктурированной информации

Для текстов (Natural Language Processing — NLP):

- **Токенизация** — разбиение текста на слова, фразы
- **Лемматизация / Стемминг** — приведение слов к начальной форме
- **Извлечение сущностей (NER)** — поиск имен, компаний, дат
- **Классификация текстов**
- **Тематика и анализ тональности**
- **Машинный перевод**

Для изображений, аудио и видео:

- **Распознавание образов (CV)** — лица, объекты, текст на изображении
- **Speech-to-Text** — перевод речи в текст
- **Video analysis** — извлечение кадров, объектов, действий
- **ML/DL** — обучение на примерах для классификации и анализа

Средства моделирования информационных потоков и структур данных

Информационные потоки — это движение данных между компонентами системы: от источника к обработке, хранению и получателю.

Основные средства моделирования:

- **DFD (Data Flow Diagrams)** — диаграммы потоков данных
👉 Показывают, **какие данные куда идут**, кто их обрабатывает
Компоненты: процессы, хранилища, внешние сущности, потоки
- **IDEF0 / IDEF1X** — методологии моделирования функций и данных
👉 Применяются в сложных системах, инженерии, управлении проектами
- **UML-диаграммы:**
 - Диаграммы классов
 - Диаграммы деятельности
 - Диаграммы последовательностей

Моделирование структур данных:

- ER-диаграммы (Entity-Relationship) — описание объектов и связей между ними
- Табличные модели для структурированных данных

JSON / XML / YAML — для слабо-структурированных

1. Первообразная и неопределенный интеграл. Основные правила интегрирования. Интегрирование рациональных функций (разложение на простейшие дроби) и некоторых иррациональных.

Первообразная

Определение 1. Функцию $F(x)$, определенную на интервале (a, b) , называют первообразной функции $f(x)$, определенной на интервале (a, b) , если для каждого $x \in (a, b)$ выполнено равенство $F'(x) = f(x)$.

Например, из справедливости равенства $(\sin 2x)' = 2 \cos 2x$ вытекает, что функция $F(x) = \sin 2x$ является первообразной функции $f(x) = 2 \cos 2x$.

Замечание. Функция $F(x) = \sin 2x$ не является единственной первообразной функции $f(x) = 2 \cos 2x$, поскольку функция $F(x) = \sin 2x + 10$, или функция $F(x) = \sin 2x - 3$, или функции вида $F(x) = \sin 2x + c$, где c – любое число, также являются первообразными функции $f(x) = 2 \cos 2x$.

Справедлива следующая теорема, доказательство которой выходит за рамки школьного курса математики.

Теорема 1. Если функция $F(x)$ является первообразной функции $f(x)$ на интервале (a, b) , то любая другая первообразная функции $f(x)$ на интервале (a, b) имеет вид $F(x) + c$, где c – некоторое число.

Неопределенный интеграл

Определение 2. Множество всех первообразных функции $f(x)$ называют неопределенным интегралом от функции $f(x)$ и обозначают

$$\int f(x) dx \quad (1)$$

Обозначение (1) читается так: «Неопределенный интеграл от функции $f(x)$ по dx ».

Если $F(x)$ является первообразной $f(x)$, то в силу теоремы 1 смысл формулы (1) заключается в следующем:

$$\int f(x) dx = \boxed{\begin{array}{l} \text{множество всех функций вида} \\ F(x) + c, \text{ где } c - \text{любое число} \end{array}} \quad (2)$$

Однако для упрощения формулу (2) принято записывать в виде

$$\int f(x) dx = F(x) + c \quad (3)$$

подразумевая, но, не указывая специально, что c – любое число.

В формуле (3) функцию $f(x)$ называют подынтегральной функцией, выражение $f(x) dx$ называют подынтегральным выражением, а число c называют постоянной интегрирования.

Операцию вычисления (взятия) интеграла по известной подынтегральной функции называют интегрированием функции.

Правила интегрирования. Замена переменной в неопределенном интеграле

Вычисление интегралов (интегрирование) основано на применении следующих правил, которые непосредственно вытекают из правил вычисления производных.

Правило 1 (интеграл от произведения числа на функцию).
Справедливо равенство

$$\int k \cdot f(x) dx = k \cdot \int f(x) dx,$$

где k – любое число.

Другими словами, интеграл от произведения числа на функцию равен произведению этого числа на интеграл от функции.

Правило 2 (интеграл от суммы функций). Интеграл от суммы функций вычисляется по формуле

$$\int [f(x) + g(x)] dx = \int f(x) dx + \int g(x) dx$$

то есть интеграл от суммы функций равен сумме интегралов от этих функций.

Правило 3 (интеграл от разности функций). Интеграл от разности функций вычисляется по формуле

$$\int [f(x) - g(x)] dx = \int f(x) dx - \int g(x) dx$$

то есть интеграл от разности функций равен разности интегралов от этих функций.

Правило 4 (интегрирование при помощи замены переменной). Из справедливости формулы

$$\int f(x) dx = F(x) + c$$

вытекает, что

$$\int f(\varphi(x)) \cdot \varphi'(x) dx = F(\varphi(x)) + c \quad (4)$$

если все входящие в формулу (4) функции $f(\varphi(x))$, $\varphi'(x)$, $F(\varphi(x))$ определены.

Доказательство правила 4. Воспользовавшись формулой для производной сложной функции, вычислим производную от правой части формулы (4):

$$[F(\varphi(x)) + c]' = F'(\varphi(x)) \cdot \varphi'(x) = f(\varphi(x)) \cdot \varphi'(x)$$

Мы получили подынтегральную функцию из левой части формулы (4), что и требовалось.

Замечание. Рассмотрим частный случай формулы (4), когда функция $\phi(x)$ является линейной функцией, то есть

$$\phi(x) = kx + b,$$

что k и b – произвольные числа, $k \neq 0$.

В этом случае

$$\phi'(x) = k,$$

и формула (4) принимает вид

$$\int f(kx + b) dx = \frac{1}{k} F(kx + b) + c \quad (5)$$

Формула (5) часто используется при решении задач.

Интегрирование рациональных функций

Для интегрирования рациональной функции $P(x)Q(x)$, где $P(x)$ и $Q(x)$ – полиномы, используется следующая последовательность шагов:

Если дробь неправильная (т.е. степень $P(x)$ больше степени $Q(x)$), преобразовать ее в правильную, выделив целое выражение;

Разложить знаменатель $Q(x)$ на произведение одночленов и/или несократимых квадратичных выражений;

Разложить рациональную дробь на простейшие дроби, используя метод неопределенных коэффициентов;

Вычислить интегралы от простейших дробей.

Рассмотрим указанные шаги более подробно.

Шаг 1. Преобразование неправильной рациональной дроби

Если дробь неправильная (т.е. степень числителя $P(x)$ больше степени знаменателя $Q(x)$), разделим многочлен $P(x)$ на $Q(x)$. Получим следующее выражение: $P(x)Q(x)=F(x)+R(x)Q(x)$, где $R(x)Q(x)$ – правильная рациональная дробь.

Шаг 2. Разложение знаменателя на простейшие дроби

Запишем многочлен знаменателя $Q(x)$ в виде $Q(x)=(x-a)^\alpha \cdots (x-b)^\beta (x^2+px+q)^\mu \cdots (x^2+rx+s)^\nu$, где квадратичные функции являются несократимыми, то есть не имеющими действительных корней.

Шаг 3. Разложение рациональной дроби на сумму простейших дробей.

Запишем рациональную функцию в следующем виде: $R(x)Q(x)=A(x-a)^\alpha + A_1(x-a)^{\alpha-1} + \dots + A_{\alpha-1}x-a + \dots + B(x-b)^\beta + B_1(x-b)^{\beta-1} + \dots + B_{\beta-1}x-b + Kx+L(x^2+px+q)^\mu + K_1x+L_1(x^2+px+q)^{\mu-1} + \dots + K_{\mu-1}x+L_{\mu-1}x^2+px+q + \dots + Mx+N(x^2+rx+s)^\nu + M_1x+N_1(x^2+rx+s)^{\nu-1} + \dots + M_{\nu-1}x+N_{\nu-1}x^2+rx+s$. Общее число неопределенных коэффициентов $A_i, B_i, K_i, L_i, M_i, N_i, \dots$ должно быть равно степени знаменателя $Q(x)$.

Затем умножим обе части полученного уравнения на знаменатель $Q(x)$ и приравняем коэффициенты при слагаемых с одинаковыми степенями x . В результате мы получим систему линейных уравнений относительно неизвестных коэффициентов $A_i, B_i, K_i, L_i, M_i, N_i, \dots$. Данная система всегда имеет единственное решение. Описанный алгоритм представляет собой метод неопределенных коэффициентов.

Шаг 4. Интегрирование простейших рациональных дробей.

Простейшие дроби, полученные при разложении произвольной правильной рациональной дроби, интегрируются с помощью следующих шести формул:

1. $\int \frac{A}{x-a} dx = \ln|x-a|$
2. $\int \frac{A}{(x-a)^k} dx = \frac{1}{(1-k)(x-a)^{k-1}}$

У дробей с квадратичным знаменателем сначала необходимо выделить полный квадрат:

$$\int \frac{Ax+B}{(x^2+px+q)^k} dx = \int \frac{At+B'}{(t^2+m^2)^k} dt,$$

где $t = x + \frac{p}{2}$, $m^2 = \frac{4q-p^2}{4}$, $B' = B - \frac{Ap}{2}$. Затем применяются следующие формулы:

3. $\int \frac{tdt}{t^2+m^2} = \frac{1}{2} \ln(t^2+m^2)$
4. $\int \frac{tdt}{(t^2+m^2)^k} = \frac{1}{2(1-k)(t^2+m^2)^{k-1}}$
5. $\int \frac{dt}{t^2+m^2} = \frac{1}{a} \arctan \frac{t}{m}$

Интеграл $\int \frac{dt}{(t^2+m^2)^k}$ может быть вычислен за k шагов с помощью **формулы редукции**

$$6. \int \frac{dt}{(t^2+m^2)^k} = \frac{t}{2m^2(k-1)(t^2+m^2)^{k-1}} + \frac{2k-3}{2m^2(k-1)} \int \frac{dt}{(t^2+m^2)^{k-1}}$$

Пример 1

Вычислить интеграл $\int \frac{2x+3}{x^2-9} dx$.

Решение.

Разложим подынтегральное выражение на простейшие дроби:

$$\frac{2x+3}{x^2-9} = \frac{2x+3}{(x-3)(x+3)} = \frac{A}{x-3} + \frac{B}{x+3}.$$

Сгруппируем слагаемые и приравняем коэффициенты при членах с одинаковыми степенями:

$$\begin{aligned} A(x+3) + B(x-3) &= 2x+3, \Rightarrow Ax+3A+Bx-3B=2x+3, \\ \Rightarrow (A+B)x+3A-3B &= 2x+3. \end{aligned}$$

Следовательно,

$$\begin{cases} A+B=2 \\ 3A-3B=3 \end{cases}, \Rightarrow \begin{cases} A=\frac{3}{2} \\ B=\frac{1}{2} \end{cases}.$$

Тогда

$$\frac{2x+3}{x^2-9} = \frac{\frac{3}{2}}{x-3} + \frac{\frac{1}{2}}{x+3}.$$

Теперь легко вычислить исходный интеграл

$$\begin{aligned} \int \frac{2x+3}{x^2-9} dx &= \frac{3}{2} \int \frac{dx}{x-3} + \frac{1}{2} \int \frac{dx}{x+3} = \frac{3}{2} \ln|x-3| + \frac{1}{2} \ln|x+3| + C \\ &= \frac{1}{2} \ln|(x-3)^2(x+3)| + C. \end{aligned}$$

Интегрирование иррациональных функций

Для интегрирования иррациональной функции, содержащей $x^{m/n}$ используется подстановка $u=x^{1/n}$.

Чтобы проинтегрировать иррациональную функцию, содержащую несколько рациональных степеней x , применяется подстановка в форме $u=x^{1/n}$, где n полагается равным наименьшему общему кратному знаменателей всех дробных степеней, входящих в данную функцию.

Рациональная функция x под знаком корня n -ой степени, т.е.

выражение вида $\sqrt[n]{\frac{ax+b}{cx+d}}$, интегрируется с помощью

подстановки $u = \left(\frac{ax+b}{cx+d}\right)^{\frac{1}{n}}$.

Пример 1

Найти интеграл $\int \frac{\sqrt{x+9}}{x} dx$.

Решение.

Сделаем подстановку:

$$u = (x+9)^{\frac{1}{2}}, \Rightarrow x+9 = u^2, \Rightarrow x = u^2 - 9, \quad dx = 2u du.$$

Вычислим интеграл

$$\begin{aligned} \int \frac{\sqrt{x+9}}{x} dx &= \int \frac{u}{u^2-9} \cdot 2u du = 2 \int \frac{u^2}{u^2-9} du = 2 \int \frac{u^2-9+9}{u^2-9} du = 2 \int \left(1 + \frac{9}{u^2-9} \right) du \\ &= 2 \int du + 18 \int \frac{du}{u^2-3^2} = 2u + 18 \cdot \frac{1}{6} \ln \left| \frac{u-3}{u+3} \right| + C = 2\sqrt{x+9} + 3 \ln \left| \frac{\sqrt{x+9}-3}{\sqrt{x+9}+3} \right| + C. \end{aligned}$$

Пример 2

Вычислить интеграл $\int \frac{\sqrt{x}-1}{\sqrt{x}+1} dx$.

Решение.

Используем следующую подстановку:

$$\sqrt{x} = u, \Rightarrow x = u^2, \quad dx = 2u du.$$

Тогда интеграл (обозначим его как I) равен

$$I = \int \frac{\sqrt{x}-1}{\sqrt{x}+1} dx = \int \frac{u-1}{u+1} 2u du = 2 \int \frac{u^2-u}{u+1} du.$$

Разделим числитель на знаменатель, выделив правильную рациональную дробь.

$$\frac{u^2-u}{u+1} = u - 2 + \frac{2}{u+1}.$$

Находим искомый интеграл:

$$\begin{aligned} I &= 2 \int \left(u - 2 + \frac{2}{u+1} \right) du = 2 \int u du - 4 \int du + 4 \int \frac{du}{u+1} = \frac{2u^2}{2} - 4u + 4 \ln|u+1| + C \\ &= x - 4\sqrt{x} + 4 \ln|\sqrt{x}+1| + C. \end{aligned}$$

2. Понятие технического зрения. Назначение технического зрения. Проблемы регистрации цифровых изображений. Компоненты технического зрения. Реконструкция изображения методом решения обратных задач.

Техническое (машинное) зрение — это направление прикладной оптики и информационных технологий, цель которого — автоматизированное получение, обработка и анализ изображений физических объектов для дальнейшего управления или принятия решений.

В широком смысле включает и аппаратную часть (камеры, оптика, освещение) и программную (алгоритмы обработки).

Назначение технического зрения

- **Контроль качества и инспекция**

- Проверка деталей на дефекты (царапины, трещины, загрязнения)
- Сортировка изделий по размеру, форме, цвету

- **Ориентация и навигация робототехники**

- Определение положения и ориентации объектов (pose estimation)
- Построение 3D-карт окружающей среды

- **Измерительные и диагностические системы**

- Бесконтактные измерения геометрических параметров (дистанция, объём)
- Медицинская визуализация (анализ снимков, эндоскопия)

- **Автоматизация управления процессами**

- Контроль уровня жидкостей, потоков на конвейере
- Чтение и распознавание маркировки (OCR, штрих-коды, QR-коды)

Проблемы регистрации цифровых изображений

- **Шумовые искажения**

- Фотонный (приближён к пуассоновскому) и термический шум сенсора
- Шум считывания при преобразовании заряда в напряжение
- **Неправильное освещение**
 - Неравномерное, слишком жёсткое или слишком слабое освещение
 - Блики и тени, создающие ложные контрасты
- **Оптические искажения**
 - Дисторсия (бочкообразная, подушкообразная)
 - Хроматические аберрации, виньетирование
- **Ограниченное разрешение**
 - Размер пикселя сенсора задаёт минимальный детектируемый объект
 - Мылая оптика и расфокусировка
- **Артефакты сжатия и передачи**
 - Блочное сжатие (JPEG) создаёт «коробки» и потерю деталей
 - Потеря кадров или пакетная передача по сети

Компоненты системы технического зрения

Компонент	Функция
Источник света	Обеспечивает нужную яркость и спектр освещения
Объектив (оптика)	Фокусирует сцену на матрице, задаёт угол обзора
Матрица (сенсор)	Конвертирует свет в электрический сигнал
АЦП и контроллер	Преобразуют аналоговый сигнал в цифровой
Память и интерфейсы	Хранение изображений, передача в вычислительную систему

Компонент	Функция
Вычислительный блок	Алгоритмы предобработки, анализа и распознавания
ПО и библиотеки	OpenCV, Halcon, TensorFlow, PyTorch и т.д.
<i>Реконструкция изображения как обратная задача</i>	

При съёмке реальный объект проходит через цепочку искажений: оптика → датчик → шум. Это «прямая задача». Чтобы вернуть изображению изначальный вид или восстановить скрытую информацию, решают *обратную* задачу:

- 1. Строим математическую модель прямого процесса**
Например, размытое фото описывают свёрткой с «функцией рассеяния точки» (PSF) + добавляют шум.
- 2. Ставим задачу нахождения исходного X по наблюдению Y**
Получаем уравнение вида $Y = A X + \text{шум}$, где A — известный оператор (размытие).
- 3. Добавляем регуляризацию**
Прямая инверсия усилила бы шум, поэтому вводят ограничение «картинка должна быть гладкой/редкой/натуральной». Классические приёмы — Тихонов, TV-регуляризация; современные — обучаемые нейросети (UNet, diffusion models).
- 4. Решаем итерационным алгоритмом**
Градиентный спуск, сглаженный Лукас-Канада, метод сопряжённых градиентов, ADMM или Plug-and-Play, пока ошибка между симулированным $A X$ и наблюдением Y не станет малой.

Получаем восстановленный образ

Ради этого снимают «шевелёнку», повышают резкость (super-resolution) или строят томографическое 3-D-изображение (КТ, МРТ) из набора 2-D проекций.

1. Понятие определенного интеграла. Основные свойства и критерий интегрируемости, Классы интегрируемых функций.

Определённый интеграл

Определенный интеграл- Приращение одной из первообразных функции $f(x)$ на отрезке $[a;b]$.

Общий вид определённого интеграла: $\int_a^b f(x)dx$.

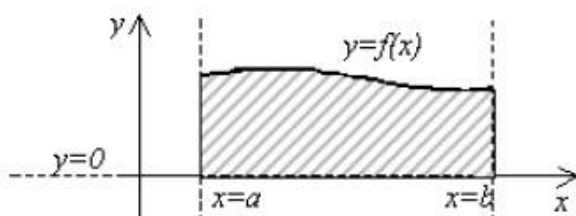
где $f(x)$ –подынтегральная функция, a и b -пределы интегрирования, dx -дифференциал

Определённый интеграл вычисляется по **формуле Ньютона –**

Лейбница: $\int_a^b f(x)dx = F(b) - F(a)$

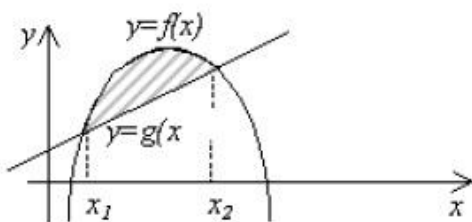
Применение определённого интеграла:

1. Нахождение площади криволинейной трапеции



Площадь фигуры, ограниченной линиями: $x=a$; $x=b$; $y=0$ и $y=f(x)$

$$S = \int_a^b f(x)dx$$



Площадь фигуры, ограниченной линиями: $y=f(x)$ и $y=g(x)$.

Найти точки пересечения x_1 и x_2 из условия: $f(x)=g(x)$

$$S = \int_{x_1}^{x_2} (f(x) - g(x))dx$$

Основные свойства определенного интеграла

Свойство 1. Постоянный множитель можно выносить за знак интеграла:

$$\int_a^b A f(x) dx = A \int_a^b f(x) dx$$

Свойство 2. Интеграл от алгебраической суммы функций равен алгебраической сумме интегралов от этих функций:

$$\int_a^b [f(x) + g(x)] dx = \int_a^b f(x) dx + \int_a^b g(x) dx$$

Свойство 3. Если отрезок интегрирования $[a, b]$ разбить на два отрезка $[a, c]$ и $[c, b]$, то интеграл по всему отрезку $[a, b]$ будет равен сумме интегралов по отрезкам $[a, c]$ и $[c, b]$:

$$\int_a^b f(x) dx = \int_a^c f(x) dx + \int_c^b f(x) dx$$

Свойство 4. При перемене местами пределов интегрирования интеграл изменяет знак:

$$\int_a^b f(x) dx = - \int_b^a f(x) dx$$

Свойство 5. Интеграл с одинаковыми пределами интегрирования равен нулю:

$$\int_a^a f(x) dx = 0$$

Свойство 6. Интеграл от постоянной величины равен этой постоянной, умноженной на длину отрезка интегрирования:

$$\int_a^b A dx = A(b - a)$$

Свойство 7. Если m и M – наименьшее и наибольшее значения функции $f(x)$ на отрезке $[a, b]$, то

$$m(b - a) \leq \int_a^b f(x) dx = M(b - a)$$

Свойство 8. Абсолютная величина интеграла от данной функции не превышает интеграла от абсолютной величины этой же функции:

$$\left| \int_a^b f(x) dx \right| \leq \int_a^b |f(x)| dx$$

Свойство 9 (Теорема о среднем). Если функция $f(x)$ непрерывна на отрезке $[a, b]$, то на этом отрезке существует такая точка c , что справедлива формула

$$f(c) = \frac{1}{b - a} \int_a^b f(x) dx,$$

называемая *формулой среднего значения* функции $f(x)$ на отрезке $[a, b]$.

Теорема (критерий интегрируемости по Риману)

Пусть функция f ограничена на отрезке $[a, b]$. Для того чтобы f была интегрируемой на этом отрезке, необходимо и достаточно, чтобы было выполнено равенство $\lim_{d(\Pi) \rightarrow 0} (\bar{S}_\Pi - \underline{S}_\Pi) = 0$. Это равенство означает, что для любого положительного ε найдется такое положительное δ , что для каждого разбиения Π , диаметр которого $d(\Pi) < \delta$, справедливо неравенство $\bar{S}_\Pi - \underline{S}_\Pi < \varepsilon$.

Доказательство. Необходимость. Пусть функция f интегрируема, т. е. существует конечный $I \equiv \lim_{d(\Pi) \rightarrow 0} \sigma$. Это означает, что для любого $\varepsilon > 0$ найдется такое $\delta > 0$, что для любого разбиения Π с $d(\Pi) < \delta$ и при любом выборе промежуточных точек ξ_i выполнено неравенство $|\sigma - I| < \varepsilon$. Это неравенство можно переписать так: $I - \varepsilon < \sigma < I + \varepsilon$. Зафиксируем произвольное разбиение Π с $d(\Pi) < \delta$. Поскольку $\bar{S}_\Pi$ — верхняя грань множества всех

интегральных сумм σ , соответствующих разбиению Π , и $\sigma < I + \varepsilon$, то $\bar{S}_\Pi \leq I + \varepsilon$. Аналогично получаем $\underline{S}_\Pi \geq I - \varepsilon$. Таким образом, $I - \varepsilon \leq \underline{S}_\Pi \leq \bar{S}_\Pi \leq I + \varepsilon$. Отсюда следует, что $\bar{S}_\Pi - \underline{S}_\Pi \leq 2\varepsilon$, если только $d(\Pi) < \delta$.

Достаточность. Заметим, что для любого разбиения Π справедливо неравенство $\underline{S}_\Pi \leq \underline{I} \leq \bar{I} \leq \bar{S}_\Pi$. Поскольку, по условию, $\bar{S}_\Pi - \underline{S}_\Pi \rightarrow 0$ при $d(\Pi) \rightarrow 0$, то $\bar{I} = \underline{I}$. Обозначим их общее значение через I . Тогда получим, что для любого разбиения Π имеет место неравенство $\underline{S}_\Pi \leq I \leq \bar{S}_\Pi$. Но и каждая интегральная сумма σ , отвечающая разбиению Π , также удовлетворяет неравенству $\underline{S}_\Pi \leq \sigma \leq \bar{S}_\Pi$. Отсюда следует, что $|\sigma - I| \leq \bar{S}_\Pi - \underline{S}_\Pi$. Поскольку правая часть последнего неравенства стремится к нулю при $d(\Pi) \rightarrow 0$, то получаем $\lim_{d(\Pi) \rightarrow 0} \sigma = I$.

Замечание. Из доказательства необходимости видно, что для интегрируемой функции ее верхняя и нижняя суммы Дарбу стремятся к интегралу от функции при стремлении к нулю диаметра разбиения.

Классы интегрируемых функций

Ответ на вопрос о том, какие функции являются интегрируемыми (т.е. существует определенный интеграл (7.2)), дают следующие теоремы:

ТЕОРЕМА 1. Если функция $f(x)$ непрерывна на отрезке $[a, b]$, то она интегрируема на нем.

ТЕОРЕМА 2. Если определенная и ограниченная на отрезке $[a, b]$ функция $f(x)$ имеет конечное число точек разрыва, то она интегрируема на этом отрезке.

ТЕОРЕМА 3. Монотонная на отрезке $[a, b]$ функция $f(x)$ интегрируема на этом отрезке.

2. Нейросетевой подход к растровому распознаванию.

Нейросетевой классификатор. Искусственные нейронные сети для распознавания границ, форм и опознание объекта, сегментации и трекинга.

Растровое распознавание — это задача извлечения информации из пиксельных изображений (сканированных документов, фотографий, картинок).

Почему нейросети?

- **Автоматическое выделение признаков.** В отличие от ручного подбора фильтров, сверточные слои сами учатся извлекать важные шаблоны (границы, текстуры, формы).
- **Глубокая иерархия.** Переход от простых признаков (края) к сложным (части объектов, целые объекты).
- **Устойчивость к шуму и вариациям.** Нейросеть может быть обучена распознавать объекты при разных углах, освещении, масштабах.

Ключевая архитектура: сверточная нейронная сеть (CNN)

1. **Свертки (Convolution).** Фильтры малого размера (3×3 , 5×5), которые сканируют изображение и добывают локальные признаковые карты.
2. **Пулинг (Pooling).** Снижение пространственного разрешения (MaxPool, AvgPool) — делает представление более инвариантным к смещениям.
3. **Нелинейности (ReLU, LeakyReLU).** Позволяют моделировать сложные распределения.
4. **Полносвязные слои (Fully Connected).** В классических классификаторах используются для объединения признаков и принятия решения.

Нейросетевой классификатор

Задача классификации — отнести всё изображение или выделенный объект к одному из заранее заданных классов.

Как это работает:

1. **Архитектура.** Обычно несколько блоков «свертка → ReLU → пулинг», затем 1–2 полносвязных слоя и софтмакс-выход.
2. **Обучение.** По размеченному набору картинок (слайс “кот”, “собака” и т. д.) минимизируется кросс-энтропийная функция потерь.
3. **Регуляризация и улучшения.** Dropout, batch-norm, data augmentation (вращение, сдвиги, шум).

Примеры популярных моделей:

- **LeNet-5** (раскладка цифр в почтовых индексах)
- **AlexNet, VGG, ResNet** — последовательные и глубокие сверточные слои
- **MobileNet, EfficientNet** — оптимизированы по скорости и размеру

ИНС для распознавания границ и форм

Распознавание границ

- **Сверточные фильтры** в первых слоях обучаются находить градиенты яркости (аналог Собеля, Превитта), но сами выбирают оптимальные комбинации.
- **Edge-detector CNN.** Небольшие сети, обученные по задачам сегментации (с метками «граница/не-граница»).

Распознавание форм

- Более глубокие сети вычленивают из краёв контуры и паттерны, соответствующие частям объектов (углы, текстуры).

- **Capsule Networks (CapsNet).** Предлагают учитывать не только наличие признаков, но и их пространственные отношения.

ИНС для опознания объектов

Object Recognition — не просто классификация целого изображения, но локализация и определение, что и где на картинке.

Подходы:

1. R-CNN и производные:

- **R-CNN:** регионы предложений (Selective Search) → CNN → классификатор
- **Fast/Faster R-CNN:** объединяют алгоритм поиска регионов и детектор в единую сеть

2. YOLO (You Only Look Once):

- Делит изображение на сетку, сразу предсказывает ограничивающие прямоугольники и классы
- Очень быстро, подходит для real-time

3. SSD (Single Shot Detector):

- Похож на YOLO, но использует признаки разных уровней свёрток для объектов разных масштабов

Сегментация

Сегментация — разбиение изображения на связные области, чаще всего по классам (фон, дорога, человек, автомобиль...).

Архитектуры:

- **FCN (Fully Convolutional Network):** заменяют полносвязные слои на свёртки, выход — карта классов
- **U-Net:** «Башенка-воронка» с пропусками (skip-connections), отлично подходит для медицинских снимков

- **Mask R-CNN:** надстройка над Faster R-CNN, добавляет ветвь сегментации к обнаружению объектов

Трекинг (tracking)

Задача трекинга — отслеживать положение объекта на последовательности кадров.

Подходы:

1. **Корреляционные трекеры:** на каждом шаге ищут наиболее похожий фрагмент (например, MOSSE, KCF)
2. **Детектор-трекер (Tracking-by-Detection):**
 - На каждом кадре детектор (например, YOLO) находит объекты
 - Алгоритм сопоставляет их с треками (IoU, appearance features)
3. **Сети с рекуррентными слоями (RNN, LSTM):** учитывают временную динамику признаков

Популярные фреймворки:

- **OpenCV Tracking API** (BOOSTING, MIL, KCF, TLD, GOTURN...)
- **DeepSORT:** сочетает детектор (YOLO) с appearance-model и алгоритмом Ханна–Канаде для ассоциации

1. Понятие двойного интеграла и его свойства. Сведение к повторному интегралу. Нахождение объемов.

Понятие двойного интеграла

Понятие интеграла может быть расширено на функции двух и большего числа переменных. Рассмотрим, например, функцию двух переменных $z = f(x, y)$. Двойной интеграл от функции $f(x, y)$ обозначается как

$$\iint_R f(x, y) dA,$$

где R - область интегрирования в плоскости Oxy . Если определенный интеграл $\int_a^b f(x) dx$ от функции одной переменной $f(x) \geq 0$ выражает площадь под кривой $f(x)$ в интервале от $x = a$ до $x = b$, то двойной интеграл выражает объем под поверхностью $z = f(x, y)$ выше плоскости O_{xy} в области интегрирования R

рис. 1.

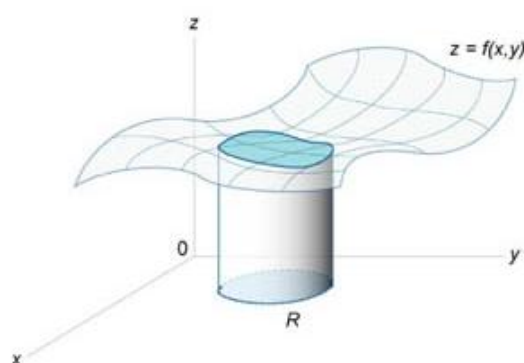


Рис.1

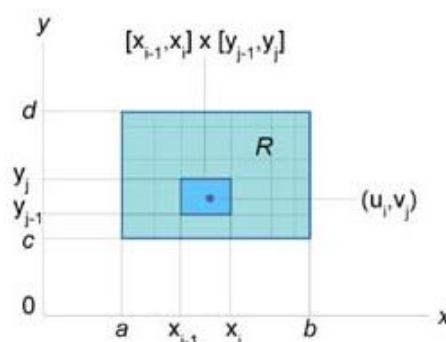


Рис.2

Формально двойной интеграл можно ввести как предел **суммы Римана**. Пусть, для простоты, область интегрирования R представляет собой прямоугольник $[a, b] \times [c, d]$ (рисунок 2).

Используя ряд чисел $\{x_0, x_1, \dots, x_m\}$, разобьем отрезок $[a, b]$ на малые интервалы таким образом, чтобы выполнялось соотношение

$$a = x_0 < x_1 < x_2 < \dots < x_i < \dots < x_{m-1} < x_m = b.$$

Аналогично, пусть множество чисел $\{y_0, y_1, \dots, y_n\}$ является разбиением отрезка $[c, d]$ вдоль оси Oy , при котором справедливы неравенства

$$c = y_0 < y_1 < y_2 < \dots < y_j < \dots < y_{n-1} < y_n = d.$$

Суммой Римана функции $f(x, y)$ над разбиением $[a, b] \times [c, d]$ называется выражение

$$\sum_{i=1}^m \sum_{j=1}^n f(u_i, v_j) \Delta x_i \Delta y_j,$$

где (u_i, v_j) - некоторая точка в прямоугольнике $(x_{i-1}, x_i) \times (y_{j-1}, y_j)$ и $\Delta x_i = x_i - x_{i-1}$, $\Delta y_j = y_j - y_{j-1}$.

Двойной интеграл от функции $f(x, y)$ в прямоугольной области $[a, b] \times [c, d]$ определяется как предел суммы Римана, при котором максимальные значения Δx_i и Δy_j стремятся к нулю:

$$\iint_{[a, b] \times [c, d]} f(x, y) dA = \lim_{\substack{\max \Delta x_i \rightarrow 0 \\ \max \Delta y_j \rightarrow 0}} \sum_{i=1}^m \sum_{j=1}^n f(u_i, v_j) \Delta x_i \Delta y_j.$$

Чтобы определить двойной интеграл в произвольной области R , отличной от прямоугольной, выберем прямоугольник $[a, b] \times [c, d]$, покрывающий область R (рисунок 3), и введем функцию $g(x, y)$, такую, что

$$\begin{cases} g(x, y) = f(x, y), & \text{если } f(x, y) \in R \\ g(x, y) = 0, & \text{если } f(x, y) \notin R \end{cases}.$$

Тогда двойной интеграл от функции $f(x, y)$ в произвольной области R определяется как

$$\iint_R f(x, y) dA = \iint_{[a, b] \times [c, d]} g(x, y) dA.$$

Повторные интегралы

Двойные интегралы вычисляются, как правило, с помощью повторных интегралов. Однако переход от двойных к повторным интегралам возможен не для произвольной области интегрирования R , а для областей определенного типа. Введем понятия областей интегрирования типа I и II.

Определение 1. Говорят, что область R на плоскости относится к *типу I* или является *элементарной относительно оси Oy*, если она лежит между графиками двух непрерывных функций, зависящих от x (рисунок 1), и описывается множеством:

$$R = \{(x, y) \mid a \leq x \leq b, p(x) \leq y \leq q(x)\}.$$

Определение 2. Говорят, что область R на плоскости относится к *типу II* или является *элементарной относительно оси Ox*, если она лежит между графиками двух непрерывных функций, зависящих от y (рисунок 2), и описывается множеством:

$$R = \{(x, y) \mid u(y) \leq x \leq v(y), c \leq y \leq d\}.$$

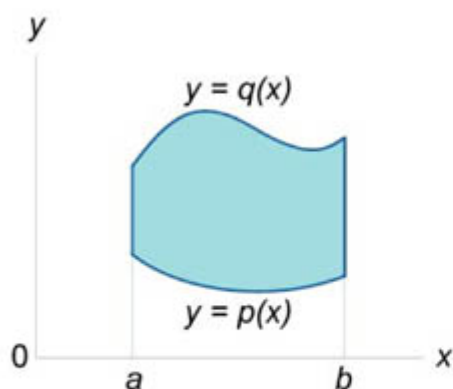


Рис.1

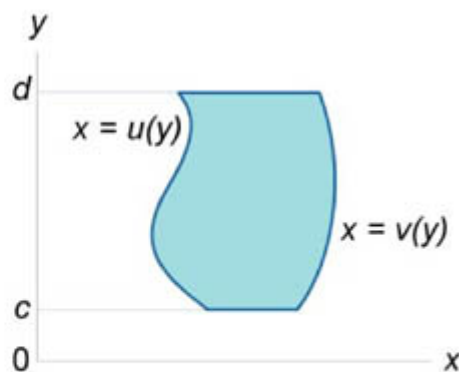


Рис.2

Связь между двойными и повторными интегралами

Пусть $f(x,y)$ является непрерывной функцией в области R типа I:

$$R = \{(x,y) \mid a \leq x \leq b, p(x) \leq y \leq q(x)\}.$$

Тогда двойной интеграл от функции $f(x,y)$ в данной области выражается через повторный интеграл в виде

$$\iint_R f(x,y) dA = \int_a^b \int_{p(x)}^{q(x)} f(x,y) dy dx.$$

Для области интегрирования типа II существует аналогичная формула. Если $f(x,y)$ является непрерывной функцией в области R типа II:

$$R = \{(x,y) \mid u(y) \leq x \leq v(y), c \leq y \leq d\},$$

то справедливо соотношение

$$\iint_R f(x,y) dA = \int_c^d \int_{u(y)}^{v(y)} f(x,y) dx dy.$$

Приведенные формулы (в англоязычной литературе они известны как *теорема Фубини*) позволяют вычислять двойные интегралы через повторные. В повторных интегралах сначала находится внутренний интеграл, а затем - внешний.

Геометрические приложения двойных интегралов

Объем тела

Если $f(x,y) > 0$ в области интегрирования R , то объем цилиндрического тела с основанием R , ограниченного сверху поверхностью $z=f(x,y)$, выражается формулой

$$V = \iint_R f(x,y) dA.$$

В случае, когда R является областью типа I, ограниченной линиями $x=a$, $x=b$, $y=g(x)$, $y=h(x)$, объем тела равен

$$V = \iint_R f(x, y) dA = \int_a^b \int_{g(x)}^{h(x)} f(x, y) dy dx.$$

Для области R типа II, ограниченной графиками функций $y=c$, $y=d$, $x=p(y)$, $x=q(y)$, объем соответственно равен

$$V = \iint_R f(x, y) dA = \int_c^d \int_{p(y)}^{q(y)} f(x, y) dx dy.$$

Если в области R выполняется неравенство $f(x, y) \geq g(x, y)$, то объем цилиндрического тела между поверхностями $z_1=g(x, y)$ и $z_2=f(x, y)$ с основанием R равен

$$V = \iint_R [f(x, y) - g(x, y)] dA.$$

Площадь поверхности

Предположим, что поверхность задана функцией $z=g(x, y)$, имеющей область определения R . Тогда площадь такой поверхности над областью R определяется формулой

$$S = \iint_R \sqrt{1 + \left(\frac{\partial z}{\partial x}\right)^2 + \left(\frac{\partial z}{\partial y}\right)^2} dx dy$$

при условии, что частные производные $\partial z/\partial x$ и $\partial z/\partial y$ непрерывны всюду в области R .

Площадь и объем в полярных координатах

Пусть S является областью, ограниченной линиями $\theta=\alpha$, $\theta=\beta$, $r=h(\theta)$, $r=g(\theta)$ (рисунок 3). Тогда площадь этой области определяется формулой

$$A = \iint_R dA = \int_{\alpha}^{\beta} \int_{h(\theta)}^{g(\theta)} r dr d\theta.$$

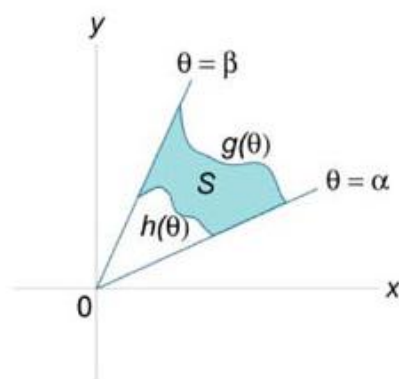


Рис.3

Объем тела, ограниченного сверху поверхностью $z=f(r,\theta)$ с основанием S , выражается в полярных координатах в виде

$$V = \iint_S f(r, \theta) r dr d\theta.$$

2.Методологии структурного системного анализа и проектирования: SADT, структурного системного анализа Гейна - Сарсона, структурного анализа и проектирования Йордона / Де марко
Общие принципы структурного анализа и проектирования

- **Цель:** получить чёткое, иерархическое представление о том, **какие функции** выполняет система, **какие данные** она обрабатывает и **как** компоненты взаимодействуют друг с другом.

Авторы предложили «рисовать картины» процесса и данных — прежде чем писать код. Смысл общий:

1. понять границы системы и её окружение;

2. разложить работу на иерархию простых функций;
3. параллельно описать, какие данные куда текут;
4. довести описание до уровня, откуда уже видно, «какие модули и какие файлы» нужны.

SADT (Structured Analysis and Design Technique)

- **Основная картинка — прямоугольник-функция с четырьмя сторонами.**
 - Слева — входы (что превращаем).
 - Справа — выходы (что получаем).
 - Сверху — управление (правила, ограничения).
 - Снизу — механизмы (чем выполняем).
- **IDEFO** — официальный стандарт, по сути тот же SADT.
- **Работа в два шага.**
 1. Рисуем Context Diagram — «система как черный ящик».
 2. Декомпозируем каждый прямоугольник до тех пор, пока подпроцесс не станет элементарным.
- **Плюсы:** легко показать, какие ресурсы нужны и какие правила действуют.
- **Минусы:** данные изображаются только стрелками; кто хранит и как структурированы данные, видно плохо.

Метод Гейна-Сарсона (Gane & Sarson SSA/SSAD)

- **Главный инструмент — DFD (Data Flow Diagram).**
 - Круг или прямоугольник — процесс.
 - Открытый прямоугольник — хранилище.
 - Стрелка — поток данных.
 - Внешний источник/приёмник — прямоугольник-«коробка».
- **Своя «любимая» эстетика.** Круг выглядит как прямоугольник с закруглёнными углами, хранилище — «половинка параллелепипеда».
- **Этапы.**

1. Контекстная диаграмма (система-один-процесс).
 2. Расчленяем на под-DFD уровня 0,1,2 ... пока не станет ясно, что будет программный модуль или ручная операция.
 3. Для каждого хранилища строим logico/physical ER-диаграмму.
 4. Процессы описываем псевдокодом (Structured English) или мини-блок-схемами.
- **Сильная сторона** — фокус на данных: видно, где они хранятся и как трансформируются.

Йордон / Де Марко (Yourdon DeMarco SA/SD)

- **Тоже DFD, но в классическом виде** — процесс круглый, хранилище = две параллельные линии.
- **Подход делится на две части.**
 1. **Structured Analysis (SA).** Чистая «логика» бизнеса без технических деталей. Декомпозиция DFD вплоть до процессов, которые выполняются одной функцией или оператором.
 2. **Structured Design (SD).** Из логических DFD формируют иерархию модулей программы (Structure Chart): вершина — главный модуль, ветки — подпрограммы, стрелки — передаваемые данные (by-value, by-reference).
- **Отличие от Гейна-Сарсона** — Yourdon делает явный мост «логическая модель → структура кода», подчёркивая, какие модули могут быть переиспользованы.
- **Инструменты** — кроме DFD, активно используется словарь данных, спецификация мини-процессов с псевдокодом и решётки «вещь-действие» (CRUD-матрицы).

Параметр	SADT	Gane & Sarson	Yourdon/DeMarco
Форма процессов	Прямоугольник и (функции)	Скруглённые прямоугольник и / круги	Круги
Потоки данных vs управление	Чёткое разделение	Только потоки данных	Только потоки данных
Иерархия	С помощью развертывания	Нумерация процессов	Нумерация + декомпозиция
Data Dictionary	Не обязательно	Не обязательно	Обязательно
Структурное проектирование	Нет (только анализ)	Только DFD	DFD + Structure Charts

1. Числовые ряды и их сумма; Критерии сходимости. Абсолютная и условная сходимость. Признаки сравнения, Коши, Даламбера, Лейбница сходимости числовых рядов.

[Здесь](#) можно более подробно почитать

Числовые ряды

Под словом "ряд" в математическом анализе понимают сумму бесконечного числа слагаемых. Рассмотрим произвольную числовую последовательность $a_1, a_2, \dots, a_n, \dots$ и образуем формальное выражение

$$a_1 + a_2 + \dots + a_n + \dots = \sum_{k=1}^{\infty} a_k.$$

Назовем это выражение числовым рядом, а числа a_k — членами ряда.

Сумма

$$S_n = \sum_{k=1}^n a_k$$

Называется частичной суммой (n-ой частичной суммой) ряда.

Определение: числовой ряд

$$\sum_{k=1}^{\infty} a_k$$

Называется сходящимся, если сходится последовательность $\{S_n\}$ его частичных сумм. При этом число

$$S = \lim_{n \rightarrow \infty} S_n = \sum_{k=1}^{\infty} a_k$$

Называется суммой ряда. Если же последовательность частичных сумм ряда расходится, то такой ряд называется расходящимся.

Теорема (критерий Коши сходимости числового ряда).

Для того, чтобы ряд

$$\sum_{k=1}^{\infty} a_k$$

сходился, необходимо и достаточно, чтобы $\forall \varepsilon > 0 \exists N, \forall n > N, \forall p \in \mathbb{N}$

$$\left| \sum_{k=n+1}^{n+p} a_k \right| < \varepsilon.$$

Доказательство:

Сходимость числового ряда — это сходимость последовательности $\{S_n\}$ его частичных сумм, а для сходимости последовательности $\{S_n\}$, как было доказано в теореме 4, необходимо и достаточно, чтобы она была фундаментальной, т.е. $\forall \varepsilon > 0 \exists N, \forall n > N, \forall p \in \mathbb{N}: |S_{n+p} - S_n| < \varepsilon$, или

$$\left| \sum_{k=n+1}^{n+p} a_k \right| < \varepsilon,$$

что и доказывает теорему.

Знакопеременные ряды. Абсолютная и условная сходимость

Числовой ряд, содержащий бесконечное множество положительных и бесконечное множество отрицательных членов, называется **знакопеременным**. Частным случаем знакопеременного ряда является **знакочередующийся ряд**, то есть такой ряд, в котором последовательные члены имеют противоположные знаки.

Признак Лейбница

Для знакочередующихся рядом действует достаточный *признак сходимости Лейбница*.

Пусть $\{a_n\}$ является числовой последовательностью, такой, что

1. $a_{n+1} < a_n$ для всех n ;
2. $\lim_{n \rightarrow \infty} a_n = 0$.

Тогда знакочередующиеся $\sum_{n=1}^{\infty} (-1)^n a_n$ и $\sum_{n=1}^{\infty} (-1)^{n-1} a_n$ ряды сходятся.

Абсолютная и условная сходимость

Ряд $\sum_{n=1}^{\infty} a_n$ называется *абсолютно сходящимся*, если ряд $\sum_{n=1}^{\infty} |a_n|$ также сходится.

Если ряд $\sum_{n=1}^{\infty} a_n$ сходится абсолютно, то он является сходящимся (в обычном смысле). Обратное утверждение неверно.

Ряд $\sum_{n=1}^{\infty} a_n$ называется *условно сходящимся*, если сам он сходится, а ряд, составленный из модулей его членов, расходится.

Признак сравнения

Теорема 7. Пусть даны два ряда с положительными членами

$$\sum_{k=1}^{\infty} p_k \quad \text{и} \quad \sum_{k=1}^{\infty} q_k$$

(обозначим их как ряд P и ряд Q соответственно), и пусть $\forall k : p_k \leq q_k$. Тогда: 1) из сходимости ряда Q следует сходимость ряда P ; 2) из расходимости ряда P следует расходимость ряда Q .

Доказательство:

Утверждение теоремы следует из неравенства

$$S_n^P = \sum_{k=1}^n p_k \leq \sum_{k=1}^n q_k = S_n^Q.$$

Пример: рассмотрим т.н. обобщенный гармонический ряд

$$\sum_{k=1}^{\infty} \frac{1}{k^{\alpha}} \quad (\alpha < 1).$$

Из сравнения с гармоническим рядом следует, что обобщенный гармонический ряд при $\alpha < 1$ расходится.

Замечания:

- 1) теорема 7 остается в силе, если неравенство $p_k \leq q_k$ выполнено, начиная не с $k=1$, а с некоторого $k=k_0$.
- 2) теорема 7 остается в силе, если вместо неравенства $p_k \leq q_k$ выполнено неравенство $p_k \leq c \cdot q_k$, где $c > 0$ — некоторое число.

Признак Даламбера

Если

$$\forall k : \quad \frac{p_{k+1}}{p_k} \leq q < 1 \quad \left(\frac{p_{k+1}}{p_k} \geq 1 \right),$$

то ряд

$$\sum_{k=1}^{\infty} p_k$$

сходится (расходится).

Доказательство:

Воспользуемся признаком сравнения (теорема 7). Из цепочки неравенств $p_{k+1} \leq q \cdot p_k \leq q \cdot q \cdot p_{k-1} \leq \dots \leq p^k p_1$ и сходимости ряда

$$\sum_{k=1}^{\infty} q^k p_1$$

закключаем, что ряд

$$\sum_{k=1}^{\infty} p_k$$

сходится.

Если

$$\frac{p_{k+1}}{p_k} \geq 1,$$

То $p_{k+1} \geq p_k \geq \dots \geq p_1 > 0$ и тем самым не выполнено необходимое условие сходимости ряда. Теорема 8 доказана.

Признак Коши

Если $\forall k: \sqrt[k]{p_k} \leq q < 1$ ($\sqrt[k]{p_k} \geq 1$), то ряд

$$\sum_{k=1}^{\infty} p_k$$

сходится (расходится).

Доказательство:

Воспользуемся теоремой 7. Из неравенства $p_k \leq q^k$ и сходимости ряда

$$\sum_{k=1}^{\infty} q^k$$

вытекает, что ряд

$$\sum_{k=1}^{\infty} p_k$$

также сходится.

Если : $\sqrt[k]{p_k} \geq 1$, то $p_k \geq 1$ и тем самым не выполнено необходимое условие сходимости ряда. Теорема 9 доказана.

Первый признак сравнения

Пусть заданы два положительных ряда $\sum_{n=1}^{\infty} u_n$ и $\sum_{n=1}^{\infty} v_n$. Если начиная с некоторого номера n_0 выполнено неравенство $u_n \leq v_n$, то: если ряд $\sum_{n=1}^{\infty} u_n$ расходится, то ряд $\sum_{n=1}^{\infty} v_n$ будет расходящимся.

если ряд $\sum_{n=1}^{\infty} v_n$ сходится, то ряд $\sum_{n=1}^{\infty} u_n$ будет сходящимся.

Второй признак сравнения

Пусть заданы два положительных ряда $\sum_{n=1}^{\infty} u_n$ и $\sum_{n=1}^{\infty} v_n$. Если при условии $v_n \neq 0$ существует предел

$$\lim_{n \rightarrow \infty} \frac{u_n}{v_n} = K,$$

где $0 < K < \infty$, то ряды $\sum_{n=1}^{\infty} u_n$ и $\sum_{n=1}^{\infty} v_n$ сходятся либо расходятся одновременно.

2. UML - унифицированный язык моделирования. Понятие диаграмм, процесса проектирования.

Что такое UML и зачем он нужен

UML (Unified Modeling Language) — это стандартизованный графический язык для визуализации, спецификации, конструирования и документирования программных систем и бизнес-процессов.

Основные цели UML:

- Облегчить понимание архитектуры и поведения системы для разных участников (аналитиков, архитекторов, разработчиков, тестировщиков, заказчиков).
- Предоставить единый набор нотаций и правил, чтобы избежать путаницы при переходе между проектами и командами.
- Поддерживать разные этапы разработки: от сбора требований до сопровождения.

Понятие «диаграмма» в UML

Диаграмма в UML — это графическое представление какого-то аспекта системы. Каждая диаграмма показывает свой взгляд (view) на модель: либо структуру, либо динамику, либо архитектуру.

2.1 Мелкомасштабные строительные блоки

- **Классификаторы (Classifier):** классы, интерфейсы, компоненты, узлы и т. д.
- **Отношения (Relationships):**
 - Ассоциация, агрегация, композиция
 - Обобщение (наследование)
 - Зависимость, реализация
- **Нотации:** прямоугольники для классов, стрелки со стереотипами для связей, пулевые лимиты, аннотации «стереотипы» («<<interface>>») и т. п.

Что такое диаграмма в UML

Диаграмма — это один лист, где с помощью специальных значков показывают определённый срез системы: структуру, поведение или взаимодействие. Каждая диаграмма отвечает на свой вопрос, поэтому нет смысла рисовать их все подряд.

Две большие группы

- **Структурные** — «кто с кем связан» (классы, компоненты, узлы).
- **Поведенческие** — «что происходит во времени» (сценарии, процессы, состояния). Внутри поведенческих выделяют поднабор «диаграмм взаимодействия» (sequence, communication).

Когда нужна	Диаграмма	Что показывает
Сбор требований	Use Case	актёры-пользователи и их цели
Быстрый обзор системы	Package или контекстная Component	крупные блоки и связи
Объектная структура	Class	классы, их атрибуты, методы и отношения Википедия
Детальный сценарий	Sequence	кто и в каком порядке посылает сообщения
Бизнес-процесс	Activity	шаги и ветвления работы
Реакция объекта	State Machine	возможные состояния и переходы
Развёртывание	Deployment	на каких узлах/серверных машинах крутятся компоненты
Поставка кода	Component	как модули упакованы в библиотеки/сервисы

Процесс проектирования с использованием UML

Процесс проектирования обычно проходит через несколько этапов (итераций), на каждом из которых выстраивается своя совокупность UML-диаграмм:

1. Сбор и анализ требований

- Составление **Use Case Diagram** для выявления бизнес-функций и актёров.

- Детализация требований текстом и постусловиями.

2. Предварительное проектирование (Architecture & Domain Modeling)

- **Class Diagram** предметной области (домена) — ключевые сущности, их атрибуты и связи.
- **Package Diagram** — группировка классов по подсистемам.

3. Дизайн поведения (Dynamic Modeling)

- **Sequence** и/или **Communication Diagrams** для критичных сценариев (транзакции, бизнес-процессы).
- **Activity Diagrams** для описания потоков работ и алгоритмов.
- **State Machine Diagrams** для жизненных циклов важных объектов.

4. Детальное проектирование компонентов

- **Component Diagram** — модули, библиотеки, их интерфейсы и зависимости.
- **Deployment Diagram** — развертывание компонентов на серверах, устройствах, контейнерах.

5. Валидация и итерация

- Проверка модели на полноту и непротиворечивость.
- Уточнение диаграмм, добавление новых или переработка существующих.
- Выход на код-генерацию (при поддержке инструментов) или на подготовку проектной документации.

6. Поддержка и сопровождение

- Обновление диаграмм при изменении требований или архитектуры.
- Использование UML в качестве справочного материала при развитии и рефакторинге.

5. Рекомендации при работе с UML

- **Не перегружать одну диаграмму:** лучше сделать несколько узконаправленных, чем одну «супер-мегадиаграмму».
- **Использовать стереотипы и метки:** они помогают сразу понять назначение элементов.
- **Следить за уровнем детализации:** на раннем этапе — только базовые классы и прецеденты, на позднем — детали алгоритмов и интерфейсов.
- **Инструменты:** Enterprise Architect, Visual Paradigm, StarUML, MagicDraw, PlantUML (текстовая генерация).

1. Степенные ряды. Область сходимости. Радиус сходимости.
Разложение функции в степенной ряд.

Степенные ряды

Определение

Ряд, членами которого являются степенные функции аргумента x , называется *степенным рядом*:

$$\sum_{n=0}^{\infty} a_n x^n = a_0 + a_1 x + a_2 x^2 + \dots + a_n x^n + \dots$$

Часто рассматривается также ряд, расположенный по степеням $(x-x_0)$, то есть ряд вида

$$\sum_{n=0}^{\infty} a_n (x - x_0)^n = a_0 + a_1 (x - x_0) + a_2 (x - x_0)^2 + \dots + a_n (x - x_0)^n + \dots,$$

где x_0 – действительное число.

Интервал и радиус сходимости

Рассмотрим функцию $f(x) = \sum_{n=0}^{\infty} a_n (x-x_0)^n$. Ее областью определения является множество тех значений x , при которых ряд сходится. Область определения такой функции называется *интервалом сходимости*.

Если интервал сходимости представляется в виде (x_0-R, x_0+R) , где $R>0$, то величина R называется *радиусом сходимости*. Сходимость ряда в конечных точках интервала проверяется отдельно.

Радиус сходимости можно вычислить, воспользовавшись радикальным признаком Коши, по формуле

$$R = \lim_{n \rightarrow \infty} \frac{1}{\sqrt[n]{a_n}}$$

или на основе признака Даламбера:

$$R = \lim_{n \rightarrow \infty} \left| \frac{a_n}{a_{n+1}} \right|.$$

Областью сходимости функционального ряда

$$\sum_{n=1}^{\infty} u_n(x) = u_1(x) + u_2(x) + u_3(x) + \dots \quad (1)$$

называется множество значений аргумента x , для которых этот ряд сходится. Функция

$$S_n(x) = \sum_{k=1}^n u_k(x) = u_1(x) + u_2(x) + \dots + u_n(x) \quad (2)$$

называется **частичной суммой** ряда; ее предельное значение

$$\lim_{n \rightarrow \infty} S_n(x) = S(x) \quad (3)$$

называется **суммой ряда**; разность

$$R_n(x) = S(x) - S_n(x) \quad (4)$$

называется **остатком** ряда.

Для нахождения области сходимости ряда (1) можно использовать известные признаки сходимости, в частности, признаки Даламбера и Коши.

Пример 1. Чтобы определить область сходимости ряда

$$\sum_{n=1}^{\infty} \frac{n}{x^n},$$

воспользуемся признаком Даламбера:

$$\lim_{n \rightarrow \infty} \left| \frac{u_{n+1}(x)}{u_n(x)} \right| = \lim_{n \rightarrow \infty} \left| \frac{(n+1)x^n}{x^{n+1}n} \right| = \frac{1}{|x|} \lim_{n \rightarrow \infty} \left| \frac{n+1}{n} \right| = \frac{1}{|x|} < 1,$$

что влечет

$$|x| > 1.$$

В граничных точках $x = \pm 1$ ряд расходится.

Пример 2. Область сходимости ряда

$$\sum_{n=1}^{\infty} \frac{1}{n^{\ln x}}$$

легко устанавливается с помощью сравнения с эталонным рядом

$$\sum_{n=1}^{\infty} \frac{1}{n^p},$$

который сходится при $p > 1$. Тогда

$$\ln x > 1 \quad \Rightarrow \quad x > e.$$

1. *Ряд Тейлора*

Если функция $f(x)$ имеет непрерывные производные до $(n+1)$ -го порядка включительно, то ее можно разложить в степенной ряд в точке $x = a$ по *формуле Тейлора*:

$$f(x) = \sum_{n=0}^{\infty} f^{(n)}(a) \frac{(x-a)^n}{n!} = f(a) + f'(a)(x-a) + \frac{f''(a)(x-a)^2}{2!} + \dots + \frac{f^{(n)}(a)(x-a)^n}{n!} + R_n,$$

где *остаточный член* R_n в форме Лагранжа определяется выражением

$$R_n = \frac{f^{(n+1)}(\xi)(x-a)^{n+1}}{(n+1)!}, \quad a < \xi < x.$$

Если данное разложение сходится в некотором интервале x с центром в точке a , т.е. $\lim_{n \rightarrow \infty} R_n = 0$, то оно называется *рядом Тейлора*, представляющим разложение функции $f(x)$ в окрестности точки a .

2. *Ряд Маклорена* является частным случаем ряда Тейлора, когда разложение в степенной ряд производится в точке $a = 0$:

$$f(x) = \sum_{n=0}^{\infty} f^{(n)}(0) \frac{x^n}{n!} = f(0) + f'(0)x + \frac{f''(0)x^2}{2!} + \dots + \frac{f^{(n)}(0)x^n}{n!} + R_n.$$

Ниже приводятся разложения некоторых функций в ряд Маклорена.

3. $e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots + \frac{x^n}{n!} + \dots$

4. $a^x = 1 + \frac{x \ln a}{1!} + \frac{(x \ln a)^2}{2!} + \frac{(x \ln a)^3}{3!} + \dots + \frac{(x \ln a)^n}{n!} + \dots$

5. $\ln(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \dots + \frac{(-1)^n x^{n+1}}{n+1} \pm \dots, \quad -1 < x \leq 1$

6. $\ln \frac{1+x}{1-x} = 2 \left(x + \frac{x^3}{3} + \frac{x^5}{5} + \frac{x^7}{7} + \dots \right), \quad |x| < 1$

7. $\ln x = 2 \left[\frac{x-1}{x+1} + \frac{1}{3} \left(\frac{x-1}{x+1} \right)^3 + \frac{1}{5} \left(\frac{x-1}{x+1} \right)^5 + \dots \right], \quad x > 0$

8. $\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots + \frac{(-1)^n x^{2n}}{(2n)!} \pm \dots$

9. $\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots + \frac{(-1)^n x^{2n+1}}{(2n+1)!} \pm \dots$

10. $\tan x = x + \frac{x^3}{3} + \frac{2x^5}{15} + \frac{17x^7}{315} + \frac{62x^9}{2835} + \dots, \quad |x| < \frac{\pi}{2}$

11. $\cot x = \frac{1}{x} - \left(\frac{x}{3} + \frac{x^3}{45} + \frac{2x^5}{945} + \frac{2x^7}{4725} + \dots \right), \quad |x| < \pi$

12. $\arcsin x = x + \frac{x^3}{2 \cdot 3} + \frac{1 \cdot 3 x^5}{2 \cdot 4 \cdot 5} + \dots + \frac{1 \cdot 3 \cdot 5 \dots (2n-1) x^{2n+1}}{2 \cdot 4 \cdot 6 \dots (2n)(2n+1)} + \dots, \quad |x| < 1$

2. Основные этапы проектирования ИЭС. Каскадная и модифицированная, спиральная модели этапов проектирования (ЖЦ) ИЭС. Пять способов создания систем.

Основные этапы проектирования ИЭС

1. Формулирование задачи и сбор требований

- Определение предметной области, целей системы, состава и формата входных/выходных данных.
- Интервью с экспертами, анализ документов, постановка функциональных и нефункциональных требований.

2. Анализ и формализация знаний

- Сбор экспертных знаний (интервью, протоколирование, анкеты).
- Формализация в виде правил, фреймов, семантических сетей или других структур.
- Построение онтологии предметной области.

3. Проектирование архитектуры ИЭС

- Высокоуровневая схема компонентов: база знаний, движок вывода, подсистемы ввода/вывода, пользовательский интерфейс.
- Определение интерфейсов между модулями и потоков данных.

4. Детальное проектирование и спецификация

- Проектирование структуры базы знаний (правила, факты, таксономии).
- Выбор и настройка механизма вывода (прямой/обратный цепочный вывод, нечёткая логика и т. д.).
- Описание алгоритмов управления консультацией и объяснений.

5. Реализация

- Кодирование базы знаний и движка (на ПРОЛОГе, CLIPS, Java + экспертная оболочка и т. п.).
- Разработка интерфейсов ввода/вывода, реализация механизмов объяснения “почему” и “зачем”.

6. Тестирование и верификация

- Проверка корректности вывода на тестовых ситуациях (контроль полноты и непротиворечивости базы знаний).

– Валидация с экспертами: сравнение рекомендаций ИЭС с решениями живых специалистов.

7. Внедрение и сопровождение

– Обучение пользователей, введение в эксплуатацию.

– Пополнение и корректировка базы знаний по мере появления новых фактов и правил.

– Мониторинг качества рекомендаций, доработка системы.

Модели жизненного цикла (ЖЦ) проектирования ИЭС

Модель	Описание	Плюсы	Минусы
Каскадная	Этапы идут строго последовательно: требования → анализ → дизайн → реализация → тестирование → внедрение.	Прозрачность, простота управления	Отсутствие возврата назад; позднее выявление ошибок
Модифицированная (с V-моделью)	Классический каскад дополнен параллельной ветвью тестирования: для каждого этапа разработки есть соответствующий этап проверки.	Раннее планирование тестов, лучшее качество	Всё ещё жёсткая структура, сложности при изменениях
Спиральная	Итеративный – каждая «виток» включает планирование, анализ рисков, реализацию	Гибкость, фокус на рисках, частые проверки	Более сложное управление, требуется опыт в

Модель	Описание	Плюсы	Минусы
	прототипа и оценку.		оценке рисков

Пять способов создания (разработки) систем

1. Ручное программирование «с нуля»

– Полная свобода в выборе алгоритмов и структур, но высокая трудоёмкость.

2. Использование стандартных библиотек и API

– Быстрая реализация «типовых» функций (работа с БД, GUI, файловой системой), остаётся большая часть ручной работы.

3. Генераторы кода (template-based development)

– Автоматическая генерация частей приложения по заранее заданным шаблонам/метамоделям (например, CRUD-модули, интерфейсные формы).

4. CASE-средства и платформы низкокода

– Визуальное моделирование (DFD, ER-диаграммы, UML) с последующей автоматической генерацией кода и метаданных.
– Снижение порога входа для специалистов бизнес-аналитиков.

5. Конфигурируемые/шаблонные решения (COTS + настройка)

– Приобретение готового ПО (базовой экспертной оболочки, CRM/ERP/BI-системы) и его параметрическая настройка под нужды заказчика.
– Минимальные доработки, быстрое внедрение, но ограничения по гибкости.

1. Матрицы, операции над ними. Определители n -го порядка, теорема Лапласа. Обратная матрица, ранг матрицы, базисный минор.

Матрицы, операции над ними.

Матрицей размера $m \times n$ называется таблица чисел, состоящая из **m строк и n столбцов**. Числа, составляющие матрицу, называются **элементами матрицы**.

Матрицы обозначаются прописными буквами латинского алфавита (например, A, B, C), а элементы матрицы – строчными буквами с двойной индексацией: a_{ij} , где i – номер строки, j – номер столбца.

Например, матрица $A_{mn} = \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1j} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2j} & \dots & a_{2n} \\ \dots & \dots & \dots & \dots & \dots & \dots \\ a_{i1} & a_{i2} & \dots & a_{ij} & \dots & a_{in} \\ \dots & \dots & \dots & \dots & \dots & \dots \\ a_{m1} & a_{m2} & \dots & a_{mj} & \dots & a_{mn} \end{pmatrix},$

или в сокращенной записи $A = (a_{ij})$, где $i = 1, 2, \dots, m$; $j = 1, 2, \dots, n$.

Элементы матрицы a_{ij} , у которых $i=j$, называются **диагональными** и образуют **главную диагональ**. Для квадратной матрицы ($m=n$) главную диагональ образуют элементы $a_{11}, a_{22}, \dots, a_{nn}$.

Равенство матриц.

$A=B$, если порядки матриц A и B одинаковы и $a_{ij}=b_{ij}$ ($i=1, 2, \dots, m$; $j=1, 2, \dots, n$)

Виды матриц

1. **Прямоугольные:** m и n - произвольные положительные целые числа.

2. **Квадратные:** $m=n$.

3. **Матрица строка:** $m=1$. Например, $(1\ 3\ 5\ 7)$ - во многих практических задачах такая матрица называется вектором.

4. **Матрица столбец:** $n=1$. Например, $\begin{pmatrix} 1 \\ 4 \\ 6 \\ 8 \end{pmatrix}$.

5. **Диагональная матрица:** $m=n$ и $a_{ij} = 0$, если $i \neq j$. Например,

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{pmatrix}.$$

6. **Единичная матрица:** $m=n$ и $E = \delta_{ij} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$,

т.е. $\delta_{ij} = 0$, если $i \neq j$

$\delta_{ij} = 1$, если $i = j$.

7. **Нулевая матрица:** $a_{ij}=0$, $i=1,2,\dots,m$, $j=1,2,\dots,n$. Пример, $\begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$.

8. **Треугольная матрица:** все элементы ниже главной диагонали равны 0.

Пример. $\begin{pmatrix} 1 & 3 & 4 \\ 0 & 2 & 5 \\ 0 & 0 & 3 \end{pmatrix}$.

9. Симметрическая матрица: $m=n$ и $a_{ij}=a_{ji}$ (т.е. на симметричных относительно главной диагонали местах стоят равные элементы), а, следовательно, $A'=A$.

Например, $\begin{pmatrix} 0 & 1 & 3 \\ 1 & 2 & 5 \\ 3 & 5 & 4 \end{pmatrix}$.

Действия над матрицами.

Некоторые операции над матрицами, такие как сложение и вычитание, допускаются только для матриц одинакового размера.

1. Сложение матриц - поэлементная операция

$$A_{mn} + B_{mn} = C_{mn} = \begin{pmatrix} a_{11} + b_{11} & a_{12} + b_{12} & \dots & a_{1n} + b_{1n} \\ \dots & \dots & \dots & \dots \\ a_{m1} + b_{m1} & a_{m2} + b_{m2} & \dots & a_{mn} + b_{mn} \end{pmatrix}$$

2. Вычитание матриц - поэлементная операция

$$A_{mn} - B_{mn} = \begin{pmatrix} a_{11} - b_{11} & a_{12} - b_{12} & \dots & a_{1n} - b_{1n} \\ \dots & \dots & \dots & \dots \\ a_{m1} - b_{m1} & a_{m2} - b_{m2} & \dots & a_{mn} - b_{mn} \end{pmatrix}$$

3. Произведение матрицы на число - поэлементная операция

$$\lambda A = \begin{pmatrix} \lambda a_{11} & \lambda a_{12} & \dots & \lambda a_{1n} \\ \lambda a_{21} & \lambda a_{22} & \dots & \lambda a_{2n} \\ \dots & \dots & \dots & \dots \\ \lambda a_{m1} & \lambda a_{m2} & \dots & \lambda a_{mn} \end{pmatrix}$$

4. Умножение $A * B$ матриц по правилу строка на столбец (число столбцов матрицы A должно быть равно числу строк матрицы B).

$A_{mk} * B_{kn} = C_{mn}$ причем каждый элемент c_{ij} матрицы C_{mn} равен сумме произведений элементов i -ой строки матрицы A на соответствующие элементы j -го столбца матрицы B , т.е. $C_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j} + \dots + a_{in}b_{nj} = \sum_{k=1}^n a_{ik}b_{kj}$.

Например,

$$C_{11} = a_{11}b_{11} + a_{12}b_{21} + a_{13}b_{31} + \dots + a_{1n}b_{n1},$$

$$C_{12} = a_{11}b_{12} + a_{12}b_{22} + a_{13}b_{32} + \dots + a_{1n}b_{n2}.$$

Пример:

$$\text{Пусть } A_{23} = \begin{pmatrix} 1 & 0 & 2 \\ 3 & 1 & 0 \end{pmatrix}, B_{33} = \begin{pmatrix} -1 & 0 & 1 \\ 5 & 1 & 4 \\ -2 & 0 & 1 \end{pmatrix}.$$

$$\begin{aligned} A_{23} * B_{33} &= C_{23} = \begin{pmatrix} 1 & 0 & 2 \\ 3 & 1 & 0 \end{pmatrix} * \begin{pmatrix} -1 & 0 & 1 \\ 5 & 1 & 4 \\ -2 & 0 & 1 \end{pmatrix} = \\ &= \begin{pmatrix} 1 * (-1) + 0 * 5 + 2 * (-2) & 1 * 0 + 0 * 1 + 2 * 0 & 1 * 1 + 0 * 4 + 2 * 1 \\ 3 * (-1) + 1 * 5 + 0 * (-2) & 3 * 0 + 1 * 1 + 0 * 0 & 3 * 1 + 1 * 4 + 0 * 1 \end{pmatrix} = \\ &= \begin{pmatrix} -5 & 0 & 3 \\ 2 & 1 & 7 \end{pmatrix} \end{aligned}$$

5. Возведение в степень

$$A^m = \underbrace{A * A * \dots * A}_{m \text{ раз}}$$

$m > 1$ целое положительное число. A - квадратная матрица ($m=n$) т.е. актуально только для квадратных матриц.

6. Транспонирование матрицы A. Транспонированную матрицу обозначают A^T или A' .

$$\text{Если } A = \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \dots & \dots & \dots & \dots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{pmatrix}, \text{ то}$$

$$A^T = A' = \begin{pmatrix} a_{11} & a_{21} & \dots & a_{m1} \\ a_{12} & a_{22} & \dots & a_{m2} \\ \dots & \dots & \dots & \dots \\ a_{1n} & a_{2n} & \dots & a_{mn} \end{pmatrix}.$$

Строки и столбцы поменялись местами.

Пример:

$$A = \begin{pmatrix} 1 & 0 & 2 \\ 3 & 1 & 0 \end{pmatrix}, A^T = \begin{pmatrix} 1 & 3 \\ 0 & 1 \\ 2 & 0 \end{pmatrix}$$

Свойства операций над матрицами

$$A+B = B+A$$

$$(A+B)+C = A+(B+C)$$

$$\lambda(A+B) = \lambda A + \lambda B$$

$$A(B+C) = AB+AC$$

$$(A+B)C = AC+BC$$

$$\lambda(AB) = (\lambda A)B = A(\lambda B)$$

$$A(BC) = (AB)C$$

$$(A')' = A$$

$$(\lambda A)' = \lambda(A)'$$

$$(A+B)' = A'+B'$$

$$(AB)' = B'A'$$

Определители n-го порядка, теорема Лапласа.

Минором M^{ij} элемента a^{ij} служит определитель $(n-1)$ -го порядка Δ_{n-1} , полученный из определителя n -го порядка Δ_n вычеркиванием i -й строки и j -го столбца.

Алгебраическим дополнением A^{ij} элемента a^{ij} называют минор M^{ij} , взятый со знаком плюс или минус в зависимости от номеров вычеркиваемых строки и столбца, т. е.

$$A^{ij} = (-1)^{i+j} * M^{ij}.$$

Свойства алгебраических дополнений:

$$1. \sum_{k=1}^n a_{ik} A_{ik} = \sum_{k=1}^n a_{kj} A_{kj} = \Delta_n.$$

Эти равенства можно принять за правила вычисления определителя. Первое из них называется разложением определителя Δ_n по элементам i -й строки, второе – по элементам j -го столбца.

$$2. \sum_{k=1}^n a_{ik} A_{jk} = 0, \quad \sum_{k=1}^n a_{kj} A_{ki} = 0 \text{ при } i \neq j.$$

Правило для вычисления определителя любого порядка основано на теореме Лапласа.

Теорема Лапласа. Определитель равен сумме попарных произведений элементов любого ряда (строки или столбца) на их алгебраические дополнения.

В соответствии с этой теоремой определитель может быть вычислен путем его разложения или по элементам любой строки или любого столбца.

В общем виде определитель n-го порядка можно разложить и вычислить следующими способами:

[illegible]

[illegible]

Вычислить определитель $\Delta = \begin{vmatrix} 3 & 3 & -1 \\ 1 & -1 & 2 \\ 5 & 2 & 1 \end{vmatrix}$ по теореме Лапласа

путем разложения его по элементам 3-ей строки и элементам 1-го столбца.

Вычисляем определитель путем разложения его по 3-ей строке

$$\Delta = \begin{vmatrix} 3 & 3 & -1 \\ 1 & -1 & 2 \\ 5 & 2 & 1 \end{vmatrix} = 5 * A_{31} + 2 * A_{32} + 1 * A_{33} =$$

$$\begin{aligned}
&= 5 * (-1)^{3+1} * M_{31} + 2 * (-1)^{3+2} * M_{32} + (-1)^{3+3} * M_{33} \\
&= 5 * M_{31} - 2 * M_{32} + M_{33} =
\end{aligned}$$

$$= 5 * \begin{vmatrix} 3 & -1 \\ -1 & 2 \end{vmatrix} - 2 * \begin{vmatrix} 3 & -1 \\ 1 & 2 \end{vmatrix} + \begin{vmatrix} 3 & 3 \\ 1 & -1 \end{vmatrix}$$

$$= 5 * (6 - 1) - 2 * (6 + 1) + (-3 - 3) =$$

$$= 25 - 14 - 6 = 5$$

Обратная матрица, ранг матрицы, базисный минор.

Квадратная матрица A порядка n называется **невырожденной** (или **неособенной**), если $\det A \neq 0$. В противном случае матрица A – **вырожденная** (или **особенная**).

Матрица A^{-1} является **обратной** для квадратной невырожденной матрицы A , если $A^{-1}A = AA^{-1} = E$, где E - единичная матрица порядка n :

$$E = \begin{pmatrix} 1 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 \\ \dots & \dots & \dots & \dots \\ 0 & 0 & \dots & 1 \end{pmatrix}.$$

Рангом матрицы A (обозначается $\text{rang } A$ или $r(A)$) является наибольший порядок порожденных ею миноров (определителей), отличных от нуля. Всякий отличный от нуля минор матрицы, порядок которого равен ее рангу, называется ее **базисным минором**. Строки и столбцы, участвующие в образовании базисного минора, также будут базисными. Матрица может иметь несколько базисных миноров, однако все их порядки одинаковы и равны рангу матрицы.

Ранг матрицы не изменится, если:

- 1) строки и столбцы матрицы поменять местами;
- 2) переставить местами два любых ее столбца (строки);
- 3) удалить из нее столбец (строку), все элементы которого равны нулю;
- 4) удалить из нее столбец (строку), являющийся линейной комбинацией остальных ее столбцов (строк);

5) умножить ее произвольный столбец (строку) на любое отличное от нуля число;

6) к любому ее столбцу (строке) прибавить произвольную линейную комбинацию остальных столбцов (строк) этой матрицы.

Преобразования 2) - 6) называются **элементарными**. Две матрицы являются **эквивалентными**, если одна получается из другой с помощью элементарных преобразований и обозначается как $A \sim B$.

Для рангов матриц справедливы следующие соотношения:

$$1) r(A + B) \leq r(A) + r(B),$$

$$2) r(A + B) \geq |r(A) - r(B)|,$$

$$3) r(AB) \leq \min\{r(A); r(B)\},$$

$$4) r(A^T A) = r(A),$$

$$5) r(AB) = r(A), \text{ если } B - \text{квадратная матрица и } \Delta(B) \neq 0,$$

$$6) r(AB) \geq r(A) + r(B) - n, \text{ где } n - \text{число столбцов матрицы } A \text{ или строк матрицы } B.$$

Пример определения ранга матрицы:

$$1) A = \begin{pmatrix} 0 & 3 & 0 \\ 0 & 0 & 0 \end{pmatrix}.$$

Очевидно, что $a_{12} = 3 \neq 0 = M_1$,

$$\begin{vmatrix} 0 & 3 \\ 0 & 0 \end{vmatrix}, \begin{vmatrix} 0 & 0 \\ 0 & 0 \end{vmatrix}, \begin{vmatrix} 3 & 0 \\ 0 & 0 \end{vmatrix} \text{ все миноры } M_2=0, \text{ следовательно, } r(A) = 1.$$

$$2) A = \begin{pmatrix} 1 & 0 & 0 & 0 & 5 \\ 0 & 0 & 0 & 0 & 0 \\ 2 & 0 & 0 & 0 & 11 \end{pmatrix} \sim \begin{pmatrix} 1 & 0 & 0 & 0 & 5 \\ 2 & 0 & 0 & 0 & 11 \end{pmatrix} \sim \begin{pmatrix} 1 & 5 \\ 2 & 11 \end{pmatrix},$$

$$\begin{vmatrix} 1 & 5 \\ 2 & 11 \end{vmatrix} = 11 - 10 = 1 \neq 0 \Rightarrow r(A) = 2.$$

В последнем примере базисный минор один - $\begin{vmatrix} 1 & 5 \\ 2 & 11 \end{vmatrix}$, потому что все остальные миноры равны 0.

2. Виды обеспечения ИЭС. Организационно, лингвистическое, правовое. Состав и структура информационного обеспечения ИЭС. Состав и роли работников группы разработки проекта.

1. Виды обеспечения ИЭС

1. Организационное обеспечение

Включает все меры и процедуры, которые обеспечивают правильное функционирование ИЭС в бизнес-процессах:

- Определение ролей и ответственности участников (эксперты, операторы, администраторы)
- Регламенты сбора и верификации знаний
- Процедуры принятия решений на основе рекомендаций ИЭС
- Планирование этапов внедрения, обучения пользователей и поддержки

2. Лингвистическое обеспечение

Отвечает за корректное представление и обработку терминологии предметной области:

- Создание и поддержка **терминологического словаря** (гlossария)
- Разработка **шаблонов фраз** и **правил разбора** естественного языка (если ИЭС общается с пользователем «по-человечески»)
- Настройка лингвистических модулей: токенизация, морфологический разбор, синтаксический парсинг
- Обеспечение однозначности терминов и синонимов, ведение «контрольного списка» допустимых формулировок

3. Правовое обеспечение

Гарантирует, что работа ИЭС соответствует законодательным и нормативным требованиям:

- Соблюдение законов о персональных данных и конфиденциальности (например, GDPR, ФЗ-152)
- Лицензирование ПО и сторонних компонентов (экспертных оболочек, баз знаний)
- Авторское право на знания и материалы экспертов
- Ответственность за решения, принимаемые на основе рекомендаций ИЭС (юридические соглашения с заказчиками)

2. Состав и структура информационного обеспечения ИЭС

Информационное обеспечение — это все **входные** данные и **модели**, на которых базируется экспертная система:

1. База знаний

- **Декларативное знание:** факты, термины, отношения (онтологии, фреймы)
- **Процедурное знание:** правила вывода, алгоритмы логики, эвристики

2. База фактов (fact base)

- Текущие данные о конкретном случае (параметры измерений, характеристики объекта)
- Исторические кейсы и прецеденты для обучения и проверки

3. Метаданные и справочники

- Словари терминов, классификаторы, коды
- Описания форматов входных/выходных документов
- Графы зависимостей между сущностями

4. Логические модели и онтологии

- Семантические сети или OWL-онтологии для формирования предметной модели
- Связи “часть–целое”, “причина–следствие”, типы процессов

5. Информационные потоки и интерфейсы

- Определение, какие данные и когда поступают в ИЭС и выводятся из неё
- Форматы обмена (XML, JSON, Web-сервисы)

6. Архитектура хранения и доступа

- Реляционные/NoSQL-хранилища для фактов и справочников
- Репозитории правил и версионный контроль базы знаний

Структурная схема информационного обеспечения обычно представляется в виде DFD/UML-диаграммы, где:

- Процессы → чтение/запись фактов, применение правил
- Хранилища → базы знаний и фактов
- Внешние сущности → источники данных (датчики, БД, файлы) и потребители результатов

3. Состав и роли работников группы разработки проекта ИЭС

Роль	Основные обязанности
Руководитель проекта	Планирование сроков, ресурсов, коммуникация с заказчиком
Бизнес-аналитик	Сбор и формализация требований, оформление ТЗ
Эксперт предметной области	Предоставляет знания, участвует в верификации базы знаний
Инженер знаний (knowledge engineer)	Выделяет, структурирует, кодирует знания в формате правил/фреймов
Системный архитектор	Проектирует общую архитектуру ИЭС, интеграцию с ИТ-ландшафтом

Роль	Основные обязанности
Разработчик ПО	Создание и настройка движка вывода, модулей обработки данных
Инженер In (лингвист)	Настройка NLP-модулей, поддержка словарей и парсеров
Тестировщик (QA-инженер)	Разработка тест-кейсов, проверка корректности выводов ИЭС
Администратор БД / знаний	Управление хранилищем фактов/прав, резервное копирование
Технический писатель	Подготовка пользовательской и эксплуатационной документации
Инженер сопровождения	Внедрение, обучение пользователей, поддержка после запуска

БИЛЕТ 13

2. Решение систем линейных уравнений по правилу Крамера, по методу Гаусса.

Решение систем линейных уравнений по правилу Крамера

Пусть нам требуется решить систему линейных алгебраических уравнений

[illegible]

в которой число уравнений равно числу неизвестных переменных и определитель основной матрицы системы отличен от нуля, то есть, $|A| \neq 0$.

Пусть Δ - определитель основной матрицы системы, а $\Delta x_1, \Delta x_2, \dots, \Delta x_n$ - определители матриц, которые получаются из A заменой 1-ого, 2-ого, ..., n -ого столбца соответственно на столбец свободных членов:

$$\Delta = \begin{vmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \dots & \dots & \dots & \dots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{vmatrix}, \quad \Delta_{x_1} = \begin{vmatrix} b_1 & a_{12} & \dots & a_{1n} \\ b_2 & a_{22} & \dots & a_{2n} \\ \dots & \dots & \dots & \dots \\ b_n & a_{n2} & \dots & a_{nn} \end{vmatrix},$$

$$\Delta_{x_2} = \begin{vmatrix} a_{11} & b_1 & \dots & a_{1n} \\ a_{21} & b_2 & \dots & a_{2n} \\ \dots & \dots & \dots & \dots \\ a_{n1} & b_n & \dots & a_{nn} \end{vmatrix}, \quad \dots, \quad \Delta_{x_n} = \begin{vmatrix} a_{11} & a_{12} & \dots & b_1 \\ a_{21} & a_{22} & \dots & b_2 \\ \dots & \dots & \dots & \dots \\ a_{n1} & a_{n2} & \dots & b_n \end{vmatrix}$$

При таких обозначениях неизвестные переменные вычисляются по формулам метода Крамера как $x_1 = \frac{\Delta_{x_1}}{\Delta}$, $x_2 = \frac{\Delta_{x_2}}{\Delta}$, ..., $x_n = \frac{\Delta_{x_n}}{\Delta}$.

Причем, существует 3 случая:

- 1) $\Delta \neq 0, \Delta x_i \neq 0 \Rightarrow \exists$ единственное решение;
- 2) $\Delta = 0, \Delta x_i \neq 0 \Rightarrow$ решений нет;
- 3) $\Delta = 0, \Delta x_i = 0 \Rightarrow$ бесконечное множество решений.

Так находится решение системы линейных алгебраических уравнений методом Крамера.

Пример

Решите систему линейных уравнений методом Крамера

$$\begin{cases} 2x_1 + 3x_2 - x_3 = 9 \\ x_1 - 2x_2 + x_3 = 3 \\ x_1 + 2x_3 = 2 \end{cases}.$$

Решение.

Основная матрица системы имеет вид, $A = \begin{pmatrix} 2 & 3 & -1 \\ 1 & -2 & 1 \\ 1 & 0 & 2 \end{pmatrix}$.

Вычислим ее определитель:

$$\Delta = \begin{vmatrix} 2 & 3 & -1 \\ 1 & -2 & 1 \\ 1 & 0 & 2 \end{vmatrix} = 2 * (-2) * 2 + 3 * 1 * 1 + (-1) * 0 * 1 -$$

$$-(-1) * (-2) * 1 - 3 * 2 * 1 - 2 * 1 * 0 = -13$$

Так как определитель основной матрицы системы отличен от нуля, то система имеет единственное решение, которое может быть найдено методом Крамера.

Составим и вычислим необходимые определители Δ_{x_1} , Δ_{x_2} , Δ_{x_3} (определитель Δ_{x_1} получаем, заменив в матрице A первый столбец на столбец свободных членов $\begin{pmatrix} 9 \\ 3 \\ 2 \end{pmatrix}$, определитель Δ_{x_2} - заменив второй столбец на столбец свободных членов, Δ_{x_3} - заменив третий столбец матрицы A на столбец свободных членов):

$$\Delta_{x_1} = \begin{vmatrix} 9 & 3 & -1 \\ 3 & -2 & 1 \\ 2 & 0 & 2 \end{vmatrix} = 9 * (-2) * 2 + 3 * 1 * 2 + (-1) * 0 * 3 -$$

$$-(-1) * (-2) * 2 - 3 * 2 * 3 - 9 * 1 * 0 = -52$$

$$\Delta_{x_2} = \begin{vmatrix} 2 & 9 & -1 \\ 1 & 3 & 1 \\ 1 & 2 & 2 \end{vmatrix} = 2 * 3 * 2 + 9 * 1 * 1 + (-1) * 2 * 1 -$$

$$-(-1) * 3 * 1 - 9 * 2 * 1 - 2 * 1 * 2 = 0$$

$$\Delta_{x_3} = \begin{vmatrix} 2 & 3 & 9 \\ 1 & -2 & 3 \\ 1 & 0 & 2 \end{vmatrix} = 2 * (-2) * 2 + 3 * 3 * 1 + 9 * 0 * 1 -$$

$$-9 * (-2) * 1 - 3 * 2 * 1 - 2 * 3 * 0 = 13$$

Находим неизвестные переменные по формулам $x_1 = \frac{\Delta_{x_1}}{\Delta}$, $x_2 = \frac{\Delta_{x_2}}{\Delta}$, $x_3 = \frac{\Delta_{x_3}}{\Delta}$:

$$x_1 = \frac{\Delta_{x_1}}{\Delta} = \frac{-52}{-13} = 4,$$

$$x_2 = \frac{\Delta_{x_2}}{\Delta} = \frac{0}{-13} = 0,$$

$$x_3 = \frac{\Delta_{x_3}}{\Delta} = \frac{13}{-13} = -1.$$

Проверка решения:

$$2 * 4 + 3 * 0 - (-1) = 9,$$

$$4 - 2 * 0 + (-1) = 3,$$

$$4 + 2 * (-1) = 2.$$

Ответ:

$$x_1 = 4, x_2 = 0, x_3 = -1.$$

Решение систем линейных уравнений по методу Гаусса

Метод Гаусса основан на элементарных преобразованиях матрицы, под которыми понимаются следующие преобразования:

- 1) сложение (вычитание) строк;
- 2) перестановка строк;
- 3) умножение строки на любое число, кроме 0;
- 4) умножение строки на любое действительное число и прибавление к любой другой строке.

Матрицы называются **эквивалентными**, если их ранги равны

$$r(A) = r(B) \Rightarrow A \sim B$$

Алгоритм нахождения ранга матрицы с помощью элементарных преобразований.

1) выбираем в матрице $\forall a_{ij} \neq 0$ и объявляем его базисным элементом

Умножим i -ю строку на подходящие числа так, чтобы в j -ом столбце получить нули, кроме a_{ij} . Если окажется, что во всех строках, кроме i -ой все элементы равны нулю, то ранг матрицы равен единице. Если найдется $\exists a_{km} \neq 0$ перейти к пункту 2)

2) $a_{km} \neq 0$, a_{km} – базисный элемент. Умножая k -ую строку на подходящие числа так, чтобы в m -ом столбце получить все нули кроме a_{km} . Если в какой-либо строке $\exists a_{qp} \neq 0$, то процесс преобразования продолжается до тех пор, пока не получим матрицу, в которой:

- а) в каждой строке по одному базисному элементу;
- б) в столбце где стоят базисные элементы, все 0 кроме базисных. Тогда ранг матрицы равен количеству базисных элементов.

Если к матрице A добавить в качестве $(n+1)$ -ого столбца матрицу-столбец свободных членов, то получим так называемую **расширенную матрицу** системы линейных уравнений. Обычно расширенную матрицу обозначают буквой $\bar{A}$, а столбец свободных

членов отделяют вертикальной линией от остальных столбцов, то есть,

$$\bar{A} = \left(\begin{array}{cccc|c} a_{11} & a_{12} & \dots & a_{1n} & b_1 \\ a_{21} & a_{22} & \dots & a_{2n} & b_2 \\ \dots & \dots & \dots & \dots & \dots \\ a_{m1} & a_{m2} & \dots & a_{mn} & b_n \end{array} \right)$$

Алгоритм решения СЛУ методом Гаусса

1. $\bar{A}$ записываем расширенную матрицу и находим $r(A)$ с помощью элементарных преобразований, при этом базисные элементы выбираем до вертикальной черты.

2. Если в матрице окажется строка, в которой до вертикальной черты всё – нули, а после не ноль, то система несовместна, т.к. $r(\bar{A}) \neq r(A)$.

Если $r(A) = r(\bar{A})$, то смотри пункт 3.

3. Переменные, коэффициенты при которых стоят на местах базисных элементов, объявляем базисными переменными, остальные – свободными переменными.

4. По последней матрице восстанавливаем систему. При этом базисные переменные оставляем в левой части, свободные переменные переносим в правую часть с противоположным знаком.

5. Разделив каждое равенство на соответствующий коэффициент при переменной, получим равенства, в которых базисные переменные выражены через свободные, что и называется общим решением СЛУ.

6. Найдем частные решения задавая свободным переменным произвольные значения и вычисляя значения базисных переменных из общего решения.

Пример.

Решить СЛУ методом Гаусса:
$$\begin{cases} 3x_1 + 2x_2 - x_3 + 4x_4 = 1 \\ 2x_1 - 2x_2 + 5x_3 - 5x_4 = 3 \\ x_1 + 4x_2 - 6x_3 + 9x_4 = -2 \end{cases}$$

$$\bar{A} = \left(\begin{array}{cccc|c} 3 & 2 & -1 & 4 & 1 \\ 2 & -2 & 5 & -5 & 3 \\ \boxed{1} & 4 & -6 & 9 & -2 \end{array} \right) \sim$$

$a_{31} = 1 \neq 0$, баз. эл-т

3 стр.*(-2) + 2 стр.

3 стр.*(-3) + 1 стр.

$$\sim \left(\begin{array}{cccc|c} 0 & -10 & 17 & -23 & 7 \\ 0 & -10 & 17 & -23 & 7 \\ \boxed{1} & 4 & -6 & 9 & -2 \end{array} \right) \sim \left(\begin{array}{cccc|c} 0 & -10 & 17 & -23 & 7 \\ 0 & 0 & 0 & 0 & 0 \\ 1 & 4 & -6 & 9 & -2 \end{array} \right) \sim$$

$a_{12} = -10 \neq 0$, баз. эл-т

1 стр.*(-1) + 2 стр.

Замечание: если в матрице есть равные или пропорциональные строки, то из них оставляем только одну, остальные вычеркиваем. Нулевую строку вычеркиваем.

$$\sim \left(\begin{array}{cccc|c} 0 & \boxed{-10} & 17 & -23 & 7 \\ \boxed{1} & 4 & -6 & 9 & -2 \end{array} \right) \begin{array}{l} * 4 \\ * 10 \end{array} \sim \left(\begin{array}{cccc|c} 0 & \boxed{-40} & 68 & -92 & 28 \\ \boxed{10} & 40 & -60 & 90 & -20 \end{array} \right) \sim$$

$$\sim \left(\begin{array}{cccc|c} 0 & \boxed{-40} & 68 & -92 & 28 \\ \boxed{10} & 0 & 8 & -2 & 8 \end{array} \right) \begin{array}{l} : 4 \\ : 2 \end{array} \sim \left(\begin{array}{cccc|c} 0 & \boxed{-10} & 17 & -23 & 7 \\ \boxed{5} & 0 & 4 & -1 & 4 \end{array} \right)$$

$r(\bar{A}) = 2 = r(A) \Rightarrow$ [теор. Кронекера Капелли](#) $\Rightarrow$ теорема совместна

x_1, x_2 – базисные переменные,

x_3, x_4 – свободные переменные.

$$\begin{cases} -10x_2 = -17x_3 + 23x_4 + 7 & |: (-10) \\ 5x_1 = -4x_3 + x_4 + 4 & |: 5 \end{cases}$$

$$\begin{cases} x_2 = \frac{17x_3 - 23x_4 - 7}{10} \\ x_1 = \frac{-4x_3 + x_4 + 4}{5} \end{cases} \text{ - общее решение.}$$

(1)

Частное решение:

$$x_3 = 0, x_4 = 1 \Rightarrow (1)$$

$$x_2 = -3, x_1 = 1$$

$(1; -3; 0; 1)$ – частное решение.

Проверка 2 ур-е:

$$2x_1 - 2x_2 + 5x_3 - 5x_4 = 3$$

$$2*1 - 2*(-3) + 5*0 - 5*1 = 2+6-5 = \mathbf{3 = 3.}$$

3. Собственные векторы и собственные значения оператора.

Рассмотрим произвольную квадратную матрицу,

например, $A = \begin{pmatrix} -1 & -6 \\ 2 & 6 \end{pmatrix}$. И умножим данную матрицу справа на какой-нибудь подходящий столбец. Мне пришёл в голову

вектор $\vec{b} = \begin{pmatrix} -1 \\ 1 \end{pmatrix}$:

$$A\vec{b} = \begin{pmatrix} -1 & -6 \\ 2 & 6 \end{pmatrix} \cdot \begin{pmatrix} -1 \\ 1 \end{pmatrix} = \begin{pmatrix} -1 \cdot (-1) - 6 \cdot 1 \\ 2 \cdot (-1) + 6 \cdot 1 \end{pmatrix} = \begin{pmatrix} -5 \\ 4 \end{pmatrix} = \vec{c}$$

Вроде ничего примечательного – умножили матрицу

$A = \begin{pmatrix} -1 & -6 \\ 2 & 6 \end{pmatrix}$ на вектор-столбец $\vec{b} = \begin{pmatrix} -1 \\ 1 \end{pmatrix}$ и получили другой вектор-

столбец $\vec{c} = \begin{pmatrix} -5 \\ 4 \end{pmatrix}$. Обычная векторная жизнь. Но в обществе таких векторов существуют особые представители, которые обладают внутренним стержнем и не желают изменять себе в трудные минуты.

Умножим ту же матрицу на $\vec{u} = \begin{pmatrix} 2 \\ -1 \end{pmatrix}$:

$$A\vec{u} = \begin{pmatrix} -1 & -6 \\ 2 & 6 \end{pmatrix} \cdot \begin{pmatrix} 2 \\ -1 \end{pmatrix} = \begin{pmatrix} -1 \cdot 2 - 6 \cdot (-1) \\ 2 \cdot 2 + 6 \cdot (-1) \end{pmatrix} = \begin{pmatrix} 4 \\ -2 \end{pmatrix} = 2 \cdot \begin{pmatrix} 2 \\ -1 \end{pmatrix}$$

На последнем шаге вынесли константу. Что произошло? В результате умножения матрицы A на вектор $\vec{u}$, данный вектор птицей Феникс возродился с числовым коэффициентом $\lambda = 2$:

$$A\vec{u} = \lambda\vec{u}$$

Определение: ненулевой вектор $\vec{u}$, который при умножении на некоторую квадратную матрицу A превращается в самого же себя с числовым коэффициентом λ , называется собственным вектором матрицы A . Число λ называют собственным значением или собственным числом данной матрицы.

Собственный вектор — понятие в линейной алгебре, определяемое для произвольного линейного оператора как ненулевой вектор, применение к которому оператора даёт коллинеарный вектор — тот же вектор, умноженный на некоторое скалярное значение. Скаляр, на который умножается собственный вектор под действием оператора, называется **собственным числом** (или **собственным значением**) линейного оператора, соответствующим данному собственному вектору. Одним из представлений линейного оператора является квадратная матрица, поэтому собственные векторы и собственные значения часто определяются в контексте использования таких матриц. (из вики, ниже более формально)

Пусть V и W — линейные пространства размерностей n и m соответственно. Будем называть **оператором**, или **преобразованием**, A , действующим из V в W , отображение вида $A: V \rightarrow W$, сопоставляющее каждому элементу $\bar{x} \in V$ некоторый элемент $\bar{y} \in W$. При этом будем использовать обозначение $\bar{y} \in A(\bar{x})$ или $\bar{y} \in A\bar{x}$.

Пусть A — линейный оператор из векторного пространства $L(V, V)$. Число λ_0 называется **собственным значением оператора** A , если существует ненулевой вектор $\bar{x}$ такой, что $A\bar{x} = \lambda_0\bar{x}$. При этом вектор $\bar{x}$ называется **собственным вектором оператора** A , отвечающим собственному значению λ_0 . Множество всех собственных значений линейного оператора A называется его **спектром**.

Определителем линейного оператора A называется $\det A$, где A – матрица линейного оператора A в любом базисе. Многочлен относительно λ называется **характеристическим многочленом оператора** A . Он не зависит от выбора базиса.

Характеристический многочлен оператора A - многочлен $P(\lambda) = -\det(A - \lambda E)$ степени n относительно λ :

$$P(\lambda) = \lambda^n - a_{n-1}\lambda^{n-1} + a_{n-2}\lambda^{n-2} + \dots + (-1)^n a_0.$$

Уравнение

$$\det(A - \lambda E) = 0 \quad (1)$$

называется **характеристическим уравнением оператора** A .

Для того чтобы число λ было **собственным значением оператора** A , необходимо и достаточно, чтобы это число было корнем характеристического уравнения (1) оператора A .

Для **тождественного** оператора ($I(u) = u$, $u \in X$ - оператор I , который каждому элементу u из линейного пространства X ставит в соответствие тот же элемент u . Тожественный оператор действует из X в X) **все** ненулевые векторы пространства являются собственными (с собственным значением, равным единице). Для **нулевого** оператора **все** ненулевые векторы пространства являются собственными (с собственным значением, равным нулю). Наиболее простой вид принимает матрица линейного оператора, имеющего n линейно независимых векторов.

Как находить собственные вектора:

Рассмотрим вопрос о построении собственного вектора, соответствующего известному собственному числу λ_k . Для этого обратимся к уравнению

$$(\alpha - \lambda_k E)u = 0.$$

Здесь E - единичная матрица, а α - матричная форма нашего линейного оператора A . Это уравнение можно понимать как систему линейных уравнений для координат вектора u - собственного вектора, соответствующего собственному числу λ_k . При этом данная система имеет нетривиальное решение, так как ранг этой системы меньше числа неизвестных. Решая эту систему методом Гаусса, можно определить координаты вектора u . Перебирая все значения λ_k , $k=1,2,\dots,n$, находим соответствующие собственные вектора u_k .

Пример. Найдем собственные значения и собственные вектора линейного преобразования, заданного в некотором базисе следующей матрицей:

$$A = \begin{pmatrix} 5 & -7 & 0 \\ -3 & 1 & 0 \\ 12 & 6 & -3 \end{pmatrix}$$

Матрица $A - \lambda E$ имеет в данном случае вид:

$$A - \lambda E = \begin{pmatrix} 5 - \lambda & -7 & 0 \\ -3 & 1 - \lambda & 0 \\ 12 & 6 & -3 - \lambda \end{pmatrix}$$

Вычисляем определитель $\det(A-\lambda E)$ и выписываем уравнение на собственные значения:

$$\det(A-\lambda E) = -(\lambda+3)(\lambda^2-6\lambda-16)=0.$$

Отсюда находим 3 собственных значения: $\lambda_1 = -3$, $\lambda_2 = 8$, $\lambda_3 = -2$. Мы получили 3 собственных значения, все они имеют кратность 1, т.е. это простые собственные числа. Вычислим соответствующие собственные вектора.

1. Рассмотрим $\lambda_1 = -3$. Соответствующее уравнение для собственного вектора $u=(u_1, u_2, u_3)^T$ имеет вид:

$$\begin{pmatrix} 8 & -7 & 0 \\ -3 & 4 & 0 \\ 12 & 6 & 0 \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \\ u_3 \end{pmatrix} = 0, \text{ где справа стоит нулевой 3-вектор. Эта}$$

система уравнений для 3 неизвестных имеет следующее решение:
 $u = (0,0,1)^T$.

2. Рассмотрим $\lambda_2 = 8$. Соответствующее уравнение для собственного вектора $u = (u_1, u_2, u_3)^T$ имеет вид:

$$\begin{pmatrix} -3 & -7 & 0 \\ -3 & -7 & 0 \\ 12 & 6 & 5 \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \\ u_3 \end{pmatrix} = 0, \text{ где справа стоит нулевой 3-вектор. Эта}$$

однородная система уравнений для неизвестных u_1, u_2, u_3 имеет решение: $u = (7,-3,0)^T$.

3. Рассмотрим $\lambda_3 = -2$. Соответствующее уравнение для собственного вектора $u = (u_1, u_2, u_3)^T$ имеет вид:

$$\begin{pmatrix} 7 & -7 & 0 \\ -3 & 3 & 0 \\ 12 & 6 & -1 \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \\ u_3 \end{pmatrix} = 0,$$
 где справа стоит нулевой 3-вектор. Эта однородная система уравнений для неизвестных u_1, u_2, u_3 имеет решение: $u = (1, 1, 0)$

Состав и структура программного обеспечения ИЭС.

Структура программных модулей. Отладка программ и программных комплексов. Верификация и документирования программного обеспечения.

1. Состав и структура программного обеспечения ИЭС

Программное обеспечение ИЭС обычно состоит из нескольких взаимосвязанных компонентов:

1. Ядро (движок вывода)

- Интерпретатор правил (прямой/обратный цепочный вывод)
- Модуль управления сессией консультации
- Механизмы нечёткой логики или вероятностного вывода (если используются)

2. Подсистема управления знаниями

- Хранилище декларативного знания (факты, онтологии, семантические сети)
- Редактор/интерфейс для ввода и редактирования правил и словарей
- Модуль версионирования и контроля целостности базы знаний

3. Модуль прикладного взаимодействия

- Компоненты пользовательского интерфейса (GUI или веб-интерфейс)
- Средства ввода «неструктурированных» данных (текстовые формы, загрузка документов)
- API (REST/SOAP) для интеграции с внешними системами

4. Инфраструктурные сервисы

- СУБД для фактов и логов сеансов (реляционная или NoSQL)
- Службы очередей или брокеры сообщений (при распределённых решениях)
- Компоненты аутентификации/авторизации и аудита безопасности

5. Вспомогательные модули

- Логирование и мониторинг
- Пакеты отчётности и визуализации результатов
- Средства резервного копирования и восстановления базы знаний

2. Структура программных модулей

(Схема: блоки показывают компоненты ядра, подсистемы знания, интерфейсы и сервисы)

1. Module Engine

- InferenceController
- RuleEvaluator
- ConflictResolver

2. Module KnowledgeBase

- FactStore
- RuleRepository
- OntologyManager

3. Module UI

- QueryForm
- ResponseRenderer
- Dashboard

4. Module Integration

- RestApiController
- SoapService

5. Module Infrastructure

- DatabaseConnector
- MessageBrokerClient
- AuthManager

6. Module Utilities

- Logger
- BackupManager
- ReportGenerator

3. Отладка программ и программных комплексов

1. Юнит-тестирование

- Проверка каждого модуля отдельно (механизм оценивания правил, загрузка фактов, парсинг запросов).

2. Интеграционное тестирование

- Сценарии «от начала до конца»: консультация, приём фактов, формирование вывода.

3. Отладка в среде разработки

- Использование точек останова и шагового выполнения в ключевых модулях (движок вывода, модуль правил).

4. Логирование с детальными сообщениями

- Записывать поток фактов, применённые правила, разрешение конфликтов — помогает воспроизвести и локализовать проблему.

5. Тестовые стенды и эмуляторы

- Подмена внешних сервисов (СУБД, брокеры) заглушками для воспроизведения ошибок интеграции.

6. Нагрузочное тестирование

- Проверка производительности при большом числе одновременных сеансов и объёмах базы знаний.

4. Верификация и документирование программного обеспечения

1. Верификация (проверка соответствия ТЗ и алгоритмическим спецификациям)

- **Code Review** — экспертная оценка кода по чек-листу требований.
- **Traceability Matrix** — матрица соответствия требований (из ТЗ, техпроекта) и реализованных функций/модулей.
- **Статический анализ кода** — инструменты проверки стилей, возможных ошибок (SonarQube, линтеры).

2. Валидация (проверка пользовательских сценариев)

- Совместное тестирование с экспертами предметной области: сравнение выводов ИЭС с эталонными решениями.

3. Документирование

- **Техническая документация**
 - Описание архитектуры и модульной структуры
 - Спецификации API и форматов данных
 - Руководство по установке и конфигурации
- **Пользовательские руководства**
 - Инструкция оператора: виды запросов, интерпретация ответов
 - Руководство администратора: резервное копирование, управление пользователями
- **Документация по тестированию**
 - Тест-кейсы, отчёты о дефектах, результаты нагрузочных и интеграционных испытаний
- **Журнал изменений (Changelog)**
 - Описание фиксированных багов, доработок, улучшений версий

1. Собственные векторы и собственные значения оператора.

Рассмотрим произвольную квадратную матрицу,

например, $A = \begin{pmatrix} -1 & -6 \\ 2 & 6 \end{pmatrix}$. И умножим данную матрицу справа на какой-нибудь подходящий столбец. Мне пришёл в голову вектор $\vec{b} = \begin{pmatrix} -1 \\ 1 \end{pmatrix}$:

$$A\vec{b} = \begin{pmatrix} -1 & -6 \\ 2 & 6 \end{pmatrix} \cdot \begin{pmatrix} -1 \\ 1 \end{pmatrix} = \begin{pmatrix} -1 \cdot (-1) - 6 \cdot 1 \\ 2 \cdot (-1) + 6 \cdot 1 \end{pmatrix} = \begin{pmatrix} -5 \\ 4 \end{pmatrix} = \vec{c}$$

Вроде ничего примечательного – умножили матрицу

$A = \begin{pmatrix} -1 & -6 \\ 2 & 6 \end{pmatrix}$ на вектор-столбец $\vec{b} = \begin{pmatrix} -1 \\ 1 \end{pmatrix}$ и получили другой вектор-столбец $\vec{c} = \begin{pmatrix} -5 \\ 4 \end{pmatrix}$. Обычная векторная жизнь. Но в обществе таких векторов существуют особые представители, которые обладают внутренним стержнем и не желают изменять себе в трудные минуты.

Умножим ту же матрицу на $\vec{u} = \begin{pmatrix} 2 \\ -1 \end{pmatrix}$:

$$A\vec{u} = \begin{pmatrix} -1 & -6 \\ 2 & 6 \end{pmatrix} \cdot \begin{pmatrix} 2 \\ -1 \end{pmatrix} = \begin{pmatrix} -1 \cdot 2 - 6 \cdot (-1) \\ 2 \cdot 2 + 6 \cdot (-1) \end{pmatrix} = \begin{pmatrix} 4 \\ -2 \end{pmatrix} = 2 \cdot \begin{pmatrix} 2 \\ -1 \end{pmatrix}$$

На последнем шаге вынесли константу. Что произошло? В результате умножения матрицы A на вектор $\vec{u}$, данный вектор птицей Феникс возродился с числовым коэффициентом $\lambda = 2$:

$$A\vec{u} = \lambda\vec{u}$$

Определение: ненулевой вектор $\vec{u}$, который при умножении на некоторую квадратную матрицу A превращается в самого же себя с числовым коэффициентом λ , называется собственным

вектором матрицы A . Число λ называют собственным значением или собственным числом данной матрицы.

Собственный вектор — понятие в линейной алгебре, определяемое для произвольного линейного оператора как ненулевой вектор, применение к которому оператора даёт коллинеарный вектор — тот же вектор, умноженный на некоторое скалярное значение. Скаляр, на который умножается собственный вектор под действием оператора, называется **собственным числом** (или **собственным значением**) линейного оператора, соответствующим данному собственному вектору. Одним из представлений линейного оператора является квадратная матрица, поэтому собственные векторы и собственные значения часто определяются в контексте использования таких матриц. (из вики, ниже более формально)

Пусть V и W — линейные пространства размерностей n и m соответственно. Будем называть **оператором**, или **преобразованием**, A , действующим из V в W , отображение вида $A: V \rightarrow W$, сопоставляющее каждому элементу $\bar{x} \in V$ некоторый элемент $\bar{y} \in W$. При этом будем использовать обозначение $\bar{y} \in A(\bar{x})$ или $\bar{y} \in A\bar{x}$.

Пусть A — линейный оператор из векторного пространства $L(V, V)$. Число λ_0 называется **собственным значением оператора** A , если существует ненулевой вектор $\bar{x}$ такой, что $A\bar{x} = \lambda_0\bar{x}$. При этом вектор $\bar{x}$ называется **собственным вектором оператора** A , отвечающим собственному значению λ_0 . Множество всех

собственных значений линейного оператора A называется его **спектром**.

Определителем линейного оператора A называется $\det A$, где A – матрица линейного оператора A в любом базисе. Многочлен относительно λ называется **характеристическим многочленом оператора** A . Он не зависит от выбора базиса.

Характеристический многочлен оператора A - многочлен $P(\lambda) = -\det(A - \lambda E)$ степени n относительно λ :

$$P(\lambda) = \lambda^n - a_{n-1}\lambda^{n-1} + a_{n-2}\lambda^{n-2} + \dots + (-1)^n a_0.$$

Уравнение

$$\det(A - \lambda E) = 0 \tag{1}$$

называется **характеристическим уравнением оператора** A .

Для того чтобы число λ было **собственным значением оператора** A , необходимо и достаточно, чтобы это число было корнем характеристического уравнения (1) оператора A .

Для **тождественного** оператора ($I(u) = u$, $u \in X$ - оператор I , который каждому элементу u из линейного пространства X ставит в соответствие тот же элемент u . Тождественный оператор действует из X в X) **все** ненулевые векторы пространства являются собственными (с собственным значением, равным единице). Для **нулевого** оператора **все** ненулевые векторы пространства являются собственными (с собственным значением, равным нулю). Наиболее простой вид принимает матрица линейного оператора, имеющего n линейно независимых векторов.

Как находить собственные вектора:

Рассмотрим вопрос о построении собственного вектора, соответствующего известному собственному числу λ_k . Для этого обратимся к уравнению

$$(\alpha - \lambda_k E)u = 0.$$

Здесь E - единичная матрица, а α - матричная форма нашего линейного оператора A . Это уравнение можно понимать как систему линейных уравнений для координат вектора u - собственного вектора, соответствующего собственному числу λ_k . При этом данная система имеет нетривиальное решение, так как ранг этой системы меньше числа неизвестных. Решая эту систему методом Гаусса, можно определить координаты вектора u . Перебирая все значения λ_k , $k=1,2,\dots,n$, находим соответствующие собственные вектора u_k .

Пример. Найдём собственные значения и собственные вектора линейного преобразования, заданного в некотором базисе следующей матрицей:

$$A = \begin{pmatrix} 5 & -7 & 0 \\ -3 & 1 & 0 \\ 12 & 6 & -3 \end{pmatrix}$$

Матрица $A - \lambda E$ имеет в данном случае вид:

$$A - \lambda E = \begin{pmatrix} 5 - \lambda & -7 & 0 \\ -3 & 1 - \lambda & 0 \\ 12 & 6 & -3 - \lambda \end{pmatrix}$$

Вычисляем определитель $\det(A - \lambda E)$ и выписываем уравнение на собственные значения:

$$\det(A - \lambda E) = -(\lambda + 3)(\lambda^2 - 6\lambda - 16) = 0.$$

Отсюда находим 3 собственных значения: $\lambda_1 = -3$, $\lambda_2 = 8$, $\lambda_3 = -2$. Мы получили 3 собственных значения, все они имеют кратность 1, т.е. это простые собственные числа. Вычислим соответствующие собственные вектора.

1. Рассмотрим $\lambda_1 = -3$. Соответствующее уравнение для собственного вектора $u = (u_1, u_2, u_3)^T$ имеет вид:

$$\begin{pmatrix} 8 & -7 & 0 \\ -3 & 4 & 0 \\ 12 & 6 & 0 \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \\ u_3 \end{pmatrix} = 0, \text{ где справа стоит нулевой 3-вектор. Эта}$$

система уравнений для 3 неизвестных имеет следующее решение:
 $u = (0, 0, 1)^T$.

2. Рассмотрим $\lambda_2 = 8$. Соответствующее уравнение для собственного вектора $u = (u_1, u_2, u_3)^T$ имеет вид:

$$\begin{pmatrix} -3 & -7 & 0 \\ -3 & -7 & 0 \\ 12 & 6 & 5 \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \\ u_3 \end{pmatrix} = 0, \text{ где справа стоит нулевой 3-вектор. Эта}$$

однородная система уравнений для неизвестных u_1, u_2, u_3 имеет решение: $u = (7, -3, 0)^T$.

3. Рассмотрим $\lambda_3 = -2$. Соответствующее уравнение для собственного вектора $u = (u_1, u_2, u_3)^T$ имеет вид:

$$\begin{pmatrix} 7 & -7 & 0 \\ -3 & 3 & 0 \\ 12 & 6 & -1 \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \\ u_3 \end{pmatrix} = 0, \text{ где справа стоит нулевой 3-вектор. Эта}$$

однородная система уравнений для неизвестных u_1, u_2, u_3 имеет решение: $u = (1, 1, 0)$

Методы представления и обработки нечетких знаний в продукционных системах. Достоинства и недостатки правил-продукций как метод представления знаний. Примеры систем продукций.

1. Методы представления и обработки нечётких знаний в продукционных системах

1. Нечёткие правила-продукции

vbnet

Копировать Редактировать

IF X IS A AND Y IS B

THEN Z IS C

где A, B, C — нечёткие множества (лингвистические термы) с определёнными функциями принадлежности.

2. Функции принадлежности

– Треугольные, гауссовы, трапециевидные и т. д. задают степень истинности «X IS A» как число в $[0, 1]$.

3. Механизмы вывода

- **Mamdani-вывод**

1. Вычисление степени срабатывания каждого правила:

$$\alpha = \min(\mu_A(x), \mu_B(y)) \quad \alpha = \min(\mu_A(x), \mu_B(y))$$

2. Применение оператора «AND» и «OR» (минимум/максимум).

3. Агрегация выходных нечётких множеств по всем правилам (максимум).

4. Дефаззификация (обычно метод центра масс) для получения численного результата.

- **TS–(Takagi–Sugeno)–Kang**

выход правила — аналитическая функция (линейная или константная),

взвешенная по степени срабатывания, итог — усреднённая сумма.

4. Управление конфликтами

– Если несколько правил срабатывают одновременно, выбирают по наибольшей степени α или по приоритету (ранжирование правил).

5. Интеграция с классическим цепочным выводом

– Факты в Базе фактов могут носить вид «X IS A to degree α ».
– При проверке LHS правила вместо булева True/False используется число α .

2. Достоинства и недостатки правил-продукций

Достоинства

Простота и прозрачность

Легко читать и объяснять логику «если–то».

Недостатки

Плохо масштабируются

При большом числе правил сложнее поддерживать и искать конфликтующие.

Достоинства

Недостатки

Модульность

Правила можно добавлять/удалять без изменения остальной системы.

Прямая вовлечённость экспертов

Правила формулируются языком предметной области.

Гибкость обработки нечёткости

Могут использоваться и классические, и нечёткие правила.

Универсальность

Применимы для диагностики, управления, планирования.

Бриттлнесс

Система не знает, что делать за границами описанных правил.

Конфликт разрешения

Сложности при выборе одного правила из многих с разными α .

Производительность

При большом числе правил и фактов вывод может тормозить.

Трудоёмкость верификации

Трудно гарантировать полноту и непротиворечивость всех комбинаций.

3. Примеры систем-оболочек продукционного типа

1. CLIPS (NASA)

– Поддерживает правила-продукции, факты, модули, иерархию; имеет отладчик и трассировку с нечёткими расширениями.

2. Jess (Java Expert System Shell)

– JVM-реализация, вместе с Drools может работать с нечёткими правилами через допочки (fuzzyJess).

3. **Drools** (Red Hat)

– Бизнес-правила для корпоративных приложений; плагин Drools Fuzzy позволяет описывать и исполнять нечёткие правила.

4. **OPS5**

– Классическая продукционная система с Rete-алгоритмом, которую расширяли для работы с нечёткостью.

5. **FuzzyCLIPS / FuzzyOPS**

– Специализированные ветки CLIPS и OPS5 с встроенным движком Mamdani и TSK для нечётких правил.

БИЛЕТ 15

1. Квадратичные формы, их невырожденные преобразования. Канонический вид. квадратичной формы. Приведение квадратичной формы к главным осям.

Квадратичные формы, их невырожденные преобразования

.

Однородный многочлен второй степени от n переменных с действительными коэффициентами

$$\sum_{i=1}^n a_{ii}x_i^2 + 2 \sum_{1 \leq i < j \leq n} a_{ij}x_i x_j, a_{ij} \in R, \quad (1)$$

называют **квадратичной формой**.

Для нас квадратичная форма представляет интерес как способ задания некоторой функции векторного аргумента, определенной в n -мерном *линейном пространстве* L . Если в этом пространстве выбрать какой-либо *базис*, то квадратичную форму (1) можно трактовать как функцию, значение которой определено через *координаты* $x_1, x_2, \dots, x_n$ *вектора* x . Эту функцию часто отождествляют с квадратичной формой.

Квадратичную форму (1) можно записать в матричном виде:

$$x^T A x, \quad (2)$$

где $x = (x_1 \ x_2 \ \dots \ x_n)^T$ — столбец, составленный из переменных; $A = (a_{ij})$ — симметрическая матрица порядка n , называемая **матрицей квадратичной формы** (1).

Ранг матрицы A квадратичной формы называют **рангом квадратичной формы**. Если матрица A имеет максимальный ранг, равный числу переменных n , то квадратичную форму называют **невырожденной**, а если $\text{rang}(A) < n$, то ее называют **вырожденной**.

Пример. Квадратичная форма от трех переменных $x_1^2 + 4x_1x_3$

имеет матрицу $\begin{pmatrix} 1 & 0 & 2 \\ 0 & 0 & 0 \\ 2 & 0 & 0 \end{pmatrix}$.

Так как $\text{rang}(A) = 2 < 3$, то эта квадратичная форма является вырожденной. В матричной записи квадратичная форма имеет вид

$$x_1^2 + 4x_1x_3 = (x_1 \ x_2 \ x_3) \begin{pmatrix} 1 & 0 & 2 \\ 0 & 0 & 0 \\ 2 & 0 & 0 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}.$$

Преобразование квадратичных форм

Пусть дана квадратичная форма $x^T A x$, где $x = (x_1 \ x_2 \ \dots \ x_n)^T$. В n -мерном линейном пространстве L с фиксированным базисом **b** она определяет функцию $f(x) = x_b^T A x_b$, заданную через координаты x_b вектора **x** в базисе **b**. Найдем представление этой же функции в некотором другом базисе **e**. Пусть U — матрица перехода от **b** к **e**. Тогда координаты x_b вектора **x** в старом базисе **b** и координаты x_e того же вектора в новом базисе **e** будут связаны соотношением

$$x_b = U x_e. \quad (3)$$

Функция $f(x)$ в новом базисе будет выражаться через новые координаты вектора **x** следующим образом:

$$x_b^T A x_b = (U x_e)^T A (U x_e) = x_e^T (U^T A U) x_e = x_e^T A' x_e$$

Итак, функция f в новом базисе также записывается при помощи квадратичной формы, причем матрица A' этой квадратичной формы связана с матрицей A исходной квадратичной формы соотношением

$$A' = U^T A U. \quad (4)$$

Преобразование матрицы квадратичной формы вызывается заменой переменных (переходом от переменных x_b к переменным x_e) в соответствии с формулой (3).

Замечание. Замену переменных вида (3) с произвольной матрицей U называют **линейной**. Изменение базиса в линейном пространстве приводит к линейной замене переменных с невырожденной матрицей.

Пример. Квадратичную форму

$$f(x_1, x_2, x_3) = 7x_1^2 + 5x_2^2 + 2x_3^2 - 8x_1x_2 + 2x_1x_3 - 6x_2x_3$$

преобразуем к новым переменным y_1, y_2, y_3 , где

$$\begin{cases} x_1 = y_1 + y_2 + y_3 \\ x_2 = y_1 + 2y_2 + 2y_3 \\ x_3 = y_1 + y_2 + 2y_3 \end{cases}$$

Эта замена переменных в матричной записи имеет вид $x = Uy$, где

$$U = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 2 & 2 \\ 1 & 1 & 2 \end{pmatrix}.$$

Согласно (4) имеем

$$A' = U^T A U = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 2 & 2 \end{pmatrix} \begin{pmatrix} 7 & -4 & 1 \\ -4 & 5 & -3 \\ 1 & -3 & 2 \end{pmatrix} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 2 & 2 \\ 1 & 1 & 2 \end{pmatrix} =$$

$$\begin{pmatrix} 2 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & -1 \end{pmatrix},$$

и квадратичная форма принимает вид

$$f(y_1, y_2, y_3) = 2y_1^2 + 3y_2^2 - y_3^2,$$

т.е. все коэффициенты при попарных произведениях переменных обнуляются и остаются слагаемые с квадратами переменных.

Канонический вид. квадратичной формы.

Квадратичную форму

$$\alpha_1 x_1^2 + \dots + \alpha_n x_n^2, \alpha_i \in R, i = \overline{1, n}, \quad (5)$$

не имеющую попарных произведений переменных, называют **квадратичной формой канонического вида**. Переменные $x_1, \dots, x_n$, в которых квадратичная форма имеет канонический вид, называют **каноническими переменными**.

Один из методов преобразования (или, как говорят, приведения) квадратичной формы к каноническому виду путем замены переменных состоит в последовательном выделении полных квадратов. Такой метод называют **методом Лагранжа**. Проиллюстрируем этот метод на простом примере.

Пример. Рассмотрим квадратичную форму $x_1^2 - 4x_1x_2$ от двух переменных. Для преобразования ее к каноническому виду выделим полный квадрат по x_1 . Для этого соберем все слагаемые, содержащие x_1 , и дополним до полного квадрата:

$$x_1^2 - 4x_1x_2 = x_1^2 - 4x_1x_2 + 4x_2^2 - 4x_2^2 = (x_1 - 2x_2)^2 - 4x_2^2$$

Введя новые переменные $z_1 = x_1 - 2x_2$, $z_2 = 2x_2$, получим квадратичную форму канонического вида: $z_1^2 - z_2^2$. #

http://mathmod.bmstu.ru/Docs/Eduwork/la_fnp/LA-06.pdf (стр. 4-5 (69-70) - применение метода Лагранжа в общем виде)

Приведение квадратичной формы к главным осям (то же приведение к каноническому виду, только через ортогональное преобразование)

Сформулируем задачу приведения квадратичной формы к главным осям: требуется найти ортогональную замену переменных $x = Sy$ ($S^{-1} = S^T$), приводящую квадратичную форму (1) к каноническому виду (5).

Матрица A квадратичной формы при переходе к новому базису изменяется по формуле $A' = U^T A U$, где U — матрица перехода. Если рассматривается евклидово пространство, а старый и новый базисы выбраны ортонормированными, то матрица перехода U является ортогональной, и мы имеем дело с **ортогональным преобразованием квадратичной формы**, т.е. преобразованием $A' = U^T A U$, в котором матрица U ортогональна.

Теорема 1. При ортогональном преобразовании квадратичной формы характеристическое уравнение ее матрицы не изменяется.

Теорема 2. Любую квадратичную форму ортогональным преобразованием можно привести к каноническому виду.

Диагональными элементами матрицы A' квадратичной формы канонического вида, получающейся в результате ортогонального преобразования, являются *собственные значения матрицы A квадратичной формы*. Из этого следует, что мы можем записать матрицу A' канонического вида, не находя соответствующего ортогонального преобразования.

Находя ортогональное преобразование, приводящее квадратичную форму к каноническому виду, мы фактически ищем базис из собственных векторов соответствующего самосопряженного оператора. Действительно, если квадратичная форма и самосопряженный оператор имели в исходном ортонормированном базисе одинаковую матрицу, то и в новом ортонормированном базисе их матрицы будут совпадать.

Мы предполагаем, что квадратичная форма представляет собой запись функции, заданной в евклидовом пространстве, через *координаты вектора* в некотором ортонормированном базисе. На самом деле такая интерпретация носит чисто вспомогательный характер, помогающий смотреть на процесс с геометрической

точки зрения, но она никак не используется в самом алгоритме построения ортогонального преобразования. Достаточно лишь записать матрицу квадратичной формы и применить к этой матрице процедуру приведения к диагональному виду.

Пример. Квадратичную форму $f(x, y) = x_1^2 - 4x_1x_2$ от двух переменных мы приводили к каноническому виду методом Лагранжа. Теперь попробуем привести ее к каноническому виду ортогональным преобразованием.

Матрица нашей квадратичной формы имеет вид

$$A = \begin{pmatrix} 1 & -2 \\ -2 & 0 \end{pmatrix}.$$

Найдем *характеристическое уравнение этой матрицы*:

$$\begin{vmatrix} 1 - \lambda & -2 \\ -2 & 0 - \lambda \end{vmatrix} = (1 - \lambda)(-\lambda) - 4 = \lambda^2 - \lambda - 4 = 0.$$

Вычисляем корни характеристического уравнения, они же собственные значения матрицы A:

$$\lambda_{1,2} = \frac{1 \pm \sqrt{17}}{2}.$$

Теперь можем записать канонический вид нашей квадратичной формы:

$$\frac{1 + \sqrt{17}}{2} y_1^2 + \frac{1 - \sqrt{17}}{2} y_2^2.$$

Нейронные сети. Общие понятия. Элементы нейронных сетей.

Архитектура нейронных сетей. Обучение нейронных сетей.

**Методы построения правил классификации: Деревья решений
- общие принципы работы.**

1. Нейронные сети

1.1 Общие понятия

Нейронная сеть (НС) — это вычислительная модель, вдохновлённая биологическими нейронными сетями мозга. Представляет собой **связное «плетение»** простых вычислительных элементов (искусственных нейронов), объединённых в слои и обучающихся на примерах.

- **Цель:** автоматически выявлять сложные нелинейные зависимости в данных.
- **Применение:** распознавание образов, речь, прогнозирование, обработка сигналов.

1.2 Элементы нейронных сетей

1. Искусственный нейрон

- Входы $x_1, x_2, \dots, x_n$ и веса $w_1, w_2, \dots, w_n$
- Сумматор: $u = \sum_i w_i x_i + b$ (где b — смещение)
- Активационная функция $\phi(u)$:
 - Сигмоида, ReLU, tanh, Softmax (для выходного слоя в задачах классификации)

2. Весовые связи

- Определяют «силу» влияния каждого входного сигнала
- Обучаются в процессе тренировки сети

3. Слои сети

- **Входной слой** — передаёт исходные признаки
- **Скрытые слои** — обрабатывают информацию, формируя промежуточные представления
- **Выходной слой** — выдаёт окончательный результат (классы, значения)

1.3 Архитектура нейронных сетей

1. Feedforward (прямого распространения)

- Сигнал идёт от входа к выходу без обратных связей.
- Примеры: многослойный перцептрон (MLP).

2. Convolutional Neural Network (CNN)

- Сверточные слои для обработки изображений и сигналов.
- Свертка → пулинг → свертка → ... → полносвязные слои.

3. Recurrent Neural Network (RNN)

- Обратные связи между слоями, сохраняют состояние.
- LSTM и GRU — модификации для долгосрочных зависимостей.

4. Гибридные и специализированные

- Autoencoder, GAN, Transformer, Graph Neural Networks и др.

1.4 Обучение нейронных сетей

1. Целевая функция (loss)

- MSE (регрессия), кросс-энтропия (классификация), специальные метрики.

2. Оптимизация

- Градиентный спуск (SGD, Adam, RMSProp):

$$w \leftarrow w - \eta \frac{\partial L}{\partial w} \quad w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

где η — скорость обучения.

3. Процесс обучения

- **Инициализация** весов (случайная, Xavier, He)
- **Эпохи и батчи**: одно полное прохождение через все данные = эпоха
- **Forward pass**: вычисляем выход и loss

- **Backward pass (backpropagation)**: считаем градиенты и обновляем веса

4. Регуляризация

– Dropout, L1/L2-нормы, ранняя остановка (early stopping) — для борьбы с переобучением.

2. Методы построения правил классификации: деревья решений

2.1 Основная идея

Дерево решений — это модель, которая рекурсивно **разбивает** пространство признаков по простым правилам (условиям), формируя древовидную структуру:

- **Внутренние узлы**: тест по одному признаку (например, «возраст ≥ 30 ?»)
- **Ветви**: результаты теста (да/нет или несколько интервалов)
- **Листья**: итоговые решения (классы или средние значения)

2.2 Индукция дерева (построение)

1. Выбор лучшего признака

– Критерии качества разбиения:

- **Information Gain (IG)** на основе энтропии

$$IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

$$IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

- **Gini Impurity**

$$Gini(S) = 1 - \sum_k p_k^2$$

$$Gini(S) = 1 - \sum_k p_k^2$$

- **Gain Ratio, Chi-squared** и др.

2. Рекурсивное разбиение

- Делим выборку по порогу в лучшем признаке
- Повторяем для каждой полученной части до достижения критерия остановки (глубина, число объектов, качество)

3. Критерии остановки

- Максимальная глубина дерева
- Минимальное число объектов в узле
- Минимальный прирост качества при разбиении

2.3 Обработка и оптимизация

1. Прореживание (Pruning)

- **Докорневое (pre-pruning)**: останавливаем разбиение заранее
- **Послеродшино (post-pruning)**: строим полное дерево, затем удаляем «слабые» ветви по статистическим тестам или кросс-валидации

2. Преимущества деревьев

- Прозрачность и наглядность
- Легко интерпретировать правила
- Работают с разными типами данных без масштабирования

3. Недостатки

- Склонность к переобучению (если не регулировать глубину)
- Нестабильность: небольшие изменения в данных могут сильно изменить структуру
- Ограниченная «гладкость» границ (оси разбиения перпендикулярны признакам)

1. Дифференциальные уравнения первого порядка, разрешенные относительно производной. Уравнения с разделяющимися переменными, однородные уравнения, уравнения в полных дифференциалах.

В общем случае дифференциальное уравнение первого порядка имеет вид $F(x, y, y') = 0$, в котором x — независимая переменная, $y(x)$ — неизвестная функция, y' — её производная, наличие которой обязательно. Дифференциальным уравнением первого порядка, разрешенным относительно производной, называется уравнение

$$y' = f(x, y) \quad (2.1.)$$

Для уравнений вида (2.1) справедлива следующая теорема.

ТЕОРЕМА 2.1. Пусть в уравнении

$$y' = f(x, y) \quad (2.1)$$

функция $f(x, y)$ непрерывна в некоторой области D плоскости XOY , а ее частная производная $f'_y(x, y)$ в этой области ограничена. Тогда для любой точки $M_0(x_0, y_0) \in D$ существует единственное решение $y = \varphi(x)$ уравнения (2.1), определенное в некотором интервале (a, b) , содержащем точку x_0 , и удовлетворяющее условию $y_0 = \varphi(x_0)$.

Числа x_0, y_0 называются **начальными значениями** для решения $y = \varphi(x)$, а условие $y_0 = \varphi(x_0)$ – **начальным условием** решения. Задача нахождения решения $y = \varphi(x)$ дифференциального уравнения (2.1), удовлетворяющего начальному условию $y_0 = \varphi(x_0)$, называется **задачей Коши**. Поэтому теорему 2.1 называют **теоремой существования и единственности решения задачи Коши**.

Геометрически задание начального условия означает, что на плоскости XOY задается точка (x_0, y_0) , через которую проходит интегральная кривая. Согласно теореме 2.1, через каждую точку области D проходит, и притом единственная, интегральная кривая уравнения (2.1). Закрепляя значение x_0 и изменяя в некоторых пределах значение y_0 (так, чтобы точка (x_0, y_0) принадлежала области D), для каждого числа y_0 будем получать свое решение. В результате вся область D будет покрыта интегральными кривыми, которые нигде между собой не пересекаются.

Таким образом, теорема 2.1 подтверждает высказанное нами ранее предположение о том, что дифференциальное уравнение имеет множество решений и говорит о том, что эта совокупность решений зависит от произвольной постоянной.

ОПРЕДЕЛЕНИЕ. **Общим решением** дифференциального уравнения

$$y' = f(x, y) \quad (2.1)$$

в области D существования и единственности решения задачи Коши называется функция $y = \varphi(x, C)$, зависящая от x и одной произвольной постоянной C , которая удовлетворяет следующим двум условиям:

- 1) при любом допустимом значении постоянной C она удовлетворяет уравнению (2.1);
- 2) каково бы ни было начальное условие $y_0 = y(x_0)$ (где $(x_0, y_0) \in D$), можно найти единственное значение $C = C_0$ такое, что функция $y = \varphi(x, C_0)$ удовлетворяет данному начальному условию.

Уравнение $\Phi(x, y, C) = 0$, задающее общее решение в неявном виде, называется **общим интегралом** уравнения.

Решение (интеграл) дифференциального уравнения, получаемое из общего решения (общего интеграла) при конкретном

значении постоянной C , называется частным решением (частным интегралом) уравнения.

Уравнения с разделяющимися переменными.

Дифференциальное уравнение вида

$$M_1(x) \cdot N_1(y)dx + M_2(x) \cdot N_2(y)dy = 0, \quad (2.5)$$

т.е. уравнение, в котором коэффициенты при дифференциалах распадаются на множители, зависящие только от x и только от y , называется **уравнением с разделяющимися переменными**. (Функции $M_1(x)$, $N_1(y)$, $M_2(x)$, $N_2(y)$ предполагаются непрерывными).

Уравнение (2.5) может быть приведено к уравнению с разделенными переменными путем деления обеих его частей на выражение $N_1(y) \cdot M_2(x)$. Действительно, в этом случае получим

$$\begin{aligned} \frac{M_1(x) \cdot N_1(y)}{N_1(y) \cdot M_2(x)} dx + \frac{M_2(x) \cdot N_2(y)}{N_1(y) \cdot M_2(x)} dy &= 0 \\ \Rightarrow \frac{M_1(x)}{M_2(x)} dx + \frac{N_2(y)}{N_1(y)} dy &= 0 \quad - \quad \text{уравнение с} \\ &\quad \text{разделенными} \\ &\quad \text{переменными.} \end{aligned}$$

Замечания. 1) Деление на $N_1(y) \cdot M_2(x)$ может привести к потере решений, обращающих в нуль произведение $N_1(y) \cdot M_2(x)$. Поэтому, чтобы получить полное решение, необходимо рассмотреть корни уравнений $N_1(y) = 0$, $M_2(x) = 0$.

2) Уравнение с разделяющимися переменными может быть задано в виде, разрешенном относительно y' . Тогда оно имеет вид

$$y' = f(x) \cdot \varphi(y).$$

Действительно, разделив уравнение (2.5) на $M_2(x) \cdot N_2(y)dx$, получим

$$\frac{M_1(x) \cdot N_1(y)}{M_2(x) \cdot N_2(y)} + \frac{dy}{dx} = 0$$

или
$$y' = -f(x) \cdot \varphi(y),$$

где $f(x) = \frac{M_1(x)}{M_2(x)}, \quad \varphi(y) = \frac{N_1(y)}{N_2(y)}.$

Простыми словами: чтобы решить такое уравнение, необходимо разделить переменные, то есть, привести уравнение к такой форме, чтобы при дифференциале dx стояла

функция, зависящая лишь от x , а при дифференциале dy — функция, зависящая от y .

Однородные уравнения

Функция $f(x, y)$ называется *однородной порядка k* , если выполняется тождество $f(tx, ty) = t^k f(x, y)$.

При $k = 0$ имеем однородную функцию нулевого порядка.

Уравнение первого порядка $y' = f(x, y)$ называется однородным относительно x и y , если функция $f(x, y)$ есть однородная функция нулевого измерения.

Для того, чтобы определить, является ли дифференциальное уравнение первого порядка однородным, нужно ввести постоянную t и заменить y на ty и x на tx : $y \rightarrow ty$, $x \rightarrow tx$. Если t сократится, то это **однородное дифференциальное уравнение**. Производная y' при таком преобразовании не меняется.

$$y' = \frac{dy}{dx} \rightarrow \frac{d ty}{d tx} = \frac{t dy}{t dx} = \frac{dy}{dx}.$$

Уравнения в полных дифференциалах

Уравнение $M(x, y)dx + N(x, y)dy = 0$ (2.22)

называется уравнением в полных дифференциалах, если его левая часть является полным дифференциалом некоторой функции $u(x, y)$ т.е. если

$$M(x, y)dx + N(x, y)dy = du(x, y).$$

Очевидно, что общий интеграл уравнения в полных дифференциалах будет иметь вид $u(x, y) = C$.

Таким образом, задача интегрирования дифференциального уравнения в полных дифференциалах фактически сводится к

задаче нахождения функции двух переменных по ее полному дифференциалу.

Критерий, когда выражение $M(x,y)dx + N(x,y)dy$ представляет собой дифференциал некоторой функции $u(x,y)$ и один из возможных способов ее нахождения, дает следующая теорема.

ТЕОРЕМА 2.2. Пусть функции $M(x,y)$, $N(x,y)$ определены и непрерывны в области D плоскости xOy и имеют в ней непрерывные частные производные $\frac{\partial M}{\partial y}$ и $\frac{\partial N}{\partial x}$. Для того чтобы выражение

$$M(x,y)dx + N(x,y)dy$$

представляло собой полный дифференциал некоторой функции $u(x,y)$, необходимо и достаточно, чтобы во всех точках области D выполнялось условие

$$\frac{\partial M}{\partial y} = \frac{\partial N}{\partial x}.$$

При этом функция $u(x,y)$ может быть найдена по одной из следующих формул

$$u(x,y) = \int_{x_0}^x M(x,y)dx + \int_{y_0}^y N(x_0,y)dy \quad (2.23)$$

$$u(x,y) = \int_{x_0}^x M(x,y_0)dx + \int_{y_0}^y N(x,y)dy, \quad (2.24)$$

где (x_0, y_0) – любая точка области D .

Замечание. В формулах (2.23) и (2.24) интегрирование производится по одной из переменных, в то время как вторая считается константой.

Подготовка исходных данных для анализа: формирования и сбор данных, построение моделей - анализ, трансформация данных.

1. Формирование и сбор данных

1. Определение источников данных

- **Внутренние:** операционные БД, CRM, ERP, журналы приложений.
- **Внешние:** публичные API, открытые датасеты, веб-скрапинг, IoT-датчики.

2. Сбор данных (экстракция)

- **ETL-процессы:** Extract → Transfer → Load.
- **Инструменты:** SQL-запросы, Python (pandas, requests), Apache NiFi, Talend.

3. Интеграция и согласование

- **Сшивание таблиц** (Join, Merge) по ключам (ID, время, гео).
- **Разрешение конфликтов:** единицы измерения, форматы дат, дубликаты.

2. Предобработка и очистка данных

1. Обработка пропущенных значений

- Удаление строк/столбцов, если пропусков много.
- Замена средним/медианой/модой или с помощью моделей (KNN-импутация).

2. Удаление и корректировка выбросов

- Статистические методы (Z-score, IQR).

- Визуальный контроль (ящик с усами, scatter plot) и ручная проверка.

3. Нормализация и масштабирование признаков

- **Min–Max scaling** (в $[0,1]$)
- **Standard scaling** (центрирование и деление на σ)
- **Robust scaling** (на основе медианы и IQR)

4. Кодирование категориальных переменных

- **One-Hot Encoding** для небольшого числа категорий.
- **Label Encoding** или **Target Encoding** для высококардинальных.

5. Форматирование дат и временных меток

- Приведение к единому часовому поясу.
- Выделение признаков: год, месяц, день недели, час, сезонность.

3. Трансформация и обогащение данных

1. Агрегация и группировка

- Сводные таблицы (pivot) по ключевым измерениям.
- Скользящие окна, кумулятивные суммы, скользящие средние.

2. Создание новых признаков (feature engineering)

- Комбинации старых признаков: отношение, разница, произведение.
- Логарифмирование, корень, степень (для стабилизации разброса).
- Texto- и граф-фичи: длина текста, частотные словари, centrality measures.

3. Выявление и удаление мультиколлинеарности

- Корреляционные матрицы, VIF (Variance Inflation Factor).
- Удаление или объединение сильно коррелированных.

4. Снижение размерности

- **PCA** (анализ главных компонент) для числовых данных.

- **t-SNE, UMAP** для визуализации высокоразмерных.
-

4. Построение моделей и анализ

1. Exploratory Data Analysis (EDA)

- **Статистические сводки:** mean, median, std, quartiles.
- **Визуализация:** гистограммы, ящики с усами, scatter plot, heatmap корреляций.

2. Выбор целевой переменной и признаков

- Уяснить тип задачи: классификация, регрессия, кластеризация.
- Отбор важных признаков: **Feature Selection** (RFE, Lasso).

3. Разбиение на тренировочный и тестовый наборы

- Обычно 70/30 или 80/20.
- При временных данных — **Time-Series Split** (скользящий контрольный период).

4. Построение и оценка базовых моделей

- Линейная регрессия, решающие деревья, kNN, наивный Байес.
- Оценка метрик: MAE/MSE/RMSE для регрессии; accuracy, precision/recall, ROC-AUC для классификации.

5. Улучшение и подбор гиперпараметров

- **Grid Search, Random Search, Bayesian Optimization.**
 - Кросс-валидация (K-Fold, ShuffleSplit).
-

5. Итоги подготовки данных

- **Качество модели** напрямую зависит от качества подготовки данных.
- **Автоматизация ETL-пайплайна** (Airflow, Luigi) гарантирует воспроизводимость.

- **Документирование шагов** (notebooks, скрипты, readme) — залог прозрачности и поддержки.

БИЛЕТ 17

1. Линейное однородное дифференциальное уравнение n-го порядка.

Опр: Линейным дифференциальным уравнением n-го порядка называется уравнение, линейное относительно неизвестной функции и ее производных и, следовательно, имеющее вид (1)

$$a_0(x)y^{(n)} + a_1(x)y^{(n-1)} + \dots + a_{n-1}(x)y' + a_n(x)y = f(x) \quad \text{где } a_0(x), a_1(x), \dots, a_n(x) - \text{коэф. уравнения}$$

Опр: Если правая часть $f(x) \equiv 0$, то уравнение (1) называется линейным однородным, так как оно однородно относительно неизвестной функции y и ее производных.

$$a_0(x)y^{(n)} + a_1(x)y^{(n-1)} + \dots + a_n(x)y = 0 \quad (2)$$

Если $a_0(x) \neq 0 \forall x \in [a, b]$, то разделив обе части (1) на эту функцию: $y^{(n)} + p_1(x)y^{(n-1)} + \dots + p_n(x)y = F(x)$ (3)

Опр: Ур. (3) наз. линейным n-порядка в нормальной форме (коэф. при старшей произв. = 0)

$$y^{(n)} = f(x) - \sum_{k=1}^n p_k(x)y^{(n-k)} = F(x, y, y' \dots y^{(n-k)}) \quad (4)$$

Если все коэф. (3) фун. $p_i(x)$ ($i=0 \dots n$) и $f(x)$ непрерывны на $x \in [a, b]$, то ур (3) в любой окрестности начальных значений

$$y(x_0) = y_0, y'(x_0) = y_0', \dots, y^{(n-1)}(x_0) = y_0^{(n-1)} \quad \text{где } x_0 \in [a, b] \text{ удовлетворяет условиям теоремы существования единственности решений.}$$

Условия выполнения:

1) $F(x, y, y' \dots y^{(n-1)})$ явл. непр. относительно всех аргументов т.к. она линейная комбинация непрерывных функций.

2) $\frac{\partial F}{\partial y^{(k)}} = -p_{n-k}(x)$ Т.К. коэф.рi непрерывны на $[a, b]$, то по теореме Вейерштрасса, они на этом же отрезке ограничены.

Основные св-ва частных решений лин. однор.ур.

Речь пойдет об ур (2).

1)Если фун. $y(x)$ явл. решением лин. однор. ур(2), то новая функция $Cy(x)$ явл. решением этого же уравнения.

Док-во.

$$\begin{aligned} & a_0(x)(Cy)^{(n)} + a_1(x)(Cy)^{(n-1)} + \dots \\ & + a_n(x)Cy \equiv C[a_0(x)y^{(n)} + \\ & + a_1(x)y^{(n-1)} + \dots + a_n(x)y] \equiv 0 \end{aligned}$$

Подставим $Cy(x)$ в левую часть (2).

2)Если $y_1(x), y_2(x)$ явл. решением одного и того же лин. однор. ур(2), то их сумма $y_1(x)+y_2(x)$, то же решение его же.

Док-во.

Подставим новую фун. в левую часть ур(2).

$$\begin{aligned} & a_0(x)(y_1 + y_2)^{(n)} + a_1(x)(y_1 + y_2)^{(n-1)} + \\ & + \dots + a_n(x)(y_1 + y_2) \equiv [a_0(x)y_1^{(n)} + \\ & + a_1(x)y_1^{(n-1)} + \dots + a_n(x)y_1] + \\ & + [a_0(x)y_2^{(n)} + a_1(x)y_2^{(n-1)} + \dots + a_n(x)y_2] \equiv 0 \end{aligned}$$

3)Если фун. $m: y_1(x), y_2(x) \dots y_n(x)$ явл. решением одного и того же

лин. однор. ур. (2), то их линейная комбинация $\sum_{i=1}^m C_i y_i(x)$ также явл. решением (2)

Док-во.

Очевидно.

4) Если (2) с действ. коэффициентами $a_i (i=0...n)$ имеет комплексно значное решение $y(x)=u(x)+iv(x)$, то $u(x), v(x)$ – вещ-ые по отдельности явл. решениями того же ур-я.

Док-во.

$$a_0(x)(u+iv)^{(n)} + a_1(x)(u+iv)^{(n-1)} + \dots + a_n(x)(u+iv) \equiv 0$$

Подставим y в Ур-е.

Воспользуемся свойством линейности производной любого порядка.

$$[a_0(x)u^{(n)} + a_1(x)u^{(n-1)} + \dots + a_n(x)u] + i[a_0(x)v^{(n)} + a_1(x)v^{(n-1)} + \dots + a_n(x)v] \equiv 0$$

[]-вещ.функция.

Основное св-во комплексно значных функции.

Комплексно значные фун $\equiv 0$. Тогда, когда $\equiv 0$ ее вещественная и мнимая часть.

$$a_0(x)u^{(n)} + a_1(x)u^{(n-1)} + \dots + a_n(x)u \equiv 0$$

$$a_0(x)v^{(n)} + a_1(x)v^{(n-1)} + \dots + a_n(x)v \equiv 0$$

u, v -решения (2). $y_1(x) \dots y_n(x)$ наз лин. зав. на $x \in [a, b]$, если $\exists a_1 \dots a_n, a_i \neq 0$:

$$\forall x \in [a, b]$$

справедливо $a_1 y_1(x) + \dots + a_n y_n(x) \equiv 0$. (5)

Опр (лин. нез.): $y_1(x) \dots y_n(x)$ наз. лин. незав. на $[a, b]$, для них (5) выполнено когда $a_1 = \dots = a_n = 0$.

Пр: рассм $1, x, x^2 \dots x^n$. Докажем, что явл. лин. нез. на любом отрезке.

Док-во

Пусть $x \in [a, b]$. Фун лин. зав., то по опр $\exists a_1 \dots a_n, a_i \neq 0$:

$$a_0 + a_1x + \dots + a_nx^n \equiv 0$$

Т.к среди коэф. есть один $\neq 0$, то левая часть есть многочлен не выше n , то из алгебры известна т-ма: Мн-ны степени n имеют не более n различных корней, то мн-н может быть $=0$ не более чем в n различных точках, то тождество невозможно, то противоречие.

$$\exists a_i, a_i \neq 0$$

Опр(лин.зав.): $p_1(x)e^{k_1x} + \dots + p_l(x)e^{k_lx} \equiv 0$

Алгоритмы поиска в структурах данных. Поиск в массиве. Поиск в графе.

Поиск — одна из важнейших операций в работе с данными. В зависимости от структуры данных (массив, дерево, граф и т.д.) применяются разные алгоритмы поиска, чтобы найти нужный элемент наиболее эффективно.

Поиск в массиве — это задача нахождения элемента по его значению или по индексу. Существует несколько основных методов поиска в массивах:

- **Линейный поиск** — самый простой способ. Элементы массива проверяются один за другим, пока не будет найден нужный элемент или не закончится массив.
Время работы линейного поиска — $O(n)$, где n — количество элементов в массиве.
- **Бинарный поиск** — более быстрый метод, применяемый только к **отсортированным** массивам.
Массив делится пополам, и в зависимости от значения искомого элемента выбирается левая или правая половина, затем процесс повторяется.
Время работы бинарного поиска — $O(\log_2 n)$.
Это значительно быстрее линейного поиска на больших массивах.

Пример:

Линейный поиск подходит для небольших массивов или когда данные не отсортированы. Бинарный поиск предпочтительнее для больших отсортированных массивов.

Поиск в графе — это задача нахождения вершины или пути между вершинами в структуре данных типа граф. Графы могут быть ориентированными или неориентированными, взвешенными или невзвешенными.

Существуют два базовых алгоритма поиска в графах:

- **Поиск в ширину (BFS — Breadth-First Search):**
 - Изучаются все вершины на текущем уровне перед переходом на следующий.
 - Используется очередь (FIFO) для хранения вершин.
 - Подходит для поиска кратчайшего пути в невзвешенных графах.
 - Время работы — $O(V+E)$, где V — количество вершин, E — количество рёбер.
- **Поиск в глубину (DFS — Depth-First Search):**
 - Углубляемся как можно дальше по одному пути, прежде чем вернуться назад.
 - Используется стек (неявно через рекурсию или явно).
 - Хорош для задач поиска всех путей, обходов всех вершин и компонент связности.
 - Время работы также — $O(V+E)$.

Пример:

BFS нужен, если задача — найти кратчайший путь от одной вершины к другой.

DFS удобен для обхода всех вершин, например, в задачах проверки связности графа.

БИЛЕТ 18

1. Линейное однородное дифференциальное уравнение с постоянными коэффициентами.

Рассмотрим линейное однородное дифференциальное уравнение n -ого порядка с постоянными коэффициентами:

$$a_0 y^{(n)} + a_1 y^{(n-1)} + \dots + a_n y = 0 .$$

Теорема. Если $y_1, \dots, y_n$ являются линейно независимыми решениями линейного однородного дифференциального уравнения n -ого порядка с постоянными коэффициентами, то общее решение этого уравнения имеет вид $y = C_1 y_1 + \dots + C_n y_n$, где $C_1, \dots, C_n$ – произвольные постоянные.

Для нахождения линейно независимых решений линейного однородного дифференциального уравнения n -ого порядка необходимо выполнить следующие действия:

1. Составляем характеристическое уравнение:

$$a_0 k^n + a_1 k^{n-1} + \dots + a_{n-1} k + a_n = 0;$$

2. Находим корни этого характеристического уравнения: $k_1, k_2, \dots, k_n$;

3. По характеру корней выписываем частные линейно независимые решения, руководствуясь тем, что:

а) каждому действительному однократному корню соответствует частное решение e^{kx} ;

б) каждой паре комплексных сопряженных однократных корней $k^{(1)} = \alpha + i\beta$, $k^{(2)} = \alpha - i\beta$ соответствуют два частных действительных решения: $e^{\alpha x} \cos \beta x$ и $e^{\alpha x} \sin \beta x$;

в) каждому действительному корню k кратности r соответствуют r линейно независимых частных решений: $e^{kx}, xe^{kx}, \dots, x^{r-1}e^{kx}$;

г) каждой паре комплексных сопряженных корней $k^{(1)} = \alpha + i\beta, k^{(2)} = \alpha - i\beta$ кратности μ соответствуют 2μ действительных частных решений:

$$e^{\alpha x} \cos(\beta x), x \cdot e^{\alpha x} \cos(\beta x), \dots, x^{\mu-1} \cdot e^{\alpha x} \cos(\beta x),$$

$$e^{\alpha x} \sin(\beta x), x \cdot e^{\alpha x} \sin(\beta x), \dots, x^{\mu-1} \cdot e^{\alpha x} \sin(\beta x).$$

Этих частных решений будет ровно столько, какова степень характеристического уравнения. Самостоятельно основываясь на определении линейной независимости для функций, предлагается убедиться, что полученные частные решения будут линейно независимыми.

4. Получив n линейно независимых частных решений $y_1, \dots, y_n$, строим общее решение исходного линейного однородного дифференциального уравнения с постоянными коэффициентами: $y_{oo} = C_1 y_1 + C_2 y_2 + \dots + C_n y_n$.

Пример:

Пример 6. $y^{IV} - y = 0$.

Решение. Составляем характеристическое уравнение:

$$k^4 - 1 = 0 \Rightarrow (k^2 + 1)(k^2 - 1) = 0 \Rightarrow k_1 = i; k_2 = -i; k_3 = 1; k_4 = -1.$$

Записываем линейно независимые частные решения

$$y_1 = \cos x, y_2 = \sin x, y_3 = e^x, y_4 = e^{-x}.$$

Получаем общее решение: $y_{oo} = C_1 \cos x + C_2 \sin x + C_3 e^x + C_4 e^{-x}$.

Информационный поиск и индексирование. Принципы организации полнотекстового поиска. Создание и использование индексов (прямые и инвертированные). Лемматизация, стемминг.

Информационный поиск — это процесс поиска нужной информации в больших массивах данных, таких как базы документов, сайты, текстовые архивы. Основная задача информационного поиска — быстро найти документы, релевантные запросу пользователя.

Когда документов становится много, простой перебор всех текстов становится неэффективным. Для ускорения поиска используется **индексирование** — это процесс подготовки специальных структур данных, которые позволяют находить информацию быстрее.

Принципы организации полнотекстового поиска:

- Документы разбиваются на отдельные слова (токены).
- Слова нормализуются (например, приводятся к нижнему регистру, удаляются знаки препинания).
- Создаётся индекс, в котором указано, в каких документах и в каких местах встречаются слова.
- При поступлении поискового запроса система быстро обращается к индексу, а не перечитывает весь текст.

Типы индексов:

- **Прямой индекс:**
Для каждого документа хранится список всех слов, которые в нём встречаются.
Прямые индексы удобны для обработки документов, например, для обновления текста.
- **Инвертированный индекс:**
Основной механизм для быстрого поиска. Для каждого слова хранится список документов, в которых это слово встречается.

Например, если в базе 10000 документов, то инвертированный индекс по слову "машина" быстро выдаст только те документы, где есть это слово.

Этот принцип лежит в основе большинства поисковых систем, включая Google.

Лемматизация и стемминг — это техники обработки слов для улучшения качества поиска.

- **Лемматизация** — приведение слова к нормальной (словарной) форме — лемме.
Например, слова "бегу", "бежал", "бежать" приводятся к общей форме "бежать".
Лемматизация учитывает грамматические правила языка и обычно требует более сложной обработки.
- **Стемминг** — это более простое и грубое приближение: обрезание окончаний у слов для получения основы (стема).
Например, "running", "runner", "runs" можно привести к "run".
Стемминг работает быстрее, но иногда может давать ошибки (например, разные слова могут иметь одинаковый стем).

Зачем нужны лемматизация и стемминг?

Они помогают находить документы даже в случае, если в тексте слово используется в другой форме, чем в запросе. Это повышает полноту поиска и делает результаты более полезными для пользователя.

2. Линейное неоднородное дифференциальное уравнение n -ого порядка. Метод вариации постоянных нахождения решения.

Линейное неоднородное дифференциальное уравнение n -го порядка имеет вид:

$$L[y] = y^{(n)} + a_{n-1}(x)y^{(n-1)} + \dots + a_1(x)y' + a_0(x)y = f(x),$$

где $y^{(n)}$ — n -я производная функции $y(x)$, $a_i(x)$ — заданные функции от x , и $f(x)$ — известная функция.

Общая структура решения

Общее решение неоднородного линейного дифференциального уравнения состоит из двух частей:

1. **Общее решение соответствующего однородного уравнения:**
 $L[y] = 0$.
2. **Частное решение неоднородного уравнения.**

Метод вариации постоянных позволяет найти частное решение неоднородного уравнения.

Метод вариации постоянных

Предположим, что общее решение однородного уравнения $L[y] = 0$ известно и имеет вид:

$$y_h = c_1 y_1(x) + c_2 y_2(x) + \dots + c_n y_n(x),$$

где $y_1(x), y_2(x), \dots, y_n(x)$ — линейно независимые решения однородного уравнения, а $c_1, c_2, \dots, c_n$ — произвольные постоянные.

Метод вариации постоянных предполагает, что в частном решении неоднородного уравнения постоянные c_i могут быть функциями от x . То есть частное решение можно искать в виде:

$$y_p = u_1(x)y_1(x) + u_2(x)y_2(x) + \dots + u_n(x)y_n(x),$$

где $u_i(x)$ — неизвестные функции, которые нужно найти.

Шаги метода вариации постоянных

1. **Записать общее решение однородного уравнения:**

$$y_h = c_1 y_1(x) + c_2 y_2(x) + \dots + c_n y_n(x).$$

2. **Подставить частное решение в виде:**

$$y_p = u_1(x)y_1(x) + u_2(x)y_2(x) + \dots + u_n(x)y_n(x).$$

3. **Найти производные частного решения.** Например, для первого порядка производная будет:

$$y_p' = u_1'y_1 + u_1y_1' + u_2'y_2 + u_2y_2' + \dots + u_n'y_n + u_ny_n'.$$

4. **Ввести условие на функции $u_i(x)$:**

$$u_1'y_1 + u_2'y_2 + \dots + u_n'y_n = 0.$$

Это условие позволяет упростить выражение для производных, и остаются только члены, содержащие производные y_i :

$$y_p' = u_1y_1' + u_2y_2' + \dots + u_ny_n'.$$

5. **Подставить полученные производные в исходное неоднородное уравнение** и решить систему уравнений для функций $u_i(x)$.

6. **Найти $u_i(x)$, проинтегрировав полученные выражения.**

Пример

Рассмотрим конкретный пример неоднородного уравнения второго порядка:

$$y'' - y = e^x.$$

1. **Общее решение однородного уравнения:**

$$y_h = c_1e^x + c_2e^{-x}.$$

2. **Частное решение неоднородного уравнения:**

$$y_p = u_1(x)e^x + u_2(x)e^{-x}.$$

3. **Производные частного решения:**

$$y_p' = u_1'e^x + u_1e^x + u_2'e^{-x} - u_2e^{-x},$$

$$y_p'' = u_1''e^x + 2u_1'e^x + u_1e^x + u_2''e^{-x} - 2u_2'e^{-x} + u_2e^{-x}.$$

4. **Условие на функции $u_i(x)$:**

$$u_1'e^x + u_2'e^{-x} = 0.$$

5. **Подстановка производных в исходное уравнение и упрощение:**

$$y_p'' - y_p = (u_1e^x + u_2e^{-x})' = e^x.$$

6. **Решение системы уравнений:**

$$\begin{aligned}u_1' e^x + u_2' e^{-x} &= 0, \\u_1' e^x - u_2' e^{-x} &= e^x.\end{aligned}$$

Из первого уравнения:

$$u_2' = -u_1' e^{2x}.$$

Подставим во второе уравнение:

$$u_1' e^x - (-u_1' e^{2x}) e^{-x} = e^x,$$

$$u_1' e^x + u_1' e^x = e^x, 2u_1' e^x = e^x,$$

$$u_1' = \frac{1}{2},$$

$$u_1 = \frac{x}{2} + C_1.$$

Теперь найдем u_2 :

$$u_2' = -\frac{1}{2} e^{2x},$$

$$u_2 = -\frac{1}{4} e^{2x} + C_2.$$

Общее решение

С учетом полученных функций $u_1(x)$ и $u_2(x)$, частное решение будет:

$$y_p = \left(\frac{x}{2} + C_1\right) e^x + \left(-\frac{1}{4} e^{2x} + C_2\right) e^{-x}.$$

Однако, C_1 и C_2 — произвольные постоянные, которые можно включить в общее решение. Поэтому частное решение примет вид:

$$y_p = \frac{x}{2}e^x - \frac{1}{4}e^x.$$

Общее решение неоднородного уравнения:

$$y = c_1 e^x + c_2 e^{-x} + \frac{x}{2}e^x - \frac{1}{4}e^x.$$

Итак, метод вариации постоянных предоставляет систематический подход к нахождению частного решения линейного неоднородного дифференциального уравнения n -го порядка, что позволяет составить общее решение этого уравнения.

1. Понятие системы. Классификация систем

Система – единство взаимосвязанных, взаимовлияющих элементов, которые расположены в определенной закономерности в пространстве и времени и совместно функционируют для достижения определенной цели.

Системы классифицируются по различным признакам:

1. По иерархии уровня сложности:

- уровень статической организации (структура – состояние чего-либо),
- уровень простой динамической структуры с заранее запланированными обязательными движениями,
- уровень термостата (цикл обратной связи),
- уровень одноклеточного живого организма (клетки),
- уровень генетически общественных организаций (растения),
- уровень животных – системы, которые обладают нервной системой, мозгом и имеют целенаправленные поведения,
- уровень индивидуального человека – система обладает самосознанием, языком как средством общения, способностью воспроизводить, создавать и передавать сложные символы,

- уровень социальных организаций,
- уровень трансцендентальной системы – разнообразные живые и неживые структуры во вселенной, которые в настоящий момент исследованы недостаточно и вряд ли будут изучены полностью.

2. По восприятию влияния факторов внешней среды:

- **открытые** – которые обмениваются со своим окружением информацией, движениями, материалами, энергией, веществом. Открытые системы имеют входы и выходы. *Входы* – потоки информации, энергии и вещества, идущие из окружения в систему, а так же факторы, влияющие на нее. *Выходы* – потоки информации, энергии и вещества, идущие из системы во внешнюю среду. Пример открытой системы: фирма, техника, человек;
- **замкнутые** – внутреннее состояние которых не зависит от внешней среды. В них отсутствуют входы и выходы. Замкнутые системы в чистом виде в природе не существуют, но при определенных условиях, когда влияние внешних факторов незначительно и им можно пренебречь, некоторые системы можно назвать замкнутыми.

3. По содержанию:

- **технические** – набор решений в таких системах ограничен, а последствия решений обычно predeterminedены и зафиксированы в инструкциях по эксплуатации;
- **биологические** – флора и фауна планеты. Набор решений ограничен, а последствия решений бывают непредсказуемы;
- **социальные** – разнообразные группы людей. Набор решений более широк, а последствия непредсказуемы.

4. По предсказуемости поведения:

- **детерминированные** – функционируют по заранее спланированным правилам с заранее установленным результатом;
- **стохастические** – поведение которых носит вероятностный характер.

5. По происхождению:

- **естественные** – сформировались в итоге естественных природных процессов,
- **искусственные** – результат человеческой деятельности.

Один и тот же объект может иметь множество разных систем. Если рассматривать производственное предприятие как совокупность машин и прочего, то предприятие – технологическая система; если как коллектив людей, имеющих общую систему ценностей, моделей поведения, то предприятие – социальная система.

Предприятие можно рассматривать как экономическую систему – при этом различают отношения к средствам производства и результатам совместной деятельности.

БИЛЕТ 20

1. Линейное неоднородное дифференциальное уравнение n -ого порядка с постоянными коэффициентами, подбор частных решений при специальном виде правой части.

2. Неоднородное уравнение

Линейным неоднородным дифференциальным уравнением n -го порядка с постоянными коэффициентами называется уравнение вида

$$y^{(n)} + a_1 y^{(n-1)} + \dots + a_{n-1} y' + a_n y = f(x), \quad (4)$$

где $a_1, a_2, \dots, a_n$ – постоянные, $f(x)$ – непрерывная на интервале (a, b) функция.

Общее решение уравнения (4) представляет собой сумму общего решения соответствующего однородного уравнения (1) и некоторого частного решения неоднородного уравнения (4).

В общем случае интегрирование уравнения (4) может быть осуществлено методом вариации произвольных постоянных.

Пример 8. Найти общее решение уравнения

$$y'' - 3y' + 2y = e^{3x}.$$

2.

Решение. Фундаментальная система решений уравнения: e^x, e^{2x} . Общее решение уравнения будем искать в виде

$$y = C_1(x)e^x + C_2(x)e^{2x}.$$

Для нахождения функций $C_1'(x)$ и $C_2'(x)$ составим систему

$$\begin{cases} C_1'(x)e^x + C_2'(x)e^{2x} = 0, \\ C_1'(x)e^x + 2C_2'(x)e^{2x} = e^{3x}, \end{cases}$$

из которой найдем $C_1'(x) = -e^{2x}$, $C_2'(x) = e^x$. Таким образом,

$$C_1(x) = -\frac{1}{2}e^{2x} + \tilde{C}_1, \quad C_2(x) = e^x + \tilde{C}_2$$

и общее решение данного уравнения

$$y = \frac{1}{2}e^{3x} + \tilde{C}_1 e^x + \tilde{C}_2 e^{2x}.$$

□

В некоторых случаях бывает легко подобрать частное решение по виду правой части $f(x)$ уравнения (4), используя метод неопределенных коэффициентов (метод Эйлера).

Метод неопределенных коэффициентов. Вид частного решения $\tilde{y}$ зависит от структуры правой части уравнения, т.е. функции $f(x)$. В частности, если правая часть уравнения

$$f(x) = e^{ax}(P_n(x) \cos bx + Q_m(x) \sin bx), \quad (5)$$

где $P_n(x)$, $Q_m(x)$ — полиномы степеней n , m , то решение $\tilde{y}$ следует искать в виде

$$\tilde{y} = x^r e^{ax}(M_s(x) \cos bx + N_s(x) \sin bx),$$

где r — кратность корня $a + ib$ характеристического уравнения (если $a \pm ib$ не является корнем характеристического



9

уравнения, то $r = 0$), а $M_s(x)$, $N_s(x)$ — полиномы степени $s = \max\{m, n\}$. Коэффициенты полиномов $M_s(x)$, $N_s(x)$ находятся из тождества, получаемого после подстановки $\tilde{y}$ в такое же уравнение.

Пример 9. Найти общее решение уравнения

$$y'' - 3y' + 2y = e^{3x}.$$

Решение. Составим характеристическое уравнение для соответствующего однородного уравнения $\lambda^2 - 3\lambda + 2 = 0$, оно имеет корни $\lambda_1 = 1$, $\lambda_2 = 2$. Общее решение данного уравнения тогда будет иметь вид

$$y^0 = C_1 e^x + C_2 e^{2x}.$$

Правая часть уравнения $f(x) = e^{3x}$. Для принятых в формуле (5) обозначений имеем $a = 3$, $b = 0$, $P_n(x) = 1$, $Q_m(x) = 0$, $a + bi = 3$ – не корень характеристического уравнения. Тогда частное решение неоднородного уравнения будем искать в виде

$$\tilde{y} = A e^{3x}.$$

Подставив $\tilde{y}$ в данное уравнение, найдем $A = 1/2$. Таким образом, общее решение неоднородного уравнения

$$y = y^0 + \tilde{y} = C_1 e^x + C_2 e^{2x} + \frac{1}{2} e^{3x}.$$

□

2. Основные положения теории моделирования: понятие модели, последовательность разработки математических моделей,

Модель — это объект-заместитель объекта-оригинала, обеспечивающий изучение некоторых свойств оригинала. Замещение одного объекта другим с целью получения информации о важнейших свойствах объекта-оригинала с помощью объекта-модели называется **моделированием**. Под **математическим моделированием** будем понимать процесс установления соответствия данному реальному объекту некоторого математического объекта, называемого **математической моделью**, и исследование этой модели, позволяющее получать характеристики рассматриваемого реального объекта. Вид

математической модели зависит как от природы реального объекта, так и задач исследования объекта и требуемой достоверности и точности решения этой задачи.

Процесс построения любой математической модели можно представить последовательностью этапов:

Обследование объекта моделирования и формулировка технического задания на разработку модели (содержательная постановка задачи);

Концептуальная и математическая постановка задачи;

Качественный анализ и проверка корректности модели;

Выбор и обоснование выбора методов решения задачи;

Поиск решения;

Разработка алгоритма решения и исследование его свойств, реализация алгоритма в виде программ;

Проверка адекватности модели;

Практическое использование построенной модели.

Обследование объекта моделирования

Математические модели, требуют для своего построения значительных интеллектуальных, финансовых и временных затрат.

Необходимость в новой модели может появиться в связи с проведением научных исследований, особенно — на стыке различных областей знания. После принятия решения о необходимости построения новой математической модели заказчик ищет исполнителя своего заказа. В качестве исполнителя, как правило, может выступать рабочая группа, включающая специалистов разного профиля: прикладных математиков, специалистов, хорошо знающих особенности объекта

моделирования, программистов. Если решение о создании модели принято и рабочая группа сформирована, то приступают к этапу обследования объекта моделирования. Основной целью данного этапа является подготовка содержательной постановки задачи моделирования. Перечень сформулированных в содержательной (словесной) форме основных вопросов об объекте моделирования, интересующих заказчика, составляет содержательную постановку задачи моделирования.

Подготовка списка вопросов, на которые должна ответить новая модель, зачастую является самостоятельной проблемой, требующей для своего решения специалистов со специфическими знаниями и способностями. Они должны не только хорошо разбираться в предметной области моделирования, знать возможности современной вычислительной математики и техники, но и уметь общаться с людьми, «разговорить» практиков, хорошо «чувствующих» объект моделирования, нюансы его поведения. К таким специалистам например относят системных аналитиков, системных инженеров, специалистов по исследованию операций.

На основании анализа всей собранной информации постановщик задачи должен сформулировать такие требования к будущей модели, которые, с одной стороны, удовлетворяли бы заказчика, а с другой — позволяли бы реализовать модель в заданные сроки и в рамках выделенных материальных средств. Системные аналитики (или операционисты) должны обладать способностью из большого объема слабо формализованной разнообразной информации об объекте моделирования, из различных нечетко высказанных и сформулированных пожеланий и требований заказчика к будущей модели выделить то главное, что может быть действительно реализовано.

На основе собранной информации об объекте моделирования системный аналитик (инженер, операционист) совместно с заказчиком формулируют содержательную постановку задачи моделирования, которая, как правило, не бывает окончательной и может уточняться и конкретизироваться в процессе разработки модели. Однако, все последующие уточнения и изменения содержательной постановки должны носить частный, не принципиальный характер.

Весь собранный в результате обследования материал о накопленных к данному моменту знаниях об объекте, содержательная постановка задачи моделирования, дополнительные требования к реализации модели и представлению результатов оформляются в виде технического задания на проектирование и разработку модели. Техническое задание является итоговым документом, заканчивающим этап обследования. Чем более полную информацию удастся собрать об объекте на этапе обследования, тем более четко можно выполнить содержательную постановку задачи, более полно учесть накопленный опыт и знания, избежать многих сложностей на последующих этапах разработки модели.

Концептуальная постановка задачи моделирования

В отличие от содержательной концептуальная постановка задачи моделирования, как правило, формулируется членами рабочей группы без привлечения представителей заказчика, на основании разработанного на предыдущем этапе технического задания, с использованием имеющихся знаний об объекте моделирования и требований к будущей модели. Анализ и совместное обсуждение членами рабочей группы всей имеющейся информации об объекте моделирования позволяет сформировать содержательную модель

объекта, являющуюся синтезом когнитивных моделей, сложившихся у каждого из членов рабочей группы.

Концептуальная постановка задачи моделирования — это сформулированный в терминах конкретных дисциплин перечень основных вопросов, интересующих заказчика, а также совокупность гипотез относительно свойств и поведения объекта моделирования.

Математическая постановка задачи

Законченная концептуальная постановка позволяет сформулировать математическую постановку задачи моделирования, включающую совокупность различных математических соотношений, описывающих поведение и свойства объекта моделирования.

Математическая постановка задачи моделирования — это совокупность математических соотношений, описывающих поведение и свойства объекта моделирования.

Математическая модель является корректной, если для нее осуществлен и получен положительный результат всех контрольных проверок: размерности, порядков, характера зависимостей, экстремальных ситуаций, граничных условий, предметного смысла и математической замкнутости.

Математическая постановка задачи еще более абстрактна, чем концептуальная, так как сводит исходную задачу к чисто математической, методы решения которой достаточно хорошо разработаны.

Выбор и обоснование выбора решения задачи

Все методы решения задач, составляющих «ядро» математических моделей, можно подразделить на аналитические и алгоритмические.

Следует отметить, что при использовании аналитических решений для получения результатов «в числах» также часто требуется разработка соответствующих алгоритмов, реализуемых на вычислительной технике.

Однако исходное решение при этом представляет собой аналитическое выражение (или их совокупность). Решения же, основанные на алгоритмических методах, принципиально не сводимы к точным аналитическим решениям рассматриваемой задачи.

Выбор того или иного метода исследования в значительной степени зависит от квалификации и опыта членов рабочей группы. Аналитические методы более удобны для последующего анализа результатов, но применимы лишь для относительно простых моделей. В случае, если математическая задача допускает аналитическое решение, последнее, без сомнения, предпочтительнее численного.

Алгоритмические методы сводятся к некоторому алгоритму, реализующему вычислительный эксперимент с использованием вычислительной техники. Точность моделирования в подобном эксперименте существенно зависит от выбранного метода и его параметров. Алгоритмические методы, как правило, более трудоемки в реализации, требуют от членов рабочей группы хорошего знания методов вычислительной математики, обширной библиотеки специального программного обеспечения и мощной вычислительной техники.

Численные методы применимы лишь для корректных математических задач, что существенно ограничивает использование их в математическом моделировании. Общим для всех численных методов является сведение математической задачи к конечномерной. Это чаще всего достигается

дискретизацией исходной задачи, то есть переходом от функции непрерывного аргумента к функциям дискретного аргумента. Применение любого численного метода неминуемо приводит к погрешности результатов решения задачи. **Выделяют три основных составляющих возникающей погрешности при численном решении исходной задачи: неустраняемая погрешность, связанная с неточным заданием исходных данных (начальные и граничные условия, коэффициенты и правые части уравнений); погрешность метода, связанная с переходом к дискретному аналогу исходной задачи; ошибка округления, связанная с конечной разрядностью чисел, представляемых в вычислительной машине.**

Естественным требованием для конкретного вычислительного алгоритма является согласованность в порядках величин перечисленных трех видов погрешностей.

Численный, или приближенный, метод реализуется всегда в виде вычислительного алгоритма. Поэтому все требования, предъявляемые к алгоритму, применимы и к вычислительному алгоритму. Прежде всего, алгоритм должен быть реализуем — обеспечивать решение задачи за допустимое машинное время. Важной характеристикой алгоритма является его точность, то есть возможность получения решения исходной задачи с заданной точностью за конечное число действий.

Время работы алгоритма зависит от числа действий, необходимых для достижения заданной точности. Для любой математической задачи, как правило, можно предложить несколько алгоритмов, позволяющих получить решение с заданной точностью, но за разное число действий. Алгоритмы, включающие меньшее число действий для достижения одинаковой точности, называют более экономичными, или более эффективными.

В процессе работы вычислительного алгоритма на каждом акте вычислений возникает некоторая погрешность. При этом от действия к действию она может возрастать или не возрастать (а в некоторых случаях даже уменьшаться). Если погрешность в процессе вычислений неограниченно возрастает, то такой алгоритм называется неустойчивым, или расходящимся. В противном случае алгоритм называется устойчивым, или сходящимся.

Реализация математической модели в виде компьютерной программы

При создании различных программных комплексов, используемых для решения разнообразных исследовательских, проектно-конструкторских и управленческих задач, в настоящее время, основой, как правило, служат математические модели. В связи с этим возникает необходимость реализации модели в виде компьютерной программы. **Успешное решение данной задачи возможно лишь при уверенном владении современными алгоритмическими языками и технологиями программирования, знаний возможностей вычислительной техники, имеющегося программного обеспечения, особенностей реализации методов вычислительной математики.**

Процесс создания программного обеспечения можно разбить на несколько этапов:

составление технического задания на разработку программного обеспечения;

проектирование структуры программного комплекса;

кодирование алгоритма;

тестирование и отладка;

сопровождение и эксплуатация.

Техническое задание на разработку программного обеспечения оформляют в виде спецификации. На этапе проектирования формируется общая структура программного комплекса. Вся программа разбивается на программные модули. Для каждого программного модуля формулируются требования по реализуемым функциям и разрабатывается алгоритм, выполняющий эти функции. Определяется схема взаимодействия программных модулей, называемая схемой потоков данных программного комплекса. Разрабатывается план и задаются исходные данные для тестирования отдельных модулей и программного комплекса в целом.

Большинство программ, реализующих математические модели, состоят из трех основных частей:

препроцессора (подготовка и проверка исходных данных модели);

процессора (решение задачи, реализация вычислительного эксперимента);

постпроцессора (отображение полученных результатов).

Проверка адекватности модели

Адекватность математической модели - степень соответствия результатов, полученных по разработанной модели, данным эксперимента или тестовой задачи.

Проверка адекватности модели преследует две цели:

убедиться в справедливости совокупности гипотез, сформулированных на этапах концептуальной и математической постановок. Переходить к проверке гипотез следует лишь после проверки использованных методов решения, комплексной

отладки и устранения всех ошибок и конфликтов, связанных с программным обеспечением;

установить, что точность полученных результатов соответствует точности, оговоренной в техническом задании.

Проверка разработанной математической модели выполняется путем сравнения с имеющимися экспериментальными данными о реальном объекте или с результатами других, созданных ранее и хорошо себя зарекомендовавших моделей. В первом случае говорят о проверке путем сравнения с экспериментом, во втором — о сравнении с результатами решения тестовой задачи.

Решение вопроса о точности моделирования зависит от требований, предъявляемых к модели, и ее назначения. При этом должна учитываться точность получения экспериментальных результатов или особенности постановок тестовых задач. В моделях, предназначенных для выполнения оценочных и прикидочных расчетов, удовлетворительной считается точность 10-15%. В моделях, используемых в управляющих и контролирующих системах, требуемая точность может быть 1-2% и даже более.

При возникновении проблем, связанных с адекватностью модели, ее корректировку требуется начинать с последовательного анализа всех возможных причин, приведших к расхождению результатов моделирования и результатов эксперимента. В первую очередь требуется исследовать модель и оценить степень ее адекватности при различных значениях варьируемых параметров (начальных и граничных условиях, параметров, характеризующих свойства объектов моделирования). Если модель неадекватна в интересующей исследователя области параметров, то можно попытаться уточнить значения констант и исходных параметров модели. Если же и в этом случае нет положительных результатов,

то единственной возможностью улучшения модели остается изменение принятой системы гипотез.

Практическое использование модели и анализ результатов моделирования

Дескриптивные модели, предназначены для описания исследуемых параметров некоторого явления или процесса, а также для изучения закономерностей изменения этих параметров. Эти модели могут использоваться для изучения свойств и особенностей поведения исследуемого объекта при различных сочетаниях исходных данных и разных режимах; при построении оптимизационных моделей и моделей-имитаторов сложных систем.

Модели, разрабатываемые для исследовательских целей, как правило, не доводятся до уровня программных комплексов, предназначенных для передачи сторонним пользователям. Время их существования чаще всего ограничено временем выполнения исследовательских работ по соответствующему направлению. Эти модели отличает поисковый характер, применение новых вычислительных процедур и алгоритмов, неразвитый программный интерфейс.

Модели и построенные на их основе программные комплексы, предназначенные для последующей передачи сторонним пользователям или коммерческого распространения, имеют развитый дружественный интерфейс, мощные пре- и постпроцессоры. Данные модели обычно строятся на апробированных и хорошо себя зарекомендовавших постановках и вычислительных процедурах. Однако следует помнить, что такие модели предназначены только для решения четко оговоренного класса задач.

Независимо от области применения созданной модели группа разработчиков обязана провести качественный и количественный анализ результатов моделирования.

Работая с моделью, разработчики становятся специалистами в области, связанной с объектом моделирования. Они достаточно хорошо представляют свойства объекта, могут предсказать и объяснить его поведение.

Поэтому всесторонний анализ результатов моделирования позволяет:

выполнить модификацию рассматриваемого объекта, найти его оптимальные характеристики или, по крайней мере, лучшим образом учесть его поведение и свойства;

обозначить область применения модели;

проверить обоснованность гипотез, принятых на этапе математической постановки, оценить возможность упрощения модели с целью повышения ее эффективности при сохранении требуемой точности;

показать, в каком направлении следует развивать модель в дальнейшем.

1. Условная вероятность. Формула полной вероятности, формула Байеса.

Условная вероятность (англ. *conditional probability*): Пусть задано вероятностное пространство (Ω, P) . Условной вероятностью события A при условии, что произошло событие B , называется число $P(A|B) = P(A \cap B) / P(B)$, где $A, B \subset \Omega$.

Замечания

Если $P(B) = 0$, то изложенное определение условной вероятности неприменимо.

Прямо из определения очевидно следует, что вероятность произведения двух событий равна:

$$P(A \cap B) = P(A|B)P(B).$$

Если события A и B независимые, то $P(A|B) = P(A \cap B) / P(B) = P(A)$

Пример

Пусть имеется 12 шариков, из которых 5 — чёрные, а 7 — белые. Пронумеруем чёрные шарики числами от 1 до 5, а белые — от 6 до 12. Случайным образом из мешка достаётся шарик. Требуется посчитать вероятность того, что шарик чёрный, если известно, что он имеет чётный номер.

Обозначим за A событие "достали чёрный шар" и за B событие "достали шар с чётным номером". Тогда $P(B) = 1/2$, так как ровно половина шариков имеют чётный номер, а $P(A \cap B) = 2/12 = 1/6$, так как только два шарика из двенадцати являются чёрными и имеют чётным номер одновременно.

Тогда по определению вероятность случайно вытащенного шарика с чётным номером оказаться чёрным равна $P(A|B)=P(A \cap B)/P(B)=1/3$

Формула полной вероятности, формула Байеса.

Если событие A может произойти только при выполнении одного из событий $B_1, B_2, \dots, B_n$, которые образуют **полную группу несовместных событий**, то вероятность события A вычисляется по формуле $P(A) = P(B_1)P(A|B_1) + P(B_2)P(A|B_2) + \dots + P(B_n)P(A|B_n)$.

Эта формула называется **формулой полной вероятности**.

Вновь рассмотрим полную группу несовместных событий $B_1, B_2, \dots, B_n$, вероятности появления которых $P(B_1), P(B_2), \dots, P(B_n)$. Событие A может произойти только вместе с каким-либо из событий $B_1, B_2, \dots, B_n$, которые будем называть **гипотезами**. Тогда по формуле полной вероятности

$$P(A) = P(B_1)P(A|B_1) + P(B_2)P(A|B_2) + \dots + P(B_n)P(A|B_n)$$

Если событие A произошло, то это может изменить вероятности гипотез $P(B_1), P(B_2), \dots, P(B_n)$.

По теореме умножения вероятностей

$$P(AB_1) = P(B_1)P(A|B_1) = P(A)P(B|A),$$

откуда

$$P(B_1|A) = \frac{P(B_1)P(A|B_1)}{P(A)}.$$

Аналогично, для остальных гипотез

$$P(B_i|A) = \frac{P(B_i)P(A|B_i)}{P(A)}, i = 1, \dots, n.$$

Полученная формула называется **формулой Байеса (формулой Бейеса)**. Вероятности гипотез $P(B_i|A)$ называются **апостериорными вероятностями**, тогда как $P(B_i)$ - **априорными вероятностями**.

Пример. В магазин поступила новая продукция с трех предприятий. Процентный состав этой продукции следующий: 20% - продукция первого предприятия, 30% - продукция второго предприятия, 50% - продукция третьего предприятия; далее, 10% продукции первого предприятия высшего сорта, на втором предприятии - 5% и на третьем - 20% продукции высшего сорта. Найти вероятность того, что случайно купленная новая продукция окажется высшего сорта.

Решение. Обозначим через B событие, заключающееся в том, что будет куплена продукция высшего сорта, через A_1, A_2, A_3 обозначим события, заключающиеся в покупке продукции, принадлежащей соответственно первому, второму и третьему предприятиям.

Можно применить формулу полной вероятности, причем в наших обозначениях:

$$P(A_1) = 0,2 \quad P(B|A_1) = 0,1$$

$$P(A_2) = 0,3 \quad P(B|A_2) = 0,05$$

$$P(A_3) = 0,5 \quad P(B|A_3) = 0,2$$

Подставляя эти значения в формулу полной вероятности, получим искомую вероятность:

$$P(B) = 0,2 \cdot 0,1 + 0,3 \cdot 0,05 + 0,5 \cdot 0,2 = 0,135.$$

1. Основные положения и области применения имитационного моделирования.

Имитационная модель - модель реального явления или процесса, построенная с помощью компьютерных технологий и позволяющая производить многократные имитационные эксперименты с целью получения новых знаний об исследуемом объекте. **Компьютерное** моделирование имеет дело с абстрактными (знаковыми, математическими) моделями.

Имитация есть подражание чему-либо.

следовательно, **имитационным** нужно называть моделирование, сохраняющее внешнее сходство с исходным процессом.

В *имитационной модели* изменения процессов и данных ассоциируются с событиями. "Проигрывание" модели заключается в последовательном переходе от одного события к другому.

Обычно *имитационные модели* строятся в случае, когда другие математические модели оказываются слишком сложными.

ИМ может быть разработана или с помощью программной реализации аналитической модели поведения объекта (*функциональная модель* класса "вход - выход"), или с помощью системы взаимосвязанных программных блоков, каждый из которых моделирует либо часть функций объекта (модель из функциональных блоков), либо подобъект (модель объектных блоков).

Статистическим считается вид моделирования, при котором воспроизводятся аналоги массовых явлений с последующей обработкой результатов наблюдений методами математической статистики.

Имитационный процесс - проведение расчетного эксперимента с помощью *имитационной модели* реального объекта.

Сценарий имитационного эксперимента - это совокупность входных (исходных) параметров модели, заданных условий проведения имитационного исследования. Получая выходные данные на каждом сценарии, исследователь имеет возможность сравнить их с данными других сценариев и принять решение о необходимости внесения коррекции в те или иные условия проведения имитационного эксперимента - иначе говоря, сформировать условия нового сценария.

Имитационные модели разделяют на дискретные и непрерывные.

Дискретные имитационные модели представляют реальный мир и моделируемые в нем процессы как дискретные, т.е. проявляющие свои функции и свойства в определенные моменты времени. Можно предполагать, что это происходит через равные интервалы времени, но в некоторых моделях могут существовать и асинхронные объекты, которые проявляют себя в случайные моменты времени. Асинхронные *дискретные модели* могут быть реализованы в классе синхронных, но с более сложными функциями управления имитационным экспериментом. Интервал времени, через который происходит анализ изменения имитационной ситуации, называется **шагом моделирования**. Шаг моделирования определяет точность, сходимость и время имитационного эксперимента.

Непрерывные модели предполагают развитие имитационной ситуации в модели как в непрерывной (аналоговой) среде.

Переменная времени в *имитационной модели* может быть либо непрерывной, либо дискретной в зависимости от того, могут ли дискретные изменения зависимых переменных происходить в любые моменты времени или только в определенные моменты.

При "непрерывной" имитации зависимые переменные модели изменяются непрерывно в течение имитационного времени. Непрерывный процесс может имитироваться либо непрерывной моделью, либо дискретной в зависимости от того, будут ли значения независимых переменных доступны в любой точке или только в определенные моменты времени.

В "комбинированной" имитации зависимые переменные модели могут изменяться дискретно, непрерывно или непрерывно с наложенными дискретными скачками. Наиболее важный аспект комбинированной имитации заключается в возможности взаимодействия между дискретно и непрерывно изменяющимися переменными. Компонентами таких моделей могут быть не только материальные потоки, но и люди, оборудование, заказы, состояния системы (которые представляются с помощью непрерывно изменяющихся зависимых переменных).

Имитационные модели также подразделяются на статические и динамические.

Статические модели имеют стабильную, не изменяющуюся во времени ("жесткую") модель функций и блоков исследуемого реального объекта. Обычно такие модели строятся на базе аналитической разработки, которая реализуется в виде программы. Такие модели близки по своим свойствам к расчетным моделям, а режим имитации здесь используется только для "прогона" расчетов в заданных интервалах входных параметров. Как правило, модели данного класса применяют для подтверждения гипотетических догадок о предполагаемых и аналитически выраженных свойствах исследуемого явления или объекта.

Динамические модели, как правило, имеют более сложную схему реализации и проведения имитационного эксперимента. Модели

данного класса предполагают, что "ядро" схемы моделирования, определяющее основные связи и свойства объект, меняется в процессе имитационного эксперимента и модифицируется в зависимости от промежуточных данных.

Здесь возможны два варианта развития: динамика изменения схемы имитационного эксперимента известна и динамика изменения модели является целью имитационного исследования.

Очевидно, что последний вариант *имитационной модели* является наиболее сложным случаем в практике исследований с помощью *имитационных моделей*, поскольку, по существу, представляет собой компьютерный инструментарий исследования нового, неизвестного свойства реального объекта. Примером таких сложных *имитационных моделей* с динамической структурой являются модели, исследующие ситуации катастроф, "переломных" экономико-политических ситуаций и т.п.

Области применения имитационных моделей

Имитационное моделирование является универсальным и особенно успешно может применяться в вероятностных процессах и в исследовании переходных режимов экономических явлений. Лица, ответственные за принятие решений в области создания экономических систем, могут оценить их эффективность одним из трех следующих способов.

Во-первых, есть возможность проводить управляемые эксперименты с экономической системой фирмы, отрасли или страны.

Во-вторых, если есть данные о развитии экономической системы за некоторый период времени в прошлом, то можно провести мысленный эксперимент на этих данных.

В-третьих, можно построить *математическую модель системы*, связывающую входные (независимые) переменные с выходными (зависимыми) переменными, а также с экономической стратегией. Если есть основания для того, чтобы считать разработанную математическую модель *адекватной* рассматриваемой экономической системе, то с помощью модели можно производить расчеты или машинные эксперименты. По результатам этих экспериментов можно выработать рекомендации по повышению эффективности существующей или проектируемой экономической системы.

К недостаткам систем *имитационного моделирования* можно отнести следующие:

сложность описания;

сложность проведения экспериментов;

слабость средств моделирования конфликтов за общие ресурсы;

необходимость перебора большого количества сравниваемых вариантов с целью отыскания оптимального.

Существующие *имитационные модели* можно условно разделить на три группы.

К *первой группе* можно отнести модели, которые достаточно точно отражают какую-либо одну сторону определенного экономического процесса, происходящего в системе сравнительно малого масштаба. **С точки зрения математики они представляют собой весьма простые соотношения между двумя-пятью переменными. Обычно это алгебраические уравнения не выше 2-й или 3-й степени, в крайнем случае, система алгебраических уравнений.**

Ко *второй группе* можно отнести модели, которые описывают реальные процессы, протекающие в экономических системах малого и среднего масштаба, подверженные воздействию случайных и неопределенных факторов. Разработка таких моделей требует принятия допущений, позволяющих разрешить неопределенности. Например, требуется задать распределения случайных величин, относящихся к входным переменным. Эта искусственная операция в известной степени порождает сомнение в достоверности результатов моделирования.

Среди моделей данной группы наибольшее распространение получили модели систем массового обслуживания.

К *третьей группе* относятся модели больших и очень больших (макроэкономических) систем: крупных торговых и промышленных предприятий и объединений, отраслей народного хозяйства и экономики страны в целом. Создание математической модели экономической системы такого масштаба представляет собой сложную научную проблему, решение которой под силу лишь крупному научно-исследовательскому учреждению.

Областями целесообразного применения компьютерных *имитационных моделей* в экономической политике на уровне федерального органа управления можно считать следующие:

макроэкономическая модель России - для прогнозирования агрегированных показателей (ВВП, инфляция, объем производства важнейших отраслей), **динамическая модель межотраслевого баланса России, модель платежного баланса России, модель консолидированного бюджета России.**

На уровне региональных органов власти целесообразно использование комплексной *имитационной модели* региона,

отражающей особенности развития региональной экономики. Данную модель можно представить в виде следующих структурных блоков:

"Население, "Рынок труда, "Производство и сфера услуг", "Финансы", "Социальная сфера" и "Уровень жизни населения, "Комплексная модель экономики субъекта РФ" .

В управлении финансово-хозяйственной деятельностью крупного предприятия возможно использование комплекса *динамических моделей*, состоящего из блоков:

Планирование производственных показателей, Планирование экономических показателей, Планирование финансовых показателей, Планирование активов, Планирование пассивов.

Комплекс *динамических моделей* в управлении финансово-хозяйственной деятельностью предприятия позволяет планировать финансово-хозяйственные потоки, построить прогнозный *бухгалтерский баланс*, сформировать план социально-экономического развития и бюджет предприятия.

Последовательность разработки имитационных моделей

Технологическая схема разработки *имитационной модели* может быть представлена в виде этапов следующим образом.

Этап 1: разработка математического (или аналитического) описания моделируемого объекта и формулировка основных положений и требований к программной реализации *имитационной модели*. Здесь разработчик оценивает сложность модели, решает задачу выбора математических, аналитических и программных средств. Определяет основные пути проектирования *имитационной модели*. Этот этап называется составление *концептуальной модели*. Он включает следующие подэтапы:

постановка задачи моделирования;
представление результатов моделирования;
интерпретация результатов моделирования;
выдача рекомендаций по оптимизации режима работы реальной системы.

Перед проведением расчетов на ПК должен быть составлен план проведения эксперимента с указанием комбинаций переменных и параметров, для которых должно проводиться моделирование системы. Задача заключается в разработке *оптимального плана* эксперимента, реализация которого позволяет при сравнительно небольшом числе испытаний модели получить достоверные данные о закономерностях функционирования системы.

Результаты моделирования могут быть представлены в виде таблиц, графиков, диаграмм, схем и т.п.

Интерпретация результатов моделирования имеет целью переход от информации, полученной в результате машинного эксперимента с моделью, к выводам, касающимся процесса функционирования объекта-оригинала.

На основании анализа результатов моделирования принимается решение о том, при каких условиях система будет функционировать с наибольшей эффективностью.

Этап 2: выбор средств описания реального объекта, методов проектирования, среды программирования.

Этап 3: разработка и создание программной реализации имитационного расчета для одного шага имитации. Определение функций изменения *имитационной модели* на шаге.

Для *динамических моделей* разрабатывается алгоритм изменения

расчетной модели при переходе от одного шага имитации к другому.

Этап 4: определение среды и условия проведения имитационного эксперимента. Разрабатывается программа управления имитационным процессом и выдачи промежуточных и окончательных результатов эксперимента по заданному сценарию.

Этап 5: анализ вариантов сценариев, принятие решения о путях совершенствования модели, имитационного процесса и выбор новых (или уточнение старых) путей исследования.

1. Независимые испытания. Формула Бернулли. Локальная предельная теорема. Теорема Муавра-Лапласа.

Независимые испытания. Формула Бернулли

Одно и тоже испытание повторяется многократно и исход каждого испытания независим от исходов других. Такой эксперимент называется *схемой повторных независимых испытаний* или *схемой Бернулли*.

Примеры повторных испытаний:

бросание монеты или игрального кубика (вероятности выпадения герба/решки или определенной цифры одинаковы в каждом броске);

извлечение из урны шара при условии, что вынутый шар после записи его цвета кладется обратно в урну (то есть состав шаров в урне не меняется и не меняется вероятность вынуть шар нужного цвета);

включение приборов (ламп, станков и т.п.) с заранее заданной одинаковой вероятностью выхода из строя каждого;

повторение стрелком выстрелов по одной и той же мишени при условии, что вероятность удачного попадания при каждом выстреле принимается одинаковой и т.д.

Итак, пусть в результате испытания возможны *два исхода*: либо появится событие A , либо противоположное ему событие.

Проведем n испытаний Бернулли. Это означает, что все n испытаний независимы; вероятность появления события A в каждом отдельно взятом или единичном испытании постоянна и от испытания к испытанию не изменяется (т.е. испытания

проводятся в одинаковых условиях). Обозначим вероятность появления события A в единичном испытании буквой p , т.е. $p=P(A)$, а вероятность противоположного события (событие A не наступило) - буквой $q = P(\bar{A}) = 1 - p$.

Тогда вероятность того, что событие A появится в этих n испытаниях ровно k раз, выражается *формулой Бернулли*

$P_n(k) = C_n^k \cdot p^k \cdot q^{n-k}$, $q = 1 - p$. Распределение числа успехов (появлений события) носит название *биномиального распределения*.

Пример. В урне 20 белых и 10 черных шаров. Вынули 4 шара, причем каждый вынутый шар возвращают в урну перед извлечением следующего и шары в урне перемешивают. Найти вероятность того, что из четырех вынутых шаров окажется 2 белых.

Решение. Событие A – достали белый шар. Тогда вероятности $P(A) = \frac{2}{3}$, $P(\bar{A}) = \frac{1}{3}$.

По формуле Бернулли требуемая вероятность равна

$$P_4(2) = C_4^2 \left(\frac{2}{3}\right)^2 \left(\frac{1}{3}\right)^2 = \frac{8}{27}$$

Локальная предельная теорема. Теорема Муавра-Лапласа.

Пусть в каждом из n независимых испытаний событие A может произойти с вероятностью p , $q = 1 - p$ (условия схемы Бернулли).

Обозначим как и раньше, через $P_n(k)$ вероятность ровно k появлений события A в n испытаниях. кроме того, пусть $P_n(k_1; k_2)$ – вероятность того, что число появлений события A находится между k_1 и k_2 .

Локальная теорема Лапласа.

Если n – велико, а p – отлично от 0 и 1, то

$$P_n(k) \approx \frac{1}{\sqrt{npq}} \varphi\left(\frac{k-np}{\sqrt{npq}}\right), \quad \text{где} \quad \varphi(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2} - \text{функция Гаусса}$$

Интегральная теорема Лапласа.

Если n – велико, а p – отлично от 0 и 1, то

$$P(n; k_1, k_2) \approx \Phi\left(\frac{k_2-np}{\sqrt{npq}}\right) - \Phi\left(\frac{k_1-np}{\sqrt{npq}}\right), \quad \text{где} \quad \Phi(x) = \frac{1}{\sqrt{2\pi}} \int_0^x e^{-t^2/2} dt - \text{функция Лапласа (функция табулирована, таблицу можно скачать на странице [формул по теории вероятностей](#)).$$

Функции Гаусса и Лапласа обладают *свойствами*, которые необходимо знать при использовании таблиц значений этих функций:

а) $\varphi(-x) = \varphi(x)$, $\Phi(-x) = -\Phi(x)$

б) при больших x верно $\varphi(x) \approx 0$, $\Phi(x) \approx 0,5$.

Теоремы Лапласа дают удовлетворительное приближение при $npq \geq 9$. Причем чем ближе значения q, p к 0,5, тем точнее данные формулы. При маленьких или больших значениях вероятности (близких к 0 или 1) формула дает большую погрешность (по сравнению с исходной формулой Бернулли).

Пример. Для мастера определенной квалификации вероятность изготовить деталь отличного качества равна 0,75. За смену он изготовил 400 деталей. Найти вероятность того, что в их числе 280 деталей отличного качества.

Решение. По условию $n = 400$, $p = 0,75$, $q = 0,25$, $k = 280$,

откуда $t = \frac{k-np}{\sqrt{npq}} = \frac{280-300}{\sqrt{75}} = -2,31$

По таблицам найдем $\varphi(-2,31) = \varphi(2,31) = 0,0277$.

Искомая вероятность равна: $P_{400}(280) = \frac{1}{\sqrt{npq}} \varphi(t) = \frac{0,0277}{\sqrt{75}} \approx 0,0032$.

2. Случайные величины и функции распределения. Дискретные случайные величины, абсолютные непрерывные случайные величины.

Случайной величиной называется такая величина, которая случайно принимает какое-то значение из множества возможных значений.

Случайные величины обозначаются: $X, Y, Z, \dots$. Значения, которые они принимают: x, y, z .

По множеству возможных значений различают дискретные и непрерывные случайные величины.

Дискретными называются **случайные величины**, значениями которых являются только отдельные точки числовой оси. (Число их может быть как конечно, так и бесконечно).

Пример: Число родившихся девочек среди ста новорожденных за последний месяц- это дискретная случайная величина, которая может принимать значения $1, 2, 3, \dots$

Непрерывными называются **случайные величины**, которые могут принимать все значения из некоторого числового промежутка.

Пример: Расстояние, которое пролетит снаряд при выстреле- это непрерывная случайная величина, значения которой принадлежат некоторому промежутку $[a; b]$.

Функцией распределения случайной величины X называется функция $F(x)$, выражающая для каждого x вероятность того, что случайная величина X примет значение, меньшее x : $F(x) = P(X < x)$
Функцию $F(x)$ иногда называют интегральной функцией распределения или интегральным законом распределения.

Общие свойства функции распределения:

1. Функция распределения случайной величины есть неотрицательная функция, заключенная между нулем и единицей
2. Функция распределения случайной величины есть неубывающая функция на всей числовой оси
3. На минус бесконечности функция распределения равна нулю, на плюс бесконечности равна единице
4. Вероятность попадания случайной величины в интервал $[x_1, x_2)$ (включая x_1) равна приращению ее функции распределения на этом интервале.

Закон распределения дискретной случайной величины (ДСВ)

представляет собой соответствие между значениями $x_1, x_2, \dots, x_n$ этой величины и их вероятностями $p_1, p_2, \dots, p_n$

Может быть задан аналитически, графически или таблично.

Самый простой способ представления закона распределения дискретной случайной величины — в виде таблицы ряда распределения, то есть

X	x_1	x_2		x_n
P	p_1	p_2		p_n

$x_1, x_2, \dots, x_n$ — значения дискретной случайной величины;
 $p_1, p_2, \dots, p_n$ — вероятности значений X дискретной случайной величины.

Также должно выполняться условия, что сумма вероятностей равна 1, то есть

$$\sum p = p_1 + p_2 + \dots + p_n = 1$$

Пример 1

Монета подбрасывается 10 раз, герб выпал 6 раз, а орел — 4 раза.
 Составить закон распределения дискретной случайной величины.

Решение

Вероятности равны:

$$p_1(6)=6/10=0,6;$$

$$p_2(4)=4/10=0,4$$

X	6	4
P	0.6	0.4

Под **понятием случайной величины** подразумевается величина, значения которой зависят от случая и для которой определена функция распределения вероятностей. *Функцией распределения вероятностей* случайной величины ξ называется вероятность того, что ξ примет значение, меньшее, чем произвольное число x :

$$F(x) = P\{\xi \leq x\}.$$

Дискретные случайные величины могут принимать только конечное или счетное множество значений. Соответствие между всеми возможными значениями дискретной случайной величины и их вероятностями называется *законом распределения* данной случайной величины.

Пусть ξ – дискретная случайная величина, единственно возможными значениями которой являются числа $x_1, x_2, \dots, x_n$.

Обозначим через

$$p_i = P(\xi = x_i) (i=1, 2, \dots, n)$$

вероятности этих значений. Тогда закон распределения случайной величины ξ задает таблица

ξ	x_1	x_2	...	x_n
p	p_1	p_2	...	p_n

Непрерывной называется случайная величина, все возможные значения которой целиком заполняют некоторый конечный или бесконечный промежуток числовой оси. Для любой непрерывной случайной величины существует неотрицательная функция $f(x)$, при любых x удовлетворяющая равенству: $F(x) = \int_{-\infty}^x f(t) dt$ $F(x) = \int_{-\infty}^x f(t) dt$.

Функция $f(x)$ называется плотностью распределения вероятностей непрерывной случайной величины.

Функция распределения и плотность вероятности непрерывной случайной величины

Если функция распределения $F_x(x)$ непрерывна, то случайная величина x называется непрерывной случайной величиной.

Если функция распределения непрерывной случайной величины дифференцируема, то более наглядное представление о случайной величине дает плотность вероятности случайной величины $p_x(x)$, которая связана с функцией распределения $F_x(x)$ формулами

$$F_x(x) = \int_{-\infty}^x p_x(t) dt \quad \text{и} \quad p_x(x) = \frac{dF_x(x)}{dx}.$$

Отсюда, в частности, следует, что для любой случайной

величины $\int_{-\infty}^{\infty} p(x) dx = 1$.

Плотность распределения вероятностей обладает следующими свойствами:

1. $f(x) \geq 0$.

2. При любых x_1 и x_2 удовлетворяет

равенству $P\{x_1 \leq \xi < x_2\} = \int_{x_1}^{x_2} f(x) dx$ $P\{x_1 \leq \xi < x_2\} = \int_{x_1}^{x_2} f(x) dx$.

$$3. \int f(x) dx = 1.$$

$F(x)$ является первообразной функцией для функции $f(x)$:

$$F'(x) = f(x),$$

поэтому функцию $F(x)$ называют также *интегральным*, а плотность вероятностей $f(x)$ - *дифференциальным законом* распределения случайной величины ξ . Функция распределения имеет также смысл для дискретных случайных величин.

В теории вероятностей в отношении непрерывных случайных величин доказана следующая важная теорема [27].

Теорема. Вероятность (до опыта) того, что непрерывная случайная величина ξ примет заранее указанное строго определенное значение a , равна нулю.

Рассмотрим теперь вероятностное пространство $\{\Omega, \Theta, P\}$, на котором определены n случайных величин

$$\xi_1 = f_1(\omega), \xi_2 = f_2(\omega), \dots, \xi_n = f_n(\omega).$$

Тогда вектор $(\xi_1, \xi_2, \dots, \xi_n)$ называется *случайным вектором* или *n -мерной случайной величиной*.

Обозначим через $\{\xi_1, \xi_2, \dots, \xi_n\}$ множество тех элементарных событий ω , для которых одновременно выполняются неравенства $\xi_1, \xi_2, \dots, \xi_n$.

Это событие принадлежит множеству Θ , т.е.

$$\{\xi_1, \xi_2, \dots, \xi_n\} \in \Theta.$$

Таким образом, при любом наборе чисел определена вероятность

$$F(x_1, x_2, \dots, x_n) = P\{\xi_1, \xi_2, \dots, \xi_n\}.$$

Функция $F(x_1, x_2, \dots, x_n)$ от n аргументов называется *n -мерной функцией распределения случайного вектора $(\xi_1, \xi_2, \dots, \xi_n)$* .

Геометрическая интерпретация рассматривает вектор $(\xi_1, \xi_2, \dots, \xi_n)$ как координаты точки n -мерного евклидова пространства. Функция $F(x_1, x_2, \dots, x_n)$ при такой интерпретации дает вероятность попадания точки $(\xi_1, \xi_2, \dots, \xi_n)$ в n -мерный параллелепипед $\xi_1 1, \xi_2 2, \dots, \xi_n n$ с ребрами, параллельными осям координат.

Функция распределения, указывая на то, какие значения может принимать случайная величина и с какими вероятностями, представляет собой наиболее полную характеристику последней. Однако общую количественную характеристику о случайной величине могут дать некоторые постоянные числа, получаемые по определенным правилам из функций распределения. К числу этих постоянных относятся математическое ожидание и дисперсия.

Для произвольной случайной величины ξ с функций распределения $f(x)$ математическим ожиданием называется интеграл

$$M\xi = \int x dF(x).$$

Для непрерывных случайных величин, распределенных по нормальному закону, математическое ожидание равно генеральному среднему, а его наилучшей точечной оценкой является выборочное среднее. Наилучшей точечной оценкой асимметрично распределенных непрерывных случайных величин является медиана.

Дисперсией случайной величины ξ называется математическое ожидание квадрата отклонения ξ от $M\xi$:

$$D\xi = M(\xi - M\xi)^2.$$

Таким образом, математическое ожидание представляет собой характеристику центральной тенденции (типичного значения) случайной величины, а дисперсия – меры ее рассеяния.

В заключение следует также охарактеризовать независимые и зависимые случайные величины. Две случайные величины считаются *независимыми*, если возможные значения и закон распределения каждой из них один и тот же при любом выборе допустимых значений другой. В противном случае случайные величины называются *зависимыми*. Несколько случайных величин являются *взаимно независимыми*, если возможные значения и законы распределения любой из них не зависят от того, какие возможные значения приняли остальные случайные величины.

Беспроводная связь. Передача радиосигнала, передача в видимом световом диапазоне и ИК-диапазоне. Спутниковые системы связи. Мобильная связь.

Беспроводная вычислительная сеть — вычислительная сеть, основанная на беспроводном (без использования кабельной проводки) принципе, полностью соответствующая стандартам для обычных проводных сетей (например, Ethernet). В качестве носителя информации в таких сетях могут выступать радиоволны СВЧ-диапазона.

Существует два основных направления применения беспроводных компьютерных сетей:

Работа в замкнутом объеме (офис, выставочный зал и т. п.);

Соединение удаленных локальных сетей (или удаленных сегментов локальной сети).

Для организации беспроводной сети в замкнутом пространстве применяются передатчики со всенаправленными антеннами.

Стандарт IEEE 802.11 определяет два режима работы сети — Ad-hoc и клиент-сервер.

Режим Ad-hoc (иначе называемый «точка-точка») — это простая сеть, в которой связь между станциями (клиентами) устанавливается напрямую, без использования специальной точки доступа.

В режиме клиент-сервер беспроводная сеть состоит, как минимум, из одной точки доступа, подключенной к проводной сети, и некоторого набора беспроводных клиентских станций. Без подключения дополнительной антенны устойчивая связь для оборудования IEEE 802.11b достигается в среднем на следующих расстояниях: открытое пространство — 500 м, комната, разделенная перегородками из неметаллического материала — 100 м, офис из нескольких комнат — 30 м.

Для соединения удаленных локальных сетей (или удаленных сегментов локальной сети) используется оборудование с направленными антеннами, что позволяет увеличить дальность связи до 20 км (а при использовании специальных усилителей и большой высоте размещения антенн — до 50 км).

Беспроводные персональные сети (WPAN - Wireless Personal Area Networks). Примеры технологий – Bluetooth. Беспроводные персональные сети (англ. Wireless personal area network, WPAN) - применяется для связи различных устройств, таких как компьютерная и бытовая техника между собой, а также с сетями более высокого уровня. WPAN разворачивается с применением сетевых технологий Bluetooth, infrared или Wi-Fi и имеет небольшой радиус действия от десятков сантиметров до нескольких метров.

Беспроводные локальные сети (WLAN - Wireless Local Area Networks). Примеры технологий – Wi-Fi. Беспроводная локальная сеть (WLAN) - объединение беспроводных устройств в сеть

происходит без использования кабелей, передача данных осуществляется через радиоэфир. Наиболее распространенной сетевой технологией для построения беспроводных локальных сетей является Wi-Fi. Данная технология обеспечивает необходимое покрытие помещений для работы конечных пользователей, например, добавлением в сеть дополнительных точек доступа. WLAN сеть может быть транслирована по всей площади, как небольших офисов, так и целых городов. Чаще всего, точка доступа WLAN, предоставляет доступ в радиусе до 20–200 м.

Беспроводные сети масштаба города (WMAN - Wireless Metropolitan Area Networks). Примеры технологий - WiMAX. Реализуется широкополосный доступ к сети через радиоканал с возможностью передачи звука и видео. WMAN используется для соединения территориально распределенных объектов (до 50 км) при помощи технологии WiMAX.

Беспроводные глобальные сети (WWAN - Wireless Wide Area Network). Примеры технологий – GPRS, EDGE. Главным отличием от локальных беспроводных сетей WLAN является использование беспроводных технологий сотовой связи для передачи данных (таких как UMTS, GPRS, CDMA, GSM, CDPD, Mobitex, HSDPA, 3G и др.). Технологии WWAN дают возможность пользователям получать доступ к Интернету, электронной почте и подключаться к виртуальным частным сетям из любой точки в пределах зоны действия оператора беспроводной связи.

Передача радиосигнала, передача в видимом световом диапазоне и ИК-диапазоне.

Радиосигнал - электромагнитное излучение, создаваемое каким-либо радиопередатчиком.

Передача происходит следующим образом: на передающей стороне (в радиопередатчике) формируются высокочастотные колебания (несущий сигнал) определенной частоты. На него накладывается сигнал, который нужно передать (звука, изображения и т. д.) - происходит *модуляция* (процесс изменения одного или нескольких параметров модулируемого несущего сигнала при помощи модулирующего сигнала) несущей полезным сигналом. Сформированный таким образом высокочастотный сигнал излучается антенной в пространство в виде радиоволн. На приёмной стороне радиоволны наводят модулированный сигнал в приемной антенне, он поступает в радиоприёмник. Здесь система фильтров выделяет из множества наведенных в антенне токов от разных передатчиков сигнал с нужной несущей частотой, а детектор выделяет из него модулирующий полезный сигнал.

Получаемый сигнал может несколько отличаться от передаваемого передатчиком вследствие влияния разнообразных помех.

Радиоволны распространяются в пустоте и в атмосфере; земная твердь и вода для них непрозрачны. Однако, благодаря эффектам дифракции и отражения, возможна связь между точками земной поверхности, не имеющими прямой видимости

Радиосигналы, на которых основывается, в частности спутниковая связь и другие типы связи, представляют собой электромагнитные волны. Система связи использует различные виды радиосигналов для передачи информации через воздушную среду от одной точки к другой.

Радиосигнал распространяется от антенны передающей станции к антенне приемной. Передача радиосигнала осуществляется благодаря нескольким факторам. Сигнал, подаваемый на антенну, характеризуется амплитудой, частотой и фазой. За счет изменения

этих параметров можно посредством радиосигналов передавать информацию.

Передача радиосигнала происходит через воздушную среду, что обуславливает уменьшение его амплитуды. В случае отсутствия препятствий радиосигналы испытывают потери в свободном пространстве, они являются одной из причин затухания сигнала, и передача радиосигнала теряет прежнее качество.

Инфракрасное излучение и излучение в миллиметровом диапазоне используется на небольших расстояниях в блоках дистанционного управления. Основной недостаток при этом - излучение не способно проходить через преграды. С другой стороны, этот недостаток одновременно является и преимуществом, когда излучение в одной комнате не интерферирует с излучением в другой и на использование данного частотного диапазона не надо получать разрешения.

Инфракрасная связь поддерживает передачу данных через инфракрасные соединения с компьютерами и любым периферийным оборудованием по протоколам IrDA(Infrared Data Association). Она является дешевым способом соединения компьютеров друг с другом и с различными устройствами.

Инфракрасный канал передачи данных устанавливается между двумя инфракрасными устройствами. Все данные по этому каналу передаются с основного (запрашивающего) устройства на вторичное (принимающее). Роль основного устройства присваивается динамически при установке связи и сохраняется до закрытия подключения. Ее может выполнять любое пригодное для этого устройство. Некоторые периферийные устройства способны выполнять только вторичную роль.

Устройство создает связь при автоматическом обнаружении другого устройства или по запросу пользователя. Запрашивающая

станция отправляет другому устройству запрос на соединение, включающий такие сведения, как адрес, скорость передачи данных и другие параметры. Отвечающее устройство, в ответ, сообщает сведения о своем адресе и возможностях. Затем основное и вторичное устройства изменяют параметры связи и скорость передачи данных на общий набор, заданный при начальной передаче данных. Наконец, первичная станция передает на вторичную сообщение, подтверждающее подключение. После этого устройства связаны и начинают передачу данных под управлением первичного устройства.

Видимый диапазон также используется для передачи данных. Обычно источником света является лазер. Монохромное когерентное излучение легко фокусируется. К недостаткам относится зависимость качества передачи от атмосферных осадков и даже от конвекционных воздушных потоков.

Спутниковые каналы связи

Спутники в системах связи могут находиться на геостационарных орбитах или низкоорбитальных. Геостационарные спутниковые системы дают заметные задержки прохождения сигналов более 500 мс. В низкоорбитальных системах обслуживание пользователей происходит поочередно разными спутниками. Чем ниже орбита, тем меньше площадь покрытия и, следовательно, нужно больше спутников и наземных станций, а также требуется система межспутниковой связи.

Спутниковые системы связи имеют отличия от наземных систем.

1. Большая задержка сигнала;
2. Спутниковые системы являются системами принципиально вещательного типа;

3. Имеют место серьезные проблемы, связанные с обеспечением безопасности передаваемой информации;
4. Стоимость передачи не зависит от расстояния;
5. Низкий коэффициент ошибок при передаче;
6. Коечному пользователю с индивидуальной антенной доступна вся пропускная способность спутника, в отличие от проводных каналов связи, где за счет мультиплексирования пропускная способность делится между множеством абонентов;
7. Спутник доступен практически всегда;
8. Возможность организации мобильных систем связи на больших расстояниях;
9. Возможность организации систем связи на больших расстояниях и в труднодоступных местах, на водных пространствах;
10. Большая скорость организации системы передачи данных.

Существует несколько способов работы наземных терминалов со спутником. При этом может использоваться мультиплексирование по частоте, по времени, метод доступа с коллизиями, маркерный метод доступа. Последняя схема предполагает, что наземные станции образуют логическое кольцо, вдоль которого движется маркер. Станция может начать передачу на спутник, лишь получив этот маркер.

Системы сотовой связи

Телефонные системы, даже на основе высокоскоростного ISDN (англ. Integrated Services Digital Network — данная технология обеспечивает передачу цифрового сигнала по телефонным каналам с предоставлением различных служб.), не в состоянии удовлетворить потребности в мобильной связи. Системы мобильной связи осуществляют передачу информации между

пунктами, один из которых или оба являются подвижными. Характерным признаком систем мобильной связи является применение радиоканала. К основным технологиям мобильной связи относятся пейджинг, твейджинг, транкинг, сотовая телефония, спутниковая телефония.

Пейджинг – это система односторонней связи, при которой передаваемое сообщение поступает на пейджер пользователя, извещая его о необходимости предпринять то или действие, или просто информируя его о тех или иных текущих событиях (симплексный режим). Это наиболее дешевый вид мобильной связи, который требует небольшой полосы пропускания.

Твейджинг - это система двухстороннего пейджинга. В отличие от пейджинга здесь возможно подтверждение получения сообщения и даже проведение некоторого подобия диалога.

Транкингом (транковой связью) называется мобильная связь, для которой характерны следующие особенности:

1. Наличие связи внутри некоторой группы и возможности группового вызова от центра ко всем членам группы;
2. Наличие приоритетности;
3. Повышенная скорость соединения;
4. Возможность выхода в телефонную сеть общего пользования может отсутствовать;
5. Преимущественная передача данных в полудуплексном режиме.

Сотовая телефония обеспечивает телефонную связь между подвижными абонентами посредством зон покрытия связи (сот). Связь осуществляется через базовые станции, выполняющие функции коммутаторов.

В общем случае преимущества использования сотовой телефонной сети следующие:

1. Выигрыш в использовании частот из-за их повторного использования;
2. Увеличенная емкость сети, т.е. число одновременно обслуживаемых пользователей;
3. Возможность использования маломощного сигнала;
4. Компактность приемопередатчика;
5. Малая мощность источников питания;
6. Хорошая масштабируемость сети за счет разделения сот при нарастании числа отказов.

Соты имеют базовую станцию, состоящую из процессора и приемно-передающей части. Базы подключаются к центральной станции коммутации. Центральная станция соединяется с наземной телефонной сетью и, при необходимости, коммутирует звонок с мобильного телефона на обычный телефон. Такие системы могут обслуживать пейджинговую и сотовую сеть.

В каждый момент времени мобильный телефон логически находится в одной определенной соте и управляется одной базовой станцией. Когда телефон покидает ее, базовая станция обнаруживает падение уровня сигнала и запрашивает окружающие станции об уровне сигнала для данного аппарата. Управление аппаратом передается станции с наибольшим входным сигналом. Телефон информируется о смене управляющей станции, при этом предлагается переключиться на новый частотный канал, так как в смежных сотах используются разные частотные каналы. Процесс переключения занимает менее 300

мсек, что незаметно для пользователя. Сигнал передатчика падает по мере удаления от базовой станции соты

БИЛЕТ 23

1. Случайные величины и функции распределения.
Дискретные случайные величины, абсолютные непрерывные случайные величины.

Случайной величиной называется такая величина, которая случайно принимает какое-то значение из множества возможных значений.

Случайные величины обозначаются: $X, Y, Z, \dots$. Значения, которые они принимают: x, y, z .

По множеству возможных значений различают дискретные и непрерывные случайные величины.

Дискретными называются **случайные величины**, значениями которых являются только отдельные точки числовой оси. (Число их может быть как конечно, так и бесконечно).

Пример: Число родившихся девочек среди ста новорожденных за последний месяц- это дискретная случайная величина, которая может принимать значения $1, 2, 3, \dots$

Непрерывными называются **случайные величины**, которые могут принимать все значения из некоторого числового промежутка.

Пример: Расстояние, которое пролетит снаряд при выстреле- это непрерывная случайная величина, значения которой принадлежат некоторому промежутку $[a; b]$.

Функцией распределения случайной величины X называется функция $F(x)$, выражающая для каждого x

вероятность того, что случайная величина X примет значение, меньшее x : $F(x)=P(X < x)$ Функцию $F(x)$ иногда называют интегральной функцией распределения или интегральным законом распределения.

Общие свойства функции распределения:

1. Функция распределения случайной величины есть неотрицательная функция, заключенная между нулем и единицей
2. Функция распределения случайной величины есть неубывающая функция на всей числовой оси
3. На минус бесконечности функция распределения равна нулю, на плюс бесконечности равна единице
4. Вероятность попадания случайной величины в интервал $[x_1, x_2)$ (включая x_1) равна приращению ее функции распределения на этом интервале.

Закон распределения дискретной случайной величины (ДСВ) представляет собой соответствие между значениями $x_1, x_2, \dots, x_n$ этой величины и их вероятностями $p_1, p_2, \dots, p_n$

Может быть задан аналитически, графически или таблично.

Самый простой способ представления закона распределения дискретной случайной величины — в виде таблицы ряда распределения, то есть

X	x_1	x_2		x_n
P	p_1	p_2		p_n

$x_1, x_2, \dots, x_n$ — значения дискретной случайной величины;
 $p_1, p_2, \dots, p_n$ — вероятности значений X дискретной случайной величина.

Также должно выполняться условия, что сумма вероятностей равна 1, то есть

$$\sum p = p_1 + p_2 + \dots + p_n = 1$$

Пример 1

Монета подбрасывается 10 раз, герб выпал 6 раз, а орел — 4 раза. Составить закон распределения дискретной случайной величины.

Решение

Вероятности равны:

$$p_1(6) = 6/10 = 0,6;$$

$$p_2(4) = 4/10 = 0,4$$

X	6	4
P	0.6	0.4

Под понятием **случайной величины** подразумевается величина, значения которой зависят от случая и для которой определена функция распределения вероятностей. *Функцией распределения вероятностей* случайной величины ξ называется вероятность того, что ξ примет значение, меньшее, чем произвольное число x :

$$F(x) = P\{\xi \leq x\}.$$

Дискретные случайные величины могут принимать только конечное или счетное множество значений. Соответствие

между всеми возможными значениями дискретной случайной величины и их вероятностями называется *законом распределения* данной случайной величины.

Пусть ξ – дискретная случайная величина, единственно возможными значениями которой являются числа $x_1, x_2, \dots, x_n$. Обозначим через

$$p_i = P(\xi = x_i) (i=1, 2, \dots, n)$$

вероятности этих значений. Тогда закон распределения случайной величины ξ задает таблица

ξ	x_1	x_2	...	x_n
p	p_1	p_2	...	p_n

Непрерывной называется случайная величина, все возможные значения которой целиком заполняют некоторый конечный или бесконечный промежуток числовой оси. Для любой непрерывной случайной величины существует неотрицательная функция $f(x)$, при любых x удовлетворяющая равенству: $F(x) = \int_{-\infty}^x f(x) dx$ $F(x) = \int_{-\infty}^x f(x) dx$.

Функция $f(x)$ называется плотностью распределения вероятностей непрерывной случайной величины.

Функция распределения и плотность вероятности непрерывной случайной величины

Если функция распределения $F_x(x)$ непрерывна, то случайная величина x называется непрерывной случайной величиной.

Если функция распределения непрерывной случайной величины дифференцируема, то более наглядное представление о случайной величине дает плотность вероятности случайной величины $p_X(x)$, которая связана с функцией распределения $F_X(x)$ формулами

$$F_X(x) = \int_{-\infty}^x p_X(t) dt \quad \text{и} \quad p_X(x) = \frac{dF_X(x)}{dx}.$$

Отсюда, в частности, следует, что для любой случайной

величины $\int_{-\infty}^{\infty} p(x) dx = 1$.

Плотность распределения вероятностей обладает следующими свойствами:

1. $f(x) \geq 0$.
2. При любых x_1 и x_2 удовлетворяет равенству $P\{x_1 \leq \xi < x_2\} = \int_{x_1}^{x_2} f(x) dx$ $P\{x_1 \leq \xi < x_2\} = \int_{x_1}^{x_2} f(x) dx$.
3. $\int f(x) dx = 1$.

$F(x)$ является первообразной функцией для функции $f(x)$:

$$F'(x) = f(x),$$

поэтому функцию $F(x)$ называют также *интегральным*, а плотность вероятностей $f(x)$ - *дифференциальным законом* распределения случайной величины ξ . Функция распределения имеет также смысл для дискретных случайных величин.

В теории вероятностей в отношении непрерывных случайных величин доказана следующая важная теорема [27].

Теорема. Вероятность (до опыта) того, что непрерывная случайная величина ξ примет заранее указанное строго определенное значение a , равна нулю.

Рассмотрим теперь вероятностное пространство $\{\Omega, \Theta, P\}$, на котором определены n случайных величин

$$\xi_1 = f_1(\omega), \xi_2 = f_2(\omega), \dots, \xi_n = f_n(\omega).$$

Тогда вектор $(\xi_1, \xi_2, \dots, \xi_n)$ называется *случайным вектором* или *n -мерной случайной величиной*.

Обозначим через $\{\xi_1 1, \xi_2 2, \dots, \xi_n n\}$ множество тех элементарных событий ω , для которых одновременно выполняются неравенства

$$\xi_1 1, \xi_2 2, \dots, \xi_n n.$$

Это событие принадлежит множеству Θ , т.е.

$$\{\xi_1 1, \xi_2 2, \dots, \xi_n n\} \in \Theta.$$

Таким образом, при любом наборе чисел определена вероятность

$$F(x_1, x_2, \dots, x_n) = P\{\xi_1 1, \xi_2 2, \dots, \xi_n n\}.$$

Функция $F(x_1, x_2, \dots, x_n)$ от n аргументов называется *n -мерной функцией распределения случайного вектора $(\xi_1, \xi_2, \dots, \xi_n)$* .

Геометрическая интерпретация рассматривает вектор $(\xi_1, \xi_2, \dots, \xi_n)$ как координаты точки n -мерного евклидова пространства. Функция $F(x_1, x_2, \dots, x_n)$ при такой интерпретации дает вероятность попадания точки $(\xi_1, \xi_2, \dots, \xi_n)$ в n -мерный параллелепипед $\xi_1 1, \xi_2 2, \dots, \xi_n n$ с ребрами, параллельными осям координат.

Функция распределения, указывая на то, какие значения может принимать случайная величина и с какими вероятностями, представляет собой наиболее полную характеристику последней. Однако общую количественную характеристику о случайной величине могут дать некоторые постоянные числа, получаемые по определенным правилам из функций распределения. К числу этих постоянных относятся математическое ожидание и дисперсия.

Для произвольной случайной величины ξ с функций распределения $f(x)$ математическим ожиданием называется интеграл

$$M\xi = \int x dF(x).$$

Для непрерывных случайных величин, распределенных по нормальному закону, математическое ожидание равно генеральному среднему, а его наилучшей точечной оценкой является выборочное среднее. Наилучшей точечной оценкой асимметрично распределенных непрерывных случайных величин является медиана.

Дисперсией случайной величины ξ называется математическое ожидание квадрата отклонения ξ от $M\xi$:

$$D\xi = M(\xi - M\xi)^2.$$

Таким образом, математическое ожидание представляет собой характеристику центральной тенденции (типичного значения) случайной величины, а дисперсия – меры ее рассеяния.

В заключение следует также охарактеризовать независимые и зависимые случайные величины. Две

случайные величины считаются *независимыми*, если возможные значения и закон распределения каждой из них один и тот же при любом выборе допустимых значений другой. В противном случае случайные величины называются *зависимыми*. Несколько случайных величин являются *взаимно независимыми*, если возможные значения и законы распределения любой из них не зависят от того, какие возможные значения приняли остальные случайные величины.

Методы повышения достоверности передачи.

ОБЕСПЕЧЕНИЕ ДОСТОВЕРНОСТИ ПЕРЕДАЧИ ИНФОРМАЦИИ

Проблема обеспечения безошибочности (**достоверности**) передачи информации в сетях имеет очень большое значение. Если при передаче обычной телеграммы возникает в тексте ошибка или при разговоре по телефону слышен треск, то в большинстве случаев ошибки и искажения легко обнаруживаются по смыслу. Но при передаче данных одна ошибка (искажение одного бита) на тысячу переданных сигналов может серьезно отразиться на качестве информации.

Существует множество методов обеспечения достоверности передачи информации (методов защиты от ошибок), отличающихся по используемым для их реализации средствам, по затратам времени на их применение на передающем и приемном пунктах, по затратам дополнительного времени на передачу фиксированного объема данных (оно обусловлено изменением объема трафика пользователя при реализации

данного метода), по степени обеспечения достоверности передачи информации. Практическое воплощение методов состоит из двух частей – программной и аппаратной.

Соотношение между ними может быть самым различным, вплоть до почти полного отсутствия одной из частей. Чем больше удельный вес аппаратных средств по сравнению с программными, тем при прочих равных условиях сложнее оборудование, реализующее метод, и меньше затрат времени на его реализацию, и наоборот.

Выделяют две основные причины возникновения ошибок при передаче информации в сетях:

❑ сбои в какой-то части оборудования сети или возникновение неблагоприятных объективных событий в сети (например, коллизий при использовании метода случайного доступа в сеть). Как правило, система передачи данных готова к такого рода проявлениям и устраняет их с помощью планово предусмотренных средств;

❑ помехи, вызванные внешними источниками и атмосферными явлениями.

Помехи – это электрические возмущения, возникающие в самой аппаратуре или попадающие в нее извне.

Трудности борьбы с помехами заключаются в беспорядочности, нерегулярности и в структурном сходстве помех с информационными сигналами. Поэтому защита информации от ошибок и вредного влияния помех имеет большое практическое значение и является одной из серьезных проблем современной теории и техники связи.

Среди многочисленных методов защиты от ошибок выделяются три группы методов: *групповые методы, помехоустойчивое кодирование и методы защиты от ошибок в системах передачи с обратной связью.*

Из **групповых методов** получили широкое применение **мажоритарный метод**, реализующий принцип Вердана, и **метод передачи информационными блоками с количественной характеристикой блока.**

Суть **мажоритарного метода**, давно используемого в телеграфии, состоит в следующем. Каждое сообщение ограниченной длины передается несколько раз, чаще всего три раза. Принимаемые сообщения запоминаются, а потом производится их поразрядное сравнение. Суждение о правильности передачи выносится по совпадению большинства из принятой информации методом «два из трех». Например, кодовая комбинация 01101 при трехразовой передаче была частично искажена помехами, поэтому приемник принял такие комбинации: 10101, 01110, 01001. В результате проверки каждой позиции отдельно правильной считается комбинация 01101.

Другой групповой метод, также не требующий перекодирования информации, предполагает передачу данных блоками с количественной характеристикой блока.

Таковыми характеристиками могут быть: *число единиц или нулей в блоке, контрольная сумма передаваемых символов в блоке, остаток от деления контрольной суммы на постоянную величину и др.* На приемном пункте эта характеристика вновь подсчитывается и сравнивается с переданной по каналу связи. Если характеристики

совпадают, считается, что блок не содержит ошибок. В противном случае на передающую сторону передается сигнал с требованием повторной передачи блока.

Помехоустойчивое (избыточное) кодирование, предполагающее разработку и использование корректирующих (помехоустойчивых) кодов, применяется для защиты от ошибок при передаче информации между устройствами. Оно позволяет получить более высокие качественные показатели работы систем связи. Его основное назначение – в обеспечении малой вероятности искажений передаваемой информации, несмотря на присутствие помех или сбоев в работе сети.

Существует довольно большое количество различных помехоустойчивых кодов, отличающихся друг от друга по ряду показателей и, прежде всего по своим корректирующим возможностям.

К числу **наиболее важных показателей корректирующих кодов** относятся:

- ▣ значность кода, или длина кодовой комбинации, включающей информационные символы (m) и проверочные, или контрольные символы (K). Обычно значность кода n есть сумма $m+K$;
- ▣ избыточность кода $K_{изб}$, выражаемая отношением числа контрольных символов в кодовой комбинации к значности кода;
- ▣ корректирующая способность кода $K_{кс}$, представляющая собой отношение числа кодовых комбинаций L , в которых ошибки были обнаружены и исправлены, к общему числу

переданных кодовых комбинаций M в фиксированном объеме информации.

При выборе кода надо стремиться, чтобы он имел меньшую избыточность. Чем больше коэффициент ***Кизб***, тем менее эффективно используется пропускная способность канала связи и больше затрачивается времени на передачу информации, но зато выше помехоустойчивость системы.

Корректирующие коды в основном применяются для обнаружения ошибок, исправление которых осуществляется путем повторной передачи искаженной информации. С этой целью в сетях используются *системы передачи с обратной связью* (наличие между абонентами дуплексной связи облегчает применение таких систем).

Системы передачи с обратной связью делятся на системы с решающей обратной связью и системы с информационной обратной связью.

Особенностью систем ***с решающей обратной связью (системы с перезапросом)*** является то, что решение о необходимости повторной передачи информации (сообщения, пакета) принимает приемник. **Здесь обязательно применяется помехоустойчивое кодирование**, с помощью которого на приемной станции осуществляется проверка принимаемой информации. При обнаружении ошибки на передающую сторону по каналу обратной связи посылается сигнал пере запроса, по которому информация передается повторно. Канал обратной связи используется также для посылки сигнала подтверждения правильности приема, автоматически определяющего начало следующей передачи.

В системах с информационной обратной связью передача информации осуществляется без помехоустойчивого кодирования. Приемник, приняв информацию по прямому каналу и зафиксировав ее в своей памяти, передает ее в полном объеме по каналу обратной связи передатчику, где переданная и возвращенная информация сравниваются. При совпадении передатчик посылает приемнику сигнал подтверждения, в противном случае происходит повторная передача всей информации. Таким образом, здесь решение о необходимости повторной передачи принимает передатчик.

Обе рассмотренные системы обеспечивают практически одинаковую достоверность, однако, **в системах с решающей обратной связью пропускная способность каналов используется эффективнее**, поэтому они получили большее распространение.

7.2. Проверка по четности и код Хемминга. Циклические коды

<https://habr.com/ru/post/111336/>

[https://ru.bmstu.wiki/Помехоустойчивое кодирование и д
екодирование в дискретных КПС](https://ru.bmstu.wiki/Помехоустойчивое_кодирование_и_д_екодирование_в_дискретных_КПС)

<https://textarchive.ru/c-2444135-pall.html>

<https://habr.com/ru/post/140611/> (подробнее про Код Хэмминга. С примером)

[https://siblec.ru/telekommunikatsii/teoriya-peredachi-
signalov/7-korrektiruyushchie-kody#7.5](https://siblec.ru/telekommunikatsii/teoriya-peredachi-signalov/7-korrektiruyushchie-kody#7.5)

7.4. Код с чётным числом единиц. Инверсионный код

Рассмотрим некоторые простейшие систематические коды, применяемые только для обнаружения ошибок. Одним из кодов подобного типа является код с четным числом единиц. Каждая комбинация этого кода содержит, помимо информационных символов, один контрольный символ, выбираемый равным 0 или 1 так, чтобы сумма единиц в комбинации всегда была четной. Примером могут служить пятизначные комбинации кода Бодо, к которым добавляется шестой контрольный символ: 10101,1 и 01100,0. Правило вычисления контрольного символа можно выразить на

основании (7.9) в следующей форме: $e = \sum_{i=1}^k c_i$. Отсюда вытекает, что для любой комбинации сумма всех символов по модулю два будет равна нулю ($\sum_0$ — суммирование по модулю):

$$e \oplus \sum_{i=1}^k c_i = 0 \quad (7.12)$$

Это позволяет в декодирующем устройстве сравнительно просто производить обнаружение ошибок путем проверки на четность. Нарушение четности имеет место при появлении однократных, трехкратных и в общем, случае ошибок нечетной кратности, что и дает возможность их обнаружить. Появление четных ошибок не изменяет четности суммы (7.12), поэтому такие ошибки не обнаруживаются. На основании (7.8) вероятность необнаруженной ошибки равна:

$$P_{ош} = 1 - (1 - P_0)^n - \sum_{g=1,3,\dots} C_n^g P_0^g (1 - P_0)^{n-g}$$

$$(g \leq n)$$

К **достоинствам** кода следует отнести простоту кодирующих и декодирующих устройств, а также малую избыточность $\chi = \frac{1}{1+k}$, однако последнее определяет и его основной **недостаток** — сравнительно низкую корректирующую способность.

Значительно лучшими корректирующими способностями обладает **инверсный код**, который также применяется только для обнаружения ошибок. С принципом построения такого кода удобно ознакомиться на примере двух комбинаций: 11000, 11000 и 01101, 10010. В каждой комбинации символы до запятой являются информационными, а последующие — контрольными. Если количество единиц в информационных символах четное, т. е. сумма этих символов

$$c_{\Sigma} = \sum_{i=1}^k c_i$$

(7.13)

равна нулю, то контрольные символы представляют собой простое повторение информационных. В противном случае, когда число единиц нечетное и сумма (7.13) равна 1, контрольные символы получаются из информационных посредством инвертирования, т. е. путем замены всех 0 на 1, а 1 на 0. Математическая форма записи образования контрольных символов имеет вид $e_j = c_j \oplus c_{\Sigma}$. При

декодировании происходит сравнение принятых информационных и контрольных символов. Если сумма единиц в принятых информационных символах четная, т.

е. $c'_{\Sigma} = \sum_{i=1}^k c'_i = 0$, то соответствующие друг другу информационные и контрольные символы суммируются по модулю два. В противном случае, когда $c'_z = 1$, происходит такое же суммирование, но с инвертированными контрольными символами. Другими словами, в соответствии с (7.10) производится r проверок на четность: $x_j = e'_j + c''_j = e'_j \oplus c'_j \oplus c'_{\Sigma}$. Ошибка обнаруживается, если хотя бы одна проверка на четность дает 1.

Инверсный код обладает высокой обнаруживающей способностью, однако она достигается ценой сравнительно большой избыточности, которая, как нетрудно определить, составляет величину $\chi = 0,5$.

7.5. Коды Хэмминга

К этому типу кодов обычно относят систематические коды с расстоянием $d=3$, которые позволяют исправить все одиночные ошибки (7.3).

Рассмотрим построение семизначного кода Хэмминга, каждая комбинация которого содержит четыре информационных и три контрольных символа. Такой код, условно обозначаемый (7.4), удовлетворяет

неравенству (7.11) и имеет избыточность $\chi = \frac{r}{k+r} = \frac{3}{7}$

Если информационные символы c занимают в комбинация первые четыре места, то последующие три контрольных символа образуются по общему правилу (7.9) как суммы:

$$e_j = \alpha_{j1}c_1 \oplus \alpha_{j2}c_2 \oplus \alpha_{j3}c_3 \oplus \alpha_{j4}c_4$$

(7.14)

Декодирование осуществляется путем трех проверок на четность (7.10):

$$\begin{aligned} x_1 &= e_1' \oplus e_1'' = e_1' \oplus \sum_{i=1}^4 \alpha_{1i} c_i', \\ x_2 &= e_2' \oplus e_2'' = e_2' \oplus \sum_{i=1}^4 \alpha_{2i} c_i', \\ x_3 &= e_3' \oplus e_3'' = e_3' \oplus \sum_{i=1}^4 \alpha_{3i} c_i'. \end{aligned}$$

(7.15)

Так как x равно 0 или 1, то всего может быть восемь контрольных чисел $X=x_1x_2x_3$: 000, 100, 010, 001, 011, 101, 110 и 111. Первое из них имеет место в случае правильного приема, а остальные семь появляются при наличии искажений и должны использоваться для определения местоположения одиночной ошибки в семизначной комбинации. Выясним, каким образом устанавливается взаимосвязь между контрольными числами и искаженными символами. Если искажен один из контрольных символов: e_1' , e_2' или e_3' , то, как следует из (7.15), контрольное число примет соответственно одно из трех значений: 100, 010 или 001. Остальные четыре контрольных числа используются для выявления ошибок в информационных символах.

Таблица 7.1

Искаженный символ	c_1	c_2	c_3	c_4	e_1	e_2	e_3
Контрольное число	011	101	110	111	100	010	001

Порядок присвоения контрольных чисел ошибочным информационным символам может устанавливаться любой, например, как показано в табл. 7.1. Нетрудно показать, что этому распределению контрольных чисел соответствуют коэффициенты α_{ji} , приведенные в табл. 7.2.

Таблица 7.2

e_1	$\alpha_{11} = 0$	$\alpha_{12} = 1$	$\alpha_{13} = 1$	$\alpha_{14} = 1$
e_2	$\alpha_{21} = 1$	$\alpha_{22} = 0$	$\alpha_{23} = 1$	$\alpha_{24} = 1$
e_3	$\alpha_{31} = 1$	$\alpha_{32} = 1$	$\alpha_{33} = 0$	$\alpha_{34} = 1$

Если подставить коэффициенты α_{ji} в выражение (7.15), то получим:

$$\begin{aligned} x_1 &= e_1 \oplus c_2 \oplus c_3 \oplus c_4, \\ x_2 &= e_2 \oplus c_1 \oplus c_3 \oplus c_4, \\ x_3 &= e_3 \oplus c_1 \oplus c_2 \oplus c_4 \end{aligned} \quad (7.16)$$

При искажении одного из информационных символов становятся равными единице те суммы x , в которые входит этот символ. Легко проверить, что получающееся в этом случае контрольное число $X = x_1 x_2 x_3$ согласуется с табл. 7.1. Нетрудно заметить, что первые четыре контрольные числа табл. 7.1 совпадают со столбцами табл. 7.2. Это свойство дает возможность при выбранном распределении контрольных чисел составить таблицу коэффициентов α_{ji} . Таким образом, при одиночной ошибке можно вычислить контрольное число, позволяющее по табл. 7.1 определить тот символ кодовой комбинации, который претерпел искажения. Исправление искаженного символа двоичной системы состоит в простой замене 0 на 1 или 1 на 0. В

качестве примера рассмотрим передачу комбинации, в которой информационными символами являются $c_1c_2c_3c_4 = 1011$, Используя ф-лу (7.14) и табл. 7.2, вычислим контрольные символы:

$$e_1 = 0 \oplus 1 \oplus 1 = 0,$$

$$e_2 = 1 \oplus 1 \oplus 1 = 1,$$

$$e_3 = 1 \oplus 0 \oplus 1 = 0.$$

Передаваемая комбинация при этом будет $c_1c_2c_3c_4e_1e_2e_3 = 1011,010$. Предположим, что принята комбинация — 1001, 010 (искажен символ c_3). Подставляя соответствующие значения в (7.16), получим:

$$x_1 = 0 \oplus 0 \oplus 0 \oplus 1 = 1,$$

$$x_2 = 1 \oplus 1 \oplus 0 \oplus 1 = 1,$$

$$x_3 = 0 \oplus 1 \oplus 0 \oplus 1 = 0.$$

Вычисленное таким образом контрольное число $X = x_1x_2x_3 = 110$ позволяет согласно табл. 7.1 исправить ошибку в символе.

Здесь был рассмотрен простейший способ построения и декодирования кодовых комбинаций, в которых первые места отводились информационным символам, а соответствие между контрольными числами и ошибками определялось таблице. Вместе с тем существует более изящный метод отыскания одиночных ошибок, предложенный впервые самим Хэммингом. При этом методе код строится так, что контрольное число в двоичной системе счисления сразу указывает номер искаженного символа. Правда, в этом случае контрольные символы необходимо располагать среди информационных, что усложняет процесс кодирования. Для кода (7.4) символы в комбинации должны размещаться в следующем

порядке: $e_1 e_2 e_3 e_4 c_5 c_6 c_7$, а контрольное число вычисляться по формулам:

$$\begin{aligned} x_1 &= e_1' \oplus c_3' \oplus c_5' \oplus c_7' \\ x_2 &= e_2' \oplus c_3' \oplus c_6' \oplus c_7', \\ x_3 &= e_3' \oplus c_5' \oplus c_6' \oplus c_7' \end{aligned}$$

(7.17)

Так, если произошла ошибка в информационном символе c_5 то контрольное число $X = x_1 x_2 x_3 = 101$, что соответствует числу 5 в двоичной системе.

В заключение отметим, что в коде (7.4) при появлении многократных ошибок контрольное число также может отличаться от нуля. Однако декодирование в этом случае будет проведено неправильно, так как оно рассчитано на исправление лишь одиночных ошибок.

http://project.net.ru/others/article7/net3_9.html

Глава 3. Кодирование информации

9. Циклические коды.

К числу эффективных кодов, обнаруживающих одиночные, кратные ошибки и пачки ошибок, относятся *циклические коды* (CRC - Cyclic Redundance Code). Они высоконадежны и могут применяться при блочной синхронизации, при которой выделение, например, бита нечетности было бы затруднительно.

Один из вариантов циклического кодирования заключается в умножении исходного кода на образующий полином $g(x)$, а декодирование - в делении на $g(x)$. Если остаток от деления не равен нулю, то произошла ошибка. Сигнал об

ошибке поступает на передатчик, что вызывает повторную передачу.

Образующий полином есть двоичное представление одного из простых множителей, на которые раскладывается число $X^n - 1$, где X^n обозначает единицу в n -м разряде, n равно числу разрядов кодовой группы. Так, если $n = 10$ и $X = 2$, то $X^n - 1 = 1023 = 11 \cdot 93$, и если $g(X) = 11$ или в двоичном коде 1011, то примеры циклических кодов $A_i \cdot g(X)$ чисел A_i в кодовой группе при этом образующем полиноме можно видеть в следующей табл. 3.1.

Основной вариант циклического кода, широко применяемый на практике, отличается от предыдущего тем, что операция деления на образующий полином заменяется следующим алгоритмом: 1) к исходному кодируемому числу A справа приписывается K нулей, где K - число битов в образующем полиноме, уменьшенное на единицу; 2) над полученным числом $A \cdot (2^K)$ выполняется операция O , отличающаяся от деления тем, что на каждом шаге операции вместо вычитания выполняется поразрядная операция "исключающее ИЛИ": 3) полученный остаток B и есть CRC - избыточный K -разрядный код, который заменяет в закодированном числе C приписанные справа K нулей, т.е. $C = A \cdot (2^K) + B$.

На приемном конце над кодом C выполняется операция O . Если остаток не равен нулю, то при передаче произошла ошибка и нужна повторная передача кода A .

П р и м е р. Пусть $A = 1001\ 1101$, образующий полином 11001.

Так как $K = 4$, то $A \cdot (2^K) = 100111010000$. Выполнение операции O расчета циклического кода показано на рис. 3.2.

Таблица 3.1

Число	Циклический код	Число	Циклический код
0	0000000000.	13	0010001111
1	0000001011	14	0010011010
2	0000010110	15	0010100101
3	0000100001	16	0011000110
5	0000110111	18	0011000110
6	0001000010	19	0011010001
.....			

Положительными свойствами циклических кодов являются малая вероятность необнаружения ошибки и сравнительно небольшое число избыточных разрядов.

Операция O в передатчике:

$$\begin{array}{r}
 1001\ 1101\ 0000 \quad | \quad 11001 \\
 \underline{1100\ 1} \\
 101\ 01 \\
 \underline{110\ 01} \\
 11\ 000 \\
 \underline{11\ 001} \\
 11\ 000 \\
 \underline{11\ 001} \\
 10 \Rightarrow \text{CRC}
 \end{array}$$

Операция O в приемнике:

$$\begin{array}{r}
 1001\ 1101\ 0010 \quad | \quad 11001 \\
 \underline{1100\ 1} \quad \text{CRC} \\
 101\ 01 \\
 \underline{110\ 01} \\
 11\ 000 \\
 \underline{11\ 001} \\
 11\ 001 \\
 \underline{11\ 001} \\
 00 \Rightarrow \text{ошибки нет}
 \end{array}$$

Рис. 3.2. Пример получения циклического кода

Общепринятое обозначение образующих полиномов дает следующий пример:

$$g(X) = X^{16} + X^{12} + X^5 + 1,$$

что эквивалентно коду 1 0001 0000 0010 0001. Этот полином используется в протоколе V.42 для кодирования кодовых групп в 240 разрядов с двумя избыточными байтами. В этом протоколе возможен и образующий полином для четырех избыточных байтов

$$g(X) = X^{32} + X^{26} + X^{23} + X^{22} + X^{16} + X^{12} + X^{11} + X^{10} + X^8 + X^7 + X^5 + X^4 + X^2 + 1.$$

??? СДЕЛАТЬ 1 ВОПРОС

Алгоритмы сжатия информации.

Все методы сжатия можно разделить на две большие группы: сжатие с потерями и сжатие без потерь. **Сжатие без потерь применяется в тех случаях, когда информацию нужно восстановить с точностью до бита.** Такой подход является единственно возможным при сжатии, например, текстовых данных.

В некоторых случаях, однако, не требуется точного восстановления информации и допускается использовать алгоритмы, реализующие сжатие с потерями, которое, в отличие от сжатия без потерь, обычно проще реализуется и обеспечивает более высокую степень архивации.

Сжатие с потерями

Лучшие степени сжатия, при сохранении «достаточно хорошего» качества данных. Применяются в основном для сжатия аналоговых данных — звука, изображений. В таких случаях распакованный файл может очень сильно отличаться от оригинала на уровне сравнения «бит в бит», но практически неотличим для человеческого уха или глаза в большинстве практических применений.

Сжатие без потерь

Данные восстанавливаются с точностью до бита, что не приводит к каким-либо потерям информации. Однако,

сжатие без потерь показывает обычно худшие степени сжатия.

Итак, перейдем к рассмотрению алгоритмов сжатия без потерь.

Универсальные методы сжатия без потерь

В общем случае можно выделить три базовых варианта, на которых строятся алгоритмы сжатия.

Первая группа методов – преобразование потока. Это предполагает описание новых поступающих несжатых данных через уже обработанные. При этом не вычисляется никаких вероятностей, кодирование символов осуществляется только на основе тех данных, которые уже были обработаны, как например в ***LZ – методах*** (названных по имени Абрахама Лемпеля и Якоба Зива). **В этом случае, второе и дальнейшие вхождения некой подстроки, уже известной кодировщику, заменяются ссылками на ее первое вхождение.**

Вторая группа методов – это статистические методы сжатия. В свою очередь, эти методы делятся на адаптивные (или поточные), и блочные.

*В первом (адаптивном) варианте, вычисление вероятностей для новых данных происходит по данным, уже обработанным при кодировании. К этим методам относятся **адаптивные варианты алгоритмов Хаффмана и Шеннона-Фано.***

Во втором (блочном) случае, статистика каждого блока данных высчитывается отдельно, и добавляется к самому сжатому блоку. Сюда можно отнести **статические варианты методов Хаффмана, Шеннона-Фано, и арифметического кодирования.**

Третья группа методов – это так называемые методы **преобразования блока**. Входящие данные разбиваются на блоки, которые затем трансформируются целиком. При этом некоторые методы, особенно основанные на перестановке блоков, могут не приводить к существенному (или вообще какому-либо) уменьшению объема данных. Однако после подобной обработки, структура данных значительно улучшается, и последующее сжатие другими алгоритмами проходит более успешно и быстро.

Общие принципы, на которых основано сжатие данных

Все методы сжатия данных основаны на простом логическом принципе. Если представить, что наиболее часто встречающиеся элементы закодированы более короткими кодами, а реже встречающиеся – более длинными, то для хранения всех данных потребуется меньше места, чем если бы все элементы представлялись кодами одинаковой длины.

Точная взаимосвязь между частотами появления элементов, и оптимальными длинами кодов описана в так называемой теореме Шеннона о источнике шифрования (Shannon's source coding theorem), которая определяет

предел максимального сжатия без потерь и энтропию Шеннона.

Простейшие алгоритмы сжатия: RLE и LZ77

<https://habr.com/ru/post/141827/> (подробнее про RLE и LZ77)

Алгоритм RLE (Кодирование длин серий (англ. *run-length encoding, RLE*)) является, наверное, самым простейшим из всех: суть его заключается в кодировании повторов. Другими словами, мы берём последовательности одинаковых элементов, и «схлопываем» их в пары «количество/значение». Например, строка вида «AAAAAAAAABCCCC» может быть преобразована в запись вроде «8×A, B, 4×C». Это, в общем-то, всё, что надо знать об алгоритме.

LZ77 — один из наиболее простых и известных алгоритмов в семействе LZ. Назван так в честь своих создателей: Абрахама Лемпеля (Abraham Lempel) и Якоба Зива (Jacob Ziv). Цифры 77 в названии означают 1977 год, в котором была опубликована статья с описанием этого алгоритма.

Основная идея заключается в том, чтобы кодировать одинаковые последовательности элементов. Т.е., если во входных данных какая-то цепочка элементов встречается более одного раза, то все последующие её вхождения можно заменить «ссылками» на её первый экземпляр.

LZ77 использует скользящее по сообщению окно. Допустим,

на текущей итерации окно зафиксировано. С правой стороны окна наращиваем подстроку, пока она есть в строке <скользящее окно + наращиваемая строка> и начинается в скользящем окне. Назовем наращиваемую строку буфером. После наращивания алгоритм выдает код состоящий из трех элементов:

- смещение в окне;
- длина буфера;
- последний символ буфера.

В конце итерации алгоритм сдвигает окно на длину равную длине буфера+1.

Пример kabababababz

Содержимое окна	Содержимое буфера	КОД
kabababababz	k	<0,0,k>
kabababababz	a	<0,0,a>
kabababababz	b	<0,0,b>
kabababababz	aba	<2,2,a>
kabababababz	bababz	<2,5,z>

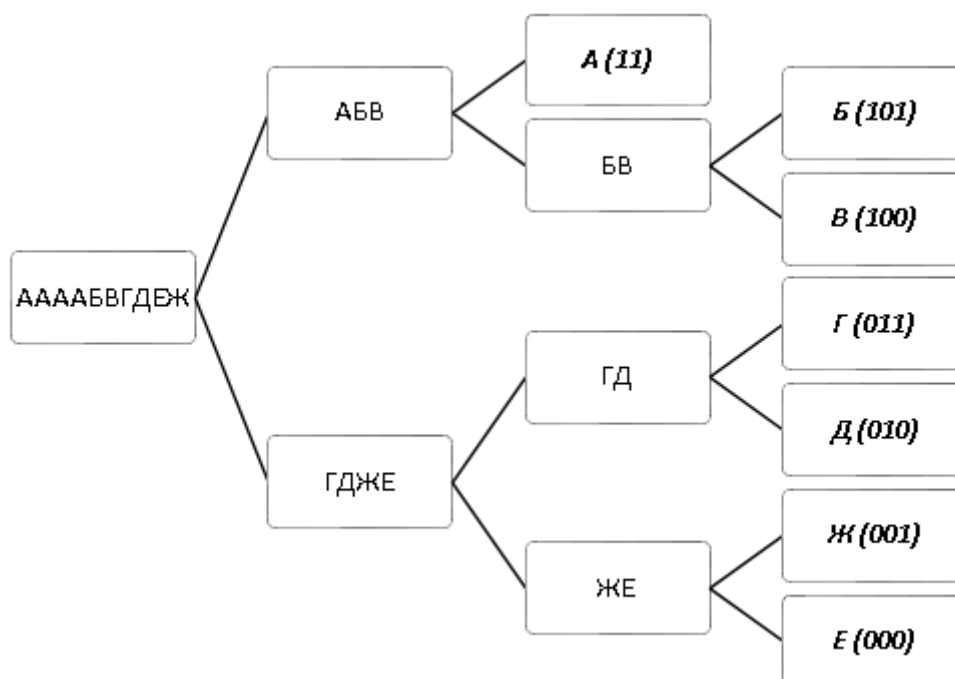
Кодирование с помощью деревьев Хаффмана

Алгоритм кодирования Хаффмана, разработанный через несколько лет после алгоритма Шеннона-Фано, тоже обладает свойством префиксности, а, кроме того,

доказанной минимальной избыточностью, именно этим обусловлено его крайне широкое распространение. Для получения кодов Хаффмана используют следующий алгоритм:

1. Все символы алфавита представляются в виде свободных узлов, при этом вес узла пропорционален частоте символа в сообщении;
2. Из множества свободных узлов выбираются два узла с минимальным весом и создаётся новый (родительский) узел с весом, равным сумме весов выбранных узлов;
3. Выбранные узлы удаляются из списка свободных, а созданный на их основе родительский узел добавляется в этот список;
4. Шаги 2-3 повторяются до тех пор, пока в списке свободных больше одного узла;
5. На основе построенного дерева каждому символу алфавита присваивается префиксный код;
6. Сообщение кодируется полученными кодами.

Рассмотрим тот же пример, что и в случае с алгоритмом Шеннона-Фано. Дерево Хаффмана и коды, полученные для сообщения «ААААБВГДЕЖ», представлены на Рис. 2:



Легко подсчитать, что объём закодированного сообщения составит 26 бит, что меньше, чем в алгоритме Шеннона-Фано. Отдельно стоит отметить, что ввиду популярности алгоритма Хаффмана на данный момент существует множество вариантов кодирования Хаффмана, в том числе и адаптивное кодирование, которое не требует передачи частот символов.

Среди недостатков алгоритма Хаффмана значительную часть составляют проблемы, связанные со сложностью реализации. Использование для хранения частот символов вещественных переменных сопряжено с потерей точности, поэтому на практике часто используют целочисленные переменные, но, т.к. вес родительских узлов постоянно растёт, рано или поздно возникает переполнение. Т.о., несмотря на простоту алгоритма, его корректная реализация до сих пор может вызывать некоторые затруднения, особенно для больших алфавитов.

ПОЛНЫЙ ОТВЕТЫ НА ВОПРОСЫ ГОСУДАРСТВЕННОГО ЭКЗАМЕНА

**по дисциплинам: Информатика,
Информационные технологии, Анализ
данных, Моделирование, Сети и
телекоммуникации**

ИНФОРМАТИКА, ИНФОРМАЦИОННЫЕ ТЕХНОЛОГИИ

**1. Понятие информации; виды информации;
математические основы информатики;
подходы к оценке количества информации;
структура и закономерности протекания
информационных процессов**

Понятие информации

В соответствии с Федеральным законом № 149-ФЗ «Об информации, информационных технологиях и о защите информации», информация — это сведения (сообщения, данные) независимо от формы их представления.

В научном контексте информация рассматривается как:

- **Философский аспект:** мера организации материи, отражение реальности
- **Кибернетический аспект:** сведения, уменьшающие неопределенность знаний (по Н. Винеру)
- **Технический аспект:** последовательность сигналов, символов, подлежащих обработке

Виды информации (классификация по различным признакам):

Критерий классификации	Виды информации
По способу восприятия	Визуальная, аудиальная, тактильная, обонятельная, вкусовая
По форме представления	Текстовая, числовая, графическая, звуковая, видео
По области возникновения	Элементарная (неживая природа), биологическая, социальная

По способу передачи	Дискретная, аналоговая
По доступу	Открытая, ограниченного доступа, конфиденциальная, государственная тайна

Математические основы информатики

Основу составляют:

- **Алгебра логики (булева алгебра):** операции И ($\wedge$), ИЛИ ($\vee$), НЕ ($\neg$), импликация ($\rightarrow$)
- **Теория множеств:** операции объединения, пересечения, дополнения
- **Теория алгоритмов:** формализация понятия алгоритма (машина Тьюринга, нормальные алгоритмы Маркова)

- **Теория информации:** количественная оценка информации

Подходы к оценке количества информации

1. Содержательный подход (формула

Хартли) Применяется для равновероятных событий. Если имеется равновероятных сообщений, то количество информации в одном из них:

2. *Пример:* При бросании монеты (): бит.

3. Вероятностный подход (формула

Шеннона) Для событий с различными вероятностями :

4. где — вероятность -го события, .

5. **Алфавитный подход** Количество

информации в сообщении длиной n символов алфавита мощностью M :

Структура и закономерности информационных процессов

Информационный процесс — совокупность последовательных действий над информацией.

Основные закономерности:

- **Закон возрастания объёма информации:**
- **Закон старения информации:** ценность информации убывает со временем по экспоненте
- **Принцип двойственности:** информация существует только в единстве с носителем

- **Принцип адекватности:** полнота отражения объективной реальности в информации

Схема информационного процесса:

```
graph LR; A[Источник] --> B[Кодирование]; B --> C[Передача]; C --> D[Приём]; D --> E[Декодирование]; E --> F[Потребитель]; G[Помехи/Шум] --> C;
```

2. Общая характеристика процессов сбора, передачи, обработки и накопления информации; технические и программные средства реализации информационных процессов

Процессы работы с информацией

Процесс	Характеристика	Технические средства	Программные средства
Сбор	Получение данных из внешних источников	Датчики, сканеры, клавиатура, микрофоны, камеры	Драйверы устройств, ПО ввода, API сенсоров
Передача	Перемещение данных между узлами	Сетевые адаптеры, маршрутизаторы,	Сетевые протоколы (TCP/IP), FTP, HTTP

		ОПТОВОЛОКН О	
Обработка	Преобразование данных для получения результата	Процессор (CPU), сопроцессоры, GPU	ОС, компиляторы, СУБД, прикладное ПО
Накопление	Долговременное хранение информации	Жёсткие диски, SSD, облачные хранилища, ленты	Файловые системы, СУБД, системы резервного о

			копирова ния
--	--	--	-----------------

Закономерности реализации процессов

- **Принцип иерархичности:** процессы организованы по уровням (физический → логический → прикладной)
- **Принцип параллелизма:** возможность одновременного выполнения нескольких процессов
- **Принцип надёжности:** обеспечение достоверности через контроль чётности, CRC, избыточное кодирование

Пример архитектуры реализации (по ГОСТ 34.003-90):



3. Содержание информационной технологии как составной части информатики; общая классификация видов информационных технологий и их реализация в промышленности, административном управлении, обучении

Информационная технология (ИТ) — по ГОСТ Р 52653-2006: «Процессы, методы и средства сбора, обработки, хранения, передачи и

использования информации для снижения трудоёмкости, повышения надёжности и оперативности принятия решений».

Классификация ИТ по сферам применения (по методике Минцифры России):

Промышленные ИТ

- **CAD/CAM/CAE-системы:** автоматизированное проектирование и производство (Компас-3D, T-FLEX)
- **ERP-системы:** планирование ресурсов предприятия (1С:Предприятие, Галактика)
- **SCADA-системы:** диспетчеризация технологических процессов (Trace Mode, MasterSCADA)

- **MES-системы:** управление

производственными исполнениями

Административные ИТ

- **СЭД (системы электронного**

документооборота): Дело, ЕВФРАТ-

Документационное управление

- **ГИС (геоинформационные системы):**

КосмосАгро, ИнГео

- **Системы поддержки принятия решений**

(СППР): на основе экспертных знаний и

аналитики

- **Порталы госуслуг:** интеграция ведомственных

информационных систем

Образовательные ИТ

- **Электронные образовательные среды (ЭОС):**

МЭШ, Российская электронная школа

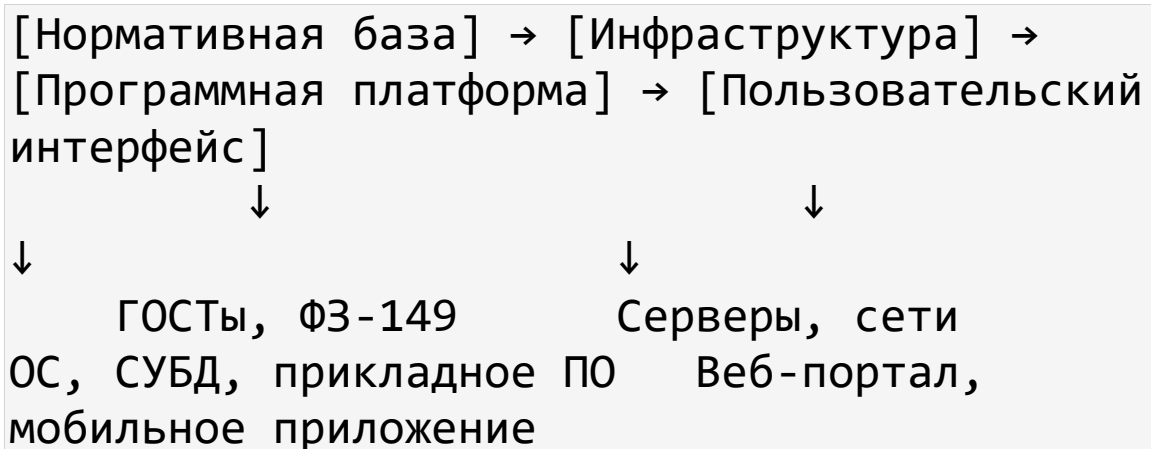
- **СДО (системы дистанционного обучения):**

Moodle, iSpring, WebTutor

- **Интеллектуальные обучающие системы:**

адаптивное обучение на основе ИИ

Схема реализации ИТ в организации:



4. Роль развития средств вычислительной техники и методологии компьютерной

обработки информации в эволюции информационных технологий

Этапы развития СВТ и соответствующие ИТ (по классификации Академии наук РФ):

Поколение ЭВМ	Период	Элементная база	Методология обработки	Развитие ИТ
I	1940 - 1950-е	Электронные лампы	Машинные коды, ассемблер	Пакетная обработка, перфокарты

II	1950 - 1960 -е	Транзисторы	Алгоритмические языки (Fortran, COBOL)	Разделение времени, первые СУБД
III	1960 - 1970 -е	Интегральные схемы	Структурное программирование	Сетевые технологии, мини-ЭВМ
IV	1970 -	БИС/СБИ С	ООП, реляционные БД	Персональные компьютеры

	1990 -е			еры, локальн ые сети
V	1990 -н.в.	Паралле льные архитект уры, ИИ- ускорите ли	Распределё нные вычисления, облачные технологии	Интернет вещей, большие данные, ИИ

Ключевые методологические сдвиги:

**6. От последовательной к параллельной
обработке — закон Амдала:**

7. От централизованной к распределённой

архитектуре — модель клиент-сервер →

микросервисы

8. От детерминированных к вероятностным

алгоритмам — машинное обучение,

нейросети

Влияние на ИТ:

- Рост производительности по закону Мура (удвоение каждые 2 года) позволил перейти от расчётов к интеллектуальной обработке
- Миниатюризация → мобильность → повсеместные вычисления (ubiquitous computing)

- Развитие сетевых технологий → глобализация информационных процессов
-
-

5. Основные этапы решения задач на ЭВМ; критерии качества программы; диалоговые программы; дружелюбность, жизненный цикл программы

Этапы решения задач на ЭВМ (по ГОСТ 19.003-80 ЕСПД):

1. Постановка задачи → 2. Математическое моделирование → 3. Разработка алгоритма
↓
4. Программирование → 5. Отладка и тестирование → 6. Эксплуатация и сопровождение

Критерии качества программы (по ГОСТ Р ИСО/МЭК 9126-2002):

Характеристика	Показатели
Функциональность	Полнота, корректность, соответствие требованиям
Надёжность	Отказоустойчивость, восстанавливаемость, соответствие времени
Удобство использования	Обучаемость, понятность, оперируемость, привлекательность
Эффективность	Использование ресурсов (память, процессорное

	время), пропускная способность
Сопровождаемость	Анализируемость, изменяемость, стабильность, тестируемость
Портируемость	Адаптируемость, устанавливаемость, совместимость

Диалоговые программы и дружелюбность интерфейса

Диалоговая программа — ПО, обеспечивающее
интерактивное взаимодействие с пользователем

в реальном времени. Принципы

дружественности (по ГОСТ Р 50886-96):

- **Естественность:** использование терминологии предметной области
- **Последовательность:** логичная структура диалога
- **Гибкость:** возможность выбора темпа и способа ввода
- **Наглядность:** визуализация состояния системы
- **Прощение ошибок:** возможность отмены действий, понятные сообщения об ошибках

Жизненный цикл программы (по ГОСТ Р

ИСО/МЭК 12207-2008):

- | |
|--|
| <ol style="list-style-type: none">1. Анализ требований2. Проектирование архитектуры |
|--|

- | |
|--|
| <ol style="list-style-type: none">3. Реализация (кодирование)4. Тестирование (модульное, интеграц.)5. Внедрение и обучение6. Эксплуатация и техническая поддержка7. Модернизация или вывод из эксплуатации |
|--|

6. Записи; файлы; динамические структуры данных; списки. Базовые алгоритмы сортировки данных

Записи и файлы

- **Запись (struct)** — именованная совокупность полей различных типов:
- **Файл** — именованная область внешней памяти. Классификация:

- По организации: последовательные, прямого доступа, индексированные
- По типу доступа: текстовые, бинарные, типизированные

Динамические структуры данных

- **Список** — последовательность элементов, связанных указателями:
- **Операции:** включение, удаление, поиск, обход.
Сложность: для вставки в начало, для поиска.
- **Стек (LIFO), Очередь (FIFO), Дерево, Граф** — специализированные структуры с определёнными правилами доступа.

Базовые алгоритмы сортировки

Алгоритм	Сложно сть (средняя)	Сложно сть (худшая)	Память	Устойчивость
Пузырьковая				Да
Вставками				Да
Выбором				Нет
Быстрая (QuickSort)				Нет
Слиянием (MergeSort)				Да

Пирамидальная (HeapSort)				Нет
-------------------------------------	--	--	--	-----

Пример алгоритма быстрой сортировки

(псевдокод):

```

procedure QuickSort(A, left, right):
    if left < right then
        pivot := Partition(A, left, right)
        QuickSort(A, left, pivot-1)
        QuickSort(A, pivot+1, right)

function Partition(A, left, right):
    pivot := A[right]
    i := left - 1
    for j := left to right-1 do
        if A[j] ≤ pivot then
            i := i + 1
            swap A[i] and A[j]
    swap A[i+1] and A[right]
    return i+1

```

7. Процедурное, логическое, функциональное и объектно-визуально-ориентированное программирование

Парадигмы программирования (по классификации В.П. Иванова, МГУ):

Процедурное программирование

- **Основа:** последовательность вызовов процедур/функций
- **Языки:** Pascal, C, Fortran
- **Принципы:** нисходящее проектирование, модульность, минимизация глобальных переменных
- **Пример** (Pascal):

Логическое программирование

- **Основа:** формальная логика предикатов
первого порядка
- **Языки:** Prolog, Mercury
- **Принципы:** декларативное описание фактов и правил, механизм унификации и поиска с возвратом
- **Пример (Prolog):**

Функциональное программирование

- **Основа:** λ -исчисление, функции как объекты первого класса
- **Языки:** Haskell, Lisp, F#, Scala
- **Принципы:** отсутствие побочных эффектов, рекурсия вместо циклов, ленивые вычисления
- **Пример (Haskell):**

Объектно-визуально-ориентированное программирование

- **Основа:** объединение ООП с визуальным конструированием интерфейса
- **Языки/среды:** Delphi, C# (Windows Forms), Python (Tkinter)
- **Принципы:**
 - Визуальные компоненты как объекты с событиями
 - Обработчики событий как методы классов
 - RAD (Rapid Application Development) — быстрая разработка
- **Пример (Delphi):**

Сравнительная таблица:

Крите рий	Процеду рное	Логи ческо е	Функцион альное	ОВ-ООП
Едини ца модул ьност и	Процеду ра	Пред икат	Функция	Класс/Ко мпонент
Поток управ ления	Последо вательны й	Поис к с возв рато м	Рекурсия/к омпозици я	Событийн ый

Состо яние	Глобальн ые перемен ные	База факт ов	Неизменяе мые данные	Инкапсул ированны е поля
Прим енени е	Системн ое ПО, алгоритм ы	ИИ, экспе ртны е систе мы	Мат. моделиро вание, анализ данных	Десктоп/ мобильн ые приложен ия

8. Тестирование и отладка; документирование и стандартизация; проектирование программного обеспечения

Тестирование и отладка (по ГОСТ Р 56907-2016)

- **Тестирование** — процесс выполнения программы с целью обнаружения ошибок:
 - *Модульное* (юнит-тесты) — проверка отдельных функций
 - *Интеграционное* — проверка взаимодействия модулей
 - *Системное* — проверка соответствия требованиям
 - *Приёмочное* — валидация пользователем
- **Методы тестирования:**

- **Белый ящик** (white-box): анализ внутренней структуры, покрытие кода
- **Чёрный ящик** (black-box): тестирование по спецификации без знания реализации
- **Серый ящик**: комбинация подходов
- **Отладка** — процесс локализации и устранения обнаруженных ошибок:
 - Инструменты: отладчики (gdb, Visual Studio Debugger), логирование, статический анализ

Документирование и стандартизация (по ЕСПД — ГОСТ 19.xxx)

Обязательные документы (по ГОСТ 19.201-78):

9. Техническое задание (ТЗ) — ГОСТ 34.602-89

10. Пояснительная записка — описание

архитектуры и алгоритмов

11. Руководство программиста — описание

интерфейсов модулей

12. Руководство оператора — инструкции по

эксплуатации

13. Описание применения — сценарии

использования

Стандарты проектирования:

● **ГОСТ Р ИСО/МЭК 12207-2008** — процессы

жизненного цикла ПО

● **ГОСТ Р 53622-2009** — стадии и этапы ЖЦ ИВС

● **ГОСТ 34.201-89** — виды, комплектность и

обозначение документов при создании АС

Проектирование программного обеспечения

Методологии:

- **Каскадная (водопадная)** — последовательные этапы, подходит для стабильных требований
- **Итеративная (RUP)** — циклическая разработка с прототипированием
- **Гибкие (Agile, Scrum)** — инкрементальная поставка, адаптация к изменениям

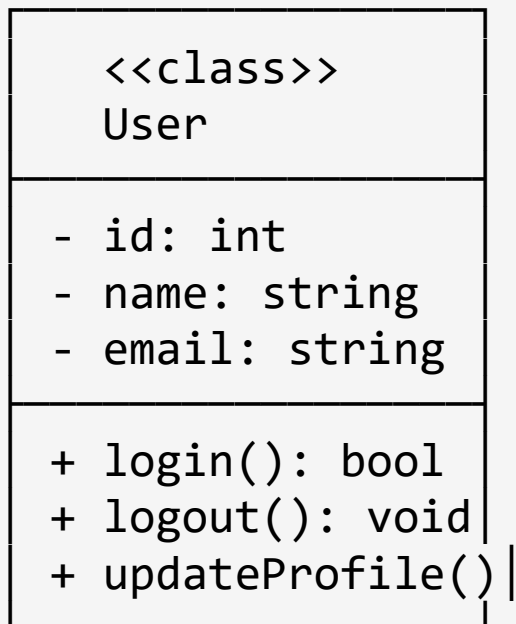
Моделирование (по ГОСТ 34.201-89):

- **IDEF0** — функциональное моделирование (блоки + стрелки)
- **DFD** — диаграммы потоков данных

- **UML** — унифицированный язык

моделирования (классы, последовательности, состояния)

Пример UML-класса:



9. Объектно-ориентированное проектирование и программирование.

Объекты и классы. Инкапсуляция, наследование, полиморфизм

Объекты и классы

- **Класс** — абстрактный тип данных, описывающий структуру и поведение объектов:
- **Объект** — экземпляр класса с конкретными значениями атрибутов:

Три кита ООП

1. Инкапсуляция

- **Суть:** объединение данных и методов в единый модуль с сокрытием внутренней реализации
- **Механизмы:** модификаторы доступа (`private`, `protected`, `public`)

- **Преимущества:** защита данных, упрощение интерфейса, снижение связности

```
class BankAccount {  
private:  
    double balance; // скрыто от внешнего  
доступа  
public:  
    void deposit(double amount) {  
        if (amount > 0) balance += amount;  
    }  
    double getBalance() const { return  
balance; }  
};
```

2. Наследование

- **Суть:** возможность создания нового класса на основе существующего с сохранением его свойств

- **Типы:**

- *Одиночное* (один базовый класс)

- *Множественное* (несколько базовых классов)

- в C++, Python

- *Иерархическое* (дерево наследования)

```
class Car : public Vehicle { // Car
наследует Vehicle
private:
    int numDoors;
public:
    void openTrunk() { /* ... */ }
    // Переопределение метода
    void move() override { cout <<
"Driving..."; }
};
```

3. Полиморфизм

- **Суть:** способность объектов с одинаковым интерфейсом иметь различную реализацию

- **Виды:**

- *Во время компиляции* (перегрузка функций, шаблоны)

- *Во время выполнения* (виртуальные функции, динамическое связывание)

```
Vehicle* vehicles[2];
vehicles[0] = new Vehicle("Boat", 30);
vehicles[1] = new Car("Toyota", 120);

for (int i = 0; i < 2; i++)
    vehicles[i]->move(); // Вызовется
    соответствующая реализация
// Вывод: "Moving..." и "Driving..."
```

Принципы проектирования (SOLID):

14. **Single Responsibility** — один класс = одна
ответственность

15. **Open/Closed** — открыт для расширения,
закрит для изменения

16. **Liskov Substitution** — подтип должен
заменять базовый тип

17. Interface Segregation — много
специализированных интерфейсов лучше
одного общего

18. Dependency Inversion — зависимость от
абстракций, а не от реализаций

**10. Логическая организация баз данных (БД);
объекты и атрибуты; схемы и подсхемы.**

Эволюция концепции БД

Логическая организация БД

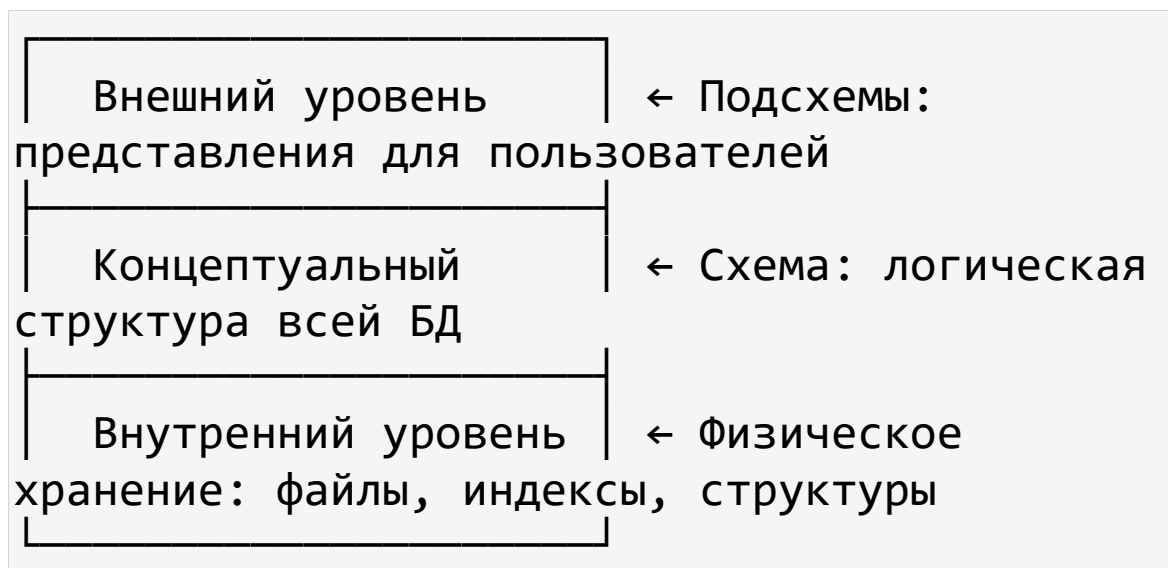
- **Объект (сущность)** — реальный или
абстрактный предмет, информация о котором
хранится (Студент, Курс, Заказ)

- **Атрибут** — свойство сущности (Студент.ФИО, Студент.ДатаРождения)

- **Связь** — ассоциация между сущностями (Студент *сдаёт* Экзамен)

Уровни абстракции данных (по стандарту

ANSI/SPARC, адаптирован в ГОСТ Р 53622-2009):



- **Схема БД** — формальное описание структуры данных на концептуальном уровне (таблицы, поля, связи, ограничения)

- **Подсхема** — часть схемы, описывающая данные, доступные конкретному приложению или пользователю

Эволюция концепции БД:

Период	Модель	Характеристика	Примеры
1960-е	Иерархическая	Древовидная структура, один родитель	IMS (IBM)
1970-е	Сетевая	Граф связей, множественные родители	IDMS, Adabas

1970- 1990- е	Реляционная	Таблицы, теория множеств, реляционная алгебра	Oracle, PostgreSQL, 1C
1990- 2000- е	Объектно- реляционная	Расширение реляционной модели объектами	PostgreSQL, Oracle
2000- н.в.	NoSQL	Документны е, ключ- значение,	MongoDB, Redis, Neo4j

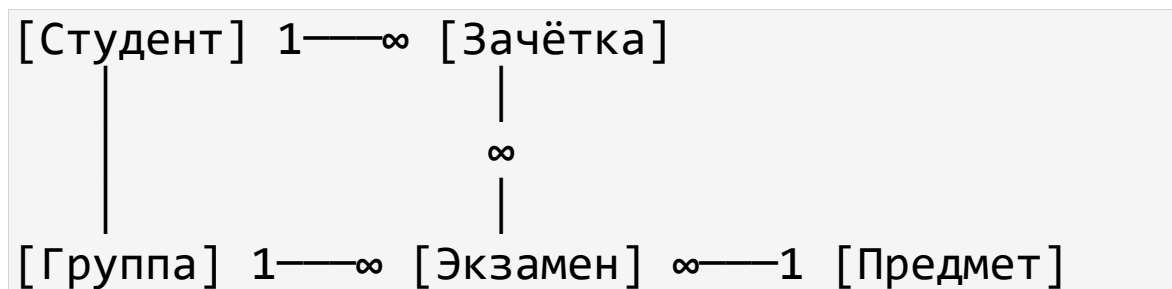
		графовые, колоночные	
2010- н.в.	Новые распределён ные	Шардирован ие, репликация, CAP-теорема	Cassandra, Cockroach DB

Реляционная модель (Э. Кодд, 1970) — основа современных СУБД:

- **Отношение** = таблица
- **Кортеж** = строка
- **Атрибут** = столбец
- **Первичный ключ** — уникальный идентификатор кортежа

- **Внешний ключ** — ссылка на первичный ключ другой таблицы

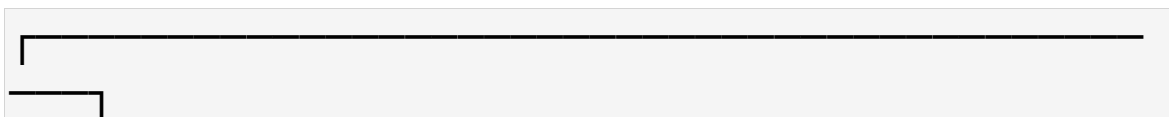
Пример схемы в виде диаграммы “сущность-связь” :



11. Типовые фазы преобразования информации. Модели базовой информационной технологии

Типовые фазы преобразования информации

(по ГОСТ Р 50886-96):



1. Регистрация: фиксация фактов в системе

2. Сбор: агрегация данных из источников

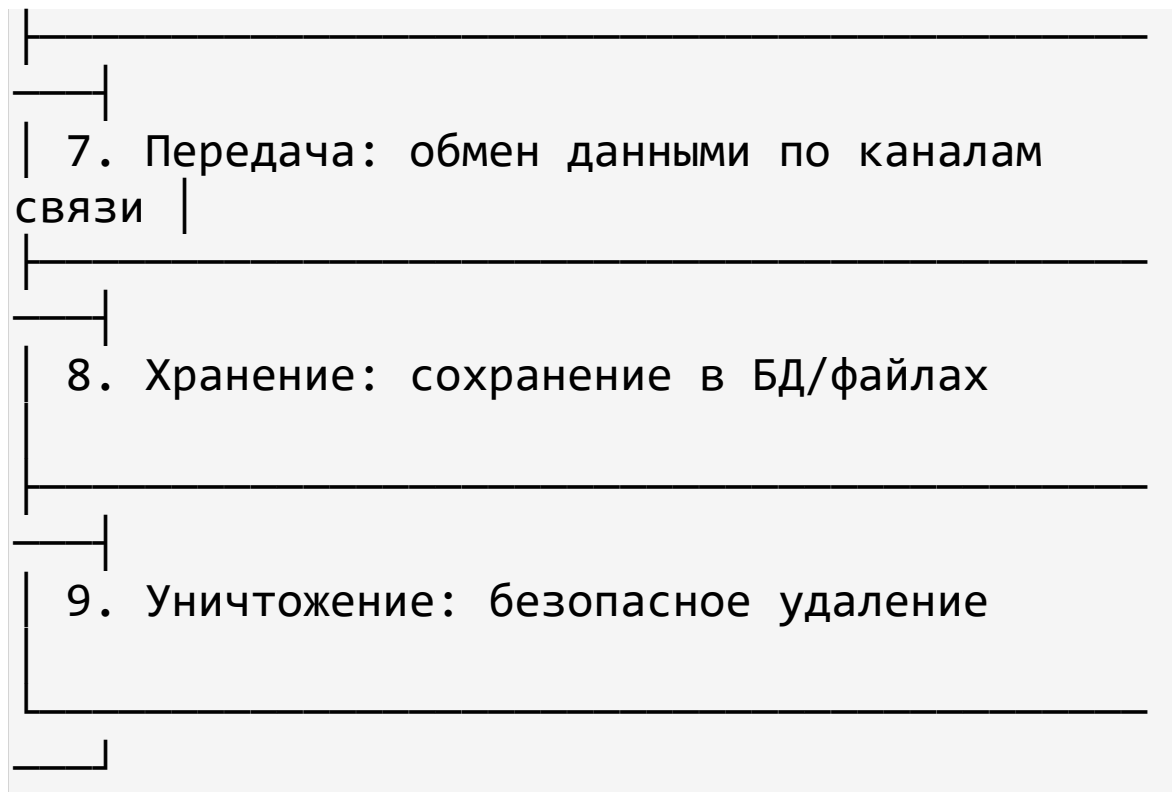
3. Подготовка: очистка, нормализация,
кодирование, проверка целостности

4. Ввод: занесение в ИС (ручной/автомат.)

5. Обработка:

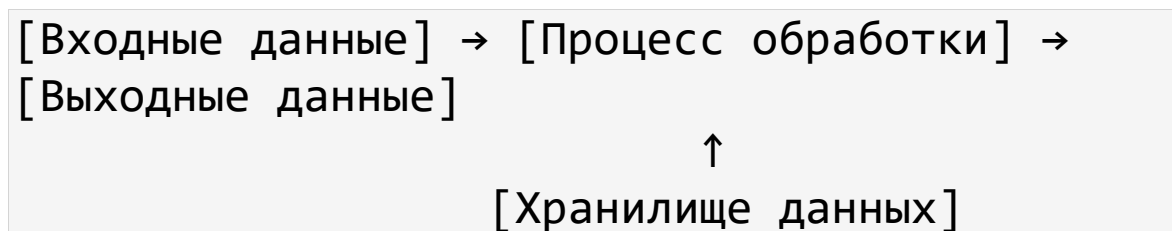
- Вычисления, агрегация, фильтрация
 - Логический анализ, принятие решений
 - Генерация отчётов, визуализация
-

6. Вывод: представление результата
пользователю или внешней системе



Модели базовой информационной технологии

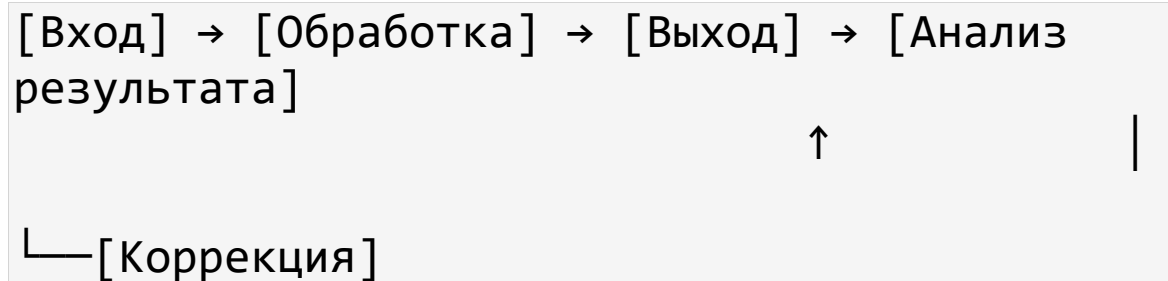
1. Модель “ввод-обработка-вывод” (IPO)



Применение: простые прикладные задачи,
расчётные системы

2. Модель с обратной связью

(кибернетическая)



Применение: системы управления, АСУ ТП,

адаптивные алгоритмы

3. Многоуровневая модель (по ГОСТ 34.003-90)



4. Модель жизненного цикла информации (по ФЗ-149)

Создание → Обработка → Хранение →
Использование → Распространение →
Уничтожение

Регулируется: требованиями к защите, сроками хранения, правами доступа

Ключевые показатели эффективности ИТ-процессов:

- **Время отклика**
 - **Пропускная способность**
 - **Надёжность** , где — интенсивность отказов
 - **Коэффициент использования**
-
-

АНАЛИЗ ДАННЫХ, ПРОЕКТИРОВАНИЕ И ЭКСПЛУАТАЦИЯ ИИиЭС

2. Типы алгоритмов кластеризации:

Иерархические алгоритмы, Итеративные алгоритмы. Плотностные алгоритмы.

Модельные алгоритмы. Самоорганизующаяся карта Кохонена. Настройка кластеров

Кластеризация — задача разбиения множества объектов на группы (кластеры) так, чтобы объекты внутри кластера были «похожи», а между кластерами — «различны» (без учителя).

Иерархические алгоритмы

- **Принцип:** построение древовидной структуры (дендрограммы)
- **Типы:**

- *Агломеративные* (снизу вверх): каждый объект — отдельный кластер → последовательное объединение ближайших
- *Дивизивные* (сверху вниз): все объекты в одном кластере → рекурсивное разделение

- **Меры близости:**

- Одиночная связь:
- Полная связь:
- Средняя связь:

- **Сложность:** времени, памяти

- **Применение:** таксономия, биоинформатика, анализ документов

Итеративные алгоритмы (на примере k-средних)

- **Алгоритм:**

- a. Задать число кластеров ,
 инициализировать центроиды

- b. **Повторять до сходимости:**

- Назначение:

- Обновление:

- **Целевая функция** (минимизируется):

- **Преимущества:** простота, скорость

- **Недостатки:** чувствительность к начальным центроидам, требует задания , плохо работает с кластерами сложной формы

Плотностные алгоритмы (DBSCAN)

- **Принцип:** кластер — область высокой плотности, отделённая от областей низкой плотности
- **Параметры:**
 - — радиус окрестности
 - — минимальное число точек в -окрестности
- **Типы точек:**
 - *Ядровые:*
 - *Граничные:* , но достижимы из ядровой
 - *Шум:* не ядровые и не граничные
- **Преимущества:** находит кластеры произвольной формы, устойчив к выбросам
- **Сложность:** с индексацией, без

Модельные алгоритмы (GMM — Gaussian Mixture Models)

- **Принцип:** данные генерируются смесью вероятностных распределений (обычно гауссовых)
- **Модель:** , где
- **Оценка параметров:** алгоритм EM (Expectation-Maximization):
 - **Е-шаг:** вычисление ответственностей
 - **М-шаг:** обновление параметров:
- **Преимущества:** вероятностная интерпретация, мягкое назначение кластеров
- **Недостатки:** чувствительность к инициализации, локальные максимумы

Самоорганизующаяся карта Кохонена (SOM)

- **Архитектура:** двумерная решётка нейронов, каждый с вектором весов
- **Алгоритм обучения** (конкурентное обучение):
 - c. Для входного вектора найти “победителя” — нейрон с минимальным расстоянием:
 - d. Обновить веса победителя и его соседей:
 - — скорость обучения (убывает со временем)
 - — функция окрестности
- **Результат:** топологически упорядоченное отображение многомерных данных на 2D-плоскость

- **Применение:** визуализация, снижение размерности, кластеризация

Настройка кластеров

- **Определение оптимального :**
 - *Метод локтя (elbow method):* график vs ,
выбор точки изгиба
 - *Силуэтный коэффициент:* , где $\bar{d}_i$ — среднее расстояние внутри кластера, d_{ij} — минимальное среднее расстояние до другого кластера
 - *Индекс Дэвиса-Болдина:* , минимум — лучше
- **Валидация:** внутренние метрики (силуэт, Дэвис-Болдин), внешние (если есть разметка:

Adjusted Rand Index, Normalized Mutual Information)

3. Технико-экономическое обоснование (ТЭО) и техническое задание (ТЗ, ГОСТ 34.602-89), изучение существующей системы

Технико-экономическое обоснование (ТЭО) — документ, оценивающий целесообразность создания ИС с точки зрения затрат и результатов.

Структура ТЭО (по методике

Минэкономразвития):

19. Резюме проекта — цели, ожидаемые результаты, сроки

20. Анализ предметной области — описание
бизнес-процессов, проблем, требований

21. Техническое решение — архитектура,
технологии, интеграция

22. Организационный план — этапы, ресурсы,
команда

23. Финансовый план:

- *Капитальные затраты (CAPEX):* оборудование,
ПО, разработка
- *Операционные затраты (OPEX):* поддержка,
лицензии, персонал
- *Экономический эффект:* прямая экономия,
рост доходов, нематериальные выгоды

24. Оценка рисков — технические,
организационные, рыночные

25. Показатели эффективности:

- *NPV (чистая приведённая стоимость):*
- *IRR (внутренняя норма доходности):* при
- *Срок окупаемости:*

Техническое задание (ТЗ) по ГОСТ 34.602-89

Обязательные разделы:

26. Общие сведения (наименование, заказчик,
разработчик, основания)

27. Назначение и цели создания системы

28. Характеристика объектов автоматизации

29. Требования к системе:

- *Функциональные:* состав функций, сценарии использования
- *К качеству:* надёжность, производительность, безопасность
- *К составу и содержанию работ:* этапы, документация
- *К видам обеспечения:* математическое, информационное, программное, техническое, организационное

30. Состав и содержание работ по созданию системы

31. Порядок контроля и приёмки

32. Требования к составу и содержанию работ по подготовке объекта автоматизации

33. Требования к документированию

34. Источники разработки, порядок
согласования и утверждения

Изучение существующей системы

(предпроектное обследование)

Методы:

- **Интервьюирование** — сбор требований от стейкхолдеров
- **Анализ документации** — регламенты, отчёты, инструкции
- **Наблюдение** — фиксация реальных бизнес-процессов
- **Прототипирование** — быстрая проверка гипотез

Результаты обследования:

- **AS-IS модель** — описание текущего состояния процессов (нотация BPMN):
- **Выявленные проблемы:** дублирование данных, ручные операции, отсутствие контроля версий
- **TO-BE модель** — целевое состояние после автоматизации
- **Перечень требований** — функциональных и нефункциональных

Пример матрицы требований:

Идентификатор	Требование	Приоритет	Источник	Критерий
---------------	------------	-----------	----------	----------

				приёмк и
FR-001	Система должна формиру вать отчёт по продажа м за период	Высок ий	Отдел продаж	Отчёт в формате PDF за 5 сек
NFR-005	Время отклика интерфе йса ≤ 2	Средн ий	Техниче ский отдел	Нагрузо чное

	сек при 100 пользова телях			тестиро вание
--	-------------------------------------	--	--	------------------

4. Стандарты международные и отечественные в области проектирования ИЭС (ГОСТ блоки 34- ЖЦ АС, 19-, ГОСТ Р 53622-2009). Стандарты ГОСТ Р ИСО-МЭК 12207, ГОСТ Р ИСО/МЭК 15288-2005, ГОСТ 34.201-89

Отечественные стандарты (ГОСТ)

Блок ГОСТ 34 — Автоматизированные системы (ЖЦ АС)

- **ГОСТ 34.601-90 — Стадии создания АС:**

- **ГОСТ 34.602-89** — Техническое задание на создание АС (основной документ для заказа)
- **ГОСТ 34.201-89** — Виды, комплектность и обозначение документов при создании АС:
 - *Технический проект* (ПЗ, ПС, РЭ)
 - *Рабочий проект* (чертежи, схемы, инструкции)

Блок ГОСТ 19 — Единая система программной документации (ЕСПД)

- **ГОСТ 19.001-77** — Общие положения
- **ГОСТ 19.201-78** — Техническое задание на разработку программы
- **ГОСТ 19.402-78** — Описание программы
- **ГОСТ 19.505-79** — Руководство системного программиста

ГОСТ Р 53622-2009

«Информационные технологии.

Информационно-вычислительные системы.

Стадии и этапы жизненного цикла, виды и комплектность документов»

- Устанавливает 8 стадий ЖЦ ИВС: от исследования до вывода из эксплуатации
- Определяет 4 вида документов:
организационно-распорядительные,
проектные, эксплуатационные, отчётные

Международные стандарты (адаптированные в РФ)

ГОСТ Р ИСО/МЭК 12207-2008

«Информационная технология. Процессы
жизненного цикла программной продукции»

- **Структура процессов:**
- **Применение:** выбор процессов под
конкретный проект, сертификация по уровню
зрелости

ГОСТ Р ИСО/МЭК 15288-2005

«Информационная технология. Системная
инженерия. Процессы жизненного цикла
систем»

- Расширяет 12207 на системы, включающие
аппаратное обеспечение, ПО, людей,
процедуры
- **Ключевые процессы:**

- *Технические*: определение требований, архитектура, реализация, интеграция, верификация, валидация
- *Управленческие*: планирование, оценка, контроль, принятие решений, управление рисками
- **Применение**: сложные ИС, критически важные системы (авионика, АСУ ТП)

ГОСТ 34.201-89

«Виды, комплектность и обозначение документов при создании автоматизированных систем»

- **Коды документов**:
 - *ТЗ* — Техническое задание

- *ПЗ* — Пояснительная записка
- *ПС* — Программная документация
- *РЭ* — Руководство по эксплуатации
- *ФО* — Формуляр

● **Правила обозначения:** АБВГ .XXXXXX .XXX Д1,

где:

- АБВГ — код организации-разработчика
- XXXXXX .XXX — код системы/подсистемы
- Д1 — код вида документа и номер выпуска

Сопоставление стандартов:

Задача	ГОСТ 34	ГОСТ 19	ГОСТ Р ИСО/МЭК 12207
--------	---------	---------	----------------------------

ЖЦ системы	✓ (стадии)	—	✓ (процессы)
ЖЦ ПО	—	✓ (ЕСПД)	✓ (процессы ПО)
Документа ция	✓ (комплек ты)	✓ (формы)	✓ (артефакты)
Управлени е проектом	Частичн о	—	✓ (полный цикл)
Применен ие	Госзаказ, крупные АС	Программ ная	Междунаро дные

		продукци я	проекты, аутсорсинг
--	--	---------------	------------------------

5. Состав и содержание проектной документации (технический проект). Рабочий проект. Разработка и адаптация программ

Проектная документация по ГОСТ 34.201-89

Технический проект (стадия «Т»)

Назначение: обоснование принятых технических и технологических решений, определение состава, структуры и характеристик создаваемой системы.

**Обязательные документы (шифры по ГОСТ
34.201-89):**

Код документа	Наименование	Содержание
ТЗ	Техническое задание	Требования заказчика, цели, ограничения
ПЗ	Пояснительная записка	Обоснование решений, расчёты, схемы, спецификации
СД	Схема данных	Логическая и физическая

		структура БД, модели ER
СП	Схема программ	Архитектура ПО, модульная структура, интерфейсы
СВ	Схема взаимосвязей	Интеграция с внешними системами, потоки данных
ПТ	Пояснения к техническому проекту	Детализация алгоритмов,

		протоколов, форматов
ИЭ	Инструкция по эксплуатации (проект)	Порядок работы, аварийные процедуры

Структура пояснительной записки (типовая):

1. Введение (цели, задачи, область применения)
2. Характеристика объекта автоматизации
3. Обоснование выбора технических и программных средств
4. Описание функциональной структуры системы
5. Информационное обеспечение (модели данных, словари)
6. Программное обеспечение (архитектура, модули, интерфейсы)
7. Математическое обеспечение (алгоритмы, модели)
8. Организационное обеспечение (роли, регламенты)
9. Оценка эффективности и рисков
10. Заключение и рекомендации

Рабочий проект (стадия «Р»)

Назначение: документация для непосредственной реализации, настройки и ввода системы в эксплуатацию.

Состав документов:

- **Рабочие чертежи** — схемы подключения, компоновки, монтажные планы
- **Программная документация** (по ЕСПД, ГОСТ 19.xxx):
 - *Текст программы* (листинг с комментариями)
 - *Описание программы* (алгоритмы, структуры данных)
 - *Руководство программиста* (интерфейсы модулей, API)

- *Руководство оператора* (пошаговые инструкции)

- **Инструкции по настройке и тестированию**

- **Протоколы приёмо-сдаточных испытаний**

Разработка и адаптация программ

Этапы адаптации типового ПО (по ГОСТ Р 53622-2009):

1. Анализ расхождений: требования ТЗ ↔ возможности типовой системы
2. Проектирование доработок: спецификация изменений, оценка трудоёмкости
3. Реализация: модификация кода, создание новых модулей, настройка параметров
4. Интеграционное тестирование: проверка взаимодействия доработок с базовой системой
5. Регрессионное тестирование: подтверждение сохранения исходной функциональности
6. Документирование изменений: ведение реестра модификаций, обновление руководств

Критерии качества адаптации:

- **Минимальная инвазивность:** изменения не должны нарушать целостность ядра системы
 - **Поддерживаемость:** доработки должны быть документированы и модульны
 - **Совместимость:** сохранение возможности обновления базовой версии ПО
-
-

6. Классические ОС и ОС типа Windows. Состав и структура комплекса технических средств. Особенности технических средств ИЭС ТП. Методы проектирования комплекса технических средств (индивидуальная техника, локальные сети, корпоративные сети, глобальная сеть и Internet)

Классические ОС и ОС типа Windows

Характеристика	Классические ОС (UNIX/Linux)	ОС семейства Windows
Архитектура	Монолитное ядро + модули	Гибридное ядро (микроядро + серверы)
Файловая система	Иерархическая, единое пространство имён (/)	Дисковая (C:, D:), реестр как централизованная БД конфигурации
Управление процессами	fork()/exec(), сигналы,	CreateProcess(), объекты ядра,

	планировщик CFS	приоритеты реального времени
Безопасность	Права rwx, SELinux/AppArmor, capabilities	ACL, Active Directory, UAC, BitLocker
Интерфейс	Преимущественно CLI, GUI как опция (X11/Wayland)	GUI-first, PowerShell для администрирования
Применение в ИЭС	Серверы, встраиваемые системы,	Рабочие станции, офисные АС,

	научные расчёты	корпоративные порталы
--	--------------------	--------------------------

Состав и структура комплекса технических средств (КТС) (по ГОСТ 34.003-90):

Уровень представления • Мониторы, панели оператора, планшеты
Уровень обработки • Серверы приложений, СУБД, шлюзы
Уровень хранения • СХД, RAID-массивы, ленточные библиотеки
Уровень ввода-вывода • Датчики, контроллеры, исполнительные механизмы, сканеры, принтеры
Уровень связи • Коммутаторы, маршрутизаторы, модемы промышленные шлюзы (Modbus, OPC UA)

Особенности технических средств ИЭС ТП

(технологических процессов):

- **Промышленная надёжность:** работа в диапазоне $-40...+70\text{ }^{\circ}\text{C}$, защита IP65, виброустойчивость
- **Детерминизм:** гарантированное время отклика (жёсткое реальное время, RTOS)
- **Интерфейсы:** поддержка промышленных протоколов (Modbus, Profibus, CAN, OPC UA)
- **Резервирование:** горячее резервирование контроллеров, дублирование каналов связи
- **Безопасность:** взрывозащита (маркировка Ex), гальваническая развязка

Методы проектирования КТС

1. Для индивидуальной техники (рабочая станция оператора)

- **Расчёт производительности:** , где — коэффициент одновременности
- **Выбор конфигурации:** баланс между стоимостью и запасом на 3–5 лет (правило 30% резерва)
- **Эргономика:** соответствие СанПиН 2.2.2/2.4.1340-03 (расстояние до монитора, освещённость)

2. Для локальных сетей (ЛВС предприятия)

- **Топология:** иерархическая звезда (ядро → распределение → доступ)
- **Расчёт пропускной способности:** , где

- **Сегментация:** VLAN по функциональному признаку (ОУ, АСУ ТП, офис), межсегментная фильтрация
- **Отказоустойчивость:** протоколы STP/RSTP, агрегация каналов (LACP)

3. Для корпоративных сетей

- **Архитектура:** трёхуровневая (кампусная) с выделенным ЦОД
- **Маршрутизация:** OSPF/BGP для динамической адаптации к изменениям топологии
- **Безопасность:** зональная модель (DMZ, внутренняя сеть, сегмент АСУ ТП), NGFW, SIEM
- **Управление:** централизованное (SDN), мониторинг через SNMP/NetFlow

4. Для глобальной сети и Internet

- **Доступ:** резервирование каналов (оптика + 4G/5G), SLA с провайдерами
- **Защита периметра:** межсетевые экраны, DDoS-митигация, WAF для веб-приложений
- **Удалённый доступ:** VPN (IPsec/IKEv2, WireGuard), MFA, Zero Trust-архитектура
- **Соответствие требованиям:** Ф3-152 (персданные), приказы ФСТЭК № 17/21, ГОСТ Р 57580

Пример расчёта пропускной способности

канала АСУ ТП: где:

- (контроллеры)
- Гц (частота опроса)

- байт (средний кадр)
 - с
 - (накладные расходы протокола)
-
-

7. Свойства знаний и отличие знаний от данных. Типы знаний: декларативные и процедурные. Нечеткие знания. Обобщенная структура экспертных систем, модели представления знаний. Правила-продукции. Структура правил продукций. Экспертные системы: классификация, инструментальные средства проектирования, разработки и отладки; этапы разработки; примеры реализации

Знания vs Данные

Критерий	Данные	Знания
Семантика	Символы без интерпретации и	Осмысленные связи, правила, закономерность и
Структура	Таблицы, файлы, потoki	Онтологии, семантические сети, фреймы, правила
Использование	Хранение, передача, агрегация	Вывод новых фактов, принятие

		решений, объяснение
Пример	<pre>{"temp": 37.2, "patient_id": 123}</pre>	«ЕСЛИ температура > 37 И лейкоциты > 10, ТО вероятна инфекция»

Свойства знаний (по Д.А. Поспелову):

35. Внутренняя интерпретируемость —

возможность извлечения смысла машиной

36. **Структурированность** — наличие

отношений между элементами (иерархия, причинность)

37. **Связность** — возможность перехода от

одного фрагмента к другому по логическим связям

38. **Активность** — способность инициировать

действия при выполнении условий

39. **Метазнания** — знания о знаниях (источник,

достоверность, область применимости)

Типы знаний

Декларативные знания («что?»)

- Факты, утверждения, описания объектов и их свойств

- **Представление:** предикаты, семантические сети, онтологии
- **Пример:** «Пациент Иванов имеет диагноз “грипп” »

Процедурные знания («как?»)

- Алгоритмы, правила действий, стратегии решения задач
- **Представление:** правила-продукции, сценарии, демонические процедуры
- **Пример:** «ЕСЛИ диагноз = грипп, ТО назначить осельтамивир 75 мг 2 р/д»

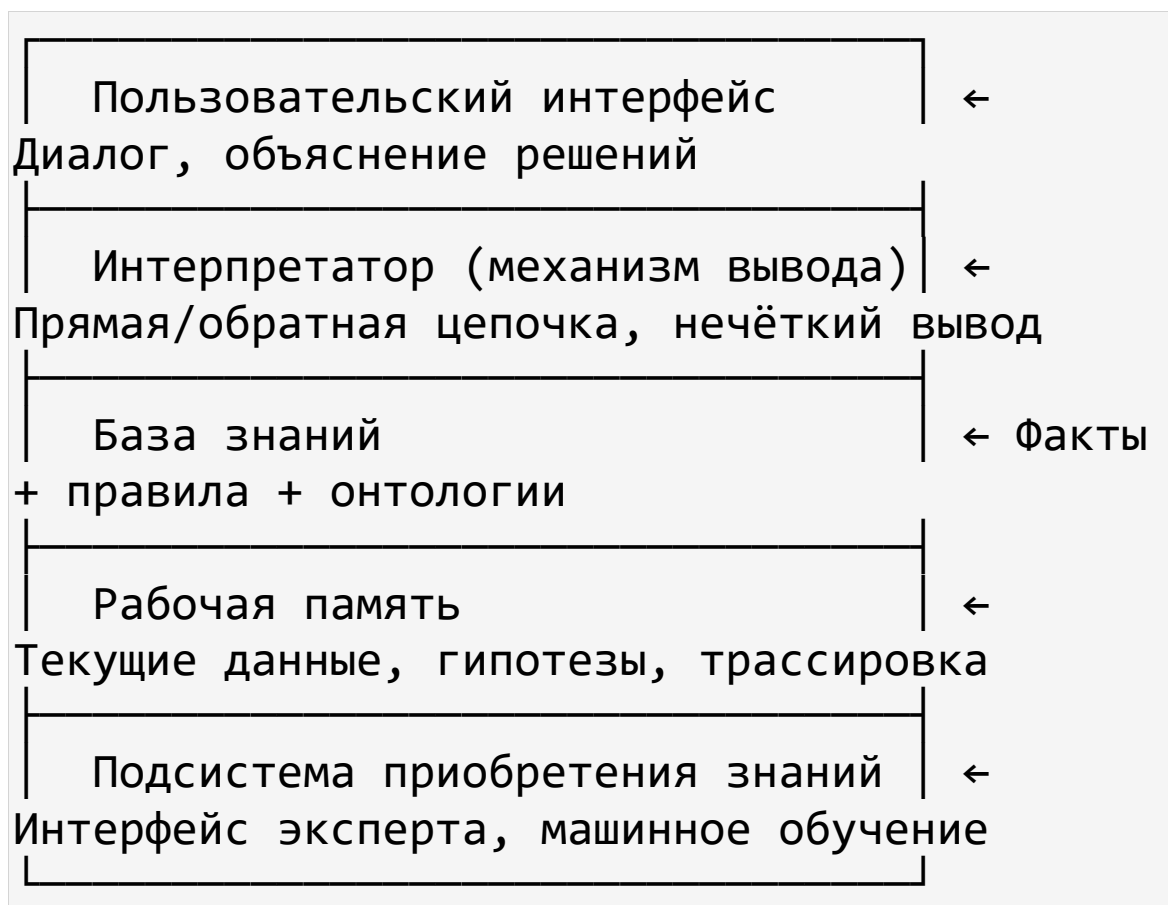
Нечёткие знания

- Знания с неопределённостью, градациями истинности

- **Математический аппарат:** теория нечётких множеств Л. Заде
- **Функция принадлежности:** , где 0 — не принадлежит, 1 — полностью принадлежит
- **Пример:** «Температура *высокая*» →

Обобщённая структура экспертной системы

(ЭС):



Модели представления знаний

Правила-продукции

- **Формат:** ЕСЛИ <условие>, ТО
<действие/закключение> [УВЕРЕННОСТЬ = α]
- **Структура правила:**
- **Механизм вывода:**
 - *Прямая цепочка (data-driven):* от фактов к заключению
 - *Обратная цепочка (goal-driven):* от гипотезы к проверке условий

Классификация экспертных систем:

Критерий	Классы

По предметной области	Медицинские, технические, финансовые, юридические
По типу знаний	Детерминированные, вероятностные, нечёткие, гибридные
По способу вывода	Продукционные, фреймовые, на основе онтологий, нейросетевые
По степени автономности	Советующие, автоматические, обучающие

Инструментальные средства разработки ЭС

(российские и адаптированные):

- **CLIPS** (адаптирован в ИС «Эксперт-М») — язык правил, прямой/обратный вывод
- **JESS** (Java-версия CLIPS) — интеграция с корпоративными системами
- **Ontology Editor «ОнтоЛаб»** (Россия) — визуальное построение онтологий, экспорт в OWL
- **PyKE** (Python Knowledge Engine) — открытая система для прототипирования

Этапы разработки ЭС (по методике В.Ф.

Хорошевского):

40. Идентификация — определение границ задачи, критериев успеха

41. Концептуализация — выделение понятий,
связей, типов знаний

42. Формализация — выбор модели
представления, кодирование правил

43. Реализация — программирование в
инструментальной среде

44. Тестирование — проверка на контрольных
примерах, настройка коэффициентов
уверенности

45. Эксплуатация и сопровождение —
обновление базы знаний, мониторинг качества
решений

Примеры реализации в РФ:

- **ЭС «Диагноз-М»** (Минздрав) — поддержка принятия решений в первичном звене здравоохранения
 - **«Техэксперт: Промышленная безопасность»** — экспертная система по нормативным требованиям
 - **«Нейро-СТМ»** (РЖД) — диагностика неисправностей подвижного состава на основе нечётких правил
-
-

8. Основные понятия семантических сетей:
представление объектов и отношений между ними в виде ориентированного графа.
Основные понятия фрейма: слоты,

присоединенные процедуры-слуги и процедуры-демоны, наследование свойств

Семантические сети

- **Определение:** модель представления знаний в виде ориентированного графа , где:
 - — вершины (понятия, объекты, события)
 - — дуги (бинарные отношения между вершинами)
- **Типы отношений** (по классификации А.И. Берштейна):
- **Пример сети** (текстовое представление):
- **Механизм вывода:** поиск путей в графе, распространение активации, логический вывод на основе транзитивности отношений

Фреймы

- **Определение:** структура данных для описания стереотипной ситуации или объекта, предложенная М. Минским.

- **Структура фрейма:**

- **Типы слотов:**

- **Данные:** значения (строки, числа, ссылки)
- **Методы:** присоединённые процедуры
- **Ограничения:** условия допустимости значений

- **Присоединённые процедуры:**

- **Наследование свойств:**

- **Механизм:** если слот не определён в текущем фрейме, поиск продолжается в родительских фреймах по иерархии IS-A

- **Переопределение:** дочерний фрейм может задать собственное значение слота, скрывающее унаследованное
- **Множественное наследование:** разрешение конфликтов по приоритету родителей или явным правилам

- **Преимущества фреймов:**

- Естественность представления иерархических знаний
 - Инкапсуляция данных и поведения
 - Гибкость за счёт процедурных вложений
 - Поддержка умолчаний и исключений
-
-

9. Алгоритм обратного распространения ошибки. Достоинства и недостатки алгоритма. Понятие паралича сети и причины его возникновения. Общая идея градиентных методов решения задач безусловной оптимизации

Алгоритм обратного распространения ошибки (Backpropagation)

- **Назначение:** обучение многослойного перцептрона (MLP) путём минимизации функции потерь относительно весов .
- **Математическая основа:** цепное правило дифференцирования (chain rule).
- **Алгоритм** (для сети с одним скрытым слоем):
- **Прямой проход:**
 - а. Вход:

- b. Скрытый слой:
- c. Выходной слой:
- d. Ошибка: (квадратичная потеря)

● **Обратный проход:**

- e. Градиент на выходе:
- f. Градиент в скрытом слое:
- g. Обновление весов (градиентный спуск):
где η — скорость обучения, a_{j-1} — активация
нейрона на предыдущем слое.

● **Функция активации и её производная**

(сигмоида):

Достоинства алгоритма:  Универсальность:

применим к любой дифференцируемой
архитектуре

✓ Эффективность: сложность $O(n^2)$, где n — число

весов

✓ Теоретическое обоснование: сходимость к локальному минимуму при выпуклой функции потерь

✓ Масштабируемость: легко параллелится на GPU

Недостатки алгоритма: ✗ Локальные

минимумы и седловые точки в невыпуклых функциях потерь

✗ Проблема затухающих/взрывающихся градиентов (vanishing/exploding gradients)

✗ Чувствительность к выбору скорости обучения и инициализации весов

✗ Требуется размеченных данных (обучение с учителем)

Паралич сети (нейронное насыщение)

- **Определение:** состояние, при котором нейроны скрытого слоя выдают значения, близкие к границам функции активации (0 или 1 для сигмоиды), что приводит к и, как следствие, к нулевым градиентам.
- **Математическое выражение:** Веса перестают обновляться — обучение останавливается.
- **Причины возникновения:**
 - h. Слишком большие начальные веса → большие значения → насыщение

i. **Высокая скорость обучения** → резкие

скачки весов в область насыщения

j. **Неподходящая функция активации**

(сигмоида/тангенс) для глубоких сетей

k. **Неотмасштабированные входные**

данные (например, признаки в диапазоне [0, 1000])

● **Способы предотвращения:**

○ Инициализация весов по методу Xavier/He:

○ Использование ReLU-активации: , при

○ Пакетная нормализация (BatchNorm) для стабилизации распределения активаций

○ Адаптивные оптимизаторы (Adam, RMSProp) с автоматической настройкой шага

Градиентные методы безусловной оптимизации

- **Общая идея:** итеративное движение в направлении, противоположном градиенту функции потерь:
 - **Варианты:**
 - **Критерии остановки:**
 - (малое изменение функции потерь)
 - (градиент близок к нулю)
 - Исчерпание максимального числа эпох
-
-

10. Методы приобретения знаний. Основные понятия методов обучения. Сведение

функционирования нейронной сети к задаче минимизации целевой функции

Методы приобретения знаний (по классификации В.Ф. Хорошевского):

Метод	Описание	Инструменты	Применение
Интервьюирование эксперта	Структурированный диалог для извлечения правил и стратегий	Протоколы, кейс-стади, think-aloud	Экспертные системы, онтологии

Анализ прецеденто в	Выявление закономерн остей из базы решённых задач	Case- based reasonin g, кластер изация	Диагности ка, классифик ация
Машинное обучение	Автоматиче ское извлечение паттернов из данных	Деревья решени й, SVM, нейросе ти	Прогнозир ование, распознав ание

Онтологическое моделирование	Формализация понятий и отношений предметной области	Protégé, OntoUM L, OWL	Семантический поиск, интеграция данных
Краудсорсинг	Сбор знаний от сообщества пользователей	Платформы типа «Вики-знания», геймификация	Общедоступные базы знаний

Основные понятия методов обучения

Обучение с учителем (supervised)

- **Данные:** размеченная выборка
- **Цель:** найти функцию , минимизирующую ошибку на новых данных
- **Метрики качества:** точность, полнота, F1, ROC-AUC, MSE

Обучение без учителя (unsupervised)

- **Данные:** неразмеченная выборка
- **Цель:** обнаружение структуры (кластеры, аномалии, снижение размерности)
- **Методы:** k-means, DBSCAN, PCA, автоэнкодеры

Обучение с подкреплением (reinforcement)

- **Модель:** агент $\leftrightarrow$ среда, награда , состояние , действие
- **Цель:** максимизировать суммарную награду

- **Алгоритмы:** Q-learning, Policy Gradient, PPO

Сведение функционирования нейронной сети к задаче минимизации целевой функции

- **Целевая функция (функция потерь)** — мера расхождения между предсказанием сети и истинным значением :

- *Регрессия:* (MSE)

- *Классификация:* (кросс-энтропия)

- **Задача оптимизации:** где — регуляризатор (L1/L2), — коэффициент регуляризации.

- **Почему это эквивалентно обучению?**

l. Прямой проход сети = вычисление

m. Вычисление = оценка качества текущего

n. Обратный проход = вычисление

о. Обновление = шаг оптимизации →

Повторение цикла минимизирует , улучшая
предсказания.

- Визуализация процесса:

- Проблемы и решения:

**11. Системы защиты от
несанкционированного доступа в
операционных системах и локальных сетях
передачи данных. Методы и средства защиты
информации в Internet. Программно-
технические методы и средства защиты
информации**

Защита от НСД в ОС и ЛВС

В операционных системах (по ГОСТ Р 50597-2013, требованиям ФСТЭК)

● Идентификация и аутентификация:

- Пароли (требования: длина ≥ 12 , сложность, история, блокировка)
- Многофакторная аутентификация (2FA):
пароль + токен/биометрия
- Единая система аутентификации (Active Directory, FreeIPA)

● Управление доступом:

- *Дискреционное (DAC)*: права rwx, ACL
(Windows NTFS, Linux POSIX.1e)
- *Мандатное (MAC)*: метки
конфиденциальности, политики Bell-LaPadula
(SELinux, Astra Linux Special Edition)

- *Ролевое (RBAC)*: назначение прав через роли (AD Groups, sudoers)

- **Аудит и мониторинг:**

- Журналы событий (Windows Event Log, Linux auditd, rsyslog)
- SIEM-системы (MaxPatrol, RuSIEM) для корреляции событий
- Контроль целостности (AIDE, Tripwire) — хеширование критических файлов

В локальных сетях

- **Сегментация и изоляция:**

- VLAN для разделения трафика (офис / АСУ ТП / гостевой доступ)

- Политики 802.1X для аутентификации устройств при подключении

- **Контроль доступа на уровне сети:**

- Списки контроля доступа (ACL) на коммутаторах/маршрутизаторах
- Порт-безопасность (port security):
ограничение MAC-адресов на порту

- **Защита от атак внутри сети:**

- ARP-спуфинг: статические ARP-таблицы, DHCP snooping, Dynamic ARP Inspection
- MAC-флудинг: ограничение числа MAC-адресов на порту

Методы и средства защиты информации в Internet

Угроза	Метод защиты	Технические средства
Перехват трафика	Шифрование канала	TLS 1.3, VPN (IPsec, WireGuard)
Фишинг/подделка	Аутентификация сервера	SSL-сертификаты, DNSSEC, DANE
DDoS-атаки	Фильтрация трафика, масштабирование	Cloudflare, Qrator, WAF, Anycast

Внедрение кода (XSS, SQLi)	Валидация ввода, экранирование	WAF (ModSecurity , Positive Technologies PT WAF)
Утечки данных	Шифрование на уровне приложения, DLP	КриптоПро, Secret Net Studio, Solar Dozor
Компрометаци я учётных записей	MFA, поведенческий анализ	RuToken, Angara Authenticati on, BI.ZONE

Программно-технические методы и средства защиты

Криптографические средства (по ФЗ-63 и ГОСТ)

● Шифрование данных:

- Симметричное: ГОСТ 28147-89, ГОСТ Р 34.12-2015 («Кузнечик»)
- Асимметричное: ГОСТ Р 34.10-2012 (ЭЦП), ГОСТ Р 34.11-2012 («Стрибог»)

● Управление ключами:

- Аппаратные токены (RuToken, JaCarta)
- Серверы ключей (КриптоПро CSP, ViPNet Coordinator)

Средства защиты периметра

- **Межсетевые экраны (МЭ):**

- Фильтрация пакетов (статические правила)
- Stateful inspection (отслеживание состояния сессий)
- Прикладной уровень (прокси, анализ содержимого)

- **Системы обнаружения/предотвращения вторжений (IDS/IPS):**

- Сигнатурный анализ (база известных атак)
- Поведенческий анализ (аномалии трафика)
- Примеры: Snort (адаптирован), Kaspersky Anti Targeted Attack

Средства защиты от утечек (DLP)

- **Каналы контроля:** email, веб-трафик, съёмные носители, печать

- **Методы анализа:**

- Контентный (ключевые слова, регулярные выражения)
- Статистический (частотный анализ, машинное обучение)
- Цифровые отпечатки (хеш-суммы документов)

- **Российские решения:** Solar Dozor, SearchInform, InfoWatch Traffic Monitor

Принципы построения комплексной системы защиты (по приказу ФСТЭК № 21):

46. **Непрерывность:** защита на всех этапах ЖЦ информации

47. **Эшелонирование:** многоуровневая оборона
(периметр → сеть → хост → данные)

48. **Своевременность:** обнаружение и реагирование в реальном времени

49. **Гибкость:** адаптация к новым угрозам без полной перестройки

50. **Экономическая целесообразность:**
 $\text{стоимость защиты} \leq \text{возможный ущерб}$

12. Американский стандарт шифрования данных DES. Отечественный стандарт шифрования данных (ГОСТ 28147-89)

DES (Data Encryption Standard)

- **История:** разработан IBM, принят NIST в 1977 как федеральный стандарт США (отозван в 2005).

- **Тип:** симметричный блочный шифр.

- **Параметры:**

- Размер блока: **64 бита**

- Длина ключа: **56 бит** (из 64 бит 8 — контрольные чётности)

- Число раундов: **16** (сеть Фейстеля)

- **Структура алгоритма** (упрощённо):

- **Режимы работы** (по ГОСТ 34.10-2012, адаптировано):

- **ECB** (Electronic Codebook) — простой, но уязвим к анализу повторений
- **CBC** (Cipher Block Chaining) — XOR с предыдущим шифртекстом, требует IV
- **CFB/OFB** — преобразование в поточный шифр

- **Криптостойкость:**

- Перебор ключа: вариантов
- В 1998 г. машина EFF Deep Crack взломала DES за 56 часов
- **Вывод:** в настоящее время **не считается безопасным** для новых систем

- **Усиленные версии:**

- **Triple DES (3DES):** , эффективная длина ключа 112/168 бит
- Применяется в банковских стандартах (EMV), но постепенно вытесняется AES

ГОСТ 28147-89

- **Статус:** действующий национальный стандарт РФ (в части, не противоречащей ГОСТ Р 34.12-2015).
- **Тип:** симметричный блочный шифр.
- **Параметры:**
 - Размер блока: **64 бита**
 - Длина ключа: **256 бит** (32 байта)
 - Число раундов: **32** (улучшенная сеть Фейстеля)

● Особенности алгоритма:

- **Подстановочные блоки (S-блоки):** 8 таблиц по 4×16 бит, могут быть заменены (режим простой замены) или заданы пользователем (режим «замена узлов»)
- **Циклический сдвиг:** на 11 бит влево после каждого раунда
- **Режимы работы** (по ГОСТ 34.13-2015, заменившему часть 28147-89):
 - Простая замена (аналог ECB)
 - Гаммирование (аналог CFB)
 - Гаммирование с обратной связью (аналог CBC)

- Выработка имитовставки (для контроля целостности)

- **Криптостойкость:**

- Перебор ключа: — вычислительно неосуществимо
- Устойчив к линейному и дифференциальному криптоанализу (при корректных S-блоках)
- Рекомендуется для защиты информации до уровня «совершенно секретно» (при использовании сертифицированных СКЗИ)

- **Область применения в РФ:**

- Защита государственной тайны (при наличии сертификата ФСБ)

- Финансовый сектор (платёжные системы, банковские транзакции)
- Электронный документооборот (в связке с ГОСТ Р 34.10 для ЭЦП)
- Системы связи специального назначения

● **Сравнение с современными стандартами:**

Вывод: для новых разработок в РФ

рекомендуется использовать **ГОСТ Р 34.12-2015**

(«Кузнечик») как более современный и

производительный стандарт, совместимый с

инфраструктурой ГОСТ Р 34.10-2012 (ЭЦП) и

ГОСТ Р 34.11-2012 (хеш-функция).

13. Основные функции межсетевых экранов для фильтрации сообщений и защиты информации. Способы защиты информационных ресурсов от несанкционированного доступа.

Помехоустойчивое кодирование. Общие принципы построения помехоустойчивых кодов

Межсетевые экраны (МЭ / Firewall)

- **Определение** (по ГОСТ Р 57580.1-2017):

«Комплекс программно-технических средств, осуществляющий контроль и фильтрацию сетевого трафика на основе заданных правил безопасности».

- **Основные функции фильтрации:**

- **Типы МЭ по архитектуре:**

- **Пакетные фильтры (stateless):** быстрые, но не отслеживают сессии
- **Stateful inspection:** хранят таблицу состояний соединений, эффективнее против спуфинга
- **Прокси-МЭ:** полное завершение сессии на МЭ, глубокий анализ контента
- **NGFW (Next-Generation):** интеграция IPS, антивируса, контроля приложений, песочницы

- **Пример правила в формате iptables (Linux):**

Способы защиты информационных ресурсов от НСД

Организационные меры

- Регламенты доступа, инструкции, обучение персонала
- Разделение обязанностей (principle of least privilege)
- Политики паролей, блокировка после неудачных попыток

Технические меры

- **Аутентификация:** MFA, биометрия, сертификаты
- **Авторизация:** RBAC, ABAC (атрибутная модель), динамические политики
- **Шифрование:**
 - *на покое* (BitLocker, VeraCrypt, ГОСТ 28147)
 - *в передаче* (TLS, IPsec)

- **Маскирование данных:** динамическое
сокрытие чувствительных полей (например,
отображение ****1234 вместо номера карты)
- **Журналирование и аудит:** неизменяемые
логи, SIEM-корреляция

Физические меры

- Контроль доступа в серверные (СКУД,
видеонаблюдение)
- Защита от побочных электромагнитных
излучений (ПЭМИН)
- Резервное питание, пожаротушение

Помехоустойчивое кодирование

- **Цель:** обнаружение и исправление ошибок, возникающих при передаче данных по зашумлённым каналам связи.

- **Основные понятия:**

- **Кодовое слово** — разрешённая последовательность бит длины
- **Информационные биты** — полезная нагрузка
- **Проверочные биты** — избыточность для контроля
- **Кодовое расстояние** — минимальное число позиций, в которых различаются любые два кодовых слова

- **Критерии исправления ошибок:**

- Обнаружение до ошибок:
- Исправление до ошибок:
- Одновременное исправление и обнаружение ($t < d$):

● **Общие принципы построения помехоустойчивых кодов:**

a. **Добавление избыточности:**

проверочных бит вычисляются как линейные функции от информационных

b. **Линейность:** код является

подпространством в $\mathbb{F}_2^n$, что упрощает кодирование/декодирование

c. **Систематичность:** информационные

биты сохраняются в явном виде в кодовом слове

d. **Оптимальность:** максимизация при фиксированных n и k (граница Хэмминга, Плоткина)

- **Классы кодов:**

- **Пример: код Хэмминга (7,4):**

14. Представление аналогового сигнала в цифровом виде. Преобразование Фурье

Дискретизация аналогового сигнала

- **Теорема Котельникова (Найквиста-Шеннона):**

- Аналоговый сигнал с ограниченным спектром может быть точно восстановлен по своим отсчётам, если частота дискретизации .

- **Этапы аналого-цифрового преобразования (АЦП):**

- **Параметры АЦП:**

- **Частота дискретизации** : 8 кГц (телефония), 44.1 кГц (аудио), 1 МГц+ (осциллографы)
- **Разрядность** : 8–24 бит; динамический диапазон дБ
- **Ошибка квантования**: равномерно распределена в , где

- **Алиасинг (наложение спектров):**

- Возникает при

- Высокие частоты «отражаются» в низкочастотную область:
- **Борьба:** строгий антиалиасинговый фильтр перед АЦП

Преобразование Фурье

- **Непрерывное преобразование Фурье (для аналоговых сигналов):**

- — комплексный спектр: — амплитудный спектр, — фазовый спектр

- **Дискретное преобразование Фурье (ДПФ) для цифровых сигналов:**

- — дискретный сигнал длины
- — дискретный спектр, — номер гармоники,

- **Быстрое преобразование Фурье (БПФ):**

- Алгоритм Кули-Тьюки: сложность вместо у прямого ДПФ
 - Требуется (допускается дополнение нулями)
 - **Применение в ИТ и связи:**
 - **Пример: спектральный анализ аудиосигнала**
(псевдокод):
 - **Оконное преобразование Фурье (для нестационарных сигналов):** где — окно (Ханна, Хэмминга, Гаусса), позволяющее анализировать локальный спектр.
-
-

МОДЕЛИРОВАНИЕ

4. Компоненты дискретно-событийной модели и их организация (схема алгоритма функционирования)

Дискретно-событийное моделирование (DES)

— метод имитации систем, состояние которых изменяется в дискретные моменты времени при наступлении событий.

Основные компоненты модели:

Компонент	Описание	Пример в системе массового обслуживания

Сущность (Entity)	Объект, перемещающийся по системе	Заявка, клиент, пакет данных
Событие (Event)	Мгновенное изменение состояния системы	Прибытие заявки, начало обслуживания, уход
Очередь (Queue)	Структура для хранения сущностей в ожидании ресурса	Очередь заявок к серверу

Ресурс (Resource)	Ограниченный элемент, необходимый для обработки	Процессор, канал связи, оператор
Статистика (Statistics)	Счётчики и накопители для анализа результатов	Среднее время ожидания, загрузка ресурса
Генератор случайных чисел	Источник стохастичности для моделирования неопределённости	Распределение времени между прибытиями

Организация: схема алгоритма

функционирования



4. Добавить новые события в очередь
(при необходимости) |



ЗАВЕРШЕНИЕ

- Вычисление итоговых метрик
- Вывод отчёта

Очередь событий реализуется как приоритетная очередь (куча), где приоритет = время наступления события. Сложность операций: вставка/извлечение — .

Пример события в псевдокоде:

```
EVENT Arrival:
    CREATE entity with id = next_id
    IF resource.free THEN
        resource.free = FALSE
        SCHEDULE Completion at t +
service_time
    ELSE
        queue.enqueue(entity)
    ENDIF
    SCHEDULE next Arrival at t +
interarrival_time
```

5. Моделирование случайных событий и дискретных случайных величин

Моделирование случайного события с вероятностью

- **Алгоритм:**

- е. Сгенерировать (равномерное распределение)
- f. Если $\rightarrow$ событие произошло, иначе — не произошло

- **Пример:** моделирование отказа устройства с вероятностью 0.01 за час:

Моделирование дискретной случайной

величины с законом распределения:

Значение			...	
Вероятность			...	

- **Метод инверсии (для дискретного случая):**

- g. Вычислить кумулятивные вероятности: ,

- h. Сгенерировать

- i. Найти такое, что $\rightarrow$ результат

- **Оптимизация:** для частых значений

использовать поиск с подстановкой (alias method), сложность после предобработки.

- **Пример: моделирование числа клиентов в час** (распределение Пуассона с):

- **Проверка качества генерации:**

- Тест : сравнение эмпирических и теоретических частот
 - Тест Колмогорова-Смирнова: для непрерывных распределений
 - Визуализация: гистограмма + теоретическая кривая
-
-

6. Моделирование непрерывных случайных величин (метод обратной функции).

Моделирование экспоненциального распределения

Метод обратной функции

● **Теорема:** если $f(x)$ — непрерывная СВ с функцией распределения $F(x)$, то $F(x)$ — непрерывная функция. Следовательно, для генерации x_i с заданным $F(x)$ достаточно:

● **Алгоритм:**

j. Задать аналитическое выражение $F(x)$ или её численную аппроксимацию

k. Найти обратную функцию (аналитически или численно)

l. Для каждого u_i вычислить $x_i = F^{-1}(u_i)$

● **Пример: нормальное распределение :**

- не выражается в элементарных функциях
- Используют приближения: метод Бокса-Мюллера, алгоритм Зигерта, или

библиотечные функции

`(numpy.random.normal)`

Моделирование экспоненциального распределения

- **Параметры:** интенсивность (среднее время между событиями)
- **Функция распределения:**
- **Обратная функция:**
- **Алгоритм генерации:**
- **Свойства, важные для моделирования:**
 - **Отсутствие последствия:** — подходит для моделирования времени между независимыми событиями (прибытия, отказы)

- Среднее: , Дисперсия:
 - **Применение в ИТ-моделировании:**
 - Время между запросами к веб-серверу
 - Время наработки на отказ оборудования
 - Длительность сессии пользователя
 - **Проверка:** построить эмпирическую функцию распределения и сравнить с теоретической (тест Колмогорова).
-
-

7. Определение необходимого количества реализаций для оценки вероятности наступления случайного события и оценки среднего значения случайной величины

Оценка вероятности события

- Пусть событие имеет неизвестную вероятность . Проведено независимых испытаний, событие произошло раз.
- Оценка:
- Дисперсия оценки: (максимум при)
- **Требуемое число испытаний для заданной точности:** По неравенству Чебышёва: Чтобы гарантировать , достаточно:
- *Пример:* (точность 1%), (доверительная вероятность):
- **Более точная оценка (нормальное приближение):** При неизвестном берут (наихудший случай):

Оценка среднего значения СВ

- Пусть $\bar{X}$ — СВ с неизвестным средним μ и известным (или оценённым) стандартным отклонением σ .
- Выборочное среднее:
- Стандартная ошибка:
- **Требуемое число реализаций:** где Δ — допустимая абсолютная погрешность.
- *Пример:* , , :
- **Если неизвестно:**
 - м. Провести пилотную серию из m испытаний (например, 30)
 - п. Оценить выборочное стандартное отклонение

о. Использовать вместо в формуле,
заменив на -квантиль Стьюдента для
степеней свободы

- **Последовательный план** (экономия
вычислений):
-
-

8. Оперативное планирование модельных экспериментов. Оценка точности результатов моделирования

Оперативное планирование экспериментов

- **Цель:** минимизировать вычислительные
затраты при достижении требуемой точности
оценок.
- **Стратегии:**

p. **Последовательный анализ** (см. выше) —
адаптивное увеличение объёма выборки

q. **Стратифицированная выборка** —
разбиение пространства входных
параметров на слои, пропорциональное
моделирование в каждом

r. **Метод контрольных переменных** —
использование коррелированной СВ с
известным средним для снижения
дисперсии: где — контрольная переменная,
— коэффициент, оцениваемый по данным

s. **Андромеда (Antithetic Variates)** —
генерация парных траекторий с
отрицательной корреляцией для
компенсации случайных отклонений

- **План эксперимента для анализа**

- чувствительности:**

- **Полный факторный эксперимент:** все комбинации уровней факторов — точно, но экспоненциально растёт число прогонов
- **Дробный факторный эксперимент:** выбор подмножества комбинаций по принципу ортогональности
- **Метод Монте-Карло по латинским гиперкубам** — равномерное покрытие многомерного пространства при меньшем числе точек

Оценка точности результатов моделирования

- **Статистические характеристики:**

- **Точечная оценка:** (среднее, вероятность, проценты)
- **Доверительный интервал:** с доверительной вероятностью
- **Относительная погрешность:**
- **Методы построения ДИ:**
 - **Для среднего при известном :**
 - **Для среднего при неизвестном :**
 - **Для вероятности (нормальное приближение):**
 - **Бутстреп** — ресэмплинг исходной выборки для оценки распределения оценки (универсально, но затратно)
- **Критерии остановки моделирования:**

t. Фиксированное число реализаций (просто, но неэффективно)

u. Достижение заданной ширины ДИ:

v. Стабилизация оценки: за последние шаги

w. Исчерпание вычислительного бюджета (времени/памяти)

- **Визуализация сходимости:**

- **Учёт систематической ошибки (смещения):**

- Статистические методы оценивают только **случайную** погрешность

- **Смещение** возникает из-за упрощений модели, неточных входных данных, некорректных допущений

- **Методы борьбы:**

- **Верификация:** проверка соответствия модели концептуальному описанию (code review, трассировка)
 - **Валидация:** сравнение выходных данных модели с реальными наблюдениями (тест на исторических данных)
 - **Анализ чувствительности:** оценка влияния входных параметров на результат
-
-

СЕТИ И ТЕЛЕКОММУНИКАЦИИ И ТЕОРИЯ ИНФОРМАЦИИ

1. Основные понятия и классификация компьютерных сетей. Клиент-серверная и

трехзвенная архитектура; одноранговые сети.

Эталонная модель ISO/OSI

Компьютерная сеть — совокупность узлов

(компьютеров, устройств) и каналов связи,

обеспечивающих обмен данными и совместное

использование ресурсов.

Классификация сетей:

Критерий	Классы
Географический охват	• Персональная (PAN, ≤ 10 м) • Локальная (LAN, ≤ 1 км) • Городская (MAN, ≤ 50 км) • Глобальная (WAN, неограниченно)

Топология	Шина, звезда, кольцо, сетка, иерархическая
Среда передачи	Проводные (витая пара, оптика), беспроводные (Wi-Fi, 5G, спутник)
Способ управления	Централизованные (клиент-сервер), децентрализованные (одноранговые, P2P)
Скорость передачи	Низкоскоростные (<1 Мбит/с), средние (1–100 Мбит/с), высокоскоростные (>1 Гбит/с)

Архитектуры взаимодействия

Одноранговая сеть (Peer-to-Peer)

- **Принцип:** все узлы равноправны, каждый может быть и клиентом, и сервером
- **Преимущества:** простота развёртывания, отказоустойчивость, масштабируемость
- **Недостатки:** сложность управления безопасностью, низкая предсказуемость производительности
- **Применение:** малые офисы, файлообмен (BitTorrent), блокчейн-сети

Клиент-серверная архитектура

- **Принцип:** чёткое разделение ролей:

- *Клиент*: инициирует запросы, отображает результаты
- *Сервер*: обрабатывает запросы, хранит данные, обеспечивает безопасность
- **Преимущества**: централизованное управление, безопасность, масштабируемость сервера
- **Недостатки**: единая точка отказа (сервер), сложность администрирования
- **Пример**: веб-приложение (браузер ↔ Apache/NGINX ↔ PostgreSQL)

Трёхзвенная архитектура (3-tier)

- **Уровни**:

- *Уровень представления (клиент):* интерфейс пользователя (веб, мобильное приложение)
- *Бизнес-логика (сервер приложений):*
обработка запросов, правила, транзакции
- *Данные (сервер БД):* хранение, запросы, целостность

● **Преимущества:**

- Независимое масштабирование уровней
- Повышенная безопасность (БД не доступна напрямую извне)
- Гибкость замены компонентов (например, смена СУБД без изменения клиента)

● **Применение:** корпоративные ИС, интернет-банкинг, государственные порталы

Эталонная модель ISO/OSI (7 уровней)

Уров ень	Названи е	Функции	Примеры протоколов/у стройств
7	Приклад ной (Applicati on)	Интерфейс для приложений, сетевые сервисы	HTTP, FTP, SMTP, DNS
6	Представ ления (Presenta tion)	Кодирование, шифрование, сжатие данных	TLS, JPEG, ASCII/Unicod e

5	Сеансовый (Session)	Управление диалогом, синхронизация, восстановление	NetBIOS, RPC
4	Транспортный (Transport)	Сквозная доставка, контроль потока, исправление ошибок	TCP, UDP, SCTP
3	Сетевой (Network)	Логическая адресация,	IP, ICMP, OSPF,

		маршрутизация, фрагментация	маршрутизаторы
2	Канальный (Data Link)	Физическая адресация (MAC), контроль доступа к среде, обнаружение ошибок	Ethernet, Wi-Fi (802.11), коммутаторы
1	Физический (Physical)	Передача битов по среде, электрические/оптические параметры	Витая пара, оптоволокно, концентраторы

- **Принцип инкапсуляции:** данные обрамляются заголовками (и, при необходимости, концевиками) на каждом уровне при передаче вниз по стеку и освобождаются от них при приёме вверх.

- **Практическое значение:** модель OSI — концептуальная основа для понимания взаимодействия, но в реальности используется упрощённая модель **TCP/IP (4 уровня):**

2. Коммутация каналов и пакетов. Режимы коммутации пакетов

Коммутация каналов (Circuit Switching)

- **Принцип:** перед передачей данных устанавливается выделенный физический или логический канал между абонентами на всё время сеанса.

- **Этапы:**

- х. Установление соединения (signaling)
- у. Передача данных (выделенная полоса гарантирована)
- z. Разрыв соединения

- **Характеристики:**

-  Гарантированная пропускная способность, постоянная задержка
-  Неэффективное использование ресурсов при пульсирующем трафике

- **✗** Длительное время установления соединения

- **Применение:** традиционная телефония (ТфОП), выделенные линии (E1, SDH)

Коммутация пакетов (Packet Switching)

- **Принцип:** данные разбиваются на пакеты фиксированного или переменного размера, каждый пакет передаётся независимо и может следовать своим маршрутом.
- **Преимущества:**
 - Эффективное использование каналов (статистическое мультиплексирование)
 - Устойчивость к отказам (динамическая маршрутизация)

- Поддержка разнородного трафика (голос, видео, данные)

- **Недостатки:**

- Переменная задержка (jitter), возможные потери пакетов
- Требуется буферизация и механизмы контроля перегрузок

Режимы коммутации пакетов

1. Датаграммный (без установления соединения, connectionless)

- **Принцип:** каждый пакет (датаграмма) содержит полный адрес назначения и маршрутизируется независимо
- **Протокол:** IP (Internet Protocol)

- **Особенности:**

- Нет гарантии доставки, порядка, отсутствия дубликатов
- Минимальные накладные расходы, высокая скорость
- Восстановление — на транспортном уровне (TCP) или в приложении

- **Применение:** веб-трафик, стриминг, онлайн-игры (в сочетании с UDP)

2. Виртуальных каналов (с установлением соединения, connection-oriented)

- **Принцип:** перед передачей данных устанавливается логическое соединение (виртуальный канал), все пакеты следуют по

одному маршруту и имеют одинаковый идентификатор (VCI)

- **Этапы:**

- аа. Установка виртуального канала (signaling)

- bb. Передача пакетов с коротким заголовком (только VCI)

- сс. Освобождение канала

- **Протоколы: X.25, Frame Relay, ATM, MPLS**

- **Особенности:**

- Гарантированный порядок доставки
 - Возможность резервирования ресурсов (QoS)

- Более высокие накладные расходы на управление соединениями

● **Применение:** корпоративные сети, телефония по пакетным сетям (VoIP с RSVP), магистральные сети операторов

Сравнительная таблица:

Характеристика	Коммутация каналов	Датаграммы	Виртуальные каналы
Установление соединения	Обязательно	Нет	Обязательно

Маршрутизация	Фиксированный путь	Независимая для каждого пакета	Фиксированный путь после установки
Адресация в пакете	Не требуется (канал идентифицируется физически)	Полный адрес (IP)	Короткий идентификатор (VCI)

Гарантии доставки	Да (физический канал)	Нет	Частично (порядок, но не надёжность)
Эффективность при пульсирующем трафике	Низкая	Высокая	Средняя
Пример	ТфОП, E1	Интернет (IP)	MPLS-сети, ATM

3. Определение канала передачи информации; основные характеристики каналов связи. Модуляция. Представление информации при передаче; синхронный и асинхронный режимы передачи. Самосинхронизирующиеся коды

Канал передачи информации

- **Определение** (по ГОСТ 23637-79):

«Совокупность технических средств и среды распространения, обеспечивающая передачу сигнала от источника к приёмнику».

- **Математическая модель канала** (по

Шеннону): где $x(t)$ — входной сигнал, $y(t)$ —

выходной сигнал, — аддитивный шум (обычно гауссовский белый шум, AWGN).

Основные характеристики каналов связи

Характеристика	Обозначение	Единицы	Формула/Описание
Полоса пропускания		Гц	Диапазон частот, в котором затухание сигнала не превышает заданного уровня

Пропускная способность		бит/с	Максимальная скорость безошибочной передачи (теорема Шеннона):
Отношение сигнал/шум		дБ	
Задержка распространения		с	, где — длина канала, — скорость сигнала в среде

Коэффициент ошибок (BER)		—	Вероятность ошибочного приёма бита: для BPSK
Затухание		дБ	

Модуляция

- **Определение:** процесс изменения одного или нескольких параметров несущего сигнала в соответствии с передаваемым сообщением.
- **Типы аналоговой модуляции:**
- **Цифровая модуляция (манипуляция):**
- **Созвездие модуляции** (пример для QPSK):
- Каждая точка — символ, несущий 2 бита информации.

Представление информации при передаче

- **Кодирование на физическом уровне:**

преобразование битов в сигналы, пригодные для передачи по конкретной среде.

- **Требования к линейному коду:**

dd. Отсутствие постоянной составляющей (DC balance)

ee. Возможность выделения тактовой частоты (самосинхронизация)

ff. Минимальная избыточность

gg. Помехоустойчивость

Синхронный и асинхронный режимы передачи

Характеристика	Асинхронный режим	Синхронный режим
Синхронизация	По старт/стоп-битами для каждого символа	По общему тактовому сигналу или встроенной синхронизации
Структура кадра	[START][8 бит данных][PARITY][STOP]	[SYNC][HEADER][DATA][CRC]
Накладные расходы	20–30% (старт/стоп биты)	3–10% (заголовки кадров)

Скорость	Низкая (до 115.2 кбит/с для UART)	Высокая (Мбит/с – Гбит/с)
Применение	Консольные порты, простые датчики, Arduino	Ethernet, USB, PCIe, магистральные каналы

● **Пример асинхронной передачи (UART, 8N1):**

Самосинхронизирующиеся коды

- **Определение:** коды, в которых переходы сигнала ($0 \rightarrow 1$ или $1 \rightarrow 0$) происходят достаточно часто для восстановления тактовой частоты приёмником без отдельного тактового провода.

- **Популярные самосинхронизирующиеся коды:**
 - **Пример манчестерского кодирования:**
 - **Преимущество:** приёмник может восстановить тактовый сигнал по фронтам, что устраняет дрейф частот и упрощает аппаратную реализацию.
-
-

4. Проводные и кабельные линии связи.

Волоконно-оптические кабели

Классификация проводных линий связи

Тип кабеля	Конструкция	Полоса пропуск ания	Затух ание	Приме нение
Витая пара (UTP/FTP)	4 пары медных проводников, свитых для снижения помех	До 500 МГц (Cat.6A)	~20 дБ/1 00м @100 МГц	ЛВС (Ethernet), телефо ния, PoE
Коаксиа льный кабель	Центральный проводник, диэлектрик,	До 1 ГГц	~3 дБ/1 00м	Кабель ное ТВ, антенн ые

	экран, оболочка		@100 МГц	систем ы, старые сети
Волоко нно- оптический	Стеклянное/п ластиковое волокно, передача света	До 100 ТГц (теорет ически)	0.2– 0.5 дБ/к м	Магист рали, ЦОД, длинн ые линии связи

**Витая пара (по ГОСТ Р 53246-2008, аналог
ISO/IEC 11801)**

● **Категории:**

●Экранирование:

- **UTP** (Unshielded) — без экрана, для офисных условий
- **FTP** (Foiled) — общий экран из фольги, защита от внешних помех
- **S/FTP** — экран каждой пары + общий, для промышленных условий

Волоконно-оптические кабели (ВОК)

●Принцип действия: полное внутреннее

отражение света в сердцевине волокна за счёт

разницы показателей преломления

сердцевины (n_1) и оболочки (n_2), где $n_1 > n_2$.

●Типы оптических волокон:

- **Оптические окна прозрачности**

(минимальное затухание):

- **Преимущества ВОК:** Огромная пропускная

способность (до терабит в секунду на одно

волокно с использованием WDM)

-  Низкое затухание (0.2 дБ/км против 20

дБ/100м у меди)

-  Полная невосприимчивость к

электромагнитным помехам (ЭМП)

-  Гальваническая развязка (безопасность при

разности потенциалов)

-  Малый вес и диаметр (экономия места в

кабельных каналах)

-  Высокая защищённость от

несанкционированного подключения

(детектируется по потере сигнала)

- **Недостатки:** ❌ Высокая стоимость активного

оборудования (лазеры, фотодетекторы)

❌ Сложность монтажа и сварки (требуется специальное оборудование)

❌ Хрупкость волокна (требует аккуратного обращения, минимальный радиус изгиба)

- **Технологии уплотнения:**

- **WDM (Wavelength Division Multiplexing):**

передача нескольких каналов на разных длинах волн по одному волокну

- *CWDM*: 18 каналов с шагом 20 нм (1270–1610 нм), дальность до 80 км

- *DWDM*: 40–160 каналов с шагом 0.8/0.4 нм, дальность > 1000 км с усилителями

- **Формула расчёта общей пропускной**

способности: где — спектральная

эффективность (бит/с/Гц), например, 4 для 16-QAM.

- **Структура оптического кабеля (по ГОСТ Р 53786-2010):**
-
-

5. IP адресация, Протоколы ARP и RARP.

Протокол ICMP

IP-адресация (IPv4)

- **Структура адреса:** 32 бита, записывается в десятичном формате с точками: 192.168.1.10
- **Классовая адресация** (устаревшая, но важная для понимания):
- **Бесклассовая адресация (CIDR):**
 - Запись: 192.168.1.0/24 — адрес сети + длина префикса в битах
 - **Расчёт количества хостов:** (исключаются адрес сети и широковещательный)
 - **Пример:** /26 → хоста
- **Специальные адреса:**

Протокол ARP (Address Resolution Protocol, RFC 826)

- **Назначение:** преобразование IP-адреса (сетевого уровень) в MAC-адрес (канальный уровень) в пределах одной подсети.
- **Алгоритм работы:**
- **Формат ARP-пакета** (упрощённо):
- **Безопасность:** ARP не имеет аутентификации → уязвим к ARP-спуфингу (атака “человек посередине”). Защита: статические ARP-записи, DHCP snooping, Dynamic ARP Inspection (DAI) на коммутаторах.

Протокол RARP (Reverse ARP, RFC 903)

- **Назначение:** обратное преобразование — получение собственного IP-адреса по

известному MAC-адресу (использовался для бездисковых рабочих станций).

- **Статус:** устарел, заменён на **BOOTP**, а затем на **DHCP** (RFC 2131), который предоставляет не только IP, но и маску, шлюз, DNS и другие параметры.

Протокол ICMP (Internet Control Message Protocol, RFC 792)

- **Назначение:** передача служебных сообщений об ошибках и диагностике в сетях IP. Не передаёт данные приложений.
- **Типы сообщений** (наиболее важные):
- **Пример работы ping:**
- **Пример traceroute** (определение маршрута):

- **Безопасность:** злонамеренное использование ICMP (ping flood, smurf attack). Меры:
фильтрация ICMP на периметре, ограничение скорости эхо-запросов.
-
-

6. Протокол TCP, Протокол UDP. Назначение, характеристики и задачи; уровни модели ISO/OSI. В каких случаях используются и почему?

Протокол TCP (Transmission Control Protocol, RFC 793)

- **Уровень модели OSI:** Транспортный (уровень 4)
- **Основные характеристики:**

- **Структура заголовка (упрощённо):**

- **Ключевые механизмы:**

hh. **Нумерация байтов:** каждый байт данных

имеет порядковый номер, что позволяет

точно запрашивать повторную передачу

ii. **Подтверждения (ACK):** получатель

отправляет номер следующего ожидаемого

байта (cumulative ACK)

jj. **Таймауты и ретрансмиссия:** если ACK не

получен за (Retransmission Timeout), пакет

отправляется повторно

kk. **Контроль перегрузок:** динамическое

изменение размера окна на основе потерь

пакетов (индикатор перегрузки сети)

- **Применение:** веб-страницы (HTTP/HTTPS), почта (SMTP, IMAP), передача файлов (FTP, SFTP), удалённый доступ (SSH), базы данных — везде, где **целостность данных важнее скорости.**

Протокол UDP (User Datagram Protocol, RFC 768)

- **Уровень модели OSI:** Транспортный (уровень 4)
- **Основные характеристики:**
- **Структура заголовка:**
- **Преимущества перед TCP:**  Минимальная задержка (нет рукопожатия, подтверждений, ретрансмиссий)

✓ Низкие накладные расходы (8 байт против 20+ у TCP)

✓ Поддержка широковещания и многоадресной рассылки (multicast)

✓ Простота реализации на стороне приложения (полный контроль над надёжностью)

● **Недостатки:** ✗ Потеря пакетов не обнаруживается протоколом

✗ Пакеты могут приходить в произвольном порядке

✗ Нет встроенного контроля перегрузок (может "забить" канал)

● **Применение:** потоковое видео/аудио (где потеря кадра лучше задержки), онлайн-игры,

голосовая связь (VoIP), DNS-запросы, SNMP, DHCP — везде, где **скорость и низкая задержка важнее полной надёжности**, либо надёжность обеспечивается на прикладном уровне.

Сравнительная таблица: TCP vs UDP

Критерий	TCP	UDP
Установка соединения	Да (3-way handshake)	Нет
Надёжность доставки	Гарантирована (подтверждения, ретрансмиссии)	Не гарантируется

Порядок данных	Сохраняется	Не гарантируется
Контроль перегрузок	Есть (динамическое окно)	Нет
Заголовок	20–60 байт	8 байт
Задержка	Выше (из-за служебных процедур)	Минимальная
Поддержка multicast	Нет	Да

Примеры приложени й	HTTP, FTP, SSH, SMTP	DNS, VoIP, видеостриминг , онлайн-игры
------------------------------------	-------------------------	--

Выбор протокола: практические рекомендации

Требуется ли гарантированная доставка?			
ДА ↓ Есть ли строгие требования к задержке?		НЕТ ↓ Можно ли допустить потерю части данных ради скорости?	
ДА ↓ TCP с QoS (например,	НЕТ ↓ TCP	ДА ↓ UDP + прикладной надёжностью	НЕТ ↓ UDP
TLS over TCP)		(например, FEC, повторные	
		запросы)	



- **Пример 1: Веб-сервер → TCP**

Потеря даже одного байта в HTML-странице может нарушить отображение или выполнение скриптов. Задержка в 100–200 мс приемлема.

- **Пример 2: Видеоконференция → UDP**

Потеря 1–2% пакетов (небольшое “замирание” изображения) предпочтительнее задержки в 500 мс из-за ретрансмиссий, которая делает диалог неестественным.

- **Пример 3: Онлайн-игра (шутер) → UDP**

Позиция игрока актуальна только “здесь и сейчас” . Устаревший пакет о положении 200

мс назад бесполезен, поэтому его потеря не критична, а доставка должна быть максимально быстрой.

- **Пример 4: Система банковских транзакций**

→ **TCP + TLS**

Абсолютная целостность и конфиденциальность данных — приоритет №1. Любая потеря или искажение недопустимы.

Гибридные подходы (современные тенденции):

- **QUIC** (на базе UDP, используется в HTTP/3):

сочетает низкую задержку UDP с надёжностью и шифрованием уровня TCP+TLS

- **WebRTC**: использует UDP для медиапотокa и TCP (или SCTP) для сигнальных данных

- **Протоколы с адаптивной надёжностью:**

динамически переключаются между

режимами в зависимости от состояния сети

ИСТОЧНИКИ И РЕКОМЕНДУЕМАЯ ЛИТЕРАТУРА

Нормативные документы РФ:

51. Федеральный закон № 149-ФЗ «Об

информации, информационных технологиях и
о защите информации»

52. Федеральный закон № 63-ФЗ «Об

электронной подписи»

53. Федеральный закон № 152-ФЗ «О персональных данных»
54. ГОСТ 34.602-89 «Техническое задание на создание автоматизированной системы»
55. ГОСТ Р ИСО/МЭК 12207-2008 «Процессы жизненного цикла программной продукции»
56. ГОСТ Р 53622-2009 «Информационно-вычислительные системы. Стадии и этапы жизненного цикла»
57. ГОСТ 28147-89 / ГОСТ Р 34.12-2015 «Системы обработки информации. Криптографическая защита»
58. Приказ ФСТЭК России № 17 «О требованиях к защите информации в автоматизированных

системах управления производственными и технологическими процессами»

59. Приказ ФСТЭК России № 21 «Об утверждении Состава и содержания организационных и технических мер по обеспечению безопасности персональных данных»

Учебная литература:

60. Острейковский В.А. Информатика: учебник для вузов. — М.: Высшая школа, 2021

61. Хорошевский В.Ф. Инженерия знаний: учебное пособие. — М.: МГУ, 2020

62. Симонов С.В. Базы данных: учебный курс. — СПб.: Питер, 2022

63. Таненбаум Э., Уэзеролл Д. Компьютерные сети. — СПб.: Питер, 2020

64. Кнут Д. Искусство программирования. Т. 1–3. — М.: Вильямс, 2019

65. Гайдамакин Н.А. Автоматизированные информационные системы, базы и банки данных. — М.: Гелиос АРВ, 2018

Дополнительные ресурсы:

- Официальный портал нормативно-правовой информации: pravo.gov.ru
- База данных ГОСТов: gost.ru
- Методические материалы ФСТЭК: fstec.ru
- Образовательные ресурсы Минцифры: digital.gov.ru

Документ подготовлен в соответствии с требованиями к академическому изложению, с сохранением исходной структуры, нумерации и акцентом на теоретические аспекты. Все формулы представлены в формате, совместимом с редакторами, поддерживающими (Microsoft Word, LibreOffice Math, Overleaf). Для оформления итогового отчёта рекомендуется использовать стиль ГОСТ 7.32-2017 «Отчёт о научно-исследовательской работе».